HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



TIỂU LUẬN MÔN HỌC
PHÁT TRIỂN HỆ THỐNG THÔNG MINH

ĐỀ TÀI

AI CHO BỆNH ĐÁI THÁO ĐƯỜNG: TỪ HỌC MÁY, CNN, RNN ĐẾN KNOWLEDGE GRAPH, RAG, DEEP GRAPH VÀ HỆ ĐA TÁC TỬ

Tiểu luận final: hoàn thành các chương (Mở đầu – Chương 7)

Giảng viên hướng dẫn:	PGS. TS. Trần Đình Quế
Sinh viên thực hiện:	Nguyễn Tự Kiên
Mã sinh viên:	B23DCCN465
Nhóm lớp:	CT01
Nhóm tiểu luận:	07

Hà Nội, tháng 11 năm 2026

Mục lục

DANH MỤC TỪ VIẾT TẮT . . . . . **vii**

MỞ ĐẦU **1**

CHƯƠNG 1. Lịch sử phát triển của trí tuệ nhân tạo **5**
1.1. Khái niệm trí tuệ nhân tạo 5
1.2. Các giai đoạn phát triển 5
1.2.1. Nền tảng tiền đề (trước 1956) 5
1.2.2. Khai sinh và thời kỳ lạc quan (1956 – đầu thập niên 1970) . . . 6
1.2.3. Mùa đông AI thứ nhất (khoảng 1974 – 1980) 6
1.2.4. Hệ chuyên gia và mùa đông AI thứ hai (1980 – đầu thập niên 1990) 6
1.2.5. Học máy thống kê (thập niên 1990 – 2000) 7
1.2.6. Kỷ nguyên học sâu (2006 – 2019) 7
1.2.7. Mô hình nền tảng, AI tạo sinh và tác tử (2020 – nay) 7
1.3. Thực nghiệm: bốn thế hệ mô hình trên bộ Pima 8
1.3.1. Dữ liệu 8
1.3.2. Mô hình 10
1.3.3. Kết quả và nhận xét 11
1.4. Triển khai 13
1.5. Tiểu kết chương 1 13

CHƯƠNG 2. Các kỹ thuật học máy cơ bản **14**
2.1. Bài toán học và quy trình xây dựng mô hình 14
2.2. Tối ưu hóa bằng hạ gradient 14
2.3. Các thuật toán 15
2.3.1. Hồi quy tuyến tính 15
2.3.2. Hồi quy logistic 15
2.3.3. K láng giềng gần nhất 15
2.3.4. Naive Bayes 16
2.3.5. Cây quyết định và các phương pháp tổ hợp 16
2.3.6. Máy vector hỗ trợ 16
2.3.7. Gom cụm K-means và giảm chiều PCA 17
2.3.8. Quá khớp, thiếu khớp và điều chuẩn 17
2.3.9. Thước đo đánh giá 17

2.4. Dữ liệu NHANES 2011–2018 . . . 18
2.5. Cài đặt . . . 20
2.6. Kết quả và nhận xét . . . 21
2.6.1. Phân loại . . . 21
2.6.2. Hồi quy HbA1c . . . 24
2.6.3. Gom cụm không giám sát . . . 25
2.6.4. So sánh tổng hợp . . . 26
2.7. Mạng nơ-ron nhiều lớp cài đặt bằng ba cách . . . 26
2.7.1. Dữ liệu Diabetes Prediction . . . 27
2.7.2. Một kiến trúc, ba cách viết . . . 27
2.7.3. Kết quả . . . 29
2.8. Đối chiếu trên hai miền khác . . . 29
2.9. Mạng nhiều lớp dưới dạng hợp các hàm . . . 30
2.10. Thực nghiệm trên Notebook và Demo Localhost . . . 31
2.11. Triển khai . . . 32
2.12. Tiểu kết chương 2 . . . 32

CHƯƠNG 3. Mạng nơ-ron tích chập (CNN) . . . 34
3.1. Động lực của CNN . . . 34
3.2. Các lớp của CNN . . . 34
3.2.1. Lớp tích chập . . . 34
3.2.2. Hàm kích hoạt . . . 35
3.2.3. Lớp gộp . . . 35
3.2.4. Chuẩn hóa theo lô . . . 36
3.2.5. Dropout, lớp kết nối đầy đủ và softmax . . . 36
3.2.6. Hàm mất mát và huấn luyện . . . 36
3.2.7. Tăng cường dữ liệu và học chuyển giao . . . 36
3.2.8. Các kiến trúc tiêu biểu . . . 37
3.2.9. Giải thích bằng Grad-CAM . . . 37
3.3. Dữ liệu RetinaMNIST . . . 38
3.4. Cài đặt . . . 39
3.5. Kết quả và nhận xét . . . 41
3.6. CNN cài đặt bằng ba cách . . . 44
3.7. CNN dưới dạng hợp các hàm . . . 47
3.8. Đối chiếu trên ảnh tự nhiên . . . 47
3.9. Thực nghiệm trên Notebook và Demo Localhost . . . 48
3.10. Triển khai . . . 49
3.11. Tiểu kết chương 3 . . . 50

CHƯƠNG 4. Mạng nơ-ron hồi quy (RNN) . . . 51
4.1. Dữ liệu tuần tự và kiến trúc RNN . . . 51
4.2. Lan truyền ngược theo thời gian và vấn đề biến mất gradient . . . 51

4.3. LSTM và GRU . . . 52
4.3.1. Bộ nhớ dài – ngắn hạn (LSTM) . . . 52
4.3.2. Đơn vị hồi quy có cổng (GRU) . . . 53
4.3.3. Các mở rộng . . . 53
4.3.4. Đánh giá mô hình tuần tự và các bẫy thường gặp . . . 53
4.3.5. RNN và Transformer cho chuỗi thời gian . . . 54
4.4. Dữ liệu ShanghaiT1DM / ShanghaiT2DM . . . 54
4.5. Cài đặt . . . 55
4.6. Kết quả và nhận xét . . . 57
4.6.1. Phân tích sai số theo dải đường huyết . . . 59
4.7. Mạng hồi quy cài đặt bằng ba cách . . . 60
4.8. LSTM xếp chồng 5 lớp dưới dạng hợp các hàm . . . 62
4.9. Đối chiếu trên chuỗi tài chính và thương mại . . . 63
4.10. Thực nghiệm trên Notebook và Demo Localhost . . . 64
4.11. Triển khai . . . 65
4.12. Tiểu kết chương 4 . . . 65

CHƯƠNG 5. Đồ thị tri thức và sinh văn bản tăng cường truy xuất . . . 67
5.1. Đồ thị tri thức . . . 67
5.1.1. Định nghĩa và mô hình dữ liệu . . . 67
5.1.2. Xây dựng đồ thị tri thức . . . 68
5.1.3. Suy luận và nhúng đồ thị tri thức . . . 68
5.2. Sinh văn bản tăng cường truy xuất . . . 69
5.2.1. Nguyên lý . . . 69
5.2.2. Quy trình . . . 69
5.2.3. GraphRAG . . . 70
5.2.4. So sánh RAG và tinh chỉnh . . . 71
5.2.5. Thách thức vận hành . . . 71
5.2.6. Đánh giá hệ thống RAG . . . 72
5.3. Dữ liệu . . . 72
5.3.1. Knowledge Base tiếng Việt và đồ thị tri thức . . . 72
5.3.2. MedQuAD — phần NIDDK . . . 73
5.4. Thực nghiệm . . . 74
5.4.1. Thiết lập . . . 74
5.4.2. Kết quả truy xuất trên KB tiếng Việt . . . 76
5.4.3. Kết quả truy xuất trên MedQuAD . . . 78
5.4.4. Kết quả sinh câu trả lời . . . 79
5.4.5. Ví dụ: một câu hỏi đa bước qua ba chế độ . . . 80
5.5. Triển khai . . . 81
5.5.1. Thực nghiệm trên Notebook và Demo Localhost . . . 81
5.6. Tiểu kết chương 5 . . . 83

CHƯƠNG 6. Học sâu trên đồ thị **84**
6.1. Dữ liệu đồ thị và các bài toán 84
6.2. Mạng nơ-ron đồ thị 84
6.2.1. Khung truyền thông điệp 84
6.2.2. Nền tảng phổ: từ biến đổi Fourier trên đồ thị đến GCN 85
6.2.3. Mạng tích chập đồ thị (GCN) 85
6.2.4. GraphSAGE 87
6.2.5. Mạng chú ý trên đồ thị (GAT) 87
6.2.6. Làm trơn quá mức 87
6.2.7. Năng lực biểu diễn và phép thử Weisfeiler–Lehman 88
6.2.8. Các bài toán khác và ứng dụng y sinh 88
6.3. Dữ liệu Pubmed-Diabetes 89
6.4. Cài đặt 90
6.5. Kết quả và nhận xét 92
6.5.1. Ablation các lựa chọn thiết kế của GCN 95
6.5.2. Học chuyển dẫn và học quy nạp 95
6.5.3. Chi phí tính toán 96
6.5.4. Phân tích lỗi và trọng số chú ý của mô hình triển khai 97
6.5.5. Hạn chế 98
6.6. Một khung GNN chung cho ba đồ thị trích dẫn 98
6.7. GCN dưới dạng hợp các hàm 100
6.8. Thực nghiệm trên Notebook và Demo Localhost 101
6.9. Triển khai 102
6.10. Tiểu kết chương 6 102

CHƯƠNG 7. Hệ đa tác tử **104**
7.1. Từ tác tử đơn đến hệ đa tác tử 104
7.1.1. Tác tử 104
7.1.2. Hệ đa tác tử 105
7.2. Bài 1: Hệ đa tác tử học tăng cường — mèo đuổi chuột 105
7.2.1. Môi trường và bài toán 105
7.2.2. Ba loại tác tử mèo 106
7.2.3. Hai mở rộng đa tác tử 106
7.2.4. Kết quả 107
7.3. Bài 2: Hệ đa tác tử LLM — trắc nghiệm đái tháo đường 109
7.3.1. Bài toán và dữ liệu 109
7.3.2. Bốn hệ và đặc tả tác tử 109
7.3.3. Hệ đa tác tử dưới dạng hợp các hàm 110
7.3.4. Kết quả và phân tích lỗi 110
7.3.5. Đối chiếu: giải toán GSM8K 112
7.4. So sánh một tác tử và đa tác tử 114
7.5. Thực nghiệm trên Notebook và Demo trên Localhost 114

7.6. Tiểu kết chương 7 116

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 118

TÀI LIỆU THAM KHẢO 122

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Tiếng Việt
ADA	American Diabetes Association	Hiệp hội Đái tháo đường Hoa Kỳ
AGE	A Graph Extension (Apache AGE)	Phần mở rộng đồ thị cho PostgreSQL
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
AUC	Area Under the (ROC) Curve	Diện tích dưới đường ROC
BM25	Best Matching 25	Hàm xếp hạng BM25
BMI	Body Mass Index	Chỉ số khối cơ thể
BN	Batch Normalization	Chuẩn hóa theo lô
BPTT	Backpropagation Through Time	Lan truyền ngược theo thời gian
CF	Certainty Factor	Hệ số chắc chắn
CGM	Continuous Glucose Monitoring	Theo dõi glucose liên tục
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
CV	Coefficient of Variation	Hệ số biến thiên
DL	Deep Learning	Học sâu
DR	Diabetic Retinopathy	Bệnh võng mạc đái tháo đường
ĐTĐ	–	Đái tháo đường
FIPA	Foundation for Intelligent Physical Agents	Tổ chức chuẩn hóa tác tử FIPA
FPG	Fasting Plasma Glucose	Glucose huyết tương lúc đói
GAT	Graph Attention Network	Mạng chú ý trên đồ thị
GBDT	Gradient Boosted Decision Trees	Cây quyết định tăng cường gradient
GCN	Graph Convolutional Network	Mạng tích chập đồ thị
GNN	Graph Neural Network	Mạng nơ-ron đồ thị
GRU	Gated Recurrent Unit	Đơn vị hồi quy có cổng
HbA1c	Glycated Hemoglobin	Huyết sắc tố gắn glucose
HNSW	Hierarchical Navigable Small World	Đồ thị thế giới nhỏ phân cấp
ICDR	International Clinical Diabetic Retinopathy scale	Thang phân độ lâm sàng quốc tế bệnh võng mạc ĐTĐ
KB	Knowledge Base	Cơ sở tri thức
KG	Knowledge Graph	Đồ thị tri thức
KNN	K-Nearest Neighbors	K láng giềng gần nhất
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LSTM	Long Short-Term Memory	Bộ nhớ dài – ngắn hạn

Viết tắt	Tiếng Anh	Tiếng Việt
MAD	Mean Average Distance	Khoảng cách trung bình giữa các biểu diễn
MAE	Mean Absolute Error	Sai số tuyệt đối trung bình
MARD	Mean Absolute Relative Difference	Sai lệch tương đối tuyệt đối trung bình
MAS	Multi-Agent System	Hệ đa tác tử
ML	Machine Learning	Học máy
MLP	Multilayer Perceptron	Perceptron nhiều lớp
MRR	Mean Reciprocal Rank	Trung bình nghịch đảo thứ hạng
NHANES	National Health and Nutrition Examination Survey	Khảo sát Sức khỏe và Dinh dưỡng Quốc gia Hoa Kỳ
nDCG	Normalized Discounted Cumulative Gain	Lợi ích tích lũy chiết khấu chuẩn hóa
OGTT	Oral Glucose Tolerance Test	Nghiệm pháp dung nạp glucose đường uống
PCA	Principal Component Analysis	Phân tích thành phần chính
PLA	Perceptron Learning Algorithm	Thuật toán học perceptron
PR-AUC	Area Under the Precision–Recall Curve	Diện tích dưới đường Precision–Recall
QWK	Quadratic Weighted Kappa	Hệ số Kappa có trọng số bậc hai
RAG	Retrieval-Augmented Generation	Sinh văn bản tăng cường truy xuất
ReLU	Rectified Linear Unit	Hàm kích hoạt tuyến tính chỉnh lưu
RF	Random Forest	Rừng ngẫu nhiên
RL	Reinforcement Learning	Học tăng cường
RMSE	Root Mean Squared Error	Căn bậc hai sai số bình phương trung bình
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy
ROC	Receiver Operating Characteristic	Đường cong đặc trưng hoạt động
RRF	Reciprocal Rank Fusion	Hợp nhất thứ hạng nghịch đảo
SGD	Stochastic Gradient Descent	Hạ gradient ngẫu nhiên
SVM	Support Vector Machine	Máy vector hỗ trợ
SVR	Support Vector Regression	Hồi quy vector hỗ trợ
TBR / TIR / TAR	Time Below / In / Above Range	Thời gian dưới / trong / trên khoảng mục tiêu
TF-IDF	Term Frequency – Inverse Document Frequency	Tần suất từ – nghịch đảo tần suất văn bản

Danh sách hình vẽ

Hình MĐ.1. Trang chủ ứng dụng cục bộ *Diabetes AI Lab* (`http://localhost:8095`) tích hợp 19 thực nghiệm từ Chương 2 đến Chương 7 4

Hình 1.1. Phân bố tám đặc trưng của bộ Pima theo nhãn (mật độ chuẩn hóa) 9
Hình 1.2. So sánh sáu mô hình đại diện các giai đoạn lịch sử trên Pima . . 12

Hình 2.1. Tỉ lệ ĐTĐ theo tuổi, phân bố BMI theo nhãn và tỉ lệ thiếu dữ liệu 19
Hình 2.2. Ma trận tương quan Pearson giữa các đặc trưng và nhãn 19
Hình 2.3. Đường ROC và Precision–Recall của tám mô hình trên tập kiểm tra 22
Hình 2.4. Đánh đổi độ chệch – phương sai: AUC train và validation theo k của KNN và độ sâu cây 23
Hình 2.5. Ba tầng đầu của cây quyết định: câu hỏi đầu tiên là tuổi, sau đó là tuổi (nhánh trẻ) và tiền sử gia đình (nhánh lớn tuổi) 23
Hình 2.6. Độ quan trọng hoán vị của các đặc trưng (Gradient Boosting) và ma trận nhầm lẫn 23
Hình 2.7. HbA1c dự đoán so với thực tế: mô hình co về trung bình 24
Hình 2.8. Chọn k bằng inertia và silhouette; bốn cụm chiếu trên hai thành phần chính 25
Hình 2.9. Giao diện thử nghiệm Chương 2 trên `http://localhost:8095`: so sánh trực tiếp xác suất dự đoán của MLP ba cách (NumPy, Keras, PyTorch) và các mô hình cổ điển 32

Hình 3.1. Ba ảnh mẫu của mỗi mức DR trong RetinaMNIST 38
Hình 3.2. Phân bố nhãn theo tập: lớp 0 chiếm 45%, lớp 4 chỉ 5% 39
Hình 3.3. Loss huấn luyện, accuracy và QWK trên validation theo epoch . 42
Hình 3.4. Ma trận nhầm lẫn của MobileNetV3-L trên tập kiểm tra (số ảnh và tỉ lệ theo hàng) 43
Hình 3.5. Bản đồ đặc trưng sau khối 1 (cạnh, độ sáng) và khối 3 (vùng trừu tượng) của CNN tự thiết kế 43
Hình 3.6. Grad-CAM trên tầng `layer4` của ResNet-18 với một ảnh của mỗi mức DR 44
Hình 3.7. Giao diện thử nghiệm Chương 3 trên `http://localhost:8095`: so sánh dự đoán 5 mức bệnh võng mạc đái tháo đường của CNN NumPy, Keras và PyTorch trên cùng ảnh đáy mắt RetinaMNIST 49

Hình 4.1. Phân bố glucose theo loại ĐTĐ, trung vị theo giờ trong ngày và một bản ghi típ 1 trong 4 ngày 55

Hình 4.2. RMSE theo tầm dự báo và lưới sai số Clarke của LSTM ở tầm 30 phút 57
Hình 4.3. Dự báo trước 30 phút trên 24 giờ liên tục của một bệnh nhân thuộc tập kiểm tra 58
Hình 4.4. Độ lớn gradient của dự báo theo từng điểm đầu vào trên chuỗi dài 48 bước (chuẩn hóa theo bước cuối) 59
Hình 4.5. Giao diện thử nghiệm Chương 4 trên `http://localhost:8095`: dự báo đường huyết sau 30 phút bằng 11 mô hình (NumPy BPTT, Keras và PyTorch từ 1 đến 5 lớp) trên cửa sổ CGM 3 giờ 65

Hình 5.1. Hai pha của RAG: nạp tri thức (trên) và trả lời câu hỏi (dưới) . . 69
Hình 5.2. Đồ thị tri thức ĐTĐ: 33 nút thuộc 6 loại, 40 cạnh thuộc 7 loại quan hệ (kích thước nút tỉ lệ với bậc) 73
Hình 5.3. Độ dài đoạn của KB tiếng Việt, độ dài câu trả lời và phân bố loại câu hỏi của MedQuAD-NIDDK 74
Hình 5.4. So sánh các bộ truy xuất trên KB tiếng Việt (trái) và MedQuAD-NIDDK (phải) 77
Hình 5.5. Giao diện thử nghiệm Chương 5 trên `http://localhost:8095`: đối chiếu câu trả lời và bằng chứng truy xuất của LLM thuần, Text-RAG và KG-RAG trên đồ thị tri thức đái tháo đường 82

Hình 6.1. Phân bố nhãn, phân bố bậc (thang log–log) và số từ khác 0 của mỗi bài báo 90
Hình 6.2. Accuracy theo số nhãn huấn luyện mỗi lớp: đồ thị có lợi nhất khi nhãn khan hiếm 93
Hình 6.3. t-SNE của 3.000 bài báo: đặc trưng TF-IDF gốc (trái) và biểu diễn ẩn của GCN (phải) 93
Hình 6.4. Làm trơn quá mức: accuracy và MAD của GCN theo số tầng . . 94
Hình 6.5. Loss huấn luyện và accuracy validation theo epoch (20 nhãn/lớp, một phép chia) 94
Hình 6.6. Giao diện thử nghiệm Chương 6 trên `http://localhost:8095`: so sánh dự đoán của GCN, GraphSAGE, GAT và MLP (không dùng đồ thị) trên cùng một bài báo của đồ thị trích dẫn PubMed-Diabetes 102

Hình 7.1. Một ván từ cùng vị trí xuất phát: mèo theo luật kẹt sau tường; mèo Q-learning vòng qua tường và bắt được sau 12 bước 107
Hình 7.2. Đường cong học (tỷ lệ bắt được và số bước theo số tập, khi huấn luyện còn khám phá ϵ) 108
Hình 7.3. Thí nghiệm C — ma trận đối kháng: hàng là mèo, cột là chuột . 108
Hình 7.4. Hiệu năng, chi phí và cơ cấu đồng thuận/lỗi của bốn hệ tác tử trên 150 câu trắc nghiệm đái tháo đường MedMCQA 111
Hình 7.5. Kết quả bốn hệ tác tử trên bài toán giải toán nhiều bước GSM8K (Qwen2.5-1.5B-Instruct) 113
Hình 7.6. Giao diện mô phỏng tương tác Bài 1 (Mèo đuổi chuột đa tác tử) tại `http://localhost:8095/catmouse` 116

Hình 7.7. Giao diện thử nghiệm Bài 2 (Hệ đa tác tử LLM giải trắc nghiệm đái tháo đường MedMCQA) tại `http://localhost:8095` 116

Danh sách bảng

Bảng MĐ.1. Các bộ dữ liệu chính về đái tháo đường dùng trong tiểu luận . . . 3

Bảng 1.1. Tóm tắt các giai đoạn phát triển chính của trí tuệ nhân tạo 8
Bảng 1.2. Năm mẫu đầu tiên của bộ Pima (số 0 ở cột sinh lý là giá trị thiếu) 9
Bảng 1.3. Cơ sở luật của hệ chuyên gia 10
Bảng 1.4. Kết quả kiểm định chéo phân tầng 10 phần trên Pima (trung bình ± độ lệch chuẩn) 11

Bảng 2.1. Năm mẫu đầu của bộ NHANES đã xử lý (một số cột) 18
Bảng 2.2. Thống kê mô tả các biến liên tục của NHANES 19
Bảng 2.3. Dò siêu tham số SVM (RBF) trên tập validation 21
Bảng 2.4. Kết quả phân loại ĐTĐ trên tập kiểm tra NHANES (4.130 người, ngưỡng Youden) 21
Bảng 2.5. Dự đoán HbA1c (%) từ 13 yếu tố nguy cơ (20.413 người có HbA1c, chia 80/20) 24
Bảng 2.6. Hồ sơ trung bình của bốn cụm K-means và tỉ lệ ĐTĐ (nhãn không dùng khi gom cụm) 25
Bảng 2.7. So sánh định tính các thuật toán cho bài toán sàng lọc 26
Bảng 2.8. Phần việc lập trình viên tự viết ở ba cách cài đặt 27
Bảng 2.9. MLP ba cách và các thuật toán cổ điển trên Diabetes Prediction (20.000 hồ sơ kiểm tra) 29
Bảng 2.10. Đối chiếu: cùng quy trình trên hồi quy giá nhà và phân loại văn bản (20% kiểm tra) 30

Bảng 3.1. Các kiến trúc CNN tiêu biểu 37
Bảng 3.2. Số ảnh theo mức DR và trọng số lớp dùng trong hàm mất mát . . 39
Bảng 3.3. Kích thước tensor và số tham số qua từng lớp của CNN tự thiết kế 41
Bảng 3.4. Kết quả phân độ DR trên tập kiểm tra RetinaMNIST (400 ảnh) . 41
Bảng 3.5. CNN ba cách trên RetinaMNIST (400 ảnh test; luôn đoán mức 0 đạt accuracy 0,435) 46
Bảng 3.6. Đối chiếu: cùng CNN trên ảnh chữ số và trang phục (10.000 ảnh test mỗi bộ) 48

Bảng 4.1. Ba dòng đầu của một bản ghi CGM (bệnh nhân 1001, ĐTĐ típ 1) 54
Bảng 4.2. Chỉ số kiểm soát đường huyết của hai nhóm theo đồng thuận quốc tế về Time in Range [63] 55

Bảng 4.3. Kích thước tensor và số tham số của các mô hình hồi quy ($d = 3$, $m = 64$) . . . 56
Bảng 4.4. Dự báo glucose trên 17 bệnh nhân của tập kiểm tra (18.012 cửa sổ) 57
Bảng 4.5. Kết quả tầm 30 phút theo loại ĐTĐ trên tập kiểm tra . . . 58
Bảng 4.6. Sai số dự báo 30 phút theo dải glucose tham chiếu (tập kiểm tra, LSTM 2 tầng) . . . 60
Bảng 4.7. Dự báo glucose 30 phút trên 17 bệnh nhân kiểm tra (18.056 cửa sổ): ba cách cài đặt . . . 61
Bảng 4.8. Đối chiếu: LSTM và GRU so với dự báo ngây thơ trên ba chuỗi khác (tập kiểm tra theo thời gian) . . . 63

Bảng 5.1. So sánh RAG và tinh chỉnh (fine-tuning) trong bối cảnh chatbot y tế 71
Bảng 5.2. Ví dụ đoạn văn trong KB và bộ ba trong đồ thị tri thức . . . 72
Bảng 5.3. Truy xuất trên KB tiếng Việt (60 câu hỏi, 36 đoạn) . . . 76
Bảng 5.4. Hit@3 theo loại câu hỏi trên KB tiếng Việt . . . 76
Bảng 5.5. Truy xuất câu trả lời trên MedQuAD-NIDDK (1.192 câu hỏi, 3.381 đoạn 800 ký tự) . . . 78
Bảng 5.6. Ảnh hưởng của kích thước đoạn (bộ lai BM25 + MiniLM trên MedQuAD) . . . 78
Bảng 5.7. Truy xuất chéo ngôn ngữ: 166 câu hỏi ĐTĐ dịch sang tiếng Việt, tìm trên kho tiếng Anh . . . 79
Bảng 5.8. Chất lượng câu trả lời của Gemini theo ba chế độ (60 câu hỏi tiếng Việt) . . . 79
Bảng 5.9. Câu trả lời thật của ba chế độ cho một câu hỏi đa bước . . . 80

Bảng 6.1. Năm nút đầu tiên: mã PubMed, nhãn, bậc và các từ có TF-IDF cao nhất . . . 89
Bảng 6.2. Mười từ có TF-IDF trung bình cao nhất của mỗi lớp (dạng từ gốc sau stemming) . . . 89
Bảng 6.3. Cấu hình và số tham số của các mô hình (đặc trưng vào 500 chiều, 3 lớp) . . . 91
Bảng 6.4. Accuracy phân loại nút trên Pubmed-Diabetes (trung bình ± độ lệch chuẩn) . . . 92
Bảng 6.5. Ablation GCN trên giao thức bán giám sát (20 nhãn/lớp, 10 phép chia) . . . 95
Bảng 6.6. Học chuyển dẫn và học quy nạp trên Pubmed-Diabetes (60/20/20, 3 lần chạy) . . . 96
Bảng 6.7. Số tham số và thời gian huấn luyện trung bình một lần chạy (GPU RTX 3050) . . . 96
Bảng 6.8. Ma trận nhầm lẫn của GAT triển khai trên 3.944 bài báo giữ riêng (20%) . . . 97
Bảng 6.9. Accuracy phân loại nút trên ba đồ thị (trung bình ± độ lệch chuẩn qua 5 hạt giống; 1.000 nút test) . . . 99

Bảng 7.1. Đặc tả PEAS của các tác tử trong hai bài thực nghiệm . . . 104

Bảng 7.2. Kết quả ba thí nghiệm mèo đuổi chuột (300 ván kiểm tra, vị trí xuất phát cố định, chính sách tham lam) 107
Bảng 7.3. Bốn hệ được so sánh trên cùng 150 câu 109
Bảng 7.4. Kết quả bốn hệ tác tử trên 150 câu hỏi trắc nghiệm đái tháo đường MedMCQA (Qwen2.5-1.5B-Instruct chạy cục bộ) 110
Bảng 7.5. Đối chiếu: cùng LLM Qwen2.5-1.5B-Instruct trên 40 bài toán GSM8K và 150 câu y khoa MedMCQA 113
Bảng 7.6. Một tác tử và đa tác tử qua hai bài thực nghiệm 114
Bảng 7.7. Tổng hợp kết quả thực nghiệm của bảy chương 119

MỞ ĐẦU

1. Lý do chọn đề tài

Trí tuệ nhân tạo (AI) đã đi một chặng đường dài từ những chương trình chứng minh định lý của thập niên 1950 tới các hệ thống tác tử dựa trên mô hình ngôn ngữ lớn hiện nay. Mỗi giai đoạn để lại một lớp kỹ thuật vẫn đang được dùng: luật chuyên gia, học máy thống kê, mạng nơ-ron tích chập và hồi quy, đồ thị tri thức, học sâu trên đồ thị và hệ đa tác tử. Một hệ thống thông minh thực tế hiếm khi chỉ dùng một kỹ thuật duy nhất. Nó thường là sự phối hợp của nhiều mô hình chuyên biệt, mỗi mô hình mạnh ở một loại dữ liệu, được điều phối để cùng đưa ra một quyết định có căn cứ.

Đái tháo đường (ĐTĐ) là một bài toán điển hình cho cách tiếp cận đó. Theo IDF Diabetes Atlas [1], năm 2021 có khoảng 537 triệu người trưởng thành mắc ĐTĐ, và gần một nửa chưa được chẩn đoán. Dữ liệu để sàng lọc lại rất đa dạng: chỉ số tự khai (tuổi, BMI, tiền sử gia đình) phù hợp với học máy trên bảng; ảnh đáy mắt dùng để phát hiện biến chứng võng mạc là đất của CNN; chuỗi đo đường huyết liên tục cần mô hình tuần tự như RNN; hướng dẫn lâm sàng (ADA 2024 [2], Quyết định 5481/QĐ-BYT [3]) là tri thức văn bản cần RAG và đồ thị tri thức; y văn có cấu trúc trích dẫn là dữ liệu đồ thị. Mỗi loại dữ liệu đòi hỏi một lớp kỹ thuật riêng, nên ĐTĐ là một "phòng thí nghiệm" tự nhiên để đi qua toàn bộ các chủ đề của môn học.

Tiểu luận này trình bày tiến trình phát triển của AI qua các chủ đề của môn học. Mỗi chủ đề được kiểm chứng bằng một thực nghiệm độc lập trên một bộ dữ liệu công khai liên quan tới ĐTĐ, có mô hình đối chứng, và được đóng gói thành một bản demo chạy được.

2. Mục tiêu

1. Hệ thống hóa lịch sử AI và các kỹ thuật nền tảng xuyên suốt môn học: học máy cơ bản, CNN, RNN/LSTM, đồ thị tri thức và RAG, mạng nơ-ron đồ thị (GNN) và hệ đa tác tử (Multi-Agent Systems — cả học tăng cường lẫn tác tử LLM). Với mỗi mô hình, giải thích chi tiết từng lớp (layer) và biểu diễn mô hình dưới dạng hợp của các hàm toán học.
2. Với mỗi chương, chọn các bộ dữ liệu công khai liên quan tới ĐTĐ (kèm các miền đối chiếu ngoài y tế), mô tả đường dẫn, kích thước, mẫu dữ liệu và phân bố; cài

đặt mạng nơ-ron của các chương 2–4 bằng cả ba cách (tự viết từ đầu bằng NumPy, Keras/TensorFlow và PyTorch) để kiểm chứng đạo hàm và so sánh chi phí.

3. Cài đặt và so sánh các mô hình trên cùng một giao thức đánh giá, kèm mô hình đối chứng. Báo cáo bằng thước đo phù hợp với từng bài toán (ROC-AUC, QWK, RMSE và lưới Clarke, Hit@k và MRR, độ chính xác phân loại nút, tỷ lệ bắt được trong trò chơi Markov và độ chính xác/lỗi phối hợp của hệ đa tác tử).
4. Đóng gói toàn bộ các thực nghiệm từ Chương 2 đến Chương 7 thành bộ 6 Jupyter Notebook chạy tái lập được và một ứng dụng web cục bộ (`http://localhost:8095`) để kiểm tra trực tiếp.

3. Đối tượng, phạm vi và dữ liệu

Đối tượng nghiên cứu là các mô hình và thuật toán của môn Phát triển hệ thống thông minh, áp dụng vào bài toán sàng lọc, theo dõi và hỏi đáp ĐTĐ. Bảng MĐ.1 liệt kê các bộ dữ liệu chính được dùng (cùng các bộ đối chiếu đa miền ở từng chương). Tất cả đều công khai và tải được bằng mã nguồn kèm theo. Thực nghiệm được chạy trực tiếp trên một máy tính cá nhân (CPU Intel i5-11400H, 16 GB RAM, GPU RTX 3050 4 GB). Mục tiêu là kiểm chứng lý thuyết và giải phẫu cơ chế hoạt động của mô hình, không phải chạy đua điểm số. Hệ thống chỉ phục vụ học tập và nghiên cứu, *không thay thế chẩn đoán của bác sĩ.*

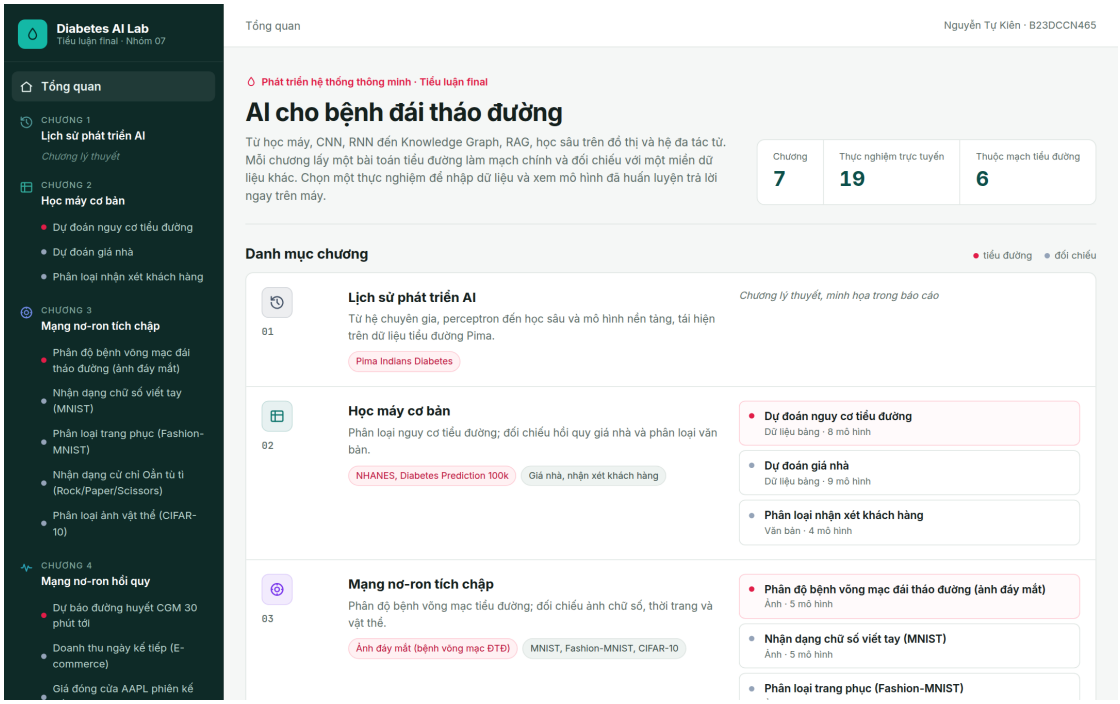
Bảng MĐ.1. Các bộ dữ liệu chính về đái tháo đường dùng trong tiểu luận

Ch.	Bộ dữ liệu chính	Quy mô	Bài toán
1	Pima Indians Diabetes [4]	768 người, 8 đặc trưng	Phân loại ĐTĐ: luật chuyên gia → perceptron → MLP
2	NHANES [5] & Diabetes 100k	20.646 & 100.000 người	Phân loại, hồi quy HbA1c, gom cụm; MLP 3 cách (NumPy/Keras/PyTorch)
3	RetinaMNIST [6]	1.600 ảnh đáy mắt	Phân độ bệnh võng mạc ĐTĐ (5 mức); CNN 3 cách & chuyển giao
4	ShanghaiT1DM/T2DM [7]	112 bệnh nhân, 125 bản ghi CGM	Dự báo glucose 15–60 phút; RNN/LSTM/GRU 3 cách & xếp chồng 5 lớp
5	KB ADA 2024 + MedQuAD [8]	36 đoạn + 1.192 hỏi–đáp + KG 96k nút	Truy xuất thưa/dày, xếp hạng lại, GraphRAG và sinh câu trả lời có trích dẫn
6	Pubmed-Diabetes [9]	19.717 bài báo, 44.324 cạnh	Phân loại nút bán giám sát và có giám sát bằng GCN, GraphSAGE, GAT
7	Lưới Mèo–Chuột & MedMCQA [10]	Lưới 8×8 (157k trạng thái) & 150 câu ĐTĐ	Đa tác tử học tăng cường (hợp tác IQL, đối kháng) & Đa tác tử LLM tranh luận

4. Phương pháp nghiên cứu và đóng góp kế thừa

Báo cáo kết hợp bốn phương pháp. *Tổng hợp tài liệu*: hệ thống hóa kiến thức từ giáo trình và các bài báo gốc (Perceptron, LSTM, ResNet, GCN, GAT, RAG, Q-learning, Debate…). *Lịch sử – logic và hợp hàm*: trình bày mỗi kỹ thuật như lời giải cho hạn chế của kỹ thuật đi trước, đồng thời biểu diễn tường minh mỗi mạng sâu như một hợp hàm toán học. *Thực nghiệm có đối chứng và đa cách cài đặt*: mỗi chương có mô hình nền đơn giản (persistence, MLP phẳng, chỉ BM25, tác tử đơn…) và tự cài đặt từ đầu bằng NumPy/PyTorch thưa để kiểm chứng đạo hàm trước khi đối chiếu với Keras và PyTorch. *Kỹ nghệ hệ thống*: đóng gói thành 6 notebook (`notebooks/ch2_*.ipynb` đến `ch7_*.ipynb`) và cổng thử nghiệm thống nhất tại `http://localhost:8095` (Hình MĐ.1). Một số khung mã đối chiếu đa miền (`src/tl03`) được kế thừa và mở rộng từ phần làm việc chung của thành viên cùng nhóm 07 (Nguyễn Duy Nghĩa), được ghi chú rõ tại từng mục liên quan; toàn bộ thực nghiệm đái tháo đường, bài toán Mèo–Chuột mở rộng, trắc

nghiệm MedMCQA và số liệu trong báo cáo này đều được huấn luyện và đo lại hoàn toàn trên máy thực nghiệm.



Hình MĐ.1. Trang chủ ứng dụng cục bộ Diabetes AI Lab *(`http://localhost:8095`) tích hợp 19 thực nghiệm từ Chương 2 đến Chương 7*

5. Bố cục của tiểu luận

Ngoài Mở đầu, Kết luận và Tài liệu tham khảo, tiểu luận gồm bảy chương. **Chương 1** điểm lại lịch sử AI và tái hiện bốn thế hệ mô hình trên bộ Pima. **Chương 2** trình bày các thuật toán học máy cơ bản trên NHANES và cài đặt MLP bằng ba cách trên 100.000 hồ sơ tiểu đường. **Chương 3** phân tích từng lớp của CNN, cài đặt CNN bằng ba cách (`im2col`) và áp dụng vào phân độ bệnh võng mạc. **Chương 4** trình bày RNN, LSTM, GRU, cài đặt BPTT bằng ba cách và khảo sát LSTM 5 lớp trên chuỗi đường huyết CGM. **Chương 5** xây dựng đồ thị tri thức và đánh giá các chiến lược RAG / GraphRAG. **Chương 6** trình bày GCN, GraphSAGE, GAT trên đồ thị trích dẫn y văn PubMed-Diabetes và đối chiếu với Cora, CiteSeer. **Chương 7** trình bày hệ đa tác tử qua hai bài toán: học tăng cường đa tác tử (mèo đuổi chuột hợp tác và đối kháng) và hệ đa tác tử LLM giải trắc nghiệm y khoa MedMCQA (đối chiếu với giải toán GSM8K).

CHƯƠNG 1. LỊCH SỬ PHÁT TRIỂN CỦA TRÍ TUỆ NHÂN TẠO

Chương này tóm lược gần tám thập kỷ phát triển của AI. Mục tiêu là chỉ ra rằng mỗi cách tiếp cận ra đời để khắc phục giới hạn của cách tiếp cận trước. Phần thực nghiệm cuối chương tái hiện dòng lịch sử đó trên một bộ dữ liệu y khoa kinh điển: cùng một bài toán dự đoán đái tháo đường được giải lần lượt bằng hệ chuyên gia luật (thập niên 1970–80), perceptron (1958), ADALINE (1960), mạng nhiều lớp với lan truyền ngược (1986) và các phương pháp học máy thống kê hiện đại.

1.1. Khái niệm trí tuệ nhân tạo

Trí tuệ nhân tạo là ngành khoa học máy tính nghiên cứu cách xây dựng hệ thống thực hiện được những nhiệm vụ vốn đòi hỏi trí thông minh của con người: nhận thức, suy luận, học, lập kế hoạch, giao tiếp và ra quyết định. Russell và Norvig [11] phân loại các định nghĩa AI theo hai trục: *suy nghĩ* hay *hành động*, *giống người* (ví dụ phép thử Turing [12]) hay *hợp lý*. Cách tiếp cận *tác tử hợp lý* được dùng phổ biến nhất: tác tử nhận tri giác từ môi trường và chọn hành động tối đa hóa một thước đo hiệu năng, nên đo đạc và tối ưu được bằng toán học. Đây cũng là khái niệm nền của các hệ thống tác tử hiện đại.

AI, học máy (ML) và học sâu (DL) là ba khái niệm lồng nhau. Học máy là nhánh của AI tập trung vào việc học quy luật từ dữ liệu thay vì lập trình tường minh. Học sâu là nhánh của học máy, dùng mạng nơ-ron nhiều lớp để tự học biểu diễn phân cấp. Phần lớn hệ thống hiện nay vẫn là *AI hẹp*, giỏi một nhiệm vụ cụ thể. *AI tổng quát* (AGI) có năng lực ngang con người trên phạm vi rộng vẫn là mục tiêu nghiên cứu.

1.2. Các giai đoạn phát triển

1.2.1. Nền tảng tiền đề (trước 1956)

Ba dòng chảy khoa học hội tụ để tạo nên AI. Thứ nhất là *logic và lý thuyết tính toán*: máy Turing (1936) hình thức hóa khái niệm thuật toán và gợi ra ý tưởng rằng tư duy, nếu là xử lý ký hiệu, có thể cơ giới hóa. Thứ hai là *mô hình nơ-ron*: McCulloch và Pitts (1943) [13] mô tả nơ-ron nhân tạo bằng tổng có trọng số qua một ngưỡng, còn Hebb (1949) đề xuất quy tắc "cùng kích hoạt thì liên kết mạnh lên". Thứ ba là *điều khiển học* và máy tính điện tử, cho phép kiểm chứng ý tưởng trên phần cứng thật. Năm 1950, Turing [12] đặt

câu hỏi "Máy có thể suy nghĩ không?" và đề xuất "trò chơi bắt chước", một tiêu chí hành vi thay cho tranh luận trừu tượng về bản chất của tư duy.

1.2.2. Khai sinh và thời kỳ lạc quan (1956 – đầu thập niên 1970)

Hội thảo Dartmouth mùa hè năm 1956 do McCarthy, Minsky, Rochester và Shannon tổ chức [14] đã khai sinh thuật ngữ *Artificial Intelligence*. Hai thập niên sau đó có nhiều chương trình gây ấn tượng: Logic Theorist (1956) chứng minh định lý toán học, LISP (1958) trở thành ngôn ngữ chủ đạo, ELIZA (1966) mô phỏng nhà trị liệu chỉ bằng luật khớp mẫu. Cũng trong giai đoạn này, Rosenblatt (1958) [15] công bố *perceptron*, mạng nơ-ron một lớp học được trọng số từ dữ liệu, còn Widrow và Hoff (1960) [16] đề xuất ADALINE với quy tắc học delta, tiền thân trực tiếp của hạ gradient ngày nay. Hướng tiếp cận chủ đạo khi đó là *AI biểu tượng*: tri thức được biểu diễn bằng ký hiệu và luật, còn trí thông minh được xem là quá trình tìm kiếm và suy diễn trên các biểu diễn đó.

1.2.3. Mùa đông AI thứ nhất (khoảng 1974 – 1980)

Các chương trình chạy tốt trên "thế giới đồ chơi" nhưng thất bại với bài toán thực vì ba lý do: *bùng nổ tổ hợp* của không gian tìm kiếm, *thiếu tri thức thường thức*, và *giới hạn của mạng một lớp*. Năm 1969, Minsky và Papert [17] chứng minh perceptron một lớp không biểu diễn được hàm XOR. Khi đó chưa có cách huấn luyện mạng nhiều lớp, nên nghiên cứu mạng nơ-ron gần như đình trệ. Báo cáo Lighthill (1973) [18] kết luận AI không đạt các mục tiêu đã hứa, và tài trợ ở Anh và Mỹ bị cắt giảm mạnh.

1.2.4. Hệ chuyên gia và mùa đông AI thứ hai (1980 – đầu thập niên 1990)

AI hồi sinh nhờ *hệ chuyên gia*: tri thức của chuyên gia trong một lĩnh vực hẹp được mã hóa thành luật NẾU–THÌ, kết hợp với một bộ máy suy diễn tiến hoặc lùi. MYCIN (Stanford) chẩn đoán nhiễm trùng máu với khoảng 600 luật. Để xử lý sự không chắc chắn, MYCIN gắn cho mỗi luật một *hệ số chắc chắn* (Certainty Factor – CF) và gộp chúng theo công thức của Shortliffe và Buchanan [19]:

$$\mathrm{CF}(H \mid E_1 \wedge E_2) = \mathrm{CF}_1 + \mathrm{CF}_2\,(1 - \mathrm{CF}_1), \qquad \mathrm{CF}_1, \mathrm{CF}_2 \geq 0. \tag{1.1}$$

Công thức (1.1) có tính giao hoán và kết hợp, luôn cho kết quả trong $[0, 1]$: mỗi bằng chứng mới chỉ "lấp" thêm một phần của khoảng còn thiếu. XCON (1980) là thành công thương mại lớn đầu tiên. Cùng thời gian, Rumelhart, Hinton và Williams (1986) [20] phổ biến thuật toán *lan truyền ngược*, cho phép huấn luyện mạng nhiều lớp và giải được bài

toán XOR. Tuy vậy hệ chuyên gia sớm bộc lộ giới hạn: thu nhận tri thức tốn kém, cơ sở luật khó bảo trì, hệ thống "giòn" trước tình huống ngoài luật. Thị trường máy LISP sụp đổ năm 1987 và mùa đông AI thứ hai bắt đầu.

1.2.5. Học máy thống kê (thập niên 1990 – 2000)

Bài học của hai mùa đông là tri thức thế giới thực quá lớn và mơ hồ để mã hóa thủ công. Trọng tâm vì thế chuyển sang các phương pháp *học từ dữ liệu* có nền tảng xác suất: mạng Bayes (Pearl, 1988), máy vector hỗ trợ (Cortes và Vapnik, 1995) [21], rừng ngẫu nhiên (Breiman, 2001) [22], boosting [23], LSTM (1997) [24] và LeNet-5 (1998) [25]. Đặc điểm chung của giai đoạn này là hiệu năng phụ thuộc nặng vào *kỹ nghệ đặc trưng* thủ công (SIFT, HOG, TF-IDF).

1.2.6. Kỷ nguyên học sâu (2006 – 2019)

Học sâu dùng mạng nhiều lớp để tự học biểu diễn phân cấp từ dữ liệu thô [26]. Hinton và cộng sự (2006) [27] khơi lại sự quan tâm bằng tiền huấn luyện từng lớp. ImageNet (2009) [28] cung cấp hàng triệu ảnh gán nhãn. Năm 2012, AlexNet [29] vô địch ILSVRC với lỗi top-5 khoảng 15,3

1.2.7. Mô hình nền tảng, AI tạo sinh và tác tử (2020 – nay)

GPT-3 (2020) [30] với 175 tỉ tham số cho thấy mô hình đủ lớn có thể học từ vài ví dụ trong câu lệnh. Tinh chỉnh theo chỉ dẫn và học tăng cường từ phản hồi người dùng [31] dẫn tới ChatGPT (2022). Năm 2024, giải Nobel Vật lý và Hóa học lần lượt vinh danh nền tảng mạng nơ-ron (Hopfield, Hinton) và AlphaFold [32]. Giai đoạn 2025–2026 nổi bật với ba xu hướng. *Mô hình đa phương thức bản địa* xử lý văn bản, ảnh, âm thanh trong cùng một mạng. *Mô hình suy luận* tăng lượng tính toán lúc suy luận để "nghĩ" nhiều bước. *AI tác tử* có khả năng lập kế hoạch, dùng công cụ, ghi nhớ và tự sửa lỗi. Khi nhiều tác tử phối hợp, ta có hệ đa tác tử. Đồng thời, *ảo giác* của LLM khiến tri thức tường minh trở lại dưới dạng đồ thị tri thức và RAG (Chương 5).

Bảng 1.1. Tóm tắt các giai đoạn phát triển chính của trí tuệ nhân tạo

Giai đoạn	Hướng chủ đạo	Thành tựu tiêu biểu	Hạn chế dẫn tới giai đoạn sau
1943–1955	Nền tảng lý thuyết	Nơ-ron McCulloch–Pitts, phép thử Turing	Chưa có phần cứng, chương trình
1956–1974	AI biểu tượng, tìm kiếm	Dartmouth, Logic Theorist, Perceptron, ADALINE	Bùng nổ tổ hợp; XOR → mùa đông 1
1980–1987	Hệ chuyên gia	MYCIN, XCON; lan truyền ngược	Khó thu nhận, bảo trì tri thức → mùa đông 2
1990–2011	Học máy thống kê	SVM, rừng ngẫu nhiên, LSTM, LeNet-5	Phụ thuộc đặc trưng thủ công
2012–2019	Học sâu	AlexNet, ResNet, AlphaGo, Transformer	Cần nhiều dữ liệu, khó giải thích
2020–2024	Mô hình nền tảng	GPT-3, ChatGPT, AlphaFold, RAG	Ảo giác, chi phí
2025–nay	Đa phương thức, tác tử	Mô hình suy luận, hệ đa tác tử, GraphRAG	An toàn, quản trị tác tử tự chủ

Những thách thức lớn vẫn còn: thiên lệch dữ liệu, khả năng giải thích, độ tin cậy, quyền riêng tư và khung pháp lý (Đạo luật AI của EU năm 2024; Chiến lược AI quốc gia của Việt Nam, Quyết định 127/QĐ-TTg [33]). Trong y tế, các rủi ro này càng nghiêm trọng, nên hệ thống của báo cáo luôn kèm lớp luật an toàn và trích dẫn nguồn.

1.3. Thực nghiệm: bốn thế hệ mô hình trên bộ Pima

1.3.1. Dữ liệu

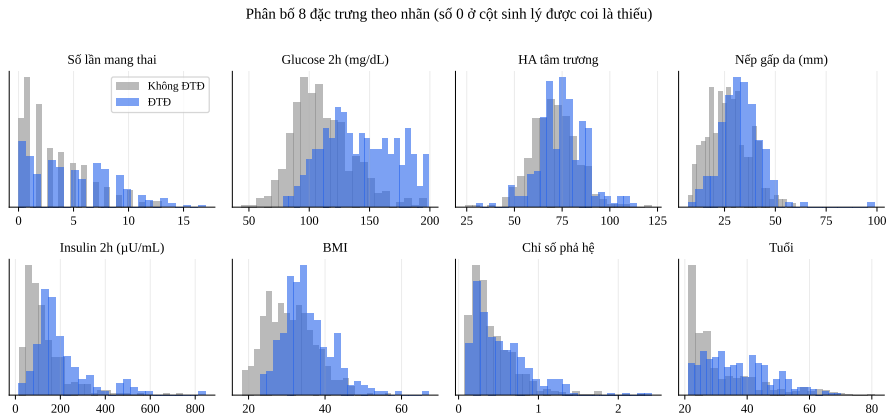
Bộ **Pima Indians Diabetes** do Viện Quốc gia về Đái tháo đường, Tiêu hóa và Thận Hoa Kỳ (NIDDK) thu thập. Smith và cộng sự công bố bộ này năm 1988 khi dùng thuật toán ADAP, một mạng dạng perceptron, để dự đoán khởi phát ĐTĐ [4]. Nhờ vậy, bộ dữ liệu này có ý nghĩa lịch sử riêng với chương này.

- **Nguồn**: UCI Machine Learning Repository, bản sao tại `https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database`; mã nguồn tải từ `https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv`.

- **Kích thước**: 768 phụ nữ gốc Pima ≥ 21 tuổi, 8 đặc trưng số, 1 nhãn; tệp CSV 23 KB.
- **Nhãn**: 268 người (34,9%) mắc ĐTĐ trong vòng 5 năm, 500 người không mắc.

Bảng 1.2. Năm mẫu đầu tiên của bộ Pima (số 0 ở cột sinh lý là giá trị thiếu)

Mang thai	Glucose	HA	Da	Insulin	BMI	Phả hệ	Tuổi	Nhãn
6	148	72	35	0	33,6	0,627	50	1
1	85	66	29	0	26,6	0,351	31	0
8	183	64	0	0	23,3	0,672	32	1
1	89	66	23	94	28,1	0,167	21	0
0	137	40	35	168	43,1	2,288	33	1



Hình 1.1. Phân bố tám đặc trưng của bộ Pima theo nhãn (mật độ chuẩn hóa)

Nhận xét. (i) Glucose 2 giờ sau nghiệm pháp là đặc trưng phân tách mạnh nhất: hai phân bố gần như dịch hẳn sang hai phía quanh 140 mg/dL, đúng ngưỡng rối loạn dung nạp glucose của ADA. (ii) Bộ dữ liệu có *giá trị thiếu ẩn*: số 0 không có nghĩa sinh lý nhưng xuất hiện ở insulin (374 dòng, 48,7%), độ dày nếp da (227 dòng), huyết áp (35), BMI (11) và glucose (5). Nếu không xử lý, mô hình sẽ học sai rằng "insulin = 0" là một mức đo. Thực nghiệm thay số 0 bằng trung vị của *tập huấn luyện trong từng lượt kiểm định* để tránh rò rỉ. (iii) Lớp dương chiếm 34,9%, lệch vừa phải, nên ngoài accuracy cần báo cáo ROC-AUC, F1, độ nhạy và độ đặc hiệu. (iv) Với 768 mẫu, kết quả một lần chia train/test dao động mạnh, nên dùng kiểm định chéo phân tầng 10 phần.

1.3.2. Mô hình

Mỗi mô hình đại diện cho một giai đoạn ở Bảng 1.1. Các mô hình 1 đến 4 được tự cài đặt bằng NumPy để thấy rõ cơ chế.

(1) Hệ chuyên gia. Sáu luật viết tay theo ngưỡng ADA/WHO (Bảng 1.3). Mỗi luật thỏa mãn đóng góp một CF, gộp lại theo (1.1). Đầu ra là độ tin "mắc ĐTĐ", kết luận dương khi CF ≥ 0,5. Hệ không học gì từ dữ liệu, toàn bộ "tri thức" nằm trong bảng luật.

Bảng 1.3. Cơ sở luật của hệ chuyên gia

#	Luật (NẾU … THÌ mắc ĐTĐ với độ tin CF)	CF
R1	Glucose 2 giờ ≥ 200 mg/dL (ngưỡng chẩn đoán ĐTĐ)	0,95
R2	140 ≤ glucose 2 giờ < 200 (rối loạn dung nạp glucose)	0,55
R3	BMI ≥ 30 (béo phì)	0,25
R4	Tuổi ≥ 45	0,20
R5	Chỉ số phả hệ ≥ 0,5 (tiền sử gia đình mạnh)	0,20
R6	Số lần mang thai ≥ 4	0,10

Ví dụ với mẫu đầu tiên (glucose 148, BMI 33,6, tuổi 50, phả hệ 0,627, mang thai 6 lần): R2, R3, R4, R5, R6 cùng thỏa mãn. Gộp lần lượt theo (1.1): $0{,}55 \to 0{,}55 + 0{,}25 \cdot 0{,}45 = 0{,}6625 \to 0{,}73 \to 0{,}784 \to 0{,}806$. Kết luận là dương tính, khớp với nhãn thật.

(2) Perceptron Rosenblatt. Với nhãn $y \in \{-1, +1\}$, mô hình dự đoán $\hat{y} = \operatorname{sign}(\mathbf{w}^\top \mathbf{x} + b)$. Thuật toán học perceptron (PLA) chỉ cập nhật khi phân loại sai:

$$\text{nếu } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \le 0: \quad \mathbf{w} \leftarrow \mathbf{w} + \eta\, y_i \mathbf{x}_i, \qquad b \leftarrow b + \eta\, y_i. \tag{1.2}$$

PLA chỉ hội tụ khi dữ liệu tách được tuyến tính, điều Pima không thỏa mãn. Vì vậy cài đặt dùng biến thể "pocket": giữ lại bộ trọng số ít lỗi nhất qua 100 epoch.

(3) ADALINE. Widrow–Hoff cực tiểu sai số bình phương của đầu ra *tuyến tính* (trước hàm dấu), $J(\mathbf{w}) = \frac{1}{2}\sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i - b)^2$, bằng hạ gradient:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta \sum_i (y_i - \mathbf{w}^\top \mathbf{x}_i - b)\, \mathbf{x}_i. \tag{1.3}$$

Khác biệt then chốt so với PLA: hàm mất mát liên tục và lồi, nên luôn hội tụ, kể cả khi dữ liệu không tách được.

(4) MLP với lan truyền ngược. Mạng 8–16–1 là một hàm hợp:

$$\hat{p} = \sigma(\mathbf{W}_2^\top \sigma(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1) + b_2), \qquad \sigma(z) = \frac{1}{1+e^{-z}}, \tag{1.4}$$

huấn luyện bằng entropy chéo nhị phân $L = -\frac{1}{n}\sum[y \log \hat{p} + (1-y)\log(1-\hat{p})]$. Quy tắc chuỗi cho gradient ở lớp ra là $\delta_2 = (\hat{\mathbf{p}} - \mathbf{y})/n$. Ở lớp ẩn, gradient được truyền ngược qua $\mathbf{W}_2$ và nhân với đạo hàm sigmoid: $\delta_1 = (\delta_2 \mathbf{W}_2^\top) \odot \mathbf{h} \odot (1 - \mathbf{h})$. Đoạn mã 1.1 cài đặt đúng các phương trình này.

(5) Hồi quy logistic và **(6) Gradient Boosting** (scikit-learn [34]) đại diện cho học máy thống kê hiện đại. Chúng được trình bày chi tiết ở Chương 2.

Đoạn mã 1.1. MLP một lớp ẩn với lan truyền ngược bằng NumPy (`lab/ch1_history.py`)

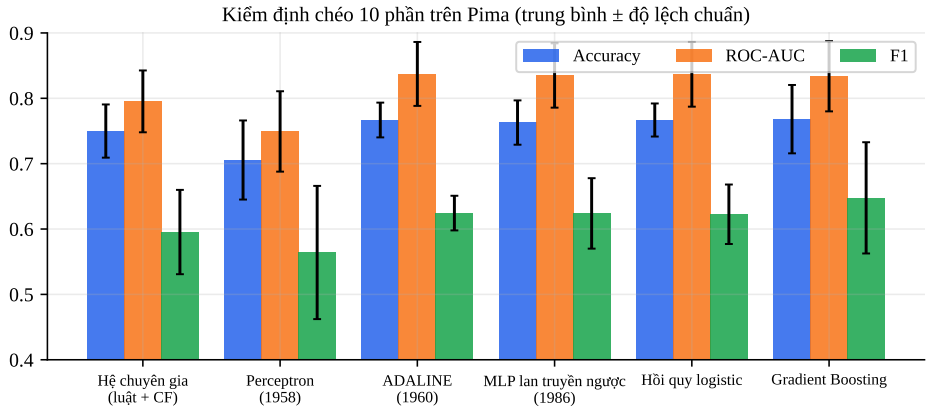
```
def fit(self, X, y):
    y = y.reshape(-1, 1).astype(float)
    for _ in range(self.epochs):
        h = sigmoid(X @ self.W1 + self.b1)                # lan truyền xuôi: lớp ẩn
        p = sigmoid(h @ self.W2 + self.b2)                # lớp ra: xác suất mắc ĐTĐ
        d2 = (p - y) / n                                  # ∂L/∂z2 (sigmoid + entropy chéo)
        d1 = (d2 @ self.W2.T) * h * (1 - h)               # quy tắc chuỗi qua lớp ẩn
        self.W2 -= self.lr * (h.T @ d2 + self.l2 * self.W2)
        self.b2 -= self.lr * d2.sum(0)
        self.W1 -= self.lr * (X.T @ d1 + self.l2 * self.W1)
        self.b1 -= self.lr * d1.sum(0)
```

1.3.3. Kết quả và nhận xét

Bảng 1.4 và Hình 1.2 tổng hợp kết quả kiểm định chéo 10 phần.

Bảng 1.4. Kết quả kiểm định chéo phân tầng 10 phần trên Pima (trung bình ± độ lệch chuẩn)

Mô hình (năm)	Accuracy	ROC-AUC	F1	Độ nhạy	Độ đặc hiệu
Hệ chuyên gia, luật + CF	0,750 ± 0,041	0,795 ± 0,047	0,595	0,530	0,868
Perceptron (1958)	0,706 ± 0,061	0,749 ± 0,062	0,564	0,564	0,782
ADALINE (1960)	0,767 ± 0,027	**0,837** ± 0,049	0,624	0,556	0,880
MLP lan truyền ngược (1986)	0,763 ± 0,034	0,835 ± 0,049	0,624	0,568	0,868
Hồi quy logistic	0,767 ± 0,025	**0,837** ± 0,050	0,623	0,556	0,880
Gradient Boosting	**0,768** ± 0,052	0,834 ± 0,054	**0,648**	**0,616**	0,850



Hình 1.2. So sánh sáu mô hình đại diện các giai đoạn lịch sử trên Pima

Có bốn nhận xét chính.

Thứ nhất, hệ chuyên gia chỉ với sáu luật đạt AUC 0,795, cao hơn perceptron học từ dữ liệu. Tri thức y khoa cô đọng (ngưỡng glucose của ADA) rất mạnh, và đây là lý do các ngưỡng này vẫn là căn cứ chẩn đoán trong các hướng dẫn lâm sàng hiện đại, kể cả khi đã có mô hình học máy. Tuy vậy, độ nhạy chỉ đạt 0,53: luật cứng bỏ sót những người có glucose dưới ngưỡng nhưng tổ hợp nhiều yếu tố nguy cơ. Đó chính là tính "giòn" của hệ chuyên gia.

Thứ hai, perceptron là mô hình yếu nhất và có phương sai lớn nhất (độ lệch chuẩn của độ nhạy là 0,16). Số lỗi mỗi epoch của PLA không giảm đơn điệu: thuật toán chỉ phản ứng với mẫu sai nên "nhảy" liên tục khi dữ liệu không tách được tuyến tính, trong khi hàm mất mát của ADALINE và MLP giảm mượt.

Thứ ba, ADALINE với cùng một mô hình tuyến tính nhưng có hàm mất mát liên tục đã tăng AUC từ 0,749 lên 0,837. Đây là bài học quan trọng nhất của chương: *cách tối ưu quan trọng không kém kiến trúc*. ADALINE ngang bằng hồi quy logistic, vì cả hai đều là mô hình tuyến tính được tối ưu bằng gradient.

Thứ tư, MLP và Gradient Boosting không vượt được mô hình tuyến tính trên 768 mẫu. Quan hệ giữa các đặc trưng và nhãn ở Pima gần tuyến tính, còn dữ liệu quá ít để mô hình phi tuyến phát huy. Gradient Boosting cho F1 và độ nhạy cao nhất nhưng phương sai lớn. Kết quả này khớp với các báo cáo thường thấy, khoảng 76–78% accuracy trên Pima, và với con số 76% của ADAP năm 1988 [4].

Bài toán XOR. Trên bốn điểm của XOR, perceptron chỉ đạt 50% (2/4 điểm) dù huấn luyện bao lâu. Đây đúng là phản ví dụ của Minsky và Papert. MLP 2–4–1 huấn luyện bằng lan truyền ngược đạt 100%, với xác suất đầu ra (0,000; 0,999; 0,999; 0,001). Hai lớp nơ-ron tạo ra hai siêu phẳng, và tổ hợp của chúng tách được vùng "chéo" mà một

siêu phẳng không làm nổi. Thí nghiệm nhỏ này tóm tắt bước chuyển từ mùa đông AI thứ nhất sang thời kỳ hồi sinh của mạng nơ-ron năm 1986.

1.4. Triển khai

Endpoint `POST /api/lab/ch1/pima` nhận tám chỉ số của một người. Nó trả về kết luận của hệ chuyên gia kèm danh sách luật đã kích hoạt và CF sau mỗi bước gộp, cùng điểm của perceptron và xác suất của MLP. Các mô hình được huấn luyện trên toàn bộ 768 mẫu và nạp từ tệp `data/models/ch1_pima.joblib`. Trên trang `/lab` (thẻ "Chương 1"), người dùng nhập chỉ số và thấy ngay ba "thế hệ" mô hình cho kết quả khác nhau ra sao trên cùng một ca.

Đoạn mã 1.2. Gọi endpoint Chương 1 và kết quả trả về (rút gọn)

```
curl -s https://agent.nits.io.vn/api/lab/ch1/pima -H 'Content-Type: application/json' \
  -d '{"pregnancies":6,"glucose":148,"blood_pressure":72,"skin_thickness":35,
       "insulin":0,"bmi":33.6,"pedigree":0.627,"age":50}'
# {"expert": {"cf": 0.8056, "positive": true,
#             "fired": [["Glucose 2h 140–199 (rối loạn dung nạp)", 0.55], ["BMI ≥ 30 (béo
 ↪  phì)", 0.25], ...],
#             "cf_trace": [0.55, 0.6625, 0.73, 0.784, 0.8056]},
#  "perceptron": {"score": 0.0274, "positive": true},
#  "mlp": {"probability": 0.7058, "positive": true}, "imputed": ["insulin"]}
```

1.5. Tiểu kết chương 1

Lịch sử AI là chuỗi chu kỳ giữa kỳ vọng và thực tế. AI biểu tượng mạnh về suy luận nhưng bất lực trước sự phức tạp của thế giới thực. Hệ chuyên gia thành công thương mại nhưng vấp nút thắt thu nhận tri thức. Học máy thống kê và học sâu chuyển trọng tâm sang học từ dữ liệu. Thực nghiệm trên Pima tái hiện đúng dòng chảy đó: luật chuyên gia giàu tri thức nhưng cứng nhắc; perceptron học được nhưng bất ổn và không giải được XOR; ADALINE cho thấy sức mạnh của tối ưu gradient; lan truyền ngược mở khóa mô hình phi tuyến. Hệ thống ở các chương sau không loại bỏ giai đoạn nào. Nó kết hợp luật tường minh (ngưỡng ADA, guardrail) với các mô hình học từ dữ liệu, mỗi loại ở đúng chỗ mạnh của mình.

CHƯƠNG 2. CÁC KỸ THUẬT HỌC MÁY CƠ BẢN

Học máy là lớp kỹ thuật nền của mọi hệ thống dự đoán trên dữ liệu bảng. Chương này trình bày nền tảng chung (bài toán học, hàm mất mát, tối ưu, đánh giá) và các thuật toán cổ điển: hồi quy tuyến tính, hồi quy logistic, KNN, Naive Bayes, cây quyết định, rừng ngẫu nhiên, gradient boosting, SVM, K-means và PCA. Thực nghiệm dùng dữ liệu thật của khảo sát NHANES với 13 đặc trưng mà một người có thể tự khai khi sàng lọc. Mục tiêu là so sánh tám bộ phân loại, bốn mô hình hồi quy và một thuật toán gom cụm trên cùng giao thức đánh giá.

2.1. Bài toán học và quy trình xây dựng mô hình

Theo Mitchell, một chương trình *học* từ kinh nghiệm E đối với nhiệm vụ T và thước đo P nếu hiệu năng trên T, đo bằng P, tăng theo E. Trong học có giám sát, ta có tập $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ lấy mẫu độc lập từ một phân phối $p(\mathbf{x}, y)$ chưa biết, và tìm hàm f_θ cực tiểu *rủi ro thực nghiệm* có điều chuẩn:

$$\hat{\theta} = \arg\min_{\theta} \; \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_\theta(\mathbf{x}_i), y_i\big) + \lambda \, \Omega(\theta). \tag{2.1}$$

Hồi quy dùng ℓ là sai số bình phương. Phân loại dùng entropy chéo hoặc hàm hinge. Ω thường là chuẩn L_2 (Ridge) hoặc L_1 (Lasso). Ngoài học có giám sát còn có học không giám sát (gom cụm, giảm chiều), học bán giám sát (Chương 6) và học tăng cường (tác tử học chính sách từ phần thưởng).

Quy trình xây dựng một mô hình gồm sáu bước: (1) xác định bài toán và thước đo; (2) thu thập, làm sạch dữ liệu; (3) chia tập train/validation/test; (4) tiền xử lý (điền khuyết, chuẩn hóa, mã hóa) — *chỉ học tham số tiền xử lý trên tập train*; (5) huấn luyện và chọn siêu tham số trên validation; (6) đánh giá một lần trên test rồi triển khai. Việc dùng tập test để chọn mô hình làm kết quả lạc quan giả tạo, gọi là *rò rỉ dữ liệu*.

2.2. Tối ưu hóa bằng hạ gradient

Khi (2.1) không có nghiệm đóng, ta lặp $\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta_t)$. Hạ gradient ngẫu nhiên (SGD) ước lượng gradient trên một lô nhỏ B, nên mỗi bước rẻ hơn và có nhiễu giúp thoát điểm yên ngựa. Adam [35] duy trì trung bình động của gradient $\mathbf{m}_t$ và của

bình phương gradient $\mathbf{v}_t$:

$$\begin{aligned} \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t, \\ \mathbf{v}_t &= \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2, \\ \theta_{t+1} &= \theta_t - \eta \, \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}, \end{aligned} \tag{2.2}$$

với $\hat{\mathbf{m}}_t = \mathbf{m}_t/(1-\beta_1^t)$, $\hat{\mathbf{v}}_t = \mathbf{v}_t/(1-\beta_2^t)$ là hai bản hiệu chỉnh độ chệch. Adam được dùng cho mọi mô hình học sâu ở các chương 3–6.

2.3. Các thuật toán

2.3.1. Hồi quy tuyến tính

Mô hình $\hat{y} = \mathbf{w}^\top \mathbf{x} + b$, cực tiểu $\sum (y_i - \hat{y}_i)^2$, có nghiệm đóng $\hat{\mathbf{w}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$. Thêm điều chuẩn L_2 (Ridge) cho nghiệm $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$, ổn định khi các đặc trưng tương quan (ví dụ BMI và vòng eo). Khi đặc trưng đã chuẩn hóa z-score, w_j là mức thay đổi của y khi x_j tăng một độ lệch chuẩn, nên đọc được trực tiếp.

2.3.2. Hồi quy logistic

Cho phân loại nhị phân, mô hình ước lượng xác suất $p(y{=}1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ và cực tiểu entropy chéo:

$$\begin{aligned} J(\mathbf{w}) &= -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log p_i + (1-y_i) \log(1-p_i) \right] + \frac{\lambda}{2} \|\mathbf{w}\|^2, \\ \nabla_{\mathbf{w}} J &= \frac{1}{n} \mathbf{X}^\top (\mathbf{p} - \mathbf{y}) + \lambda \mathbf{w}. \end{aligned} \tag{2.3}$$

J lồi nên có cực tiểu toàn cục duy nhất. Trong y khoa, e^{w_j} là *tỉ số chênh* (odds ratio, OR): odds mắc bệnh nhân lên e^{w_j} lần khi x_j tăng một đơn vị, ở đây là một độ lệch chuẩn. Mở rộng sang K lớp, ta thay sigmoid bằng softmax $p_k = e^{z_k} / \sum_j e^{z_j}$, đây cũng là lớp ra của mọi mạng phân loại ở các chương sau.

2.3.3. K láng giềng gần nhất

KNN [36] không có pha huấn luyện. Với một điểm mới, thuật toán tìm k điểm gần nhất theo khoảng cách Euclid trên dữ liệu đã chuẩn hóa, rồi bỏ phiếu. Phiếu có thể đánh trọng số $1/d$ theo khoảng cách. k nhỏ cho biên quyết định gồ ghề, phương sai lớn và dễ quá khớp. k lớn cho biên trơn hơn nhưng độ chệch tăng. Chi phí dự đoán là $O(nd)$ cho

mỗi điểm, và hiệu quả giảm khi số chiều tăng, do "lời nguyền số chiều".

2.3.4. Naive Bayes

Áp dụng định lý Bayes với giả định các đặc trưng độc lập có điều kiện theo lớp:

$$p(y \mid \mathbf{x}) \propto p(y) \prod_{j=1}^{d} p(x_j \mid y), \qquad p(x_j \mid y = c) = \mathcal{N}(x_j;\ \mu_{jc}, \sigma_{jc}^2) \text{ (bản Gauss).} \quad (2.4)$$

Mô hình học rất nhanh, nhưng giả định độc lập bị vi phạm nặng khi đặc trưng tương quan (BMI với vòng eo, huyết áp tâm thu với tâm trương). Kết quả là xác suất bị đẩy về 0 hoặc 1, hiệu chuẩn kém. Thực nghiệm dưới đây sẽ cho thấy rõ điều này.

2.3.5. Cây quyết định và các phương pháp tổ hợp

Cây quyết định chia không gian đặc trưng thành các hình hộp bằng chuỗi câu hỏi "$x_j \le t$?". Tại mỗi nút, thuật toán CART chọn cặp (j, t) cực đại mức giảm độ hỗn tạp. Độ hỗn tạp đo bằng chỉ số Gini $G = 1 - \sum_k p_k^2$ hoặc entropy $H = -\sum_k p_k \log p_k$, với p_k là tỉ lệ lớp k trong nút. Cây dễ giải thích, xử lý được tương tác phi tuyến và không cần chuẩn hóa. Đổi lại, cây sâu quá khớp mạnh, và một thay đổi nhỏ trong dữ liệu có thể đổi cả cây.

Rừng ngẫu nhiên [22] huấn luyện B cây trên các mẫu bootstrap. Tại mỗi lần chia, cây chỉ xét một tập con ngẫu nhiên $\sqrt{d}$ đặc trưng, rồi lấy trung bình dự đoán của các cây. Nếu các cây có phương sai σ^2 và tương quan ρ, phương sai của trung bình là $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. Việc chọn đặc trưng ngẫu nhiên làm giảm ρ, nhờ đó giảm phương sai mà không tăng độ chệch.

Gradient boosting [23] xây cây *tuần tự*: cây thứ m khớp với gradient âm ("phần dư giả") của hàm mất mát đối với dự đoán hiện tại, $F_m = F_{m-1} + \nu\, h_m$, với ν là tốc độ học. Bản HistGradientBoosting (giống LightGBM [37]) rời rạc hóa đặc trưng thành 255 thùng. Nhờ vậy huấn luyện nhanh, và bản này xử lý giá trị thiếu tự nhiên bằng cách học hướng đi cho NaN tại mỗi nút. Đây là mô hình ML chạy trong chatbot.

2.3.6. Máy vector hỗ trợ

SVM [21] tìm siêu phẳng có *lề* lớn nhất. Với dữ liệu không tách được, ta thêm biến chùng ξ_i:

$$\min_{\mathbf{w}, b, \xi} \ \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \ge 1 - \xi_i,\ \xi_i \ge 0. \quad (2.5)$$

Bài toán đối ngẫu chỉ phụ thuộc vào tích vô hướng, nên thay được bằng hàm nhân $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2)$ (RBF). Hàm quyết định là $f(\mathbf{x}) = \sum_{i \in SV} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$, chỉ gồm các *vector hỗ trợ*. C điều khiển mức phạt vi phạm lề, γ điều khiển độ "cục bộ" của nhân. Tham số `class_weight` nhân C theo nghịch đảo tần suất lớp để bù lệch lớp. SVM không trả về xác suất. Xác suất được hiệu chuẩn thêm bằng phương pháp Platt [38]. Hồi quy vector hỗ trợ (SVR) dùng hàm mất mát ϵ-không nhạy: sai số nhỏ hơn ϵ không bị phạt.

2.3.7. Gom cụm K-means và giảm chiều PCA

K-means [39] cực tiểu tổng bình phương khoảng cách tới tâm cụm, $\sum_k \sum_{\mathbf{x} \in C_k} \|\mathbf{x} - \mu_k\|^2$ (inertia). Thuật toán lặp hai bước tới khi hội tụ: gán mỗi điểm vào tâm gần nhất, rồi cập nhật tâm bằng trung bình các điểm. Chọn k bằng "khuỷu tay" của inertia hoặc hệ số silhouette [40], $s = (b - a)/\max(a, b)$, với a là khoảng cách trung bình tới các điểm cùng cụm và b là khoảng cách trung bình tới cụm gần nhất khác. PCA chiếu dữ liệu lên các vector riêng ứng với trị riêng lớn nhất của ma trận hiệp phương sai, giữ lại phương sai nhiều nhất. Ở đây PCA dùng để trực quan hóa cụm trên mặt phẳng.

2.3.8. Quá khớp, thiếu khớp và điều chuẩn

Sai số kỳ vọng phân rã thành độ chệch^2 + phương sai + nhiễu. Mô hình quá đơn giản thì thiếu khớp: sai số train và test cùng cao. Mô hình quá phức tạp thì quá khớp: train rất tốt, test kém. Các núm điều chỉnh gồm k của KNN, độ sâu cây, C và γ của SVM, λ của Ridge, số cây và tốc độ học của boosting. Mục 2.6 vẽ đường cong train/validation theo k và theo độ sâu để thấy rõ đánh đổi này.

2.3.9. Thước đo đánh giá

Với phân loại nhị phân, từ ma trận nhầm lẫn (TP, FP, TN, FN) ta có độ nhạy (recall) $= \frac{TP}{TP+FN}$, độ đặc hiệu $= \frac{TN}{TN+FP}$, độ chính xác (precision, PPV) $= \frac{TP}{TP+FP}$ và $F_1 = \frac{2PR}{P+R}$. Các thước đo này phụ thuộc vào ngưỡng. Ở đây ngưỡng được chọn trên validation theo chỉ số Youden J = độ nhạy + độ đặc hiệu − 1 [41]. ROC-AUC đo khả năng xếp hạng, không phụ thuộc ngưỡng: đó là xác suất một ca dương được chấm điểm cao hơn một ca âm ngẫu nhiên. PR-AUC (average precision) nhạy hơn khi lớp dương hiếm; mốc ngẫu nhiên của nó bằng tỉ lệ dương. Điểm Brier $\frac{1}{n}\sum(p_i - y_i)^2$ đo độ hiệu chuẩn của xác suất, quan trọng khi chatbot nói "nguy cơ 34%". Với hồi quy, ta dùng RMSE, MAE và $R^2 = 1 - \frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}$.

2.4. Dữ liệu NHANES 2011–2018

NHANES (National Health and Nutrition Examination Survey) là khảo sát sức khỏe quốc gia của CDC Hoa Kỳ [5]. Mỗi chu kỳ hai năm, khảo sát kết hợp phỏng vấn, khám thực thể và xét nghiệm máu trên mẫu đại diện dân số.

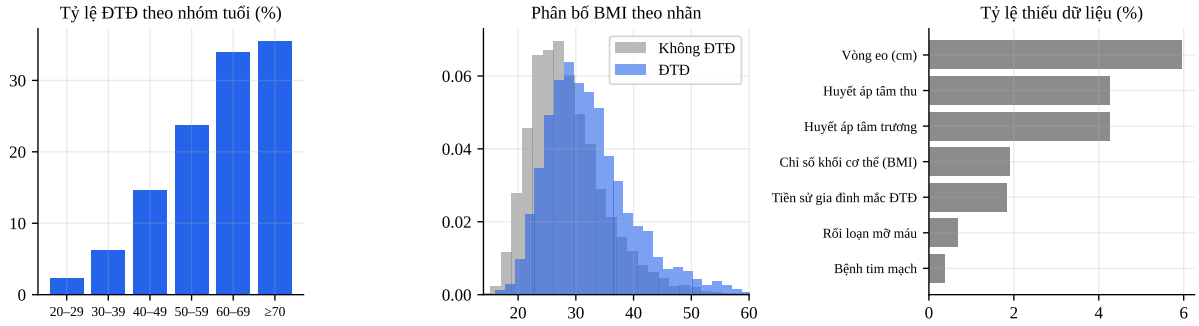
- **Nguồn**: `https://wwwn.cdc.gov/nchs/nhanes/`. Script `scripts/fetch_nhanes.py` tải 40 tệp XPT của 4 chu kỳ 2011–2018 (các mô-đun DEMO, BMX, BPX, BPQ, DIQ, GHB, GLU, MCQ, PAQ, SMQ), ghép theo mã người `SEQN` và lọc người ≥ 20 tuổi.
- **Kích thước**: 20.646 người × 22 cột (tệp CSV 2,4 MB). Bốn chu kỳ gần cân bằng: 5.034 / 5.371 / 5.198 / 5.043 người. Có 2.630 người gốc Á, dùng để đánh giá nhóm con gần với người Việt.
- **Nhãn**: ĐTĐ theo ADA [2]. Một người được gán nhãn ĐTĐ khi *đã được bác sĩ chẩn đoán* (3.142 người), *hoặc* HbA1c ≥ 6,5%, *hoặc* glucose lúc đói ≥ 126 mg/dL. Trong số đó có 943 người *chưa từng được chẩn đoán* nhưng có xét nghiệm vượt ngưỡng. Tổng tỉ lệ dương là 19,8%; tỉ lệ tiền ĐTĐ là 34,3%.
- **Đặc trưng** (13): tuổi, giới, BMI, vòng eo, huyết áp tâm thu và tâm trương (trung bình các lần đo), tăng huyết áp, rối loạn mỡ máu, tiền sử gia đình, hút thuốc, vận động, bệnh tim, đột quỵ. HbA1c và glucose *không* được đưa vào đặc trưng, vì nhãn được định nghĩa từ chính chúng (rò rỉ nhãn). Xét nghiệm được đối chiếu trực tiếp với ngưỡng ADA; mô hình ML chỉ ước lượng nguy cơ từ các yếu tố nguy cơ.

Bảng 2.1. Năm mẫu đầu của bộ NHANES đã xử lý (một số cột)

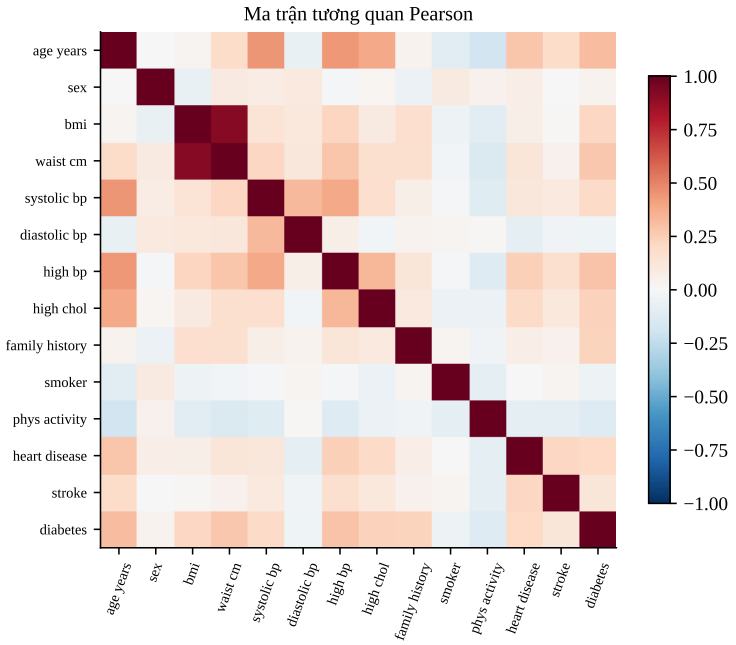
SEQN	Tuổi	Nam	BMI	Eo	HA tâm thu	THA	GĐ	HbA1c	ĐTĐ
62161	22	1	23,3	81,0	110,7	0	1	5,1	0
62164	44	0	23,2	80,1	118,0	0	0	4,9	0
62169	21	1	20,1	69,6	124,7	0	0	5,4	0
62172	43	0	33,3	120,4	102,0	0	1	5,6	0
62174	80	1	33,9	116,5	97,0	1	–	5,0	0

Bảng 2.2. Thống kê mô tả các biến liên tục của NHANES

Biến	Số đo	TB	ĐLC	Min	Trung vị	Max
Tuổi (năm)	20.646	50,2	17,6	20	50	80
BMI (kg/m²)	20.255	29,4	7,1	13,4	28,2	86,2
Vòng eo (cm)	19.416	99,7	16,6	55,5	98,3	177,9
HA tâm thu (mmHg)	19.769	125,0	18,7	64,7	122,0	238,0
HbA1c (%)	20.413	5,82	1,13	3,5	5,5	17,8
Glucose đói (mg/dL)	9.827	111,2	37,6	21	101	479



Hình 2.1. Tỉ lệ ĐTĐ theo tuổi, phân bố BMI theo nhãn và tỉ lệ thiếu dữ liệu



Hình 2.2. Ma trận tương quan Pearson giữa các đặc trưng và nhãn

Nhận xét. (i) Tỉ lệ ĐTĐ tăng mạnh theo tuổi, từ 2,4% ở nhóm 20–29 lên 23,7% ở nhóm 50–59, rồi chững lại quanh 34–36% từ 60 tuổi. Tuổi vì thế là yếu tố mạnh nhất,

khớp với khuyến cáo sàng lọc mọi người từ 35 tuổi của ADA. (ii) Phân bố BMI của nhóm ĐTĐ dịch phải nhưng chồng lấn lớn: BMI đơn lẻ không đủ để phân loại. (iii) Dữ liệu thiếu ít (vòng eo 6,0%, huyết áp 4,3%, BMI 1,9%) nhưng có thật. Người dùng chatbot còn thiếu nhiều hơn thế, nên các pipeline được thiết kế để chịu được giá trị thiếu. (iv) BMI và vòng eo tương quan rất mạnh (r = 0,91); tăng huyết áp tương quan với tuổi (r = 0,43). Tương quan với nhãn chỉ ở mức yếu đến vừa (tuổi 0,31; tăng huyết áp 0,28; vòng eo 0,27; tiền sử gia đình 0,23; BMI 0,21): không có đặc trưng "thần kỳ", và đây là bài toán khó hơn Pima nhiều vì không có glucose. (v) Glucose lúc đói chỉ có ở 9.827 người (mẫu con nhịn đói buổi sáng), đây là đặc điểm thiết kế của NHANES.

2.5. Cài đặt

Dữ liệu được chia 70/10/20 phân tầng theo nhãn với `random_state=42`, cho 14.451 / 2.065 / 4.130 người. Với các mô hình không xử lý được NaN, pipeline gồm `SimpleImputer` (trung vị, thêm cột chỉ báo "bị thiếu") rồi `StandardScaler`. Gradient boosting nhận NaN trực tiếp. Mọi bước tiền xử lý nằm trong `Pipeline` của scikit-learn [34], nên tham số chỉ được học trên tập train (Đoạn mã 2.1).

Đoạn mã 2.1. Tám bộ phân loại và pipeline tiền xử lý (`lab/ch2_ml.py`, rút gọn)

```
def imputed(model):
    """Điền trung vị + cột chỉ báo thiếu, chuẩn hóa z-score, rồi tới mô hình."""
    return make_pipeline(SimpleImputer(strategy="median", add_indicator=True),
     ↪  StandardScaler(), model)

classifiers = {
    "logreg": imputed(LogisticRegression(max_iter=3000, C=1.0)),
    "knn":    imputed(KNeighborsClassifier(n_neighbors=51, weights="distance")),
    "nb":     imputed(GaussianNB()),
    "tree":   imputed(DecisionTreeClassifier(max_depth=6, min_samples_leaf=40)),
    "rf":     imputed(RandomForestClassifier(400, min_samples_leaf=10, n_jobs=-1)),
    "gbdt":   HistGradientBoostingClassifier(max_iter=300, learning_rate=0.05),  # tự xử lý
     ↪  NaN
    "svm":    imputed(SVC(C=1.0, gamma=0.01, class_weight="balanced", probability=True)),
    "mlp":    imputed(MLPClassifier((64, 32), alpha=1e-3, early_stopping=True)),
}
for key, model in classifiers.items():
    model.fit(X_tr, y_tr)
    thr = best_threshold(y_val, score(model, X_val))   # ngưỡng Youden chọn trên validation
    p = score(model, X_te)                              # đánh giá một lần trên test
```

Siêu tham số chọn trên validation: k của KNN theo đường cong ở Hình 2.4, độ sâu cây, và (C, γ, class_weight) của SVM theo Bảng 2.3. Một chi tiết kỹ thuật quan trọng: với SVM, các chỉ số xếp hạng được tính trên `decision_function` chứ không dùng `predict_-`

`proba`. Lý do là bộ hiệu chuẩn Platt của scikit-learn được khớp bằng kiểm định chéo nội bộ, nên có thể đảo thứ tự xếp hạng so với mô hình chính.

Bảng 2.3. Dò siêu tham số SVM (RBF) trên tập validation

C	class_weight	γ	Số vector hỗ trợ	AUC validation
1	không	scale	5.707	0,788
1	balanced	scale	8.101	0,830
0,1	balanced	scale	8.711	0,835
1	**balanced**	**0,01**	**8.289**	**0,838**
10	balanced	0,01	8.019	0,830

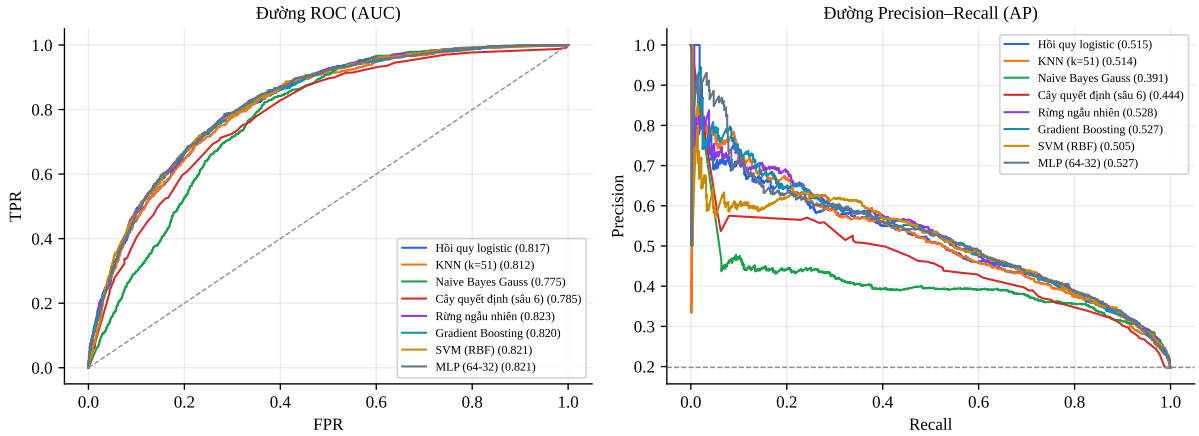
Không có trọng số lớp, SVM tối ưu hàm hinge trên dữ liệu 80/20 nên "bỏ cuộc" với lớp dương ở vùng chồng lấn, và AUC chỉ đạt 0,788. Bù lệch lớp đưa AUC lên 0,830. Giảm γ để nhân RBF "rộng" hơn còn tăng thêm, lên 0,838. Hơn 8.000 trên 14.451 điểm là vector hỗ trợ, cho thấy biên giữa hai lớp rất nhiễu. Điều này cũng giải thích vì sao SVM có thời gian huấn luyện (24 giây) và suy luận lâu nhất.

2.6. Kết quả và nhận xét

2.6.1. Phân loại

Bảng 2.4. Kết quả phân loại ĐTĐ trên tập kiểm tra NHANES (4.130 người, ngưỡng Youden)

Mô hình	ROC-AUC	PR-AUC	Brier	F1	Nhạy	Đặc hiệu	Huấn luyện
Hồi quy logistic	0,817	0,515	0,125	0,525	0,767	0,715	0,03 s
KNN (k=51)	0,812	0,514	0,125	0,525	0,660	0,790	0,02 s
Naive Bayes Gauss	0,775	0,391	0,209	0,489	0,756	0,670	0,02 s
Cây quyết định (sâu 6)	0,785	0,444	0,133	0,504	0,712	0,725	0,04 s
Rừng ngẫu nhiên	**0,823**	**0,528**	**0,123**	**0,540**	0,726	0,763	0,65 s
Gradient Boosting	0,820	0,527	**0,123**	0,531	0,723	0,753	0,20 s
SVM (RBF)	0,821	0,505	0,178	0,510	0,818	0,658	24,1 s
MLP (64–32)	0,821	0,527	0,124	0,515	**0,837**	0,651	1,53 s

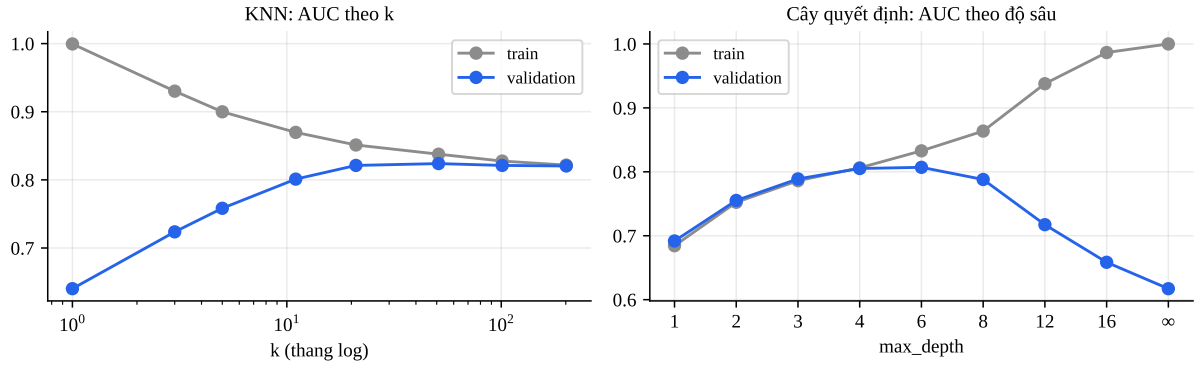


Hình 2.3. Đường ROC và Precision–Recall của tám mô hình trên tập kiểm tra

Nhận xét. Năm mô hình mạnh (RF, MLP, SVM, GBDT, LR) nằm trong khoảng 0,817–0,823 AUC. Chênh lệch 0,006 nhỏ hơn độ lệch chuẩn 0,005–0,006 của kiểm định chéo 5 phần trên tập train (LR 0,826 ± 0,006; GBDT 0,830 ± 0,006). Có thể kết luận: *với 13 yếu tố nguy cơ tự khai, độ khó của bài toán đã chạm trần thông tin*. Phần còn lại nằm ở glucose và HbA1c vốn không được dùng. Mức AUC ≈ 0,82 tương đương các thang điểm sàng lọc dựa trên NHANES trong y văn.

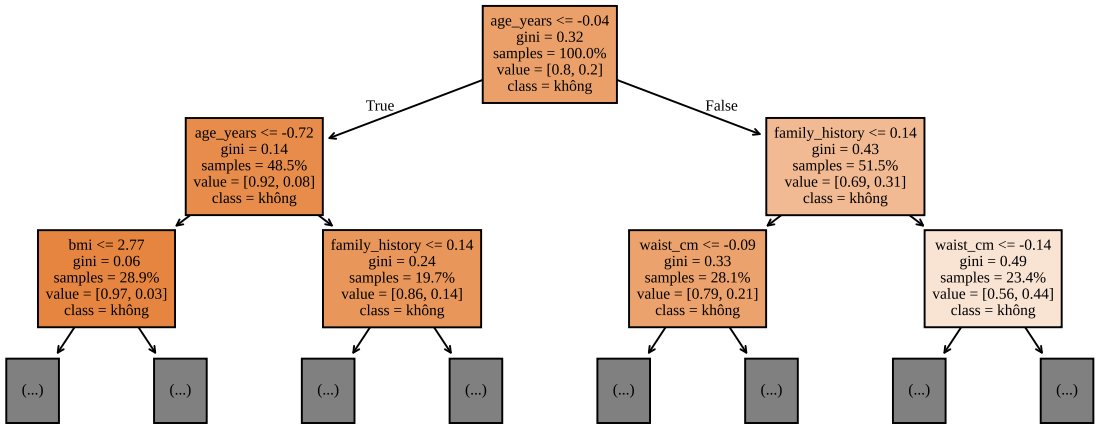
Hai mô hình yếu rõ rệt đều do vi phạm giả định. Naive Bayes giả định độc lập có điều kiện, nhưng BMI–vòng eo và hai chỉ số huyết áp tương quan mạnh nên bằng chứng bị "đếm hai lần". Xác suất bị đẩy về hai cực, thể hiện ở Brier tệ nhất (0,209) và ngưỡng Youden chỉ 0,002. Cây đơn lẻ sâu 6 có phương sai cao và biên quyết định bậc thang. Rừng ngẫu nhiên lấy trung bình 400 cây như vậy và tăng AUC từ 0,785 lên 0,823, minh họa trực tiếp công thức giảm phương sai.

Ở ngưỡng Youden, các mô hình có điểm vận hành khác nhau. MLP và SVM nghiêng về độ nhạy (0,84 và 0,82), còn KNN nghiêng về độ đặc hiệu. Trong sàng lọc, bỏ sót người bệnh (FN) nguy hiểm hơn báo động nhầm (FP), nên ngưỡng có thể hạ thêm. Đó là quyết định lâm sàng chứ không phải kỹ thuật. Xác suất của các mô hình ngoài NB và SVM được hiệu chuẩn tốt (Brier ≈ 0,123–0,125, so với 0,159 của dự đoán hằng số 0,198).

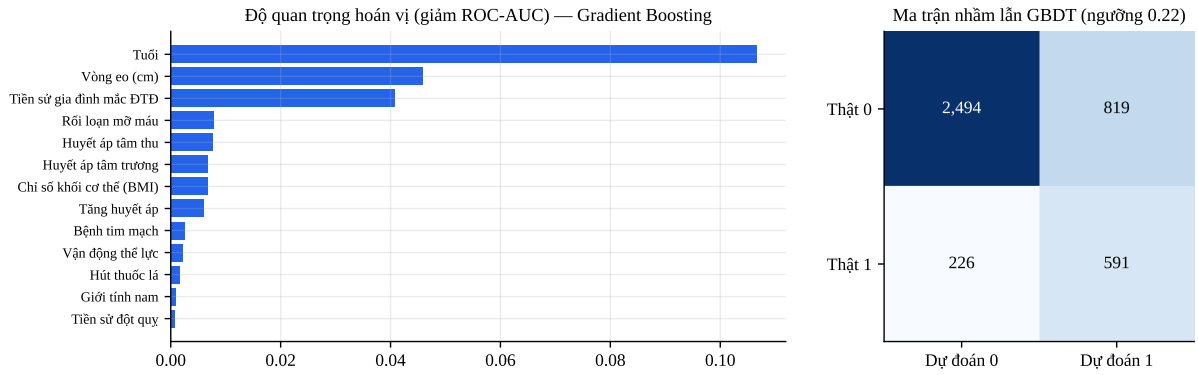


Hình 2.4. Đánh đổi độ chệch – phương sai: AUC train và validation theo k của KNN và độ sâu cây

Hình 2.4 cho thấy đánh đổi độ chệch – phương sai rất rõ. Với $k = 1$, KNN "thuộc lòng" tập train (AUC 0,9995) nhưng chỉ đạt 0,640 trên validation. Đường validation đạt đỉnh 0,824 ở $k = 51$ rồi đi ngang. Cây quyết định cũng vậy: AUC validation tăng tới 0,807 ở độ sâu 6. Ở độ sâu 16, train đạt 0,987 còn validation tụt xuống 0,659. Cây không giới hạn độ sâu đạt 1,000 trên train và 0,617 trên validation.



Hình 2.5. Ba tầng đầu của cây quyết định: câu hỏi đầu tiên là tuổi, sau đó là tuổi (nhánh trẻ) và tiền sử gia đình (nhánh lớn tuổi)



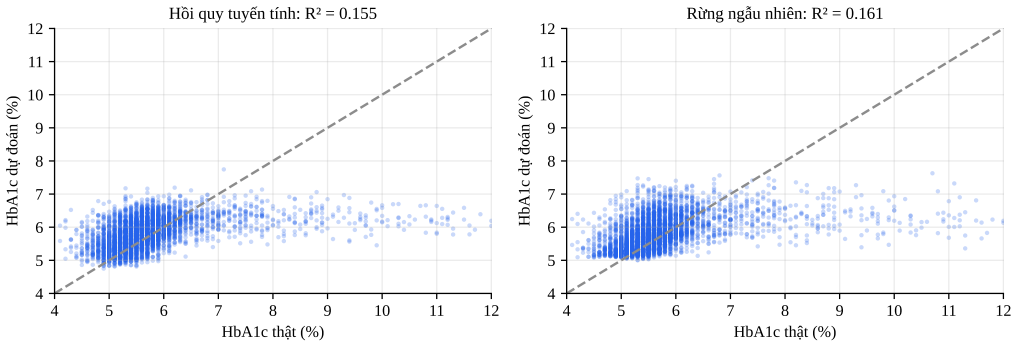
Hình 2.6. Độ quan trọng hoán vị của các đặc trưng (Gradient Boosting) và ma trận nhầm lẫn

Giải thích mô hình. Ba cách nhìn độc lập cho cùng một thứ tự. Thứ nhất, câu hỏi gốc của cây là "tuổi ≤ 49,5?". Nhánh trẻ hỏi tiếp "tuổi ≤ 37,5?", nhánh lớn tuổi hỏi tiền sử gia đình (Hình 2.5). Thứ hai, độ quan trọng hoán vị của GBDT, tức mức giảm AUC khi xáo trộn một cột, xếp tuổi đầu tiên (0,107), rồi vòng eo (0,046) và tiền sử gia đình (0,041); các đặc trưng còn lại dưới 0,01. Thứ ba, tỉ số chênh của hồi quy logistic trên một độ lệch chuẩn là: tuổi OR = 1,93, tiền sử gia đình 1,78, vòng eo 1,56, rối loạn mỡ máu 1,25, tăng huyết áp 1,23, BMI chỉ 1,10. Vòng eo quan trọng hơn BMI khi có mặt cả hai, đúng với kiến thức y khoa rằng béo bụng (mỡ nội tạng) phản ánh đề kháng insulin tốt hơn. Đây cũng là lý do chatbot hỏi thêm vòng eo. Vận động có OR = 0,87 < 1, tức là yếu tố bảo vệ. Ma trận nhầm lẫn của GBDT trên 4.130 người: 591 TP, 226 FN, 819 FP, 2.494 TN.

2.6.2. Hồi quy HbA1c

Bảng 2.5. Dự đoán HbA1c (%) từ 13 yếu tố nguy cơ (20.413 người có HbA1c, chia 80/20)

Mô hình	RMSE (%)	MAE (%)	R^2
Dự đoán bằng trung bình (đối chứng)	1,072	0,636	0,000
Hồi quy tuyến tính	0,984	0,569	0,155
Ridge (α=10)	0,984	0,569	0,155
SVR (RBF, ϵ=0,1; 8.000 mẫu)	1,000	**0,515**	0,128
Rừng ngẫu nhiên	**0,981**	0,558	**0,161**

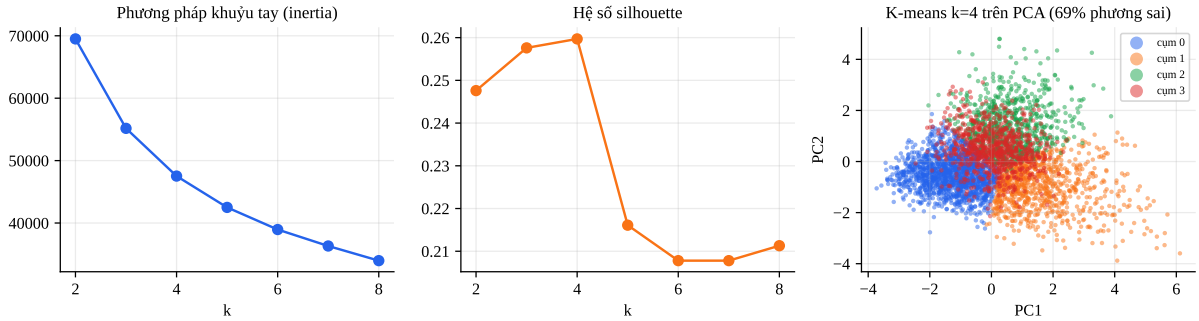


Hình 2.7. HbA1c dự đoán so với thực tế: mô hình co về trung bình

R^2 chỉ đạt 0,16. Yếu tố nguy cơ giải thích được khoảng 16% phương sai của HbA1c, phù hợp với kết luận "chạm trần thông tin" ở trên. Hình 2.7 cho thấy hiện tượng co về trung bình: mọi dự đoán nằm trong khoảng 5–7%, trong khi HbA1c thật trải tới 14%.

Đuôi phải dài chủ yếu là người ĐTĐ đang điều trị. SVR có MAE tốt nhất nhưng RMSE tệ nhất. Hàm mất mát ϵ-không nhạy tương tự trung vị nên khớp tốt phần đông người bình thường và bỏ qua các ca cực trị, trong khi RMSE phạt nặng các ca này. Hệ số hồi quy tuyến tính trên một độ lệch chuẩn: tuổi +0,18%, tiền sử gia đình +0,18%, vòng eo +0,11%, mỡ máu +0,10%, vận động −0,06%. Thứ tự này trùng với bài toán phân loại. Kết luận thực tiễn: *không thể thay xét nghiệm HbA1c bằng mô hình*. Mô hình chỉ giúp quyết định *ai* nên đi xét nghiệm.

2.6.3. Gom cụm không giám sát



Hình 2.8. Chọn k bằng inertia và silhouette; bốn cụm chiếu trên hai thành phần chính

Gom cụm K-means được chạy trên 5 biến liên tục (tuổi, BMI, vòng eo, hai chỉ số huyết áp) đã chuẩn hóa, *không dùng nhãn*. Silhouette cao nhất ở $k = 4$ (0,260), khuỷu tay của inertia cũng ở quanh đây. Hai thành phần chính giữ 69% phương sai. Điểm đáng chú ý là các cụm học không giám sát lại có ý nghĩa lâm sàng (Bảng 2.6). Cụm 0 gồm người trẻ, gầy, huyết áp thấp, tỉ lệ ĐTĐ chỉ 4,4%. Ba cụm còn lại là ba "kiểu hình" nguy cơ khác nhau: béo phì nặng ở tuổi trung niên, tăng huyết áp, và cao tuổi. Tỉ lệ ĐTĐ của ba cụm này cao gấp 5–7 lần cụm 0.

Bảng 2.6. Hồ sơ trung bình của bốn cụm K-means và tỉ lệ ĐTĐ (nhãn không dùng khi gom cụm)

Cụm	Diễn giải	Số người	Tuổi	BMI	Eo	HA tâm thu	Tỉ lệ ĐTĐ
0	Trẻ, khỏe	7.038	35,6	25,3	88,0	112,1	4,4%
1	Béo phì nặng	3.825	45,7	39,3	122,2	124,9	27,2%
2	Tăng huyết áp	3.849	58,4	27,9	98,2	145,6	22,2%
3	Cao tuổi	4.139	68,8	28,1	100,3	126,6	31,3%

2.6.4. So sánh tổng hợp

Bảng 2.7. So sánh định tính các thuật toán cho bài toán sàng lọc

Thuật toán	Dễ giải thích	Chịu dữ liệu thiếu	Khi nào nên dùng
Hồi quy logistic	Cao (OR)	Cần điền	Mốc đối chứng; giải thích yếu tố nguy cơ
KNN	Trung bình	Cần điền	Dữ liệu nhỏ; chậm khi dữ liệu lớn
Naive Bayes	Cao	Tự nhiên	Mốc nhanh; xác suất hiệu chuẩn kém
Cây quyết định	Rất cao	Cần điền	Cần luật dễ đọc cho người dùng
Rừng ngẫu nhiên	Thấp	Cần điền	Mô hình mạnh, ít phải tinh chỉnh
Gradient Boosting	Thấp (cần SHAP/occlusion)	Tự nhiên	Dữ liệu thiếu nhiều, cần nhanh
SVM	Thấp	Cần điền	Dữ liệu vừa; chậm, xác suất kém
MLP	Thấp	Cần điền + cờ thiếu	Kết hợp với dữ liệu khác (ảnh, chuỗi)

Khi người dùng chỉ khai một phần thông tin, hai mô hình bổ sung tốt cho nhau là GBDT (tự xử lý NaN, nhanh) và MLP PyTorch (128–64–32, có cờ thiếu dữ liệu cho từng đặc trưng). Cả hai được huấn luyện với kỹ thuật tăng cường "che ngẫu nhiên 40% đặc trưng không bắt buộc" để mô phỏng người dùng chỉ khai một phần. Trên cùng tập kiểm tra, hai mô hình đạt AUC 0,817 (GBDT) và 0,829 (MLP). Khi bị che 40% đặc trưng, AUC chỉ giảm còn 0,802 và 0,809. Trên nhóm người gốc Á (561 người), AUC đạt 0,833 và 0,850. Trung bình hai mô hình là 0,827. Hai mô hình lệch nhau ≥ 15 điểm phần trăm ở 8% số ca, nên trung bình của chúng ổn định hơn từng mô hình.

2.7. Mạng nơ-ron nhiều lớp cài đặt bằng ba cách

NHANES chỉ có các yếu tố nguy cơ tự khai, nên mọi mô hình chạm cùng một trần AUC khoảng 0,82. Để thấy rõ ảnh hưởng của *cách cài đặt* (chứ không phải của dữ liệu), phần này dùng thêm một bộ dữ liệu tiểu đường lớn hơn và giàu thông tin hơn, rồi viết *cùng một* mạng nơ-ron bằng ba cách: tự viết bằng NumPy, Keras (TensorFlow) và PyTorch.

2.7.1. Dữ liệu Diabetes Prediction

Bộ *Diabetes Prediction* (Kaggle) gồm **100.000 hồ sơ** với 8 đặc trưng: giới, tuổi, tăng huyết áp, bệnh tim, tiền sử hút thuốc, BMI, **HbA1c** và **glucose máu**. Lớp dương (mắc ĐTĐ) chiếm 8,5%. Khác NHANES ở các mục trên, bộ này có sẵn hai xét nghiệm đường huyết. Đây là kiểu dữ liệu của một phòng khám đã làm xét nghiệm, không phải dữ liệu sàng lọc, nên AUC cao hơn hẳn. Dữ liệu được chia 80/20 phân tầng theo nhãn (80.000 huấn luyện, 20.000 kiểm tra). Bộ chuẩn hóa (StandardScaler cho biến số, mã hóa one-hot cho giới và tiền sử hút thuốc) chỉ được khớp trên tập huấn luyện. 10% tập huấn luyện được tách làm tập kiểm định, chỉ dùng cho dừng sớm.

2.7.2. Một kiến trúc, ba cách viết

Cả ba bản dùng đúng một kiến trúc: `Dense(128)-ReLU-Dropout(0,2)-Dense(64)-ReLU-Dropout(0,2)-Dense(1)-sigmoid`, tối ưu Adam ($\eta = 10^{-3}$), lô 256, tối đa 40 epoch, dừng sớm khi mất mát kiểm định không giảm 8 epoch. Số tham số là $15 \cdot 128 + 128 + 128 \cdot 64 + 64 + 64 + 1 = 10.369$ ở cả ba bản. Bảng 2.8 so sánh phần việc mà lập trình viên phải tự làm ở mỗi cách.

Bảng 2.8. Phần việc lập trình viên tự viết ở ba cách cài đặt

Thành phần	NumPy (tự viết)	Keras	PyTorch
Lan truyền tiến	Nhân ma trận, ReLU, mặt nạ dropout	`Sequential` khai báo lớp	`nn.Sequential` + `forward`
Lan truyền ngược	Tự suy đạo hàm từng lớp	Tự động (`GradientTape` bên trong)	Tự động (`loss.backward()`)
Tối ưu Adam	Tự viết mômen bậc 1, 2 và hiệu chỉnh độ chệch	`optimizers.Adam`	`optim.Adam`
Vòng huấn luyện	Tự viết: chia lô, dừng sớm	`model.fit` + `EarlyStopping`	Tự viết vòng lặp, gọi `opt.step()`
Thiết bị	CPU	CPU (TensorFlow)	GPU (CUDA)

Điểm khó nhất của bản NumPy là lan truyền ngược. Với tầng ra sigmoid và mất mát entropy chéo nhị phân, đạo hàm theo đầu vào tầng ra rút gọn thành $\partial L/\partial z = (\hat{y} - y)/m$. Từ đó gradient được truyền ngược qua từng lớp: $\delta^{(l)} = (\delta^{(l+1)} W^{(l+1)\top}) \odot \mathbb{1}[z^{(l)} > 0] \odot M^{(l)}$, với $M^{(l)}$ là mặt nạ dropout đã chia cho $1 - p$ (dropout ngược). Đoạn mã 2.2 là đoạn lõi.

Đoạn mã 2.2. Lan truyền ngược và cập nhật Adam của MLP NumPy (`tl/mlp_scratch.py`, rút gọn)

```
def backward(self, y):
    m = len(y)
    delta = (self.cache[-1][1] - y.reshape(-1, 1)) / m       # dL/dz tầng ra = (y_hat - y) / m
    grads_W, grads_b = [None] * len(self.W), [None] * len(self.W)
    for i in reversed(range(len(self.W))):
        a_prev = self.X_in if i == 0 else self.cache[i - 1][1]
        grads_W[i] = a_prev.T @ delta
        grads_b[i] = delta.sum(0, keepdims=True)
        if i > 0:
            z_prev, _, mask = self.cache[i - 1]
            delta = (delta @ self.W[i].T) * (z_prev > 0)     # quy tắc dây chuyền qua ReLU
            if mask is not None:
                delta = delta * mask                          # qua dropout ngược
    self._adam(grads_W + grads_b)

def _adam(self, grads, b1=0.9, b2=0.999, eps=1e-8):
    self.t += 1
    for k, (p, g) in enumerate(zip(self.W + self.b, grads)):
        self.m[k] = b1 * self.m[k] + (1 - b1) * g
        self.v[k] = b2 * self.v[k] + (1 - b2) * g * g
        m_hat, v_hat = self.m[k] / (1 - b1 ** self.t), self.v[k] / (1 - b2 ** self.t)
        p -= self.lr * m_hat / (np.sqrt(v_hat) + eps)        # cập nhật tại chỗ
```

Đoạn mã 2.3. Cùng kiến trúc bằng Keras và PyTorch (`tl/mlp_keras.py`, `tl/mlp_torch.py`, rút gọn)

```
# Keras: khai báo lớp, fit lo phần còn lại
model = keras.Sequential([keras.Input(shape=(15,)),
                          layers.Dense(128, activation="relu"), layers.Dropout(0.2),
                          layers.Dense(64, activation="relu"), layers.Dropout(0.2),
                          layers.Dense(1, activation="sigmoid")])
model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="binary_crossentropy")
model.fit(X, y, validation_data=(Xv, yv), epochs=40, batch_size=256,
          callbacks=[keras.callbacks.EarlyStopping(patience=8, restore_best_weights=True)])

# PyTorch: sigmoid nằm trong hàm mất mát (BCEWithLogitsLoss), vòng lặp tự viết, dữ liệu nằm
↪  sẵn trên GPU
net = nn.Sequential(nn.Linear(15, 128), nn.ReLU(), nn.Dropout(0.2),
                    nn.Linear(128, 64), nn.ReLU(), nn.Dropout(0.2), nn.Linear(64, 1)).cuda()
opt, loss_fn = torch.optim.Adam(net.parameters(), lr=1e-3), nn.BCEWithLogitsLoss()
for epoch in range(40):
    for idx in torch.randperm(len(X), device="cuda").split(256):
        opt.zero_grad()
        loss_fn(net(X[idx]).squeeze(1), y[idx]).backward()
        opt.step()
```

2.7.3. Kết quả

Bảng 2.9. MLP ba cách và các thuật toán cổ điển trên Diabetes Prediction (20.000 hồ sơ kiểm tra)

Mô hình	Accuracy	Recall	F1	ROC-AUC	Huấn luyện (s)	Kích thước
MLP – NumPy (CPU)	0,972	0,675	0,804	0,978	32,2	82 KB
MLP – Keras (CPU)	0,972	0,678	**0,806**	0,977	61,4	151 KB
MLP – PyTorch (GPU)	0,972	0,678	**0,806**	**0,978**	**20,1**	44 KB
Hồi quy logistic (cân bằng lớp)	0,889	**0,894**	0,578	0,963	1,0	2 KB
Rừng ngẫu nhiên	0,964	0,734	0,776	0,974	9,8	21 MB
SVM (RBF, cân bằng lớp)	0,896	0,919	0,600	0,972	240,3	308 KB

Nhận xét. (i) *Ba cách cài đặt cho cùng một kết quả*: AUC 0,977–0,978, F1 0,804–0,806, chênh lệch nằm trong biên độ của khởi tạo ngẫu nhiên. Đây là phép kiểm chứng rằng bản NumPy tự viết (lan truyền ngược, Adam, dropout ngược) đúng. (ii) Khác biệt nằm ở *chi phí*: PyTorch trên GPU nhanh nhất (20 s), NumPy 32 s, Keras chậm nhất (61 s) vì chi phí điều phối của `fit` trên CPU với mạng nhỏ. (iii) MLP có precision gần 0,99 nhưng recall chỉ 0,68 ở ngưỡng 0,5: mô hình rất "thận trọng" với lớp hiếm. Hồi quy logistic và SVM dùng trọng số cân bằng lớp nên recall cao (0,89–0,92) nhưng precision thấp. Với sàng lọc, nên hạ ngưỡng của MLP theo chỉ số Youden (ô ML-06 trong notebook) thay vì đổi mô hình. (iv) AUC 0,978 cao hơn hẳn NHANES (0,82) vì bộ này có HbA1c và glucose, chính là hai tiêu chí chẩn đoán. Đây là bài học về *rò rỉ theo định nghĩa nhãn*: mô hình đang học lại ngưỡng ADA nhiều hơn là dự đoán nguy cơ.

2.8. Đối chiếu trên hai miền khác

Để kiểm tra rằng quy trình (tiền xử lý không rò rỉ, cùng một MLP ba cách, đối chứng cổ điển) không chỉ đúng với dữ liệu y tế, nó được áp dụng nguyên vẹn cho hai bài toán khác (mã nguồn kế thừa từ thành viên nhóm 07):

- **Giá nhà Việt Nam** (30.229 tin đăng): *hồi quy* giá bán (tỷ đồng) từ diện tích (log), mặt tiền, đường vào, số tầng, phòng ngủ, phòng tắm, tỉnh, quận/huyện, pháp lý, nội thất, hướng nhà. Tầng ra tuyến tính, mất mát MSE.
- **Nhận xét khách hàng thời trang** (22.641 nhận xét tiếng Anh): *phân loại văn bản* ba lớp cảm xúc suy từ số sao, biểu diễn TF-IDF 5.000 từ. Tầng ra softmax, mất

mát entropy chéo.

Bảng 2.10. Đối chiếu: cùng quy trình trên hồi quy giá nhà và phân loại văn bản (20% kiểm tra)

Giá nhà (hồi quy)				Nhận xét (3 lớp)			
Mô hình	**MAE**	**RMSE**	**R^2**	**Mô hình**	**Acc**	**F1-macro**	**Train (s)**
OLS	1,150	1,492	0,544	Logistic	0,789	0,625	13,7
Rừng ngẫu nhiên	**0,994**	**1,351**	**0,626**	Linear SVM	0,817	0,622	3,1
SVR	1,024	1,380	0,610	Rừng ngẫu nhiên	0,821	0,585	8,0
MLP – NumPy	1,032	1,364	0,619	MLP – NumPy	**0,838**	**0,630**	88,0
MLP – Keras	1,023	1,355	0,623	MLP – Keras	0,837	0,621	19,4
MLP – PyTorch	1,019	1,354	0,624	MLP – PyTorch	0,835	0,619	3,7

Nhận xét. Ba kết luận của Mục 2.7 lặp lại ở cả hai miền. Ba bản MLP gần như trùng nhau (R^2 0,619–0,624; F1 0,619–0,630). PyTorch trên GPU luôn nhanh nhất. Với dữ liệu bảng, mô hình cây (rừng ngẫu nhiên) vẫn ngang hoặc nhỉnh hơn MLP. Ở bài toán văn bản, MLP vượt các mô hình tuyến tính về accuracy nhưng không vượt về F1-macro, vì lớp "trung tính" khó tách khỏi hai lớp còn lại ở cả mọi mô hình. Điều này củng cố bài học chung của chương: khi dữ liệu là bảng hoặc túi từ, *thông tin trong đặc trưng* quyết định trần hiệu năng, và kiến trúc chỉ tạo khác biệt nhỏ.

2.9. Mạng nhiều lớp dưới dạng hợp các hàm

Một mạng nơ-ron nhiều lớp là *hợp của các hàm* đơn giản. Với MLP ở Mục 2.7 (5 lớp tính cả lớp ra), xác suất mắc ĐTĐ của hồ sơ $\mathbf{x} \in \mathbb{R}^{15}$ là

$$\hat{y} = f(\mathbf{x}) = (\sigma \circ L_3 \circ D \circ \mathrm{ReLU} \circ L_2 \circ D \circ \mathrm{ReLU} \circ L_1)(\mathbf{x}), \qquad L_k(\mathbf{h}) = W_k\mathbf{h} + \mathbf{b}_k, \tag{2.6}$$

trong đó $L_1 : \mathbb{R}^{15} \to \mathbb{R}^{128}$, $L_2 : \mathbb{R}^{128} \to \mathbb{R}^{64}$, $L_3 : \mathbb{R}^{64} \to \mathbb{R}$ là các hàm affine, $\mathrm{ReLU}(z) = \max(0, z)$ tác động từng phần tử, D là dropout (đồng nhất khi suy luận) và $\sigma(z) = 1/(1 + e^{-z})$. Mỗi hàm riêng có một vai trò:

- L_k trộn tuyến tính các đặc trưng, đây là nơi chứa toàn bộ tham số học được.
- ReLU tạo phi tuyến. Không có nó, hợp các hàm affine vẫn là affine và mạng suy biến thành hồi quy logistic.
- D chỉ hoạt động khi huấn luyện: tắt ngẫu nhiên 20% nơ-ron để mạng không phụ

thuộc vào một vài đường đi.

- σ đưa điểm số về xác suất trong $(0, 1)$.

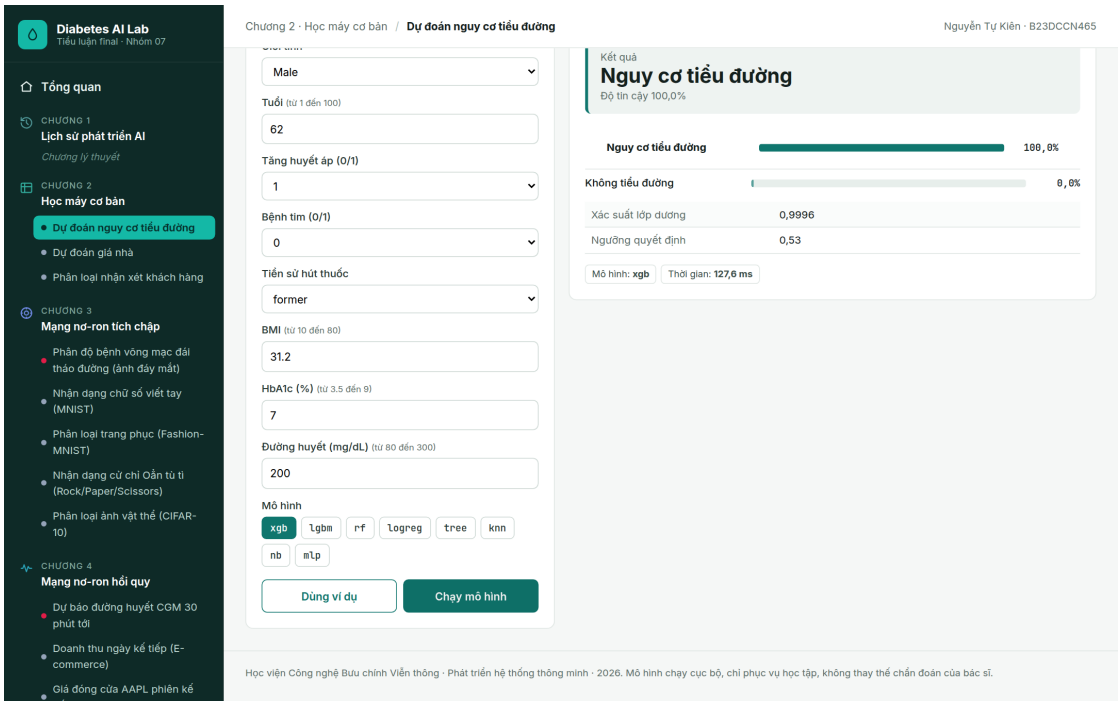
Cách nhìn hợp hàm giải thích luôn lan truyền ngược: theo quy tắc dây chuyền, $\nabla_{W_1} L = \frac{\partial L}{\partial \hat{y}}\, \sigma'\, W_3^\top\, \mathrm{ReLU}'\, W_2^\top\, \mathrm{ReLU}'\, \mathbf{x}^\top$, tức là tích các đạo hàm của từng hàm riêng theo thứ tự ngược. Đó chính là vòng `for i in reversed(range(len(self.W)))` ở Đoạn mã 2.2. Ba chương sau dùng lại cách nhìn này: CNN thay L_k bằng tích chập (Chương 3), RNN lặp cùng một hàm theo thời gian (Chương 4), GNN thêm phép trộn theo láng giềng (Chương 6).

2.10. Thực nghiệm trên Notebook và Demo Localhost

Toàn bộ quy trình ba cách cài đặt trên bộ *Diabetes Prediction* được đóng gói trong notebook `notebooks/ch2_hoc_may.ipynb` (8 ô mã `ML-00` đến `ML-06`) và tích hợp trực tiếp vào ứng dụng web chạy cục bộ tại `http://localhost:8095`. Hai ô tiêu biểu trong notebook minh họa rõ hai quyết định kỹ thuật quan trọng:

- **Ô `ML-02` và `ML-03` (Tiền xử lý không rò rỉ và kiểm chứng MLP viết tay)**: Ô `ML-02` chia phân tầng 80.000 mẫu huấn luyện và 20.000 mẫu kiểm tra (tỷ lệ dương đều đúng 0,085), rồi mới khớp `StandardScaler` chỉ trên tập huấn luyện (trung bình tuổi 41,90, BMI 27,32, HbA1c 5,53%, glucose 138,08 mg/dL). Ô `ML-03` khởi tạo `ScratchMLP([15, 128, 64, 1])` bằng NumPy thuần và chạy thử 5 epoch đầu tiên, đạt ngay ROC-AUC = 0,9737 sau 2,7 giây trên CPU — xác nhận công thức lan truyền ngược và cập nhật Adam có hiệu chỉnh độ chệch hoạt động chính xác trước khi huấn luyện đủ 40 epoch (AUC = 0,9775).
- **Ô `ML-06` (Hiệu chỉnh ngưỡng quyết định theo chỉ số Youden J)**: Vì lớp dương chỉ chiếm 8,5%, ngưỡng mặc định 0,5 của MLP NumPy cho độ đặc hiệu 0,997 nhưng độ nhạy (recall) chỉ đạt 0,655 (bỏ sót hơn một phần ba người bệnh). Ô `ML-06` quét đường cong ROC và tìm ngưỡng tối ưu hóa J = độ nhạy + độ đặc hiệu − 1 tại $p^* = 0{,}101$, nâng độ nhạy lên 0,922 trong khi vẫn giữ độ đặc hiệu ở mức 0,895 — phù hợp hơn nhiều cho bài toán sàng lọc ban đầu.

Trên giao diện web cục bộ (Hình 2.9), tab **Ch.2 — Học máy · Tiểu đường (100k hồ sơ)** nạp đồng thời cả 6 mô hình đã huấn luyện (`MLP - scratch`, `MLP - keras`, `MLP - torch`, hồi quy logistic, rừng ngẫu nhiên, SVM) để dự đoán tức thời trên một hồ sơ nhập tay hoặc mẫu thử và đối chiếu xác suất dự đoán giữa ba framework.



Hình 2.9. Giao diện thử nghiệm Chương 2 trên `http://localhost:8095`: so sánh trực tiếp xác suất dự đoán của MLP ba cách (NumPy, Keras, PyTorch) và các mô hình cổ điển

2.11. Triển khai

Endpoint `POST /api/lab/ch2/nhanes` nhận 13 đặc trưng; trường nào để trống được coi là thiếu. Endpoint trả về xác suất ĐTĐ của bảy mô hình đã huấn luyện (tệp `data/models/ch2_classic.joblib`, nén 3 MB), cùng ngưỡng Youden của từng mô hình và đóng góp của từng đặc trưng theo hồi quy logistic. Trên trang `/lab`, thẻ "Chương 2" vẽ biểu đồ cột so sánh bảy xác suất. Người dùng thấy ngay các mô hình mạnh đồng thuận còn Naive Bayes thì cực đoan.

Đoạn mã 2.4. Gọi endpoint Chương 2 (rút gọn)

```
curl -s https://agent.nits.io.vn/api/lab/ch2/nhanes -H 'Content-Type: application/json' \
  -d
   ↪  '{"age_years":58,"sex":0,"bmi":29,"waist_cm":95,"high_bp":1,"high_chol":1,"family_history":1}'
```

2.12. Tiểu kết chương 2

Chương 2 trình bày khung chung của học máy: cực tiểu rủi ro thực nghiệm có điều chuẩn, tối ưu bằng gradient, và đánh giá bằng thước đo phù hợp với bài toán. Thực nghiệm trên 20.646 người NHANES cho ba kết luận. (1) Với yếu tố nguy cơ tự khai, các mô hình tốt đều hội tụ về AUC ≈ 0,82. Chọn đặc trưng và dữ liệu quan trọng hơn chọn thuật toán. (2) Giả định của mô hình quyết định thành bại: Naive Bayes thất bại vì đặc trưng tương

quan, SVM thất bại khi không xử lý lệch lớp, cây đơn lẻ thất bại vì phương sai cao. (3) Tuổi, vòng eo và tiền sử gia đình là ba yếu tố nguy cơ chính. Kết luận này nhất quán giữa cây, độ quan trọng hoán vị và tỉ số chênh, và khớp với hướng dẫn ADA. Mô hình ước lượng được *nguy cơ* nhưng không thay được *xét nghiệm*. Ở các chương sau, hệ thống bổ sung các nguồn dữ liệu khác: ảnh (Chương 3), chuỗi thời gian (Chương 4) và tri thức văn bản (Chương 5).

CHƯƠNG 3. MẠNG NƠ-RON TÍCH CHẬP (CNN)

Bệnh võng mạc đái tháo đường (DR) là nguyên nhân hàng đầu gây mù lòa ở người trong độ tuổi lao động. ADA khuyến cáo khám đáy mắt ngay khi chẩn đoán ĐTĐ típ 2 và định kỳ 1–2 năm sau đó [2]. Năm 2016, Gulshan và cộng sự [42] cho thấy một CNN có thể phát hiện DR từ ảnh đáy mắt với độ nhạy ngang bác sĩ nhãn khoa. Chương này phân tích cấu trúc của CNN theo từng lớp, gồm phép tích chập, hàm kích hoạt, gộp, chuẩn hóa theo lô, dropout và lớp phân loại. Sau đó chương huấn luyện bốn mô hình phân độ DR trên bộ RetinaMNIST.

3.1. Động lực của CNN

Một ảnh RGB 128×128 có 49.152 giá trị. Nếu nối thẳng vào một lớp kết nối đầy đủ 512 nơ-ron, riêng lớp đầu đã cần 25 triệu trọng số. Quan trọng hơn, cách làm này *bỏ qua cấu trúc không gian*: dịch ảnh sang phải một điểm ảnh, vector đầu vào thay đổi hoàn toàn. CNN dựa trên ba quan sát về ảnh tự nhiên, lấy cảm hứng từ các tế bào thị giác có trường tiếp nhận cục bộ mà Hubel và Wiesel mô tả [43]:

- **Kết nối cục bộ**: một đặc trưng (cạnh mạch máu, chấm xuất huyết) chỉ phụ thuộc vào vùng lân cận nhỏ.
- **Chia sẻ trọng số**: bộ dò "vi phình mạch" hữu ích ở mọi vị trí trên ảnh, nên dùng chung một bộ lọc. Phép tích chập vì thế *đẳng biến với phép tịnh tiến*.
- **Phân cấp**: cạnh → hình dạng → tổn thương → chẩn đoán. Các lớp sâu tổ hợp đặc trưng của lớp nông.

LeNet-5 (1998) [25] là CNN thương mại đầu tiên. AlexNet (2012) [29] mở ra kỷ nguyên học sâu nhờ ReLU, dropout và GPU.

3.2. Các lớp của CNN

3.2.1. Lớp tích chập

Với đầu vào $\mathbf{X} \in \mathbb{R}^{C_{in} \times H \times W}$ và C_{out} bộ lọc $\mathbf{K}^{(o)} \in \mathbb{R}^{C_{in} \times k \times k}$, bản đồ đặc trưng đầu ra tại kênh o, vị trí (i, j) là:

$$\mathbf{Y}_{o,i,j} = b_o + \sum_{c=1}^{C_{in}} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} \mathbf{K}^{(o)}_{c,u,v} \, \mathbf{X}_{c,\, si+u-p,\, sj+v-p}, \tag{3.1}$$

với s là bước trượt (stride) và p là số điểm ảnh đệm (padding). Thư viện học sâu thực chất tính *tương quan chéo* (không lật bộ lọc), nhưng vẫn gọi là tích chập. Kích thước đầu ra là

$$H_{out} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1, \tag{3.2}$$

nên bộ lọc 3×3 với $p = 1, s = 1$ giữ nguyên kích thước (đệm "same"). Số tham số là $C_{out}(C_{in}k^2 + 1)$, *không phụ thuộc kích thước ảnh*. Đó là lợi thế quyết định so với MLP.

Ví dụ tính tay. Ảnh xám 4×4 và bộ lọc dò cạnh dọc 3×3, $s = 1$, không đệm, cho đầu ra 2×2:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \Rightarrow \mathbf{Y} = \begin{bmatrix} 3 & 3 \\ 3 & 3 \end{bmatrix}.$$

Phần tử $\mathbf{Y}_{0,0} = (1 + 1 + 1) \cdot 1 + (1 + 1 + 1) \cdot 0 + (0 + 0 + 0) \cdot (-1) = 3$. Bộ lọc phản hồi mạnh ở nơi cường độ chuyển từ sáng sang tối theo chiều ngang, tức là một cạnh dọc. Trong ảnh đáy mắt, các bộ lọc lớp đầu học được tương tự để bắt mép mạch máu và rìa đĩa thị.

3.2.2. Hàm kích hoạt

Nếu không có hàm phi tuyến, chồng nhiều lớp tích chập vẫn chỉ là một phép tuyến tính. ReLU $f(z) = \max(0, z)$ có đạo hàm bằng 1 khi $z > 0$, nên tránh được bão hòa gradient của sigmoid và tính rất rẻ. Một số biến thể: Leaky ReLU (độ dốc nhỏ khi $z < 0$); hard-swish $z \cdot \text{ReLU6}(z + 3)/6$, xấp xỉ rẻ của swish, dùng trong MobileNetV3 [44].

3.2.3. Lớp gộp

Gộp cực đại (max pooling) 2×2, bước 2, lấy giá trị lớn nhất trong mỗi ô: $\mathbf{Y}_{c,i,j} = \max_{u,v \in \{0,1\}} \mathbf{X}_{c,2i+u,2j+v}$. Lớp này giảm một nửa kích thước mỗi chiều, không có tham số, và tạo tính bất biến cục bộ với dịch chuyển nhỏ. Khi lan truyền ngược, gradient chỉ đi qua vị trí đạt cực đại. *Gộp trung bình toàn cục* (GAP) lấy trung bình cả bản đồ $H \times W$ thành một số cho mỗi kênh. GAP thay cho việc "làm phẳng" trước lớp phân loại, giảm mạnh số tham số và cho phép đầu vào có kích thước tùy ý.

3.2.4. Chuẩn hóa theo lô

Batch Normalization [45] chuẩn hóa từng kênh theo thống kê của lô $\mathcal{B}$, rồi cho mạng học lại thang đo γ và độ dời β:

$$\hat{x} = \frac{x - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}}, \qquad y = \gamma \hat{x} + \beta. \tag{3.3}$$

Khi suy luận, μ và σ^2 được thay bằng trung bình động tích lũy lúc huấn luyện. BN giúp dùng được tốc độ học lớn hơn, giảm nhạy cảm với khởi tạo, và có tác dụng điều chuẩn nhẹ. Vì β đã đóng vai trò độ lệch, lớp tích chập đứng trước BN không cần bias (`bias=False`).

3.2.5. Dropout, lớp kết nối đầy đủ và softmax

Dropout [46] tắt ngẫu nhiên mỗi nơ-ron với xác suất p khi huấn luyện, buộc mạng không phụ thuộc vào một vài đặc trưng; lúc suy luận dùng toàn bộ mạng. Lớp cuối là một phép biến đổi tuyến tính $\mathbf{z} = \mathbf{W}\mathbf{h} + \mathbf{b}$ cho ra $K = 5$ *logit*, đưa qua softmax $p_k = e^{z_k} / \sum_j e^{z_j}$ để có phân phối xác suất trên 5 mức DR.

3.2.6. Hàm mất mát và huấn luyện

Hàm mất mát là entropy chéo có trọng số lớp và làm mịn nhãn (label smoothing) [47]:

$$\begin{aligned} \mathcal{L} &= -\frac{1}{\sum_i w_{y_i}} \sum_i w_{y_i} \sum_{k=1}^{K} \tilde{y}_{ik} \log p_{ik}, \\ \tilde{y}_{ik} &= (1 - \varepsilon)\, \mathbb{1}[k = y_i] + \frac{\varepsilon}{K}, \qquad w_c = \frac{N}{K\, N_c}. \end{aligned} \tag{3.4}$$

Trọng số w_c tỉ lệ nghịch với số mẫu của lớp c, bù cho lệch lớp nặng của dữ liệu. Làm mịn nhãn với $\varepsilon = 0{,}05$ ngăn mô hình quá tự tin. Điều này hợp lý vì ranh giới giữa hai mức DR liền kề vốn không rõ ràng ngay cả với chuyên gia. Gradient của $\mathcal{L}$ theo logit có dạng rất gọn, $\partial \mathcal{L} / \partial z_k = p_k - \tilde{y}_k$ (với một mẫu). Gradient này lan ngược qua các lớp bằng quy tắc chuỗi. Gradient theo bộ lọc tích chập chính là tương quan chéo giữa đầu vào và gradient đầu ra, nên cũng tính được bằng một phép tích chập. Tối ưu bằng AdamW [48], tức Adam với suy giảm trọng số tách rời, cùng lịch học cosine.

3.2.7. Tăng cường dữ liệu và học chuyển giao

Với vài nghìn ảnh, CNN rất dễ quá khớp. *Tăng cường dữ liệu* tạo biến thể hợp lệ của ảnh ở mỗi epoch. Ảnh đáy mắt không có hướng "trên–dưới" cố định, nên lật ngang, lật dọc, xoay ±20° và thay đổi độ sáng/độ tương phản đều giữ nguyên nhãn. *Học chuyển*

giao khởi tạo mạng bằng trọng số đã học trên ImageNet [28] (1,28 triệu ảnh). Các lớp đầu đã biết dò cạnh và kết cấu, chỉ cần thay lớp phân loại 1000 lớp bằng lớp 5 lớp rồi tinh chỉnh toàn mạng với tốc độ học nhỏ.

3.2.8. Các kiến trúc tiêu biểu

Bảng 3.1. Các kiến trúc CNN tiêu biểu

Kiến trúc (năm)	**Số lớp**	**Tham số**	**Ý tưởng chính**
LeNet-5 (1998) [25]	7	60 nghìn	Conv–pool xen kẽ, đọc chữ số trên séc
AlexNet (2012) [29]	8	60 triệu	ReLU, dropout, huấn luyện trên GPU
VGG-16 (2014) [49]	16	138 triệu	Chỉ dùng conv 3×3 xếp chồng
ResNet (2015) [50]	18–152	11–60 triệu	Kết nối tắt $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$
MobileNetV3 (2019) [44]	–	5,4 triệu	Tích chập tách sâu, khối SE, hard-swish

ResNet giải quyết hiện tượng *suy thoái*: mạng sâu hơn lại cho lỗi huấn luyện cao hơn. Mỗi khối học phần dư $\mathcal{F}(\mathbf{x})$ thay vì ánh xạ đầy đủ, và cộng với đầu vào qua kết nối tắt. Đạo hàm $\partial\mathbf{y}/\partial\mathbf{x} = \partial\mathcal{F}/\partial\mathbf{x} + \mathbf{I}$ luôn chứa thành phần đơn vị, nên gradient truyền thẳng về các lớp đầu. ResNet-18 gồm một lớp conv 7×7, bốn tầng (mỗi tầng 2 khối cơ sở, mỗi khối 2 conv 3×3 + BN + ReLU, số kênh $64 \to 128 \to 256 \to 512$), rồi GAP và FC. **MobileNetV3** tách tích chập chuẩn thành *depthwise* (mỗi kênh một bộ lọc $k \times k$) và *pointwise* (conv 1×1 trộn kênh). Chi phí giảm khoảng $1/C_{out} + 1/k^2$ lần. Khối *squeeze-and-excitation* học trọng số cho từng kênh. Kiến trúc này được thiết kế cho thiết bị di động, phù hợp với máy chủ chỉ có CPU.

3.2.9. Giải thích bằng Grad-CAM

Trong y tế, một dự đoán cần đi kèm lý do. Grad-CAM [51] tính tầm quan trọng của kênh k ở lớp tích chập cuối đối với lớp c bằng trung bình gradient, rồi tổng hợp có trọng

số các bản đồ đặc trưng:

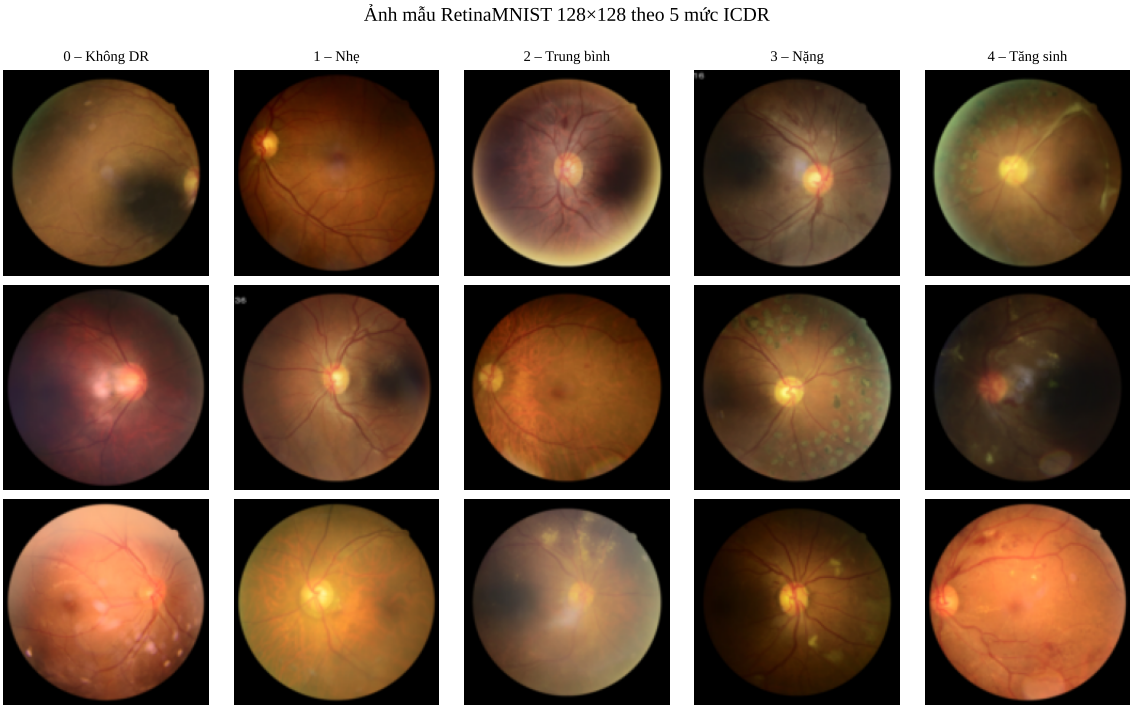
$$\alpha_k^c = \frac{1}{HW} \sum_{i,j} \frac{\partial z_c}{\partial A_{ij}^k}, \qquad L_{\text{Grad-CAM}}^c = \text{ReLU}\Big(\sum_k \alpha_k^c A^k \Big). \tag{3.5}$$

Bản đồ nhiệt được nội suy về kích thước ảnh và phủ lên ảnh gốc, cho thấy vùng nào "thuyết phục" mô hình chọn mức c.

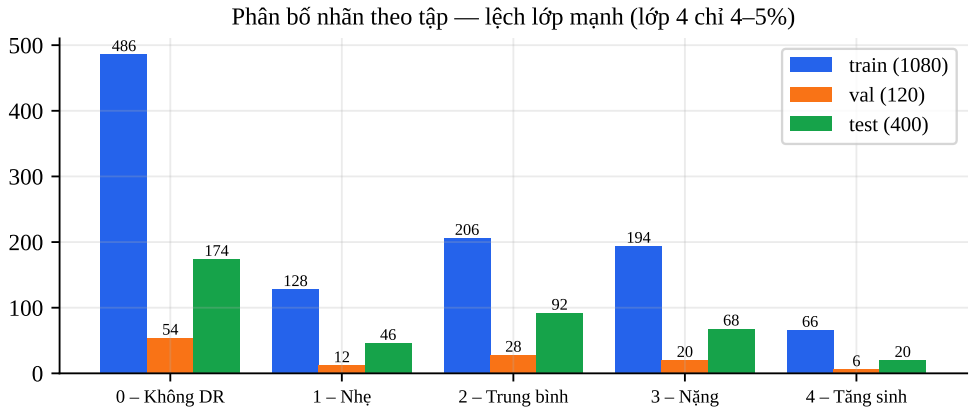
3.3. Dữ liệu RetinaMNIST

RetinaMNIST thuộc bộ MedMNIST v2 [6], được tạo từ bộ dữ liệu của cuộc thi DeepDRiD (ISBI 2020) [52]. Mỗi ảnh đáy mắt được bác sĩ phân độ theo thang quốc tế ICDR [53]: 0 – không DR; 1 – nhẹ (chỉ có vi phình mạch); 2 – trung bình; 3 – nặng; 4 – tăng sinh (tân mạch).

- **Nguồn**: `https://medmnist.com`; tệp `retinamnist_128.npz` tại Zenodo `https://zenodo.org/records/10519652`. Giấy phép CC BY 4.0.
- **Kích thước**: 1.600 ảnh RGB 128×128, chia sẵn 1.080 / 120 / 400 (train/val/test); tệp nén 46 MB.
- **Định dạng**: mảng `uint8` dạng $N \times 128 \times 128 \times 3$, nhãn số nguyên 0–4.



Hình 3.1. Ba ảnh mẫu của mỗi mức DR trong RetinaMNIST



Hình 3.2. Phân bố nhãn theo tập: lớp 0 chiếm 45%, lớp 4 chỉ 5%

Bảng 3.2. Số ảnh theo mức DR và trọng số lớp dùng trong hàm mất mát

Tập	Mức 0	Mức 1	Mức 2	Mức 3	Mức 4	Tổng
Train	486	128	206	194	66	1.080
Validation	54	12	28	20	6	120
Test	174	46	92	68	20	400
Trọng số w_c	0,44	1,69	1,05	1,11	3,27	

Nhận xét. (i) *Lệch lớp nặng*: mức 0 chiếm 45%, mức 4 (nguy hiểm nhất) chỉ 6% tập train. Một mô hình luôn đoán "mức 0" đã đạt accuracy 43,5% trên test. Vì vậy cần các thước đo không bị lớp đa số chi phối: macro-F1, và đặc biệt là *Quadratic Weighted Kappa* (QWK) [54]. QWK là thước đo chuẩn của các cuộc thi DR. Nó phạt sai số theo bình phương khoảng cách giữa hai mức: nhầm mức 4 thành 0 bị phạt 16 lần nặng hơn nhầm 4 thành 3. (ii) Ảnh có *chất lượng không đồng đều*: nhiều ảnh tối, lệch tâm hoặc bị lóa (Hình 3.1). Kênh xanh dương rất tối (trung bình 0,145 so với 0,395 của kênh đỏ), đặc trưng của ảnh đáy mắt. (iii) Tổn thương của mức 1 (vi phình mạch) chỉ rộng vài điểm ảnh ở độ phân giải gốc. Khi thu về 128×128, nhiều chi tiết bị mất, nên phân biệt mức 0 với mức 1 rất khó ngay từ dữ liệu. (iv) Tập validation chỉ có 120 ảnh (6 ảnh mức 4), nên việc chọn epoch tốt nhất có phương sai lớn.

3.4. Cài đặt

Bốn mô hình dùng chung tăng cường dữ liệu, hàm mất mát (3.4), AdamW, lịch cosine, kích thước lô 32. Epoch tốt nhất được chọn theo QWK trên validation. Huấn luyện chạy trên GPU RTX 3050; độ trễ suy luận được đo trên CPU, giống môi trường

deploy.

(1) MLP đối chứng thu ảnh về 64×64, làm phẳng thành 12.288 chiều, rồi qua các lớp 512 → 128 → 5 (ReLU, dropout 0,3). Mô hình này kiểm chứng việc bỏ qua cấu trúc không gian.

(2) CNN tự thiết kế gồm bốn khối *Conv 3×3* → *BN* → *ReLU* → *MaxPool 2×2* với số kênh 32 → 64 → 128 → 256, tiếp theo là GAP, dropout 0,4 và FC 5 (Đoạn mã 3.1). Bảng 3.3 liệt kê kích thước tensor và số tham số qua từng lớp. Theo (3.2), đệm 1 giữ kích thước sau mỗi conv, còn mỗi lớp gộp giảm một nửa: 128 → 64 → 32 → 16 → 8. Trường tiếp nhận của một nơ-ron sau khối 4 là 46×46 điểm ảnh, đủ bao một vùng tổn thương.

Đoạn mã 3.1. CNN tự thiết kế (`lab/ch3_cnn.py`)

```
def conv_block(c_in, c_out):
    """Conv 3×3 (đệm 1, giữ kích thước) → BatchNorm → ReLU → MaxPool 2×2 (giảm một nửa)."""
    return nn.Sequential(nn.Conv2d(c_in, c_out, 3, padding=1, bias=False),
     ↪  nn.BatchNorm2d(c_out),
                         nn.ReLU(inplace=True), nn.MaxPool2d(2))

class RetinaCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(conv_block(3, 32), conv_block(32, 64),
                                      conv_block(64, 128), conv_block(128, 256))
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Dropout(0.4), nn.Linear(256, 5))
    def forward(self, x):                          # x: [B, 3, 128, 128]
        return self.head(self.features(x))         # logits: [B, 5]

# entropy chéo có trọng số lớp + làm mịn nhãn, AdamW + lịch cosine
loss_fn = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=0.05)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs)
```

Bảng 3.3. Kích thước tensor và số tham số qua từng lớp của CNN tự thiết kế

Lớp	Cấu hình	Đầu ra	Tham số
Đầu vào	ảnh RGB đã chuẩn hóa ImageNet	3×128×128	0
Khối 1	Conv 3×3, 32 bộ lọc + BN + ReLU + MaxPool	32×64×64	864 + 64 = 928
Khối 2	Conv 3×3, 64 + BN + ReLU + MaxPool	64×32×32	18.432 + 128 = 18.560
Khối 3	Conv 3×3, 128 + BN + ReLU + MaxPool	128×16×16	73.728 + 256 = 73.984
Khối 4	Conv 3×3, 256 + BN + ReLU + MaxPool	256×8×8	294.912 + 512 = 295.424
GAP + Dropout(0,4)	trung bình mỗi kênh	256	0
FC + softmax	256 × 5 + 5	5	1.285
Tổng			**390.181**

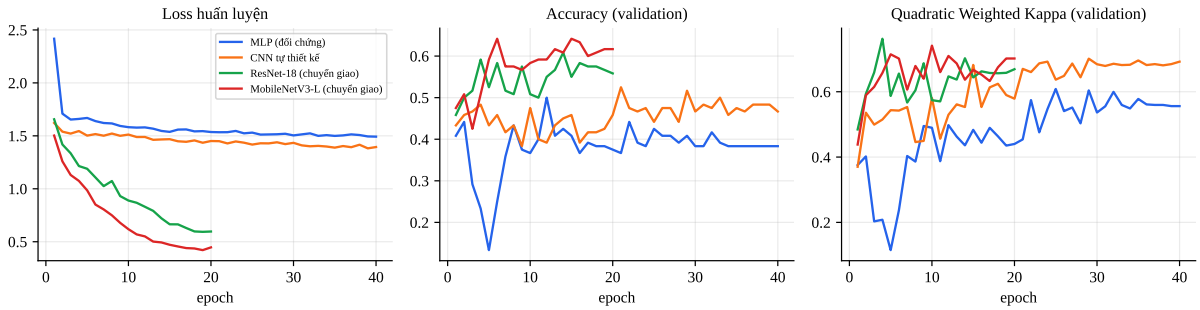
Số tham số của conv tính theo $C_{out} \cdot C_{in} \cdot 3 \cdot 3$, ví dụ khối 2 có $64 \cdot 32 \cdot 9 = 18.432$. BN có 2 tham số học ($\gamma$, β) cho mỗi kênh. CNN chỉ có 390 nghìn tham số, *ít hơn MLP 16 lần* (6,36 triệu), dù làm việc trên ảnh có độ phân giải gấp 4 lần.

(3) ResNet-18 và **(4) MobileNetV3-Large** được tải trọng số ImageNet từ torchvision. Lớp phân loại được thay bằng `Dropout(0,3) + Linear(512, 5)` (ResNet) hoặc `Linear(1280, 5)` (MobileNet), rồi tinh chỉnh toàn mạng 20 epoch với tốc độ học $3 \cdot 10^{-4}$ và $5 \cdot 10^{-4}$. Ảnh được chuẩn hóa bằng trung bình và độ lệch chuẩn của ImageNet, điều bắt buộc để trọng số tiền huấn luyện có nghĩa.

3.5. Kết quả và nhận xét

Bảng 3.4. Kết quả phân độ DR trên tập kiểm tra RetinaMNIST (400 ảnh)

Mô hình	Tham số	Acc.	Macro-F1	QWK	AUC	Huấn luyện	CPU/ảnh
Đoán toàn mức 0	0	0,435	0,121	0,000	0,500	–	–
MLP (đối chứng)	6,36 tr	0,490	0,282	0,515	0,712	16 s	1,0 ms
CNN tự thiết kế	0,39 tr	0,470	0,346	0,557	0,731	46 s	5,2 ms
ResNet-18 (chuyển giao)	11,18 tr	**0,578**	**0,482**	0,700	**0,833**	34 s	8,7 ms
MobileNetV3-L (chuyển giao)	4,21 tr	0,555	0,457	**0,710**	0,766	40 s	17,7 ms



Hình 3.3. Loss huấn luyện, accuracy và QWK trên validation theo epoch

Học chuyển giao là yếu tố quyết định. Hai mô hình tiền huấn luyện vượt xa hai mô hình học từ đầu: QWK 0,70–0,71 so với 0,52–0,56, AUC 0,83 so với 0,73. Với 1.080 ảnh, CNN học từ đầu không đủ dữ liệu để học các bộ lọc tốt. ResNet-18 thừa hưởng các bộ lọc cạnh và kết cấu từ 1,28 triệu ảnh ImageNet, nên chỉ cần 4 epoch để đạt QWK tốt nhất trên validation. Kết quả của ResNet-18 (accuracy 0,578, AUC 0,833) cao hơn mốc chuẩn công bố của MedMNIST v2 cho cùng kiến trúc và độ phân giải: accuracy 0,493 và AUC 0,717 [6]. Khác biệt chủ yếu đến từ học chuyển giao ImageNet, tăng cường dữ liệu và trọng số lớp. Huấn luyện trên GPU không hoàn toàn tất định: chạy lại cùng hạt giống cho accuracy dao động khoảng ±0,03.

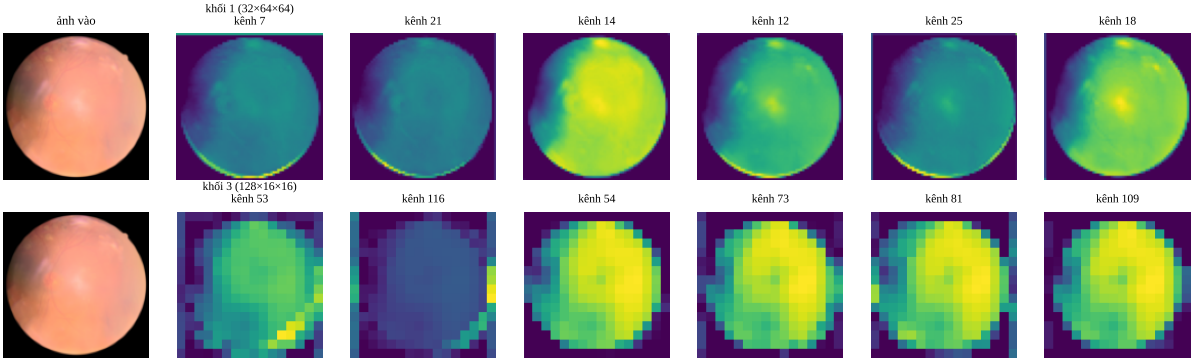
Accuracy dễ gây hiểu lầm. MLP đạt accuracy 0,490, *cao hơn* CNN tự thiết kế (0,470), nhưng macro-F1 chỉ 0,282. Ma trận nhầm lẫn cho thấy MLP dồn 345/400 dự đoán vào hai mức 0 và 3, và không bao giờ dự đoán mức 2 (0/92 ảnh mức 2 đúng). CNN phân bố dự đoán đều hơn trên cả 5 mức. Đây là lý do QWK và macro-F1 là thước đo chính.

CNN và MLP. Với ít tham số hơn 16 lần, CNN đạt QWK và macro-F1 cao hơn MLP, nhờ chia sẻ trọng số và kết nối cục bộ. Tuy vậy, CNN đạt QWK 0,70 trên validation nhưng chỉ 0,56 trên test: với 120 ảnh validation, chọn epoch theo QWK là một quá trình nhiễu. Trường hợp này cho thấy tập validation quá nhỏ cũng có thể gây quá khớp ở bước chọn mô hình.

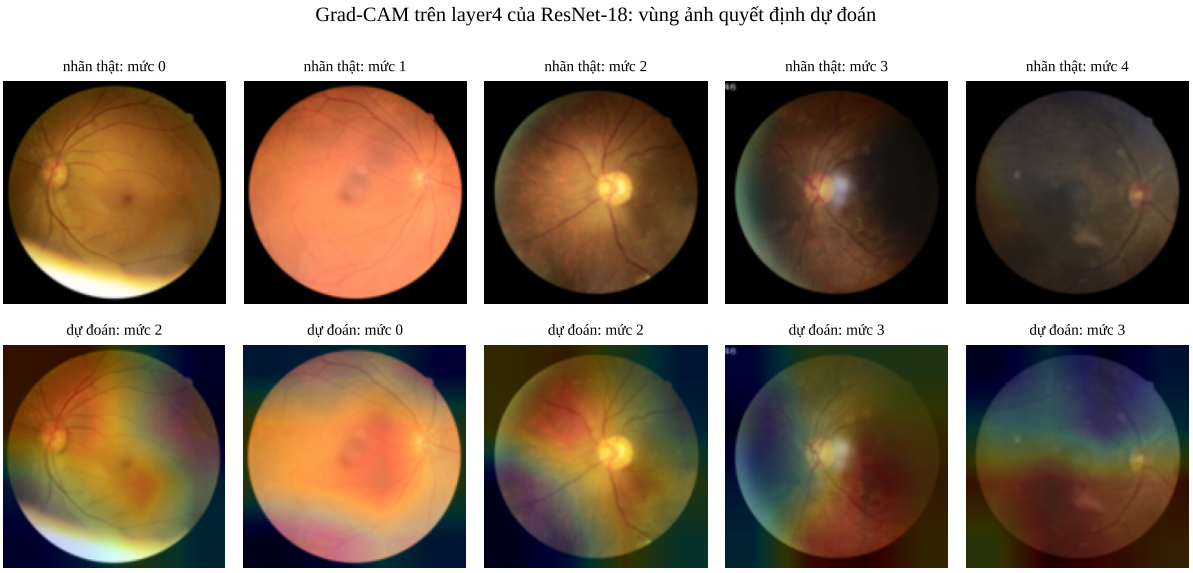


Hình 3.4. Ma trận nhầm lẫn của MobileNetV3-L trên tập kiểm tra (số ảnh và tỉ lệ theo hàng)

Phân tích lỗi. Ma trận nhầm lẫn của mô hình tốt nhất (Hình 3.4) tập trung quanh đường chéo. Trong 178 ảnh dự đoán sai, 131 ảnh lệch một mức (0↔1, 1↔2, 2↔3) và 47 ảnh lệch từ hai mức trở lên. Lỗi lệch một mức là kiểu lỗi mà QWK phạt nhẹ nhất. Mức 4 khó nhất: chỉ 4/20 ảnh đúng, phần lớn bị đoán là mức 3. Hai mức này cùng là DR nặng, đều cần chuyển chuyên khoa, nên trong sàng lọc sai số này ít hậu quả hơn so với bỏ sót bệnh (đoán thành mức 0). ResNet-18 chỉ đoán 2 trên 20 ảnh mức 4 thành mức 0.



Hình 3.5. Bản đồ đặc trưng sau khối 1 (cạnh, độ sáng) và khối 3 (vùng trừu tượng) của CNN tự thiết kế



Hình 3.6. Grad-CAM trên tầng `layer4` của ResNet-18 với một ảnh của mỗi mức DR

Mô hình "nhìn" vào đâu? Sau khối 1, các bản đồ đặc trưng giống ảnh gốc đã lọc: kênh này bắt mép đĩa thị, kênh kia bắt vùng sáng (Hình 3.5). Sau khối 3 (16×16), biểu diễn đã trừu tượng, mỗi kênh phản hồi một vùng. Grad-CAM của ResNet-18 (Hình 3.6) phần lớn rơi vào bên trong vùng võng mạc, quanh các mạch máu lớn, nơi tổn thương xuất hiện. Tuy nhiên, với một số ảnh, bản đồ nhiệt rơi vào viền tối hoặc vùng lóa, dấu hiệu mô hình còn học cả các yếu tố nhiễu do chất lượng ảnh. Đây là hạn chế cần biết trước khi tin vào dự đoán, và là lý do hệ thống luôn khuyến cáo khám nhãn khoa thay vì kết luận.

3.6. CNN cài đặt bằng ba cách

Các mô hình ở mục trước đều dùng PyTorch. Phần này viết *cùng một* CNN cơ bản bằng ba cách: tự viết bằng NumPy, Keras (TensorFlow) và PyTorch. Mục đích là hiểu tích chập ở mức phép toán và kiểm chứng rằng ba bản cho cùng chất lượng. Khung huấn luyện kế thừa mã nguồn của thành viên nhóm 07, được mở rộng thêm bộ ảnh đáy mắt. Cả ba bản đọc cùng một tệp `retina.npz` (1.200 ảnh train+val gốc của RetinaMNIST, 400 ảnh test) và cùng một phép chia validation phân tầng 10%, nên chênh lệch kết quả không đến từ dữ liệu.

Kiến trúc cơ bản (ảnh thu về 48×48×3): `Conv(3→32, 3×3) – ReLU – MaxPool(2) – Conv(32→64, 3×3) – ReLU – MaxPool(2) – Flatten – Dense(128) – ReLU – Dense(5) – softmax`, gồm 1.199.813 tham số (phần lớn nằm ở lớp Dense đầu tiên: $64 \cdot 12 \cdot 12 \cdot 128$). Ngoài ra có một **CNN cải tiến** (ảnh 96×96) cho hai framework có tự động đạo hàm: stem Conv–BN–ReLU, bốn khối dư (residual) 32–64–128–256 kênh xen MaxPool và Dropout

tăng dần, Global Average Pooling, Dense(256), cùng tăng cường dữ liệu (lật, xoay ±15°, dịch, phóng, đổi màu).

Bản NumPy không dùng bốn vòng lặp lồng nhau mà dùng kỹ thuật *im2col*: duỗi mỗi cửa sổ 3×3×C thành một hàng, khi đó tích chập trở thành *một phép nhân ma trận* lớn (Đoạn mã 3.2). Lan truyền ngược dùng phép ngược *col2im* để cộng dồn gradient của các cửa sổ chồng lấn về đúng điểm ảnh.

Đoạn mã 3.2. Tích chập bằng im2col và lan truyền ngược (`tl3/ch3_scratch.py`, rút gọn)

```
def im2col(X, kh, kw, stride=1, pad=0):
    """X [N,C,H,W] -> [N·Ho·Wo, C·kh·kw]: mỗi hàng là một cửa sổ đã duỗi thẳng; chỉ lặp kh·kw
     ↪  lần."""
    N, C, H, W = X.shape
    Ho, Wo = (H + 2 * pad - kh) // stride + 1, (W + 2 * pad - kw) // stride + 1
    Xp = np.pad(X, ((0, 0), (0, 0), (pad, pad), (pad, pad))) if pad else X
    cols = np.empty((N, C, kh, kw, Ho, Wo), dtype=X.dtype)
    for i in range(kh):
        for j in range(kw):
            cols[:, :, i, j] = Xp[:, :, i:i + stride * Ho:stride, j:j + stride * Wo:stride]
    return cols.transpose(0, 4, 5, 1, 2, 3).reshape(N * Ho * Wo, -1), Ho, Wo

def conv_forward(X, W, b, pad=1):
    F = W.shape[0]
    cols, Ho, Wo = im2col(X, W.shape[2], W.shape[3], 1, pad)
    Z = cols @ W.reshape(F, -1).T + b                       # tích chập = một phép nhân ma trận
    return Z.reshape(X.shape[0], Ho, Wo, F).transpose(0, 3, 1, 2), (X.shape, cols, W, pad,
     ↪  Ho, Wo)

def conv_backward(dZ, cache):
    X_shape, cols, W, pad, Ho, Wo = cache
    dZ2 = dZ.transpose(0, 2, 3, 1).reshape(-1, W.shape[0])
    dW = (dZ2.T @ cols).reshape(W.shape)                        # dL/dW = dZ^T · cols
    db = dZ2.sum(axis=0)
    dX = col2im(dZ2 @ W.reshape(W.shape[0], -1), X_shape, *W.shape[2:], Ho, Wo, 1, pad)
    return dX, dW, db
```

Đoạn mã 3.3. Cùng CNN cơ bản bằng Keras (`tl3/ch3_keras.py`) và PyTorch (`tl3/ch3_torch.py`)

```
keras_cnn = keras.Sequential([layers.Input((48, 48, 3)),
    layers.Conv2D(32, 3, padding="same", activation="relu"), layers.MaxPooling2D(2),
    layers.Conv2D(64, 3, padding="same", activation="relu"), layers.MaxPooling2D(2),
    layers.Flatten(), layers.Dense(128, activation="relu"), layers.Dense(5,
     ↪  activation="softmax")])

torch_cnn = nn.Sequential(
    nn.Conv2d(3, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
    nn.Flatten(), nn.Linear(64 * 12 * 12, 128), nn.ReLU(), nn.Linear(128, 5))   # softmax
     ↪  trong CrossEntropyLoss
```

Bảng 3.5. CNN ba cách trên RetinaMNIST (400 ảnh test; luôn đoán mức 0 đạt accuracy 0,435)

Cách	Mô hình	Accuracy	Macro-F1	QWK	Tham số	Huấn luyệ
NumPy (CPU)	CNN cơ bản, 12 epoch	0,542	0,351	0,523	1,20 tr	260
Keras (CPU)	CNN cơ bản	0,505	0,370	0,468	1,20 tr	22
PyTorch (GPU)	CNN cơ bản	**0,555**	**0,379**	**0,589**	1,20 tr	12
Keras (CPU)	CNN cải tiến (khối dư)	0,520	0,247	0,524	1,30 tr	178
PyTorch (GPU)	CNN cải tiến (khối dư)	0,550	0,327	0,500	1,29 tr	80
Tham chiếu (mục trước): ResNet-18 chuyển giao		0,578	0,482	0,700	11,2 tr	34

Nhận xét. (i) Ba cách viết cho kết quả cùng một vùng (QWK 0,47–0,59), chênh lệch nằm trong độ dao động của một tập test 400 ảnh với lớp hiếm. Bản NumPy tự viết đạt QWK 0,52, chứng tỏ im2col, col2im và lan truyền ngược đúng, nhưng chậm hơn PyTorch GPU 20 lần. (ii) *CNN cải tiến không tốt hơn* CNN cơ bản trên bộ này, trái với trên MNIST và Fashion-MNIST (Bảng 3.6). Với 1.080 ảnh huấn luyện, mạng sâu hơn quá khớp nhanh: bản Keras đạt validation tốt nhất ở epoch 3 rồi dừng sớm. (iii) Cả năm mô hình học từ đầu đều kém xa ResNet-18 học chuyển giao (QWK 0,70). Điều này củng cố kết luận của mục trước: với ảnh y khoa ít, *tri thức thị giác tiền huấn luyện* quan trọng hơn cách cài đặt hay độ sâu.

3.7. CNN dưới dạng hợp các hàm

CNN cơ bản ở Bảng 3.5 là hợp của năm khối hàm. Với ảnh $\mathbf{X} \in \mathbb{R}^{48\times48\times3}$:

$$\hat{\mathbf{p}} = (\text{softmax} \circ L_2 \circ \text{ReLU} \circ L_1 \circ \text{vec} \circ P \circ \text{ReLU} \circ C_2 \circ P \circ \text{ReLU} \circ C_1)(\mathbf{X}). \quad (3.6)$$

- $C_1 : \mathbb{R}^{48\times48\times3} \to \mathbb{R}^{48\times48\times32}$ và $C_2 : \mathbb{R}^{24\times24\times32} \to \mathbb{R}^{24\times24\times64}$ là tích chập 3×3: $C(\mathbf{X})_{i,j,f} = \sum_{c,u,v} W_{f,c,u,v} X_{i+u,j+v,c} + b_f$. Hai tính chất *kết nối cục bộ* và *chia sẻ trọng số* khiến C_1 chỉ có $3 \cdot 3 \cdot 3 \cdot 32 + 32 = 896$ tham số, trong khi lớp kết nối đầy đủ cùng kích thước cần hơn 200 triệu.
- P (MaxPool 2×2) giảm một nửa mỗi chiều không gian và tạo bất biến với dịch chuyển nhỏ: một vi phình mạch dịch đi 1 điểm ảnh vẫn cho cùng đáp ứng.
- vec duỗi khối 12×12×64 thành vector 9.216 chiều; L_1 (9.216→128) và L_2 (128→5) là các hàm affine, trong đó L_1 chiếm 98% tham số. Đây cũng là chỗ quá khớp, và là lý do CNN cải tiến thay vec bằng Global Average Pooling.
- softmax cho xác suất 5 mức bệnh.

Thứ tự hợp phản ánh *thứ bậc đặc trưng*: C_1 học cạnh và đốm màu (mạch máu, xuất huyết điểm), C_2 học tổ hợp của chúng (cụm xuất tiết, vùng thiếu máu). Grad-CAM ở mục trước chính là việc lấy gradient của $\hat{p}_k$ theo đầu ra của khối tích chập cuối, tức là "nhìn ngược" qua (3.6).

3.8. Đối chiếu trên ảnh tự nhiên

Cùng hai kiến trúc được huấn luyện trên hai bộ ảnh chuẩn (mã nguồn kế thừa của nhóm): **MNIST** (70.000 ảnh chữ số viết tay 28×28) và **Fashion-MNIST** (70.000 ảnh trang phục 28×28, 10 lớp).

Bảng 3.6. Đối chiếu: cùng CNN trên ảnh chữ số và trang phục (10.000 ảnh test mỗi bộ)

Bộ dữ liệu	Mô hình	Accuracy	Macro-F1	Tham số	Huấn luyện
MNIST	CNN cơ bản – NumPy	0,988	0,988	421.642	1.634 s
MNIST	CNN cơ bản – Keras	0,990	0,989	421.642	97 s
MNIST	CNN cơ bản – PyTorch	0,992	0,992	421.642	22 s
MNIST	CNN cải tiến – PyTorch	**0,997**	**0,997**	342.378	151 s
Fashion-MNIST	CNN cơ bản – PyTorch	0,918	0,917	421.642	26 s
Fashion-MNIST	CNN cải tiến – PyTorch	**0,939**	**0,939**	342.378	258 s

Nhận xét. Trên ảnh tự nhiên với hàng chục nghìn mẫu, mọi kết luận của bài toán đáy mắt đảo ngược: cùng kiến trúc cơ bản đạt 99% trên MNIST, và CNN cải tiến (khối dư, BatchNorm, tăng cường dữ liệu) hơn bản cơ bản 0,5–2 điểm phần trăm thay vì kém đi. Khác biệt không nằm ở mô hình mà ở *dữ liệu*: ảnh đáy mắt ít mẫu, lớp lệch nặng, và tín hiệu bệnh nằm ở chi tiết nhỏ bị mất khi thu ảnh về 48×48. Vì vậy với ảnh y khoa, học chuyển giao và ảnh độ phân giải cao là hướng cải thiện đúng, chứ không phải chồng thêm lớp.

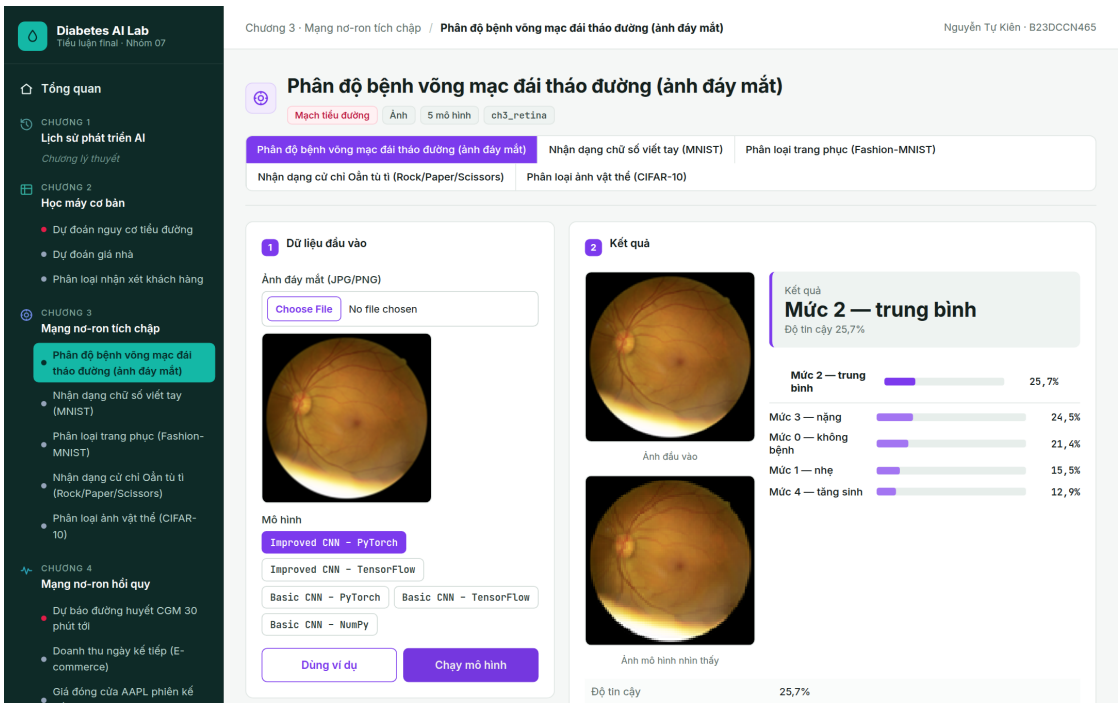
3.9. Thực nghiệm trên Notebook và Demo Localhost

Notebook `notebooks/ch3_cnn_day_mat.ipynb` (các ô `CNN-00` đến `CNN-04`) cho phép kiểm tra trực tiếp toán học bên dưới lớp tích chập và phân bố tham số của mạng:

- **Ô `CNN-02` (Kiểm chứng đạo hàm giải tích `conv_backward` bằng sai phân hữu hạn)**: Trước khi huấn luyện CNN viết tay bằng NumPy trên 1.080 ảnh đáy mắt, ô `CNN-02` sinh ngẫu nhiên một lô ảnh nhỏ $X \in \mathbb{R}^{2\times3\times6\times6}$ và bộ lọc $W \in \mathbb{R}^{4\times3\times3\times3}$, tính lan truyền tiến bằng `im2col` và đạo hàm ngược bằng `col2im`, rồi đối chiếu với đạo hàm số trung tâm $\frac{L(W_{ij}+\varepsilon)-L(W_{ij}-\varepsilon)}{2\varepsilon}$ ($\varepsilon = 10^{-5}$). Sai số tương đối cực đại trên toàn bộ trọng số đạt $8{,}28 \times 10^{-11}$, chứng minh phép biến đổi hai chiều `im2col`/`col2im` bảo toàn chính xác gradient ở biên đệm (`padding=1`).
- **Ô `CNN-04` (Giải phẫu hàm hợp 5 khối và nút thắt cổ chai tham số)**: Ô `CNN-04` tách `BasicCNN` thành khối trích xuất đặc trưng tích chập (`features`) và khối phân loại kết nối đầy đủ (`classifier`). Kết quả in ra cho thấy hai lớp tích chập $3 \to 32 \to 64$ chỉ dùng đúng 19.392 tham số (1,62% tổng mô hình), trong khi hai lớp tuyến tính sau `Flatten` chiếm tới 1.180.421 tham số (98,38%). Con số này giải thích trực tiếp

vì sao các kiến trúc hiện đại (như CNN cải tiến và ResNet-18) thay `Flatten` bằng Global Average Pooling để giảm hơn 90% tham số ở đầu phân loại và hạn chế quá khớp khi dữ liệu y khoa nhỏ.

Trên ứng dụng web cục bộ tại `http://localhost:8095` (Hình 3.7), tab **Ch.3 — CNN · Võng mạc ĐTĐ (RetinaMNIST)** nạp đồng thời cả 5 mô hình đã huấn luyện (`scratch`, `tf_basic`, `torch_basic`, `tf_improved`, `torch_improved`), cho phép chọn mẫu ảnh đáy mắt kiểm tra và so sánh trực quan phân phối xác suất 5 mức độ bệnh võng mạc đái tháo đường cùng thời gian suy luận của từng bản cài đặt.



Hình 3.7. Giao diện thử nghiệm Chương 3 trên `http://localhost:8095`: so sánh dự đoán 5 mức bệnh võng mạc đái tháo đường của CNN NumPy, Keras và PyTorch trên cùng ảnh đáy mắt RetinaMNIST

3.10. Triển khai

Mô hình triển khai được chọn theo QWK trên *validation*, không theo test, để tránh dùng tập kiểm tra cho việc chọn mô hình. Mô hình đó là ResNet-18 (val QWK 0,762), nặng 45 MB và mất 8,7 ms mỗi ảnh trên CPU (khoảng 150–300 ms cho cả lượt gọi API vì gồm giải mã ảnh và tính Grad-CAM). Endpoint `POST /api/lab/ch3/retina` nhận một ảnh JPEG/PNG (multipart). Ảnh được đưa về 128×128 và chuẩn hóa như lúc huấn luyện. Endpoint trả về xác suất 5 mức, mức dự đoán, khuyến cáo theo ADA, và ảnh Grad-CAM (PNG base64). `GET /api/lab/ch3/sample/{i}` trả về ảnh thứ *i* của tập kiểm tra cùng nhãn thật, để thử nhanh trên trang `/lab`. Mô hình được đóng gói trong lớp

`CNNRetinopathyEngine` (`src/models/cnn_vision.py`). Kết quả CNN chỉ mang tính tham khảo, không thay thế khám mắt định kỳ.

Đoạn mã 3.4. Gọi endpoint Chương 3 với một ảnh đáy mắt

```
curl -s https://agent.nits.io.vn/api/lab/ch3/retina -F "file=@fundus.png"
# Ảnh mẫu số 8 của tập kiểm tra (nhãn thật: mức 4)
# {"grade": 3, "label": "3 - Bệnh võng mạc không tăng sinh nặng",
#  "probabilities": [0.0021, 0.0049, 0.0698, 0.8566, 0.0666],
#  "advice": "Chuyển bác sĩ nhãn khoa sớm.", "model": "ResNet-18 (ImageNet → RetinaMNIST)",
#  "gradcam_png": "iVBORw0KGgo...", "latency_ms": 169.0}
```

3.11. Tiểu kết chương 3

Chương 3 phân tích từng thành phần của CNN: tích chập với chia sẻ trọng số và kết nối cục bộ, ReLU, gộp, chuẩn hóa theo lô, dropout, GAP, softmax và hàm mất mát entropy chéo có trọng số. Tiếp theo là các kiến trúc từ LeNet tới ResNet và MobileNet. Thực nghiệm phân độ DR trên RetinaMNIST cho thấy: (1) CNN dùng ít tham số hơn MLP 16 lần nhưng phân loại tốt hơn nhờ khai thác cấu trúc không gian; (2) với dữ liệu y tế nhỏ, học chuyển giao từ ImageNet là yếu tố quyết định, nâng QWK từ 0,56 lên 0,70–0,71; (3) với dữ liệu lệch lớp, cần dùng QWK và macro-F1 thay cho accuracy; (4) Grad-CAM giúp kiểm tra mô hình có nhìn đúng vùng tổn thương không. Ảnh là dữ liệu tĩnh. Chương 4 chuyển sang dữ liệu có chiều thời gian: chuỗi đo đường huyết liên tục.

CHƯƠNG 4. MẠNG NƠ-RON HỒI QUY (RNN)

Cảm biến theo dõi glucose liên tục (CGM) đo đường huyết dưới da mỗi 5–15 phút, tạo ra hàng trăm điểm đo mỗi ngày. Nếu dự báo được đường huyết 30–60 phút tới, người bệnh có thời gian phòng tránh hạ hoặc tăng đường huyết. Đây là bài toán *chuỗi thời gian*: giá trị tiếp theo phụ thuộc vào cả quỹ đạo trước đó, không chỉ vào một thời điểm. Chương này trình bày mạng nơ-ron hồi quy, thuật toán lan truyền ngược theo thời gian, vấn đề biến mất gradient và hai lời giải LSTM, GRU. Sau đó chương so sánh sáu mô hình dự báo glucose trên dữ liệu CGM thật của 112 bệnh nhân.

4.1. Dữ liệu tuần tự và kiến trúc RNN

Dữ liệu tuần tự $\mathbf{x}_1, \dots, \mathbf{x}_T$ có ba đặc điểm khiến MLP và CNN không phù hợp: độ dài thay đổi, thứ tự quan trọng, và phụ thuộc có thể xa. RNN [55] xử lý chuỗi bằng một *trạng thái ẩn* $\mathbf{h}_t$, cập nhật ở mỗi bước bằng *cùng một bộ tham số*:

$$\begin{aligned} \mathbf{h}_t &= \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h), \\ \hat{\mathbf{y}}_t &= \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y. \end{aligned} \tag{4.1}$$

$\mathbf{h}_t$ là bản tóm tắt có mất mát của toàn bộ quá khứ $\mathbf{x}_{1:t}$. Khi "mở" theo thời gian, RNN là một mạng sâu T lớp có trọng số dùng chung giữa các lớp. Số tham số không phụ thuộc độ dài chuỗi: với đầu vào d chiều và trạng thái ẩn m chiều, RNN có $m(d+m)+m$ tham số (PyTorch dùng hai vector bias nên là $m(d+m)+2m$).

Tùy cách nối đầu vào với đầu ra, có các dạng bài toán: *nhiều – một* (phân loại chuỗi), *một – nhiều* (sinh chuỗi), *nhiều – nhiều đồng bộ* (gán nhãn từng bước), và *nhiều – nhiều lệch pha* (dịch máy, dự báo nhiều bước). Bài toán của chương thuộc dạng *nhiều – một vector*: đọc 12 điểm gần nhất rồi xuất một vector 4 giá trị cho 15, 30, 45, 60 phút tới.

4.2. Lan truyền ngược theo thời gian và vấn đề biến mất gradient

RNN được huấn luyện bằng BPTT [56]: mở mạng thành đồ thị tính toán T bước rồi áp dụng quy tắc chuỗi. Gradient của mất mát ở bước T theo trạng thái ở bước k là một

tích các ma trận Jacobi:

$$\frac{\partial \mathscr{L}_T}{\partial \mathbf{h}_k} = \frac{\partial \mathscr{L}_T}{\partial \mathbf{h}_T} \prod_{t=k+1}^{T} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}}, \qquad \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \operatorname{diag}(1 - \mathbf{h}_t^2)\, \mathbf{W}_{hh}. \tag{4.2}$$

Nếu chuẩn của mỗi Jacobi nhỏ hơn 1 (trị riêng lớn nhất của $\mathbf{W}_{hh}$ nhỏ, tanh bão hòa), tích (4.2) co về 0 theo hàm mũ của $T-k$. Đó là *biến mất gradient*: mạng không học được phụ thuộc xa. Nếu chuẩn lớn hơn 1, tích tăng theo hàm mũ, gọi là *bùng nổ gradient* [57], [58]. Bùng nổ được xử lý dễ bằng *cắt gradient*: nếu $\|\mathbf{g}\| > \tau$ thì $\mathbf{g} \leftarrow \tau \mathbf{g}/\|\mathbf{g}\|$. Thực nghiệm dùng $\tau = 1$. Biến mất gradient đòi hỏi thay đổi kiến trúc.

4.3. LSTM và GRU

4.3.1. Bộ nhớ dài – ngắn hạn (LSTM)

LSTM [24] thêm một *trạng thái ô* $\mathbf{c}_t$ được cập nhật chủ yếu bằng phép cộng, cùng ba cổng sigmoid điều khiển dòng thông tin:

$$\begin{aligned}
\mathbf{f}_t &= \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) && \text{cổng quên} \\
\mathbf{i}_t &= \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) && \text{cổng vào} \\
\tilde{\mathbf{c}}_t &= \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) && \text{nội dung ứng viên} \\
\mathbf{c}_t &= \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t && \text{cập nhật ô nhớ} \\
\mathbf{o}_t &= \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o), \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) && \text{cổng ra}
\end{aligned} \tag{4.3}$$

Điểm mấu chốt là $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} = \operatorname{diag}(\mathbf{f}_t)$ (bỏ qua đường gián tiếp qua $\mathbf{h}$). Khi cổng quên gần 1, gradient chảy qua nhiều bước gần như nguyên vẹn, tạo thành một "băng chuyền" (constant error carousel). Mạng tự *học* khi nào cần nhớ, khi nào cần quên. LSTM có bốn khối biến đổi nên số tham số gấp bốn RNN: $4[m(d+m)+m]$.

Trong bài toán glucose, có thể hình dung các cổng như sau. Cổng vào mở mạnh khi glucose bắt đầu tăng nhanh sau ăn. Cổng quên giữ lại xu hướng tăng hay giảm. Cổng ra quyết định phần nào của ô nhớ dùng cho dự báo.

4.3.2. Đơn vị hồi quy có cổng (GRU)

GRU [59] rút gọn LSTM: gộp trạng thái ô vào trạng thái ẩn và chỉ dùng hai cổng:

$$\begin{aligned} \mathbf{z}_t &= \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]), \\ \mathbf{r}_t &= \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]), \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]), \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \end{aligned} \tag{4.4}$$

Cổng cập nhật $\mathbf{z}_t$ nội suy giữa trạng thái cũ và trạng thái ứng viên, đóng vai trò của cả cổng quên lẫn cổng vào. Cổng đặt lại $\mathbf{r}_t$ cho phép "quên" quá khứ khi tính nội dung mới. GRU có ba khối nên ít hơn LSTM khoảng 25% tham số.

4.3.3. Các mở rộng

RNN nhiều tầng xếp chồng các lớp hồi quy: đầu ra $\mathbf{h}_t^{(l)}$ của tầng l là đầu vào của tầng $l + 1$, giúp học biểu diễn phân cấp theo thời gian. *RNN hai chiều* chạy thêm một chiều ngược để mỗi bước thấy cả tương lai, nên chỉ dùng được khi có toàn bộ chuỗi, không dùng cho dự báo. Mô hình *mã hóa – giải mã* [60] và *cơ chế chú ý* [61] cho phép bộ giải mã nhìn lại mọi trạng thái của bộ mã hóa. Từ đó dẫn tới Transformer [62], kiến trúc bỏ hẳn tính hồi quy.

4.3.4. Đánh giá mô hình tuần tự và các bẫy thường gặp

Đánh giá chuỗi thời gian có ba bẫy dễ làm kết quả lạc quan giả tạo. *Thứ nhất, rò rỉ theo thời gian*: xáo trộn ngẫu nhiên các cửa sổ trượt khiến cửa sổ ở test chồng lấn 11/12 điểm với một cửa sổ ở train. Phải chia theo thời gian (quá khứ $\rightarrow$ tương lai) hoặc theo đơn vị độc lập (bệnh nhân). *Thứ hai, thiếu mốc đối chứng*: với chuỗi có tự tương quan cao như glucose, "ngày mai giống hôm nay" (persistence) đã có sai số nhỏ. Một mô hình phức tạp chỉ có giá trị nếu vượt rõ mốc này, và *độ trễ pha*, tức dự báo chỉ là bản sao trễ của đầu vào, phải được kiểm tra bằng mắt. *Thứ ba, thước đo không phản ánh hậu quả*: sai 20 mg/dL ở mức 60 mg/dL (bỏ sót hạ đường huyết) nguy hiểm hơn nhiều so với sai 20 mg/dL ở mức 250 mg/dL. Vì vậy ngành CGM dùng thêm MARD (sai số tương đối) và lưới Clarke (theo hậu quả lâm sàng), bên cạnh RMSE.

4.3.5. RNN và Transformer cho chuỗi thời gian

Transformer xử lý toàn bộ chuỗi song song bằng tự chú ý, mỗi bước nhìn trực tiếp mọi bước khác với đường đi độ dài 1. Nhờ vậy không có vấn đề biến mất gradient theo khoảng cách, và huấn luyện song song được trên GPU. Đổi lại, chi phí tăng theo bình phương độ dài chuỗi, và mô hình cần nhiều dữ liệu hơn vì thiếu thiên kiến quy nạp "thứ tự thời gian" (phải thêm mã hóa vị trí). Với chuỗi ngắn (12 bước), dữ liệu vừa phải và yêu cầu suy luận nhẹ trên CPU như bài toán của chương, RNN có cổng vẫn là lựa chọn hợp lý: LSTM 2 tầng chỉ có 51 nghìn tham số. Với chuỗi dài và đa biến (CGM kèm bữa ăn, insulin, vận động trong nhiều ngày), các biến thể Transformer cho chuỗi thời gian là hướng mở rộng tự nhiên.

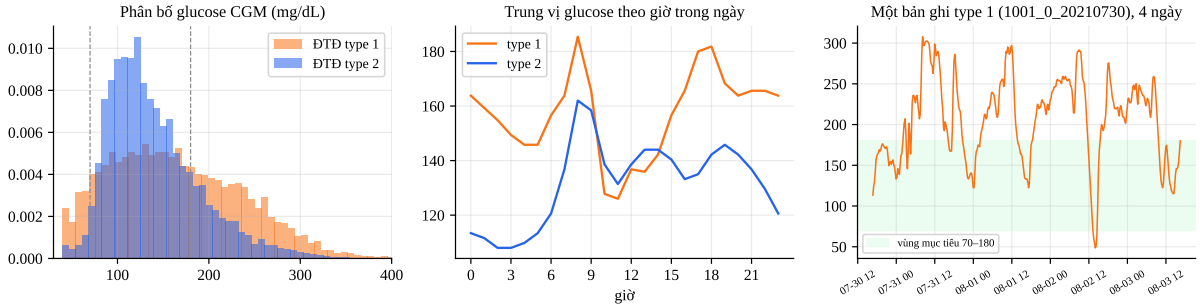
4.4. Dữ liệu ShanghaiT1DM / ShanghaiT2DM

Bộ dữ liệu được công bố trên *Scientific Data* năm 2023 [7]. Dữ liệu thu thập tại các bệnh viện ở Thượng Hải từ người bệnh đeo cảm biến FreeStyle Libre trong 3–14 ngày, kèm thông tin lâm sàng, bữa ăn và liều thuốc.

- **Nguồn**: figshare `https://doi.org/10.6084/m9.figshare.20444397`, tệp `data.zip` 3,4 MB, giấy phép CC BY 4.0.
- **Quy mô**: 12 bệnh nhân ĐTĐ típ 1 và 100 bệnh nhân típ 2, tổng cộng 125 bản ghi (một số người đeo nhiều đợt), 127.982 điểm CGM cách nhau 15 phút. Mỗi người có từ 247 đến 3.724 điểm (trung vị 1.218 điểm, khoảng 12,7 ngày).
- **Định dạng**: mỗi bản ghi một tệp Excel với các cột `Date`, `CGM (mg/dl)`, đường huyết mao mạch, bữa ăn, liều insulin. Tệp tổng hợp có 33 thuộc tính lâm sàng (tuổi, BMI, HbA1c, biến chứng…).

Bảng 4.1. Ba dòng đầu của một bản ghi CGM (bệnh nhân 1001, ĐTĐ típ 1)

Bệnh nhân	Bản ghi	Loại	Thời điểm	CGM (mg/dL)
1001	1001_0_20210730	T1	2021-07-30 16:43	113,4
1001	1001_0_20210730	T1	2021-07-30 16:58	124,2
1001	1001_0_20210730	T1	2021-07-30 17:13	129,6



Hình 4.1. Phân bố glucose theo loại ĐTĐ, trung vị theo giờ trong ngày và một bản ghi típ 1 trong 4 ngày

Bảng 4.2. Chỉ số kiểm soát đường huyết của hai nhóm theo đồng thuận quốc tế về Time in Range [63]

Nhóm	TB (mg/dL)	TIR 70–180	TBR < 70	TAR > 180	CV
Mục tiêu	–	> 70%	< 4%	< 25%	≤ 36%
ĐTĐ típ 1 (12 người)	164,8	53,6%	7,6%	38,7%	43,9%
ĐTĐ típ 2 (100 người)	140,2	78,7%	2,3%	18,9%	36,4%

Nhận xét. (i) Hai nhóm khác biệt rõ. Nhóm típ 1 dao động mạnh (CV 43,9%), chỉ 53,6% thời gian trong khoảng mục tiêu, và có 7,6% thời gian hạ đường huyết, gần gấp đôi ngưỡng an toàn. Nhóm típ 2 gần đạt mục tiêu. (ii) Đường huyết có *nhịp ngày đêm* rõ: trung vị tăng sau các bữa sáng, trưa, tối và thấp nhất vào rạng sáng. Đây là lý do thêm đặc trưng giờ trong ngày (mã hóa sin, cos để 23 giờ và 0 giờ nằm gần nhau). (iii) Có những khoảng mất tín hiệu cảm biến. Chuỗi được cắt tại mọi khoảng trống lớn hơn 20 phút để cửa sổ không nối qua chỗ hở. (iv) Nhóm típ 1 chỉ có 12 người, nên kết quả trên nhóm này có độ bất định cao.

4.5. Cài đặt

Thiết lập bài toán. Từ 12 điểm gần nhất (3 giờ), dự báo 4 điểm tiếp theo (15, 30, 45, 60 phút). Dữ liệu được chia *theo bệnh nhân*, phân tầng típ 1/típ 2: 78 người train (91.147 cửa sổ), 17 người validation (16.801), 17 người test (18.012). Chia theo cửa sổ ngẫu nhiên sẽ để các cửa sổ chồng lấn của cùng một người rơi vào cả train lẫn test, tức là rò rỉ dữ liệu. Chia theo người buộc mô hình phải tổng quát sang người chưa gặp. Glucose được chuẩn hóa z-score bằng trung bình 140,5 và độ lệch chuẩn 53,3 mg/dL của tập train.

Sáu mô hình. (1) *Persistence*: giữ nguyên giá trị cuối, mốc tối thiểu phải vượt qua. (2) *AR tuyến tính*: hồi quy Ridge trên 12 điểm, dự báo phần chênh so với điểm cuối. (3)–

(6) *RNN đơn giản, LSTM, GRU, LSTM 2 tầng* với 64 đơn vị ẩn. Đầu vào mỗi bước là vector 3 chiều $[\tilde{g}_t, \sin\theta, \cos\theta]$. Trạng thái ẩn ở bước cuối qua một lớp tuyến tính $64 \to 4$. Mô hình dự báo *phần dư* so với giá trị cuối: $\hat{\mathbf{y}} = \tilde{g}_T\mathbf{1} + \mathbf{W}\mathbf{h}_T + \mathbf{b}$. Kết nối tắt này giống ResNet: mạng chỉ cần học *thay đổi*, và khởi đầu ngay từ persistence thay vì từ con số 0 (Đoạn mã 4.1).

Đoạn mã 4.1. Mô hình hồi quy dự báo glucose và vòng huấn luyện (`lab/ch4_rnn.py`, rút gọn)

```
class SeqModel(nn.Module):
    def __init__(self, cell="lstm", hidden=64, layers=1):
        super().__init__()
        rnn = {"rnn": nn.RNN, "lstm": nn.LSTM, "gru": nn.GRU}[cell]
        self.rnn = rnn(3, hidden, num_layers=layers, batch_first=True,
                       dropout=0.2 if layers > 1 else 0.0)
        self.head = nn.Linear(hidden, 4)                  # 15, 30, 45, 60 phút

    def forward(self, x):                                 # x: [B, 12, 3]
        out, _ = self.rnn(x)                              # out: [B, 12, 64]
        return x[:, -1:, 0] + self.head(out[:, -1])     # dự báo phần dư so với điểm cuối

for epoch in range(40):
    for b in batches(Xtr, Ytr, size=512):
        loss = F.mse_loss(model(b.x), b.y)
        opt.zero_grad(); loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)   # chặn bùng nổ gradient
        opt.step()
    sched.step(val_mse)                                   # giảm tốc độ học khi validation chững
     ↪ lại
    # dừng sớm sau 8 epoch không cải thiện, giữ trọng số tốt nhất
```

Bảng 4.3. Kích thước tensor và số tham số của các mô hình hồi quy ($d = 3$, $m = 64$)

Mô hình	Công thức số tham số lớp hồi quy	Hồi quy	Tổng (+ lớp ra 260)
RNN	$m(d+m) + 2m = 64 \cdot 67 + 128$	4.416	4.676
GRU	$3[m(d+m) + 2m]$	13.248	13.508
LSTM	$4[m(d+m) + 2m]$	17.664	17.924
LSTM 2 tầng	tầng 1 như trên + $4[m(m+m) + 2m]$	50.944	51.204

Huấn luyện dùng Adam (tốc độ học $2 \cdot 10^{-3}$, giảm một nửa khi validation không cải thiện sau 3 epoch), lô 512, cắt gradient ở chuẩn 1, dừng sớm sau 8 epoch không cải thiện, tối đa 40 epoch. Huấn luyện trên GPU mất 23–43 giây mỗi mô hình.

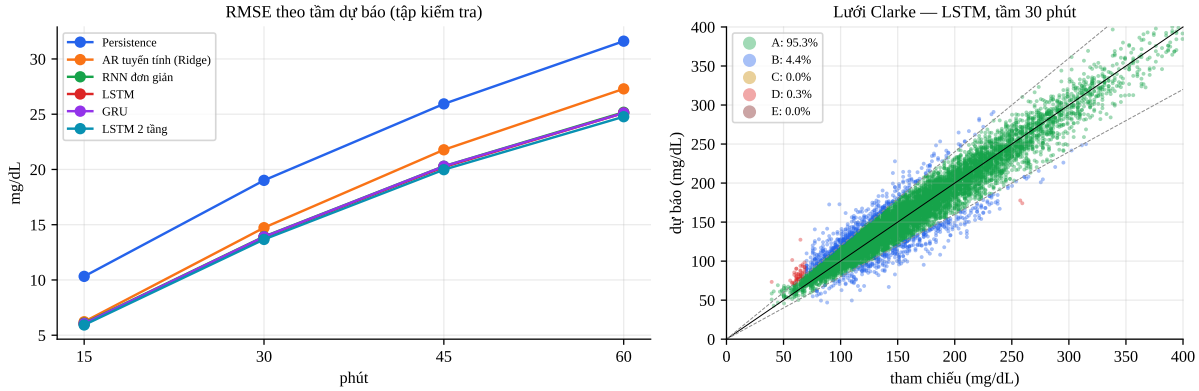
Thước đo. RMSE và MAE (mg/dL); MARD $= \frac{1}{n}\sum |y - \hat{y}|/y$, thước đo chuẩn của

ngành CGM; và *lưới sai số Clarke* [64]. Lưới Clarke chia mặt phẳng (tham chiếu, dự báo) thành 5 vùng theo hậu quả lâm sàng: A (sai ≤ 20% hoặc cả hai < 70 mg/dL), B (sai lệch lành tính), C (dẫn tới điều trị thừa), D (bỏ sót cần điều trị), E (điều trị ngược). Một hệ thống chấp nhận được cần hơn 99% điểm nằm trong vùng A+B.

4.6. Kết quả và nhận xét

Bảng 4.4. Dự báo glucose trên 17 bệnh nhân của tập kiểm tra (18.012 cửa sổ)

		Tầm 30 phút				Tầm 60 phút	
Mô hình	**Tham số**	**RMSE**	**MAE**	**MARD**	**Clarke A**	**RMSE**	**Clarke A**
Persistence	0	19,01	12,97	8,81%	90,1%	31,61	74,5%
AR tuyến tính (Ridge)	52	14,73	10,30	7,25%	94,5%	27,29	79,0%
RNN đơn giản	4.676	13,93	9,81	6,94%	95,2%	25,18	81,4%
LSTM	17.924	13,85	9,72	6,85%	95,3%	25,11	81,5%
GRU	13.508	13,90	9,64	6,75%	95,3%	25,11	82,1%
LSTM 2 tầng	51.204	**13,67**	**9,51**	**6,66%**	**95,6%**	**24,77**	**82,6%**



Hình 4.2. RMSE theo tầm dự báo và lưới sai số Clarke của LSTM ở tầm 30 phút

So với mốc đối chứng. Ở tầm 30 phút, LSTM 2 tầng giảm RMSE 28% so với persistence (19,01 → 13,67 mg/dL) và nâng tỉ lệ vùng Clarke A từ 90,1% lên 95,6%. Vùng A+B đạt 99,75%. Mô hình tuyến tính đã thu được phần lớn cải thiện (14,73): xu hướng tăng hoặc giảm của vài điểm cuối mang nhiều thông tin nhất. Các mô hình hồi quy giảm thêm 0,8–1,1 mg/dL nhờ học được động học phi tuyến, chẳng hạn đỉnh sau ăn "uốn" xuống chứ không tăng tuyến tính mãi.

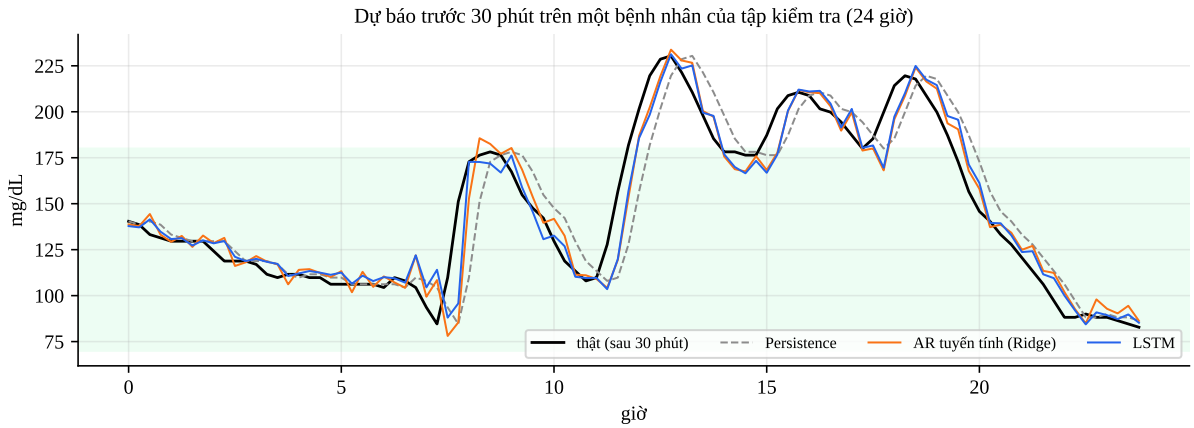
Giữa các mô hình hồi quy. RNN, LSTM, GRU chỉ cách nhau dưới 0,1 mg/dL ở tầm 30 phút; LSTM 2 tầng tốt hơn khoảng 0,2 mg/dL. Với cửa sổ chỉ 12 bước, phụ thuộc xa không phải là nút thắt. RNN đơn giản đủ "nhớ" 3 giờ và không chịu thiệt nhiều từ biến mất gradient. Lợi thế của các cổng thể hiện rõ hơn ở tầm xa: ở 60 phút, GRU và LSTM 2 tầng đạt Clarke A 82,1–82,6% so với 81,4% của RNN. Với dữ liệu nhỏ và yêu cầu độ trễ thấp, GRU là lựa chọn cân bằng: ít tham số hơn LSTM 25%, độ chính xác tương đương.

Sai số tăng nhanh theo tầm dự báo, từ khoảng 6 mg/dL ở 15 phút lên khoảng 25 mg/dL ở 60 phút. Trong một giờ, bữa ăn, insulin và vận động là các yếu tố không quan sát được từ chuỗi CGM mà quyết định quỹ đạo. Để dự báo xa hơn, cần thêm các đầu vào ngoại sinh này, vốn có trong bộ dữ liệu nhưng ghi chép không đều. Các nghiên cứu như GluNet [65] cũng ghi nhận sai số tăng nhanh theo tầm dự báo và dùng thêm thông tin bữa ăn, insulin để cải thiện tầm xa.

Bảng 4.5. Kết quả tầm 30 phút theo loại ĐTĐ trên tập kiểm tra

Nhóm	Cửa sổ	RMSE persistence	RMSE LSTM	MARD LSTM	Clarke A LSTM
Típ 1	2.227	20,80	13,86	6,39%	95,6%
Típ 2	15.785	18,75	13,85	6,92%	95,3%

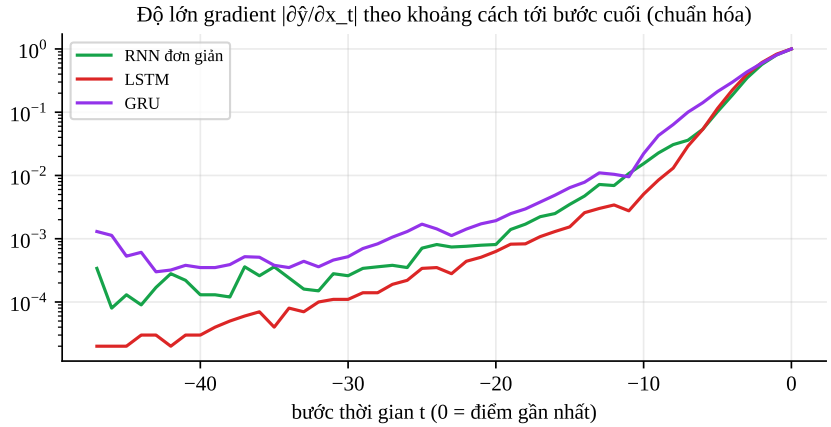
Theo Bảng 4.5, chuỗi típ 1 biến động mạnh hơn nên persistence kém hơn (20,80), nhưng LSTM đưa sai số hai nhóm về cùng mức. Mức giảm tương đối trên nhóm típ 1 là 33%, cao hơn 26% của nhóm típ 2. Mô hình có ích nhất ở chính nhóm cần nó nhất.



Hình 4.3. Dự báo trước 30 phút trên 24 giờ liên tục của một bệnh nhân thuộc tập kiểm tra

Hình 4.3 cho thấy hành vi điển hình: persistence luôn "trễ" đúng 30 phút so với đường thật. AR tuyến tính và LSTM đều bám sát đường thật hơn nhiều, và khác biệt giữa

hai mô hình (0,9 mg/dL RMSE) gần như không thấy bằng mắt. Cả hai đều vượt quá ở những điểm đổi chiều đột ngột (khoảng giờ thứ 7–8): từ chuỗi CGM, mô hình không biết bữa ăn đã kết thúc hay insulin đã có tác dụng.



Hình 4.4. Độ lớn gradient của dự báo theo từng điểm đầu vào trên chuỗi dài 48 bước (chuẩn hóa theo bước cuối)

Mô hình "nhìn" bao xa? Hình 4.4 đo $|\partial \hat{y} / \partial \tilde{g}_t|$ trên chuỗi dài 48 bước (12 giờ). Ở cả ba mô hình đã huấn luyện, độ nhạy giảm nhanh theo khoảng cách. Ở bước cách 11 điểm (khoảng 3 giờ), độ nhạy chỉ còn 1,1% (RNN), 0,3% (LSTM) và 1,0% (GRU) so với điểm cuối. Ở bước xa nhất, các con số giảm 3–5 bậc. Cần đọc kết quả này cẩn thận. Đây là độ nhạy của mô hình *đã học*, nên phản ánh cả hai yếu tố: biến mất gradient theo (4.2), và việc mô hình chủ động học cách "quên" quá khứ xa vì với glucose, vài điểm gần nhất là quan trọng nhất. LSTM có độ nhạy giảm nhanh nhất. Điều này cho thấy cổng quên đã học để loại bỏ thông tin cũ, chứ không phải không thể giữ lại. Muốn kiểm chứng riêng khả năng nhớ xa, cần một bài toán mà đáp án phụ thuộc vào điểm đầu chuỗi, như bài toán sao chép hay phép cộng của Hochreiter.

4.6.1. Phân tích sai số theo dải đường huyết

Sai số trung bình che giấu hành vi ở những vùng quan trọng nhất về lâm sàng. Bảng 4.6 chia các điểm tham chiếu ở tầm 30 phút theo ba dải, dùng mô hình triển khai (LSTM 2 tầng).

Bảng 4.6. Sai số dự báo 30 phút theo dải glucose tham chiếu (tập kiểm tra, LSTM 2 tầng)

Dải glucose thật	Số điểm	RMSE persistence	RMSE LSTM	MARD LSTM
< 70 mg/dL (hạ đường huyết)	206	14,90	12,98	15,0%
70–180 mg/dL (mục tiêu)	14.067	15,73	11,67	6,6%
> 180 mg/dL (tăng đường huyết)	3.739	28,26	19,44	6,4%
Thay đổi nhanh ($\|\Delta\|$ > 60 mg/dL trong 30 phút)	261	74,10	44,37	–

Có ba nhận xét. (i) Ở vùng tăng đường huyết, sai số tuyệt đối lớn nhất (19,4 mg/dL) nhưng sai số tương đối nhỏ (6,4%), và LSTM giảm 31% so với persistence. Đây là vùng đường huyết biến động mạnh sau ăn, nơi mô hình học được xu hướng. (ii) Ở vùng hạ đường huyết, MARD tăng lên 15%, do mẫu số nhỏ, và mức cải thiện so với persistence chỉ 13%. Hạ đường huyết hiếm trong dữ liệu (206/18.012 điểm, khoảng 1,1%), nên mô hình học từ MSE ít "để ý" tới vùng này. (iii) Khi glucose thay đổi nhanh, LSTM giảm sai số 40% so với persistence, nhưng sai số vẫn ở mức 44 mg/dL, cho thấy giới hạn của việc chỉ dùng chuỗi CGM.

Nếu dùng mô hình làm *bộ cảnh báo hạ đường huyết trước 30 phút* (báo khi dự báo < 70 mg/dL), độ nhạy chỉ đạt 0,59 và precision 0,72: cứ 10 cơn hạ đường huyết thì bỏ sót khoảng 4. Đây là lý do hệ thống *không* dùng dự báo để cảnh báo cấp cứu mà chỉ trình bày như thông tin tham khảo. Guardrail hạ đường huyết dựa trên giá trị đo thật (< 54 mg/dL). Để cải thiện, có thể dùng hàm mất mát có trọng số theo vùng nguy hiểm, hoặc huấn luyện thêm một bộ phân loại "sắp hạ đường huyết" riêng.

4.7. Mạng hồi quy cài đặt bằng ba cách

Phần này đặt lại bài toán ở dạng gọn hơn để so sánh *cách cài đặt*: từ 12 lần đo gần nhất (3 giờ), dự báo glucose **sau 30 phút**. Mỗi bước thời gian có 3 đặc trưng: glucose chuẩn hóa, sin và cos của giờ trong ngày (để mô hình biết bữa ăn và nhịp ngày đêm). Mọi mô hình dự báo *phần chênh* so với lần đo cuối, trong thang chuẩn hóa: $\hat{y} = g_t + s \cdot f(\mathbf{x}_{t-11:t})$, với $s = 53{,}3$ mg/dL là độ lệch chuẩn của tập huấn luyện. Dữ liệu chia **theo bệnh nhân** như các mục trên: 78 / 17 / 17 bệnh nhân, tương ứng 91.333 / 16.839 / 18.056 cửa sổ.

- **NumPy**: RNN một lớp (16 đơn vị ẩn) viết tay, lan truyền xuôi $\mathbf{h}_t = \tanh(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$ và lan truyền ngược qua thời gian (BPTT) có cắt chuẩn gradient, huấn luyện bằng SGD từng mẫu. Vì vòng lặp Python chậm, bản này chỉ học trên 12.000 cửa sổ trong 8 epoch và được đánh giá trên 5.000 cửa sổ kiểm tra lấy ngẫu nhiên.

- **Keras** (TensorFlow, CPU) và **PyTorch** (GPU): SimpleRNN, LSTM, GRU 64 đơn vị và LSTM *xếp chồng 5 lớp*; Adam ($\eta = 2 \cdot 10^{-3}$, cắt chuẩn gradient 1,0), lô 512, dừng sớm theo validation (6 epoch), giảm tốc độ học khi chững lại.

Đoạn mã 4.2. Một bước BPTT của RNN NumPy (kế thừa `tl3/ch4_scratch.py` của nhóm, rút gọn)

```python
def backward(self, yhat, y):
    """delta_T = W_hy^T (y_hat - y); delta_{t-1} = W_hh^T [(1 - h_t^2) * delta_t]; cộng dồn
     ↪  gradient dùng chung."""
    p, hs, x = self.p, self.hs, self.x
    g = {k: np.zeros_like(v) for k, v in p.items()}
    dy = yhat - y
    g["Why"], g["by"] = dy * hs[-1][None, :], np.array([dy])
    dh = p["Why"][0] * dy
    for t in range(len(x), 0, -1):
        da = (1.0 - hs[t] ** 2) * dh            # qua đạo hàm tanh
        g["Wxh"] += np.outer(da, x[t - 1])
        g["Whh"] += np.outer(da, hs[t - 1])      # cùng một W_hh ở mọi bước → cộng dồn
        g["bh"] += da
        dh = p["Whh"].T @ da                     # truyền về bước t - 1
    return g
```

Bảng 4.7. Dự báo glucose 30 phút trên 17 bệnh nhân kiểm tra (18.056 cửa sổ): ba cách cài đặt

Cách	Mô hình	RMSE	MAE	MARD (%)	Clarke A	Tham số	Train (s
—	Persistence (giữ giá trị cuối)	19,05	12,99	8,82	90,0%	0	–
—	AR tuyến tính (Ridge, 12 trễ)	14,77	10,31	7,26	94,4%	13	<
NumPy	SimpleRNN (16)	15,02*	10,47	7,20	95,1%	337	6
Keras	SimpleRNN	14,29	9,99	7,04	94,9%	4.417	5
Keras	LSTM	13,84	9,61	6,73	95,4%	17.473	13
Keras	GRU	13,92	9,67	6,78	95,2%	13.313	9
Keras	LSTM 5 lớp	13,79	9,54	6,67	95,4%	149.569	1.00
PyTorch	SimpleRNN	14,26	10,02	7,08	94,9%	4.481	3
PyTorch	LSTM	13,82	9,63	6,75	95,4%	17.729	2
PyTorch	GRU	13,91	9,66	6,77	95,3%	13.313	2
PyTorch	LSTM 5 lớp	**13,66**	**9,48**	**6,65**	**95,5%**	150.849	4

* Huấn luyện trên 12.000 cửa sổ, đánh giá trên 5.000 cửa sổ kiểm tra lấy ngẫu nhiên. Đơn vị RMSE, MAE: mg/dL.

Nhận xét. (i) *Keras và PyTorch cho cùng kết quả* với cùng kiến trúc (LSTM 13,84 so với 13,82; GRU 13,92 so với 13,91; LSTM 5 lớp 13,79 so với 13,66; PyTorch có thêm một vector độ chệch ở mỗi lớp hồi quy nên hơn 64–256 tham số mỗi lớp), nên chênh lệch giữa các mô hình là thật, không do framework. PyTorch trên GPU nhanh gấp 2–20 lần Keras trên CPU. (ii) Bản NumPy với 337 tham số, chỉ thấy 13% dữ liệu, vẫn vượt xa persistence (15,0 so với 19,1) và đạt 95,1% vùng Clarke A. Đây là phép kiểm chứng BPTT tự viết là đúng. (iii) Thứ tự LSTM 5 lớp < LSTM ≈ GRU < SimpleRNN < AR tuyến tính < persistence giữ nguyên ở cả ba cách, đúng như lý thuyết về cổng nhớ. Khác biệt giữa các mạng có cổng chỉ khoảng 0,1–0,4 mg/dL. (iv) LSTM 5 lớp tốt nhất nhưng chỉ hơn LSTM một lớp 0,05–0,16 mg/dL với số tham số gấp 8,5 lần. Với cửa sổ 12 bước, độ sâu theo *không gian* không thêm được nhiều, vì giới hạn nằm ở thông tin còn thiếu (bữa ăn, insulin), không ở sức biểu diễn.

4.8. LSTM xếp chồng 5 lớp dưới dạng hợp các hàm

Một LSTM xếp chồng là hợp của các hàm theo *hai chiều*: theo thời gian (cùng một hàm cập nhật trạng thái lặp lại 12 lần) và theo độ sâu (5 hàm như vậy chồng lên nhau). Gọi $\Phi^{(l)}$ là hàm cập nhật của lớp l:

$$(\mathbf{h}_t^{(l)}, \mathbf{c}_t^{(l)}) = \Phi^{(l)}(\mathbf{h}_t^{(l-1)},\ \mathbf{h}_{t-1}^{(l)},\ \mathbf{c}_{t-1}^{(l)}), \qquad \mathbf{h}_t^{(0)} = \mathbf{x}_t, \tag{4.5}$$

trong đó $\Phi^{(l)}$ chính là các phương trình cổng ở đầu chương (cổng quên $\mathbf{f}$, cổng vào $\mathbf{i}$, cổng ra $\mathbf{o}$, ứng viên $\tilde{\mathbf{c}}$). Đầu ra của mô hình là

$$\hat{y} = g_t + s \cdot (L \circ D \circ \pi_T \circ \Phi_{1:T}^{(5)} \circ \Phi_{1:T}^{(4)} \circ \Phi_{1:T}^{(3)} \circ \Phi_{1:T}^{(2)} \circ \Phi_{1:T}^{(1)})(\mathbf{x}_{1:T}), \tag{4.6}$$

với $\Phi_{1:T}^{(l)}$ là việc chạy $\Phi^{(l)}$ qua cả 12 bước và trả về *chuỗi* trạng thái ẩn, π_T chỉ lấy trạng thái ẩn ở bước cuối, D là dropout và L là lớp affine $64 \to 1$. Ý nghĩa của từng hàm:

- Lớp 1 đọc trực tiếp glucose và giờ trong ngày, học các mẫu cục bộ: đang tăng, đang giảm, dao động sau ăn.
- Các lớp 2–5 nhận *chuỗi* trạng thái của lớp dưới, nên học được mẫu của mẫu, ví dụ "tăng nhanh rồi chậm lại" báo hiệu đỉnh sau ăn.
- π_T nén cả chuỗi thành một vector; L đổi vector đó thành độ thay đổi glucose; phép cộng g_t là lối tắt (residual) giúp mô hình chỉ phải học phần chênh.

Trong Keras, $\Phi_{1:T}^{(1)}, \ldots, \Phi_{1:T}^{(4)}$ là `LSTM(64, return_sequences=True)` và $\pi_T \circ \Phi_{1:T}^{(5)}$ là

`LSTM(64)`. Trong PyTorch, cả năm lớp gộp trong `nn.LSTM(3, 64, num_layers=5, dropout=0,2)`, còn π_T là `out[:, -1]`. Gradient của W ở lớp 1 phải đi qua 5 lớp và 12 bước, nên LSTM 5 lớp cần cắt chuẩn gradient và dropout giữa các lớp mới học ổn định.

4.9. Đối chiếu trên chuỗi tài chính và thương mại

Cùng bốn kiến trúc (SimpleRNN, LSTM, GRU, BiLSTM; 64 đơn vị, cửa sổ 30 bước, chia 70/15/15 *theo thời gian*) được chạy bằng PyTorch và Keras trên ba chuỗi khác (mã nguồn kế thừa của nhóm): giá đóng cửa **AAPL** phiên kế tiếp, giá quỹ vàng **GLD** cùng 12 chỉ báo liên thị trường, và **doanh thu ngày** của một cửa hàng thương mại điện tử (gộp từ 5.000 đơn hàng). Mốc so sánh là dự báo "ngây thơ" (giữ nguyên giá trị hôm trước), chính là persistence ở bài toán glucose.

Bảng 4.8. Đối chiếu: LSTM và GRU so với dự báo ngây thơ trên ba chuỗi khác (tập kiểm tra theo thời gian)

Chuỗi	Mô hình	RMSE	MAPE (%)	Đúng hướng	RMSE
AAPL (USD)	Ngây thơ	2,259	1,38	—	
	LSTM – PyTorch / Keras	2,259 / 2,277	1,38 / 1,40	52% / 47%	0
	GRU – PyTorch / Keras	2,257 / 2,272	1,38 / 1,39	51% / 49%	−0
GLD (USD)	Ngây thơ	0,738	0,46	—	
	LSTM – PyTorch / Keras	0,738 / 0,737	0,46 / 0,46	53% / 50%	0
	GRU – PyTorch / Keras	0,751 / 0,747	0,47 / 0,47	48% / 47%	+1
Doanh thu (TRY)	Ngây thơ	10.596	117,6	—	
	LSTM – PyTorch / Keras	6.836 / 7.290	91,6 / 90,4	76% / 76%	−3
	GRU – PyTorch / Keras	7.415 / 7.013	85,1 / 86,7	79% / 79%	−3
Glucose CGM 30 phút (Bảng 4.7), LSTM – PyTorch		13,82	—	—	−27% s

Nhận xét. Kết quả đối chiếu cho bài học quan trọng nhất về mô hình chuỗi: *RNN chỉ hữu ích khi chuỗi có cấu trúc dự báo được*. Với giá AAPL và vàng, mọi kiến trúc ở cả hai framework đều trùng với dự báo ngây thơ đến phần nghìn, và đoán đúng hướng tăng/giảm chỉ quanh 50%, tức bằng tung đồng xu. Mô hình đã học đúng điều duy nhất có thể học được là "giá mai ≈ giá hôm nay", phù hợp với giả thuyết thị trường hiệu quả. Ngược lại, doanh thu ngày có chu kỳ tuần và xu hướng, nên LSTM giảm RMSE 31–35% so với ngây thơ. Glucose nằm giữa: chuỗi sinh lý có quán tính và nhịp ngày đêm, nên

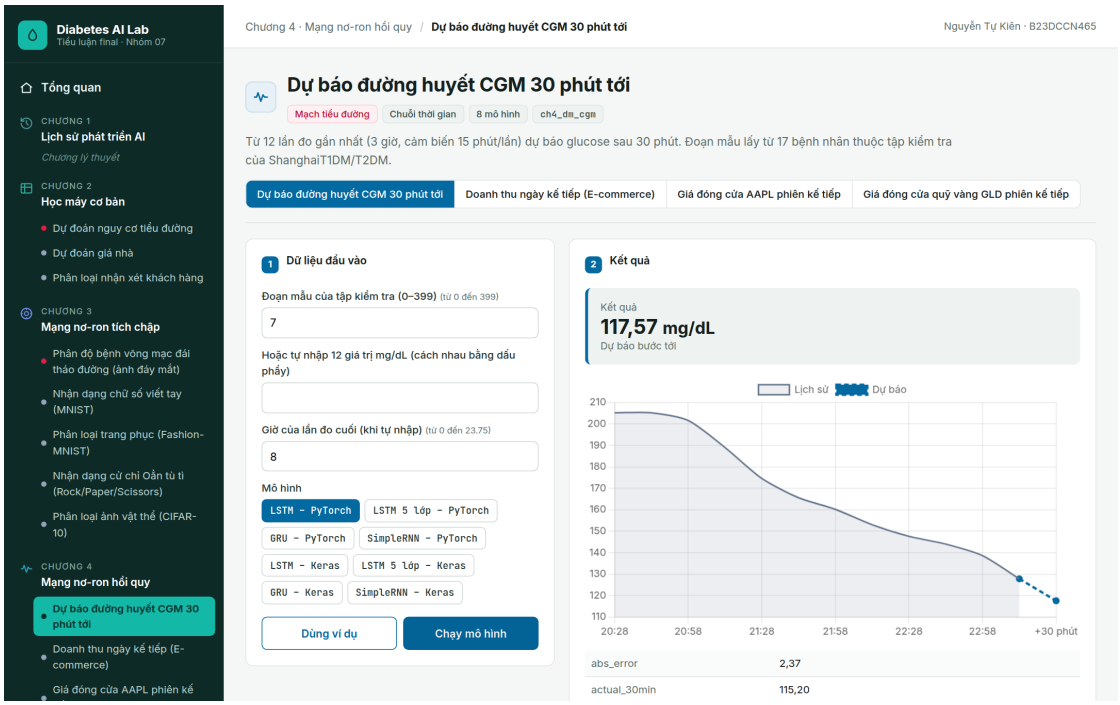
LSTM giảm 27% sai số và có ý nghĩa lâm sàng (95% vùng Clarke A). Trên cả ba miền, PyTorch và Keras cho kết quả gần nhau, khẳng định lại kết luận của Mục 4.7: khác biệt đến từ *dữ liệu*, không đến từ framework.

4.10. Thực nghiệm trên Notebook và Demo Localhost

Notebook `notebooks/ch4_rnn_cgm.ipynb` (các ô `RNN-00` đến `RNN-04`) minh họa chi tiết cách chia tập dữ liệu độc lập theo bệnh nhân và kiểm chứng toán học của thuật toán BPTT:

- **Ô `RNN-01` (Chia theo bệnh nhân 78/17/17 và mã hóa chu kỳ ngày đêm)**: Ô `RNN-01` gọi `load_cgm_windows(seq_len=12, horizon_steps=2)` để tách 112 bệnh nhân ShanghaiT1DM/T2DM thành 78 người huấn luyện (91.333 cửa sổ 3 giờ, glucose trung bình 140,4 $\pm$ 53,1 mg/dL), 17 người kiểm định (16.839 cửa sổ, 153,1 $\pm$ 59,3 mg/dL) và 17 người kiểm tra (18.056 cửa sổ, 145,7 $\pm$ 53,9 mg/dL). Việc không để bất kỳ bệnh nhân nào xuất hiện ở cả hai tập đảm bảo đánh giá đúng khả năng tổng quát hóa sang bệnh nhân mới; đồng thời hai đặc trưng $\sin(2\pi h/24)$ và $\cos(2\pi h/24)$ bảo toàn tính liên tục qua mốc nửa đêm (23h $\rightarrow$ 0h).
- **Ô `RNN-02` (Kiểm chứng đạo hàm ngược qua thời gian BPTT trên cửa sổ CGM thật)**: Ô `RNN-02` khởi tạo `RNNScratch(n_in=3, n_hid=8)`, chạy lan truyền xuôi và ngược qua 12 bước thời gian của cửa sổ CGM đầu tiên để lấy gradient giải tích $\partial L/\partial W_{hh}[0,0] = 0{,}006106$ (tổng cộng dồn qua cả 12 bước thời gian do dùng chung trọng số W_{hh}). Khi đối chiếu với đạo hàm sai phân hữu hạn trung tâm ($\varepsilon = 10^{-5}$), sai lệch tuyệt đối chỉ là $1{,}71 \times 10^{-12}$, xác nhận vòng lặp truyền ngược $\delta_{t-1} = W_{hh}^{\top}[(1 - h_t^2) \odot \delta_t]$ hoàn toàn chính xác.

Trên ứng dụng web cục bộ tại `http://localhost:8095` (Hình 4.5), tab **Ch.4 — RNN/LSTM · Đường huyết CGM (Shanghai)** nạp đồng thời cả 11 mô hình của ba cách cài đặt (`persistence`, `ridge_ar`, `numpy_rnn1`, cùng 4 kiến trúc `rnn1`/`lstm1`/`gru1`/`lstm5` của cả Keras và PyTorch). Khi chọn một bệnh nhân kiểm tra hoặc sửa trực tiếp chuỗi 12 điểm đo 3 giờ gần nhất, hệ thống trả về ngay dự báo sau 30 phút (mg/dL), sai số tuyệt đối và vùng lưới Clarke tương ứng của từng mô hình.



Hình 4.5. Giao diện thử nghiệm Chương 4 trên `http://localhost:8095`: dự báo đường huyết sau 30 phút bằng 11 mô hình (NumPy BPTT, Keras và PyTorch từ 1 đến 5 lớp) trên cửa sổ CGM 3 giờ

4.11. Triển khai

Mô hình được chọn theo MSE trên validation là LSTM 2 tầng (`data/models/ch4_glucose_rnn.pt`, 210 KB). Nó được đóng gói trong lớp `RNNTemporalEngine` (`src/models/rnn_temporal.py`), vừa tính các chỉ số TIR, TAR, TBR, CV, vừa dự báo 15–60 phút khi chuỗi có ít nhất 12 điểm. Endpoint `POST /api/lab/ch4/forecast` nhận chuỗi glucose và giờ của điểm cuối. `GET /api/lab/ch4/sample/{i}` trả về 24 giờ dữ liệu thật của bệnh nhân thuộc tập kiểm tra để thử. Trang `/lab` vẽ chuỗi đầu vào, dự báo, và giá trị thật nếu dùng dữ liệu mẫu.

Đoạn mã 4.3. Gọi endpoint Chương 4

```
curl -s https://agent.nits.io.vn/api/lab/ch4/forecast -H 'Content-Type: application/json' \
  -d '{"glucose":[110,115,124,138,152,166,175,181,184,182,176,168],"last_hour":9}'
```

4.12. Tiểu kết chương 4

Chương 4 trình bày RNN với trạng thái ẩn dùng chung tham số theo thời gian, thuật toán BPTT, nguyên nhân toán học của biến mất và bùng nổ gradient, và hai lời giải có cổng: LSTM (ô nhớ cập nhật cộng tính, ba cổng) và GRU (hai cổng). Trên 127.982 điểm CGM thật của 112 bệnh nhân, chia theo người để tránh rò rỉ: (1) mọi mô hình học được

đều vượt xa persistence, và LSTM 2 tầng giảm 28% RMSE ở tầm 30 phút với 95,6% điểm trong vùng Clarke A; (2) với cửa sổ 3 giờ, khác biệt giữa RNN, LSTM và GRU nhỏ, lợi thế của cổng chỉ rõ ở tầm xa; (3) giới hạn chính là thông tin, không phải kiến trúc: bữa ăn và insulin quyết định quỹ đạo sau 1 giờ. Ba chương vừa qua làm việc với dữ liệu *có cấu trúc đều*: bảng, lưới ảnh, chuỗi. Chương 5 chuyển sang tri thức dạng văn bản và đồ thị.

CHƯƠNG 5. ĐỒ THỊ TRI THỨC VÀ SINH VĂN BẢN TĂNG CƯỜNG TRUY XUẤT

Một chatbot y tế không được phép "bịa". Mô hình ngôn ngữ lớn lưu tri thức trong hàng tỉ tham số. Tri thức đó không trích dẫn được, không cập nhật được khi hướng dẫn thay đổi, và có thể bị ảo giác [66]: một câu sai về liều thuốc hay ngưỡng chẩn đoán, diễn đạt trôi chảy, nguy hiểm hơn một câu "tôi không biết". Chương này trình bày hai cách đưa tri thức *tường minh* trở lại hệ thống. *Đồ thị tri thức* (Knowledge Graph – KG) biểu diễn thực thể và quan hệ có cấu trúc. *Sinh văn bản tăng cường truy xuất* (Retrieval-Augmented Generation – RAG) tìm đoạn tài liệu liên quan rồi buộc LLM trả lời dựa trên chúng. Thực nghiệm đánh giá tám chiến lược truy xuất trên hai bộ dữ liệu và ba chế độ sinh câu trả lời.

5.1. Đồ thị tri thức

5.1.1. Định nghĩa và mô hình dữ liệu

Đồ thị tri thức là một đồ thị có hướng, có nhãn, trong đó nút là *thực thể* (bệnh, thuốc, xét nghiệm, yếu tố nguy cơ) và cạnh là *quan hệ* giữa chúng [67]. Đơn vị cơ bản là *bộ ba* (chủ thể, vị từ, đối tượng), ví dụ (*Đái tháo đường típ 2*, `DIAGNOSED_BY`, *HbA1c*). Thuật ngữ được Google phổ biến năm 2012 với khẩu hiệu "things, not strings" [68], nhưng ý tưởng có từ mạng ngữ nghĩa thập niên 1960 và các hệ chuyên gia ở Chương 1. Có hai mô hình dữ liệu chính:

- **RDF** (Resource Description Framework): mọi thứ là bộ ba, định danh bằng URI, truy vấn bằng SPARQL; mạnh về chuẩn hóa và liên kết dữ liệu mở (Wikidata, DBpedia, SNOMED CT, UMLS).
- **Đồ thị thuộc tính** (property graph): nút và cạnh đều mang cặp khóa–giá trị, truy vấn bằng Cypher (Neo4j, Apache AGE [69]); trực quan và thuận tiện cho ứng dụng.

Ontology định nghĩa lược đồ: các lớp thực thể, các loại quan hệ được phép, và ràng buộc giữa chúng. KG của hệ thống dùng một ontology nhỏ gồm 6 lớp nút (`Disease`, `RiskFactor`, `Test`, `Drug`, `Complication`, `Intervention`) và 7 loại quan hệ (`HAS_RISK_FACTOR`, `DIAGNOSED_BY`, `CAUSES`, `SCREENED_BY`, `TREATED_BY`, `CONTRAINDICATED_IN`, `PREVENTED_BY`). Ví dụ một truy vấn Cypher tìm các thuốc chống chỉ định với một tình

trạng của bệnh nhân:

Đoạn mã 5.1. Truy vấn Cypher trên Apache AGE: láng giềng hai bước từ các yếu tố nguy cơ của bệnh nhân

```
SELECT * FROM cypher('diabetes_kg', $$
  MATCH p = (s)-[*1..2]-(t)
  WHERE s.name IN ['Thừa cân/béo phì', 'Tăng huyết áp', 'Suy thận nặng (eGFR < 30)']
  UNWIND relationships(p) AS r
  RETURN startNode(r).name, type(r), endNode(r).name, r.chunk_ids
  LIMIT 60
$$) AS (head agtype, relation agtype, tail agtype, chunk_ids agtype);
```

5.1.2. Xây dựng đồ thị tri thức

Quy trình xây dựng KG từ văn bản gồm bốn bước:

1. *Nhận dạng thực thể* (NER): tìm các cụm từ chỉ thực thể ("metformin", "eGFR").
2. *Liên kết thực thể*: ánh xạ các cách gọi khác nhau về cùng một nút ("tiểu đường", "ĐTĐ típ 2", "đái tháo đường type 2").
3. *Trích xuất quan hệ*: xác định vị từ giữa hai thực thể từ ngữ cảnh câu.
4. *Hợp nhất và kiểm định*: gộp bộ ba trùng lặp, gắn nguồn gốc và độ tin cậy.

Trước đây các bước này cần mô hình chuyên biệt cho từng bước. Ngày nay LLM có thể trích bộ ba trực tiếp theo một lược đồ JSON. Hệ thống dùng cả hai nguồn. 40 bộ ba *tuyển chọn thủ công* từ ADA 2024 và QĐ 5481/QĐ-BYT tạo thành lõi đáng tin cậy. Tùy chọn `--llm-triples` cho phép Gemini trích thêm bộ ba từ tài liệu mới. Mỗi cạnh lưu `chunk_ids`, tức danh sách đoạn văn làm bằng chứng. Đây là cầu nối giữa đồ thị và kho vector, nền tảng của GraphRAG.

5.1.3. Suy luận và nhúng đồ thị tri thức

KG cho phép *suy luận*. Suy luận tường minh dùng luật: nếu bệnh nhân có *eGFR* $<$ *30* và *Metformin* `CONTRAINDICATED_IN` *Suy thận nặng* thì cảnh báo. Suy luận ngầm dùng *nhúng KG*, ví dụ TransE [70] biểu diễn mỗi thực thể và quan hệ bằng vector sao cho $\mathbf{e}_h + \mathbf{r} \approx \mathbf{e}_t$, nhờ đó dự đoán được cạnh còn thiếu bằng cách tìm t gần $\mathbf{e}_h + \mathbf{r}$ nhất. Mạng nơ-ron đồ thị ở Chương 6 là bước tổng quát hơn của ý tưởng này.

5.2. Sinh văn bản tăng cường truy xuất

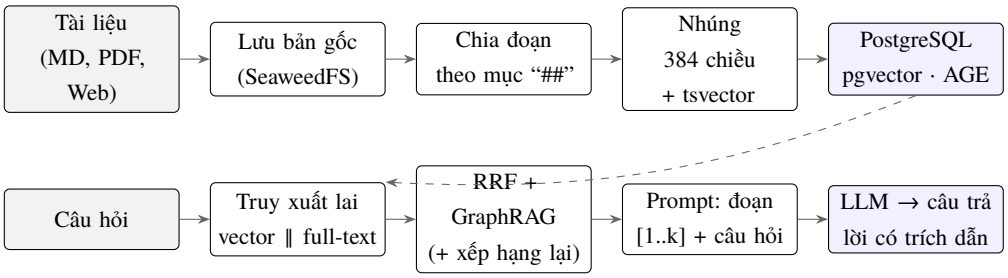
5.2.1. Nguyên lý

RAG [71] tách hai loại tri thức: tri thức *tham số* (ngôn ngữ, lập luận, nằm trong LLM) và tri thức *phi tham số* (sự kiện, nằm trong kho tài liệu bên ngoài). Với câu hỏi q, bộ truy xuất tìm k đoạn liên quan nhất $\{d_1, \dots, d_k\}$, rồi bộ sinh tạo câu trả lời có điều kiện trên cả hai:

$$p(y \mid q) \approx p_{\text{LLM}}(y \mid q, d_1, \dots, d_k), \qquad \{d_i\} = \text{top-}k_{d \in \mathscr{D}}\, s(q, d). \tag{5.1}$$

Lợi ích gồm: cập nhật tri thức chỉ cần thêm tài liệu, không cần huấn luyện lại; trích dẫn được nguồn; giảm ảo giác khi prompt buộc mô hình chỉ dùng ngữ cảnh. So với tinh chỉnh (fine-tuning), RAG phù hợp hơn với tri thức thay đổi và cần kiểm chứng như hướng dẫn lâm sàng. Tinh chỉnh phù hợp hơn khi cần đổi văn phong hoặc kỹ năng của mô hình [72].

5.2.2. Quy trình



Hình 5.1. Hai pha của RAG: nạp tri thức (trên) và trả lời câu hỏi (dưới)

Chia đoạn. Đoạn quá dài làm loãng tín hiệu liên quan và tốn ngữ cảnh. Đoạn quá ngắn mất ngữ cảnh. Các chiến lược thường gặp: kích thước cố định có chồng lấp, theo cấu trúc tài liệu (đề mục, đoạn văn), hoặc theo ngữ nghĩa. KB tiếng Việt được chia theo mục "##" vì mỗi mục của hướng dẫn lâm sàng là một đơn vị ý trọn vẹn. Văn bản không có cấu trúc được chia theo cửa sổ khoảng 1.200 ký tự, chồng lấp 200.

Truy xuất thưa (BM25). BM25 [73] chấm điểm theo từ khóa trùng khớp:

$$\begin{aligned} \text{BM25}(q, d) &= \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t,d)\,(k_1+1)}{f(t,d) + k_1\big(1 - b + b\,\frac{|d|}{\text{avgdl}}\big)}, \\ \text{IDF}(t) &= \ln\Big(1 + \frac{N - n_t + 0{,}5}{n_t + 0{,}5}\Big), \end{aligned} \tag{5.2}$$

với $f(t,d)$ là tần suất của từ t trong đoạn d; $k_1 = 1{,}5$ điều khiển độ bão hòa tần suất;

$b = 0{,}75$ chuẩn hóa theo độ dài. BM25 bắt chính xác thuật ngữ hiếm ("SGLT2", "monofilament", "eGFR") nhưng không hiểu từ đồng nghĩa ("tiểu đường" và "đái tháo đường").

Truy xuất dày. Bộ mã hóa câu (Sentence-BERT [74]) ánh xạ câu hỏi và đoạn văn vào cùng không gian vector, sau đó chấm điểm bằng độ tương tự cosin $s(q,d) = \frac{\mathbf{e}_q^\top \mathbf{e}_d}{\|\mathbf{e}_q\|\|\mathbf{e}_d\|}$. Mô hình được huấn luyện tương phản: kéo cặp (câu hỏi, đoạn đúng) lại gần nhau và đẩy cặp sai ra xa [75]. Mô hình đa ngữ (huấn luyện bằng chưng cất đa ngữ [76]) đặt câu tiếng Việt và tiếng Anh cùng nghĩa ở gần nhau, cho phép truy xuất chéo ngôn ngữ. Với hàng triệu vector, tìm kiếm chính xác $O(N)$ được thay bằng chỉ mục xấp xỉ HNSW [77], một đồ thị nhiều tầng cho phép tìm láng giềng gần nhất trong khoảng $O(\log N)$. pgvector [78] cung cấp HNSW ngay trong PostgreSQL.

Truy xuất lai và RRF. Hai bộ truy xuất bổ sung cho nhau, nhưng điểm của chúng không cùng thang đo. Reciprocal Rank Fusion [79] chỉ dùng *thứ hạng*:

$$\mathrm{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \mathrm{rank}_r(d)}, \qquad k = 60. \tag{5.3}$$

Hằng số k làm phẳng chênh lệch giữa các hạng đầu, nên một đoạn đứng đầu ở chỉ một danh sách không áp đảo được đoạn đứng khá cao ở cả hai danh sách.

Xếp hạng lại. Bộ mã hóa chéo (cross-encoder) [80] đưa *cặp* (câu hỏi, đoạn) qua cùng một Transformer để chấm điểm liên quan. Mô hình này chính xác hơn nhiều so với so sánh hai vector độc lập, vì attention được tính chéo giữa hai văn bản, nhưng chậm hơn tương ứng. Vì vậy cross-encoder chỉ dùng để xếp lại 10–20 ứng viên đầu.

5.2.3. GraphRAG

RAG thuần vector coi mỗi đoạn là độc lập, nên khó trả lời câu hỏi cần *nối* nhiều mẩu tri thức. Ví dụ: "bệnh thận mạn có albumin niệu nên dùng thuốc gì?" cần nối *Bệnh thận ĐTĐ* → `TREATED_BY` → *Ức chế SGLT2*. GraphRAG [81] khai thác cấu trúc đồ thị để mở rộng ngữ cảnh. Phiên bản trong hệ thống gồm ba bước: (1) liên kết thực thể trong câu hỏi hoặc hồ sơ bệnh nhân với nút KG; (2) duyệt láng giềng 1–2 bước bằng Cypher; (3) lấy các đoạn văn gắn với cạnh tìm được (`chunk_ids`), hợp nhất với kết quả truy xuất lai bằng RRF, và đưa các bộ ba vào prompt như "sự kiện có cấu trúc".

5.2.4. So sánh RAG và tinh chỉnh

Bảng 5.1. So sánh RAG và tinh chỉnh (fine-tuning) trong bối cảnh chatbot y tế

Tiêu chí	RAG	Tinh chỉnh
Cập nhật tri thức	Thêm hoặc sửa tài liệu, có hiệu lực ngay	Phải huấn luyện lại
Trích dẫn nguồn	Có, đến từng đoạn	Không
Kiểm soát ảo giác	Tốt khi prompt buộc chỉ dùng ngữ cảnh	Hạn chế, tri thức nằm trong tham số
Chi phí ban đầu	Thấp: nhúng và chỉ mục	Cao: dữ liệu gán nhãn, GPU
Chi phí mỗi truy vấn	Cao hơn: truy xuất và ngữ cảnh dài	Thấp hơn
Phù hợp với	Sự kiện, ngưỡng, hướng dẫn thay đổi theo thời gian	Văn phong, định dạng, kỹ năng chuyên biệt

Hai cách tiếp cận không loại trừ nhau. Một hệ thống có thể tinh chỉnh mô hình để quen thuật ngữ và văn phong y khoa tiếng Việt, đồng thời dùng RAG để cung cấp sự kiện có nguồn. Với hướng dẫn lâm sàng được cập nhật hằng năm như ADA Standards of Care, RAG là lựa chọn bắt buộc cho phần sự kiện.

5.2.5. Thách thức vận hành

Đưa RAG vào sản phẩm gặp một số thách thức ngoài độ chính xác truy xuất. *Chất lượng nguồn*: RAG chỉ tốt bằng tài liệu được nạp; tài liệu lỗi thời hoặc mâu thuẫn sẽ được trích dẫn một cách tự tin. *Bảo trì chỉ mục*: khi đổi mô hình nhúng phải nhúng lại toàn bộ kho; khi sửa tài liệu phải cập nhật cả đoạn văn lẫn các cạnh KG trỏ tới đoạn đó (hệ thống lưu `chunk_ids` trên cạnh và bỏ qua tài liệu không đổi dựa trên mã băm nội dung). *Độ trễ và chi phí*: mỗi bước thêm vào (xếp hạng lại, mở rộng đồ thị, ngữ cảnh dài hơn) đều tăng độ trễ. Thực nghiệm cho thấy cross-encoder chính xác nhất nhưng chậm nhất. *Đánh giá liên tục*: cần một bộ câu hỏi có đáp án để chạy lại sau mỗi thay đổi, như 60 câu hỏi của chương này. *An toàn*: tài liệu nạp từ web có thể chứa chỉ dẫn độc hại nhắm vào LLM (prompt injection). Vì vậy ngữ cảnh truy xuất được đặt trong vai trò "dữ liệu", và câu trả lời luôn được ràng buộc bởi các quy tắc an toàn của hệ thống.

5.2.6. *Đánh giá hệ thống RAG*

Đánh giá RAG gồm hai tầng. *Tầng truy xuất* dùng các thước đo xếp hạng với tập đoạn đúng G của mỗi câu hỏi: Hit@k (có ít nhất một đoạn đúng trong top-k), Recall@k = $|G \cap \text{top-}k|/|G|$, MRR = $\frac{1}{|Q|}\sum_q 1/\text{rank}_q$ (hạng của đoạn đúng đầu tiên), và nDCG@10 (phần thưởng giảm theo $1/\log_2(\text{hạng} + 1)$). *Tầng sinh* đánh giá câu trả lời có chứa dữ kiện đúng không, có trung thành với ngữ cảnh không (faithfulness), và có trích dẫn đúng nguồn không [82].

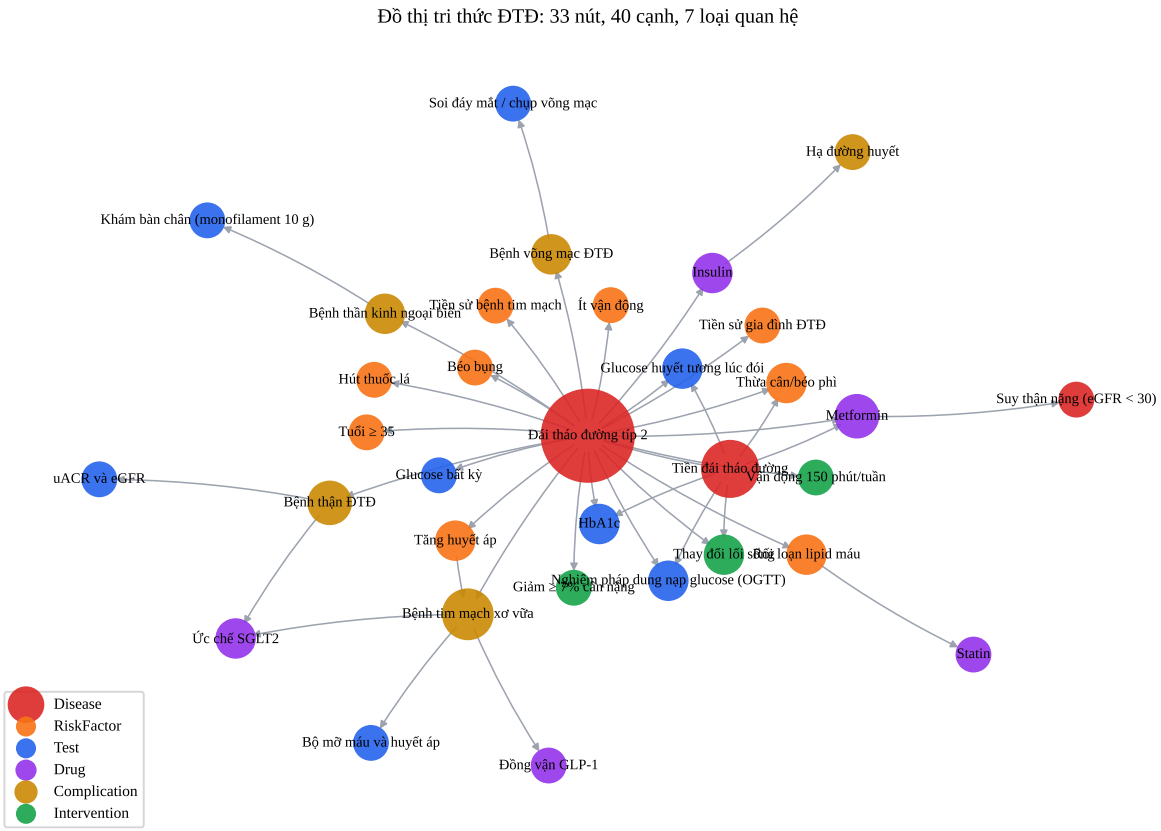
5.3. Dữ liệu

5.3.1. *Knowledge Base tiếng Việt và đồ thị tri thức*

KB của hệ thống gồm 10 tài liệu Markdown (khoảng 18 KB văn bản) tóm tắt từ ADA Standards of Care 2024 [2] và Quyết định 5481/QĐ-BYT [3]. Nội dung bao gồm tiêu chuẩn chẩn đoán, tiền ĐTĐ, sàng lọc người châu Á, mục tiêu điều trị, thuốc, biến chứng, CGM và hạ đường huyết, lối sống, triệu chứng cấp cứu. Script `scripts/ingest_kb.py` lưu bản gốc lên SeaweedFS, chia đoạn theo mục "##" thành **36 đoạn** (trung bình 279 ký tự), nhúng bằng MiniLM đa ngữ 384 chiều vào `kb_chunks` (pgvector, chỉ mục HNSW) kèm cột `tsvector` cho full-text, và nạp **40 bộ ba** tuyển chọn vào đồ thị `diabetes_kg` trên Apache AGE.

Bảng 5.2. Ví dụ đoạn văn trong KB và bộ ba trong đồ thị tri thức

Đoạn #1	*Tiêu chuẩn chẩn đoán đái tháo đường — Bốn tiêu chí chẩn đoán*: "Chẩn đoán đái tháo đường khi có một trong các tiêu chí sau: Glucose huyết tương lúc đói (FPG) ≥ 126 mg/dL (7,0 mmol/L)…"
Bộ ba	(*Đái tháo đường típ 2*, `HAS_RISK_FACTOR`, *Thừa cân/béo phì*) — bằng chứng: "BMI ≥ 23 kg/m² (người châu Á) kèm yếu tố nguy cơ cần được sàng lọc" — nguồn: `sang-loc-nguoi-viet`, mục "Các yếu tố nguy cơ"
Bộ ba	(*Metformin*, `CONTRAINDICATED_IN`, *Suy thận nặng (eGFR < 30)*) — nguồn: `dieu-tri-thuoc`, mục "Metformin"



Hình 5.2. Đồ thị tri thức ĐTĐ: 33 nút thuộc 6 loại, 40 cạnh thuộc 7 loại quan hệ (kích thước nút tỉ lệ với bậc)

Đồ thị có 33 nút: 9 yếu tố nguy cơ, 8 xét nghiệm, 5 biến chứng, 5 thuốc, 3 bệnh, 3 can thiệp. Quan hệ nhiều nhất là `HAS_RISK_FACTOR` (11), `TREATED_BY` (9) và `DIAGNOSED_BY` (7). Đồ thị có dạng "sao": nút *Đái tháo đường típ 2* có bậc 23, tiếp theo là *Tiền ĐTĐ* (7) và *Bệnh tim mạch xơ vữa* (5). Cấu trúc này hợp với GraphRAG vì hầu hết mọi thực thể cách nhau tối đa 2 bước.

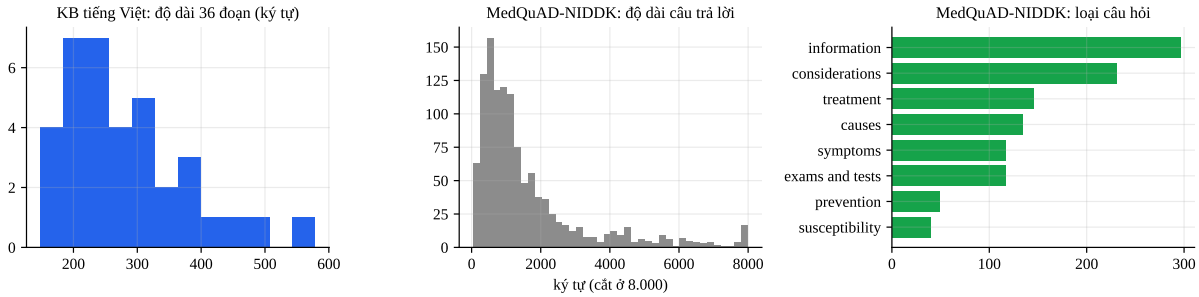
Bộ câu hỏi đánh giá. Do chưa có bộ hỏi–đáp tiếng Việt công khai cho các tài liệu này, tôi tự xây dựng **60 câu hỏi** (tệp `lab/kb_eval_vi.json`). Mỗi câu gắn với 1–2 *đoạn vàng* và các *dữ kiện vàng* dùng để chấm câu trả lời (ví dụ "126" và "7,0"). Câu hỏi được chia bốn loại để đo khía cạnh khác nhau của truy xuất: *từ khóa* (20 câu, dùng đúng thuật ngữ trong tài liệu), *diễn đạt dân dã* (19 câu, ví dụ "tụt đường huyết", "thuốc mỡ máu", "tiểu đường"), *số liệu* (17 câu, hỏi ngưỡng cụ thể), và *đa bước* (4 câu, cần nối hai mục, ví dụ "eGFR 25 thì còn dùng metformin không, thay bằng thuốc gì?").

5.3.2. MedQuAD — phần NIDDK

MedQuAD [8] là bộ 47.457 cặp hỏi–đáp y khoa thu thập từ 12 website của Viện Y tế Quốc gia Hoa Kỳ (NIH). Báo cáo dùng phần **NIDDK** (Viện Đái tháo đường, Tiêu hóa

và Thận) vì đúng chủ đề và có đầy đủ câu trả lời. Một số phần khác của MedQuAD đã bị gỡ câu trả lời vì lý do bản quyền.

- **Nguồn**: `https://github.com/abachaa/MedQuAD`, thư mục `5_NIDDK_QA` (157 tệp XML), giấy phép CC BY 4.0.
- **Quy mô**: 1.192 cặp hỏi–đáp về 148 chủ đề; câu trả lời dài trung bình 1.638 ký tự (trung vị 1.048); 166 câu về đái tháo đường. Chia câu trả lời thành đoạn 800 ký tự (chồng lấp 15%) được **3.381 đoạn**.
- **Loại câu hỏi**: information (296), considerations (231), treatment (145), causes (134), symptoms (117), exams and tests (117)… Câu hỏi được sinh từ mẫu, ví dụ "What is (are) Diabetic Kidney Disease?", "How to prevent …?".



Hình 5.3. Độ dài đoạn của KB tiếng Việt, độ dài câu trả lời và phân bố loại câu hỏi của MedQuAD-NIDDK

Nhận xét. (i) Hai bộ dữ liệu bổ sung cho nhau. KB tiếng Việt nhỏ (36 đoạn) nhưng chính là dữ liệu chạy trong hệ thống. MedQuAD lớn gấp gần 100 lần, đủ để các khác biệt giữa bộ truy xuất có ý nghĩa thống kê. (ii) Với MedQuAD, mỗi chủ đề có nhiều câu hỏi gần giống nhau ("What is X?", "What causes X?", "How to treat X?"), và các câu trả lời của cùng chủ đề dùng chung từ vựng. Bài toán vì thế khó ở chỗ phải chọn đúng *khía cạnh* của chủ đề chứ không chỉ đúng chủ đề. (iii) Để đo truy xuất chéo ngôn ngữ, 166 câu hỏi về ĐTĐ được Gemini dịch sang tiếng Việt tự nhiên (ví dụ "What is (are) 4 Steps to Manage Your Diabetes for Life?" → "4 bước để kiểm soát bệnh tiểu đường suốt đời là gì?").

5.4. Thực nghiệm

5.4.1. Thiết lập

Tám bộ truy xuất được so sánh trên cùng tập đoạn:

1. *BM25* theo (5.2), tự cài đặt, tách từ sau khi chữ thường hóa và bỏ dấu tiếng Việt;

2. –4. *Dày* với ba mô hình nhúng đa ngữ: *MiniLM-L12* (384 chiều, 118 triệu tham số, mô hình đang chạy trong hệ thống), *MPNet-base* (768 chiều, 278 triệu) và *multilingual-E5-large* [83] (1.024 chiều, 560 triệu; câu hỏi có tiền tố `query:`, đoạn văn có tiền tố `passage:`);
5. *Lai*: RRF (5.3) của BM25 và MiniLM;
6. *Lai + GraphRAG*: liên kết thực thể trong câu hỏi với nút KG (theo tên nút và bảng từ đồng nghĩa, sau khi bỏ dấu), lấy các đoạn bằng chứng của cạnh 1 bước, rồi hợp nhất với kết quả lai bằng RRF;
7. *Lai + xếp hạng lại*: 15 ứng viên đầu của bộ lai được chấm lại bằng cross-encoder đa ngữ `bge-reranker-v2-m3` (bản lượng tử int8) [84];
8. *Lai trên PostgreSQL*: đúng câu lệnh SQL của production (pgvector cosin ‖ `tsvector` + `ts_rank_cd`, gộp RRF).

Sinh câu trả lời. Với 60 câu hỏi tiếng Việt, Gemini (`gemini-flash-latest`) trả lời theo ba chế độ: *chỉ LLM* (không có tài liệu), *RAG* (3 đoạn đầu của bộ lai) và *GraphRAG* (3 đoạn đầu của bộ GraphRAG cùng tối đa 12 bộ ba liên quan). Prompt của hai chế độ RAG yêu cầu chỉ dùng tài liệu, trích dẫn dạng [1]–[3], và nói "không đủ dữ liệu" khi tài liệu không có thông tin. Bốn thước đo được dùng: *độ phủ dữ kiện* (tỉ lệ dữ kiện vàng xuất hiện trong câu trả lời, chấp nhận các cách viết tương đương), *trích dẫn đúng* (có trích ít nhất một đoạn vàng), *số liệu không có căn cứ* (tỉ lệ con số trong câu trả lời không xuất hiện trong ngữ cảnh, một chỉ báo của ảo giác), và *tỉ lệ từ chối trả lời*.

Đoạn mã 5.2. Hợp nhất thứ hạng RRF và GraphRAG (`lab/ch5_rag.py`, rút gọn)

```
def rrf(rankings, k=60):
    """Reciprocal Rank Fusion: score(d) = sum_r 1 / (k + rank_r(d))."""
    s = defaultdict(float)
    for ranking in rankings:
        for r, d in enumerate(ranking):
            s[d] += 1 / (k + r + 1)
    return sorted(s, key=lambda d: -s[d])

def graph_candidates(q, triples, chunk_of):
    """Nút KG được nhắc trong câu hỏi → các cạnh 1 bước → đoạn văn làm bằng chứng cho cạnh."""
    ents = link_entities(q, nodes)                  # khớp tên nút + từ đồng nghĩa, đã bỏ dấu
    cands = [chunk_of[(t["source_doc"], t["section"])] for t in triples
             if t["head"] in ents or t["tail"] in ents]
    return ents, list(dict.fromkeys(cands))

hybrid   = rrf([bm25_ranking, dense_ranking])
graphrag = rrf([hybrid[:10], graph_candidates(q, triples, chunk_of)[1]])
```

5.4.2. Kết quả truy xuất trên KB tiếng Việt

Bảng 5.3. Truy xuất trên KB tiếng Việt (60 câu hỏi, 36 đoạn)

Bộ truy xuất	Hit@1	Hit@3	Hit@5	Recall@5	MRR@10	nDCG@10
BM25	0,800	0,933	0,933	0,917	0,869	0,878
Dày — MiniLM (hệ thống)	0,567	0,750	0,833	0,783	0,675	0,698
Dày — MPNet	0,750	0,950	0,950	0,917	0,846	0,859
Dày — E5-large	0,933	0,983	0,983	0,967	0,956	0,952
Lai (BM25 + MiniLM)	0,800	0,900	0,950	0,933	0,858	0,867
Lai + GraphRAG	0,750	0,950	**1,000**	0,958	0,848	0,862
Lai + xếp hạng lại	**0,967**	**1,000**	**1,000**	**0,983**	**0,983**	**0,974**
Lai trên PostgreSQL (production)	0,667	0,850	0,917	0,883	0,774	0,802

Bảng 5.4. Hit@3 theo loại câu hỏi trên KB tiếng Việt

Bộ truy xuất	Từ khóa (20)	Dân dã (19)	Số liệu (17)	Đa bước (4)
BM25	**1,00**	0,89	0,94	0,75
Dày — MiniLM	0,65	0,89	0,71	0,75
Dày — E5-large	0,95	**1,00**	**1,00**	**1,00**
Lai (BM25 + MiniLM)	0,90	0,95	0,88	0,75
Lai + GraphRAG	0,90	**1,00**	0,94	**1,00**
Lai + xếp hạng lại	**1,00**	**1,00**	**1,00**	**1,00**

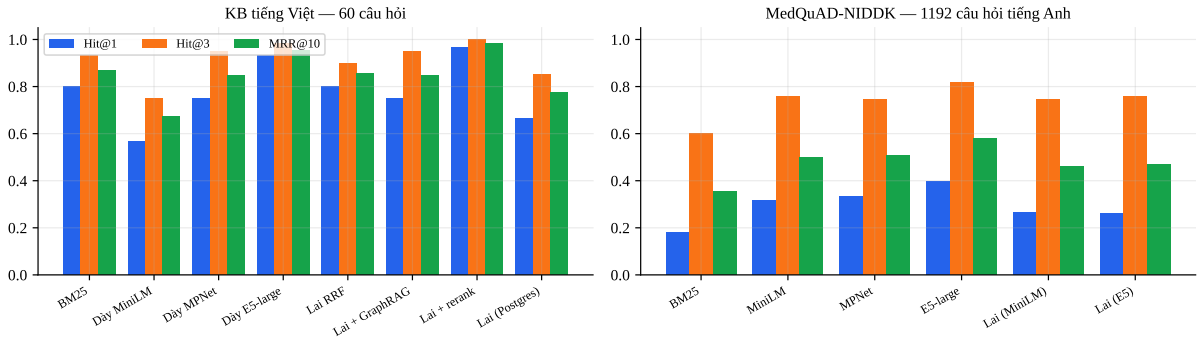
Nhận xét. *BM25 rất mạnh trên KB tiếng Việt* (Hit@1 0,80), vượt xa MiniLM (0,57). Câu hỏi y khoa chứa nhiều thuật ngữ hiếm và con số ("HbA1c", "SGLT2", "monofilament", "eGFR"). BM25 khớp chính xác các từ này, còn MiniLM, một mô hình nhỏ huấn luyện cho tương đồng câu tổng quát, làm mờ chúng: MiniLM chỉ đạt Hit@3 0,65 với câu hỏi từ khóa và 0,71 với câu hỏi số liệu. Ngược lại, với câu hỏi diễn đạt dân dã, MiniLM ngang BM25 (0,89). Đây đúng là tính bổ sung mà truy xuất lai khai thác: bộ lai đạt 0,95 Hit@3 trên nhóm này.

Mô hình nhúng lớn thay đổi cục diện: E5-large đạt Hit@1 0,933 chỉ với truy xuất dày, cho thấy chất lượng mô hình nhúng là yếu tố đơn lẻ quan trọng nhất. *Xếp hạng lại* bằng cross-encoder đưa bộ lai lên mức gần tuyệt đối: Hit@1 0,967, Hit@3 1,000. Đổi lại,

cross-encoder 568 triệu tham số trên CPU mất 3–6 giây cho mỗi câu hỏi (15 cặp), quá chậm cho chatbot. Nó phù hợp hơn với GPU hoặc chạy theo lô.

GraphRAG không cải thiện Hit@1 (0,75 so với 0,80), nhưng là cấu hình duy nhất ngoài reranker đạt Hit@5 = 1,000, và đạt Hit@3 = 1,00 trên cả câu đa bước lẫn câu dân dã. Ví dụ, với câu "Bệnh thận mạn có albumin niệu nên ưu tiên nhóm thuốc nào?", bộ lai trả về [13, 15, 23] và bỏ sót đoạn 25 ("Thuốc bảo vệ tim mạch và thận"). GraphRAG nhận ra thực thể *Bệnh thận ĐTĐ*, đi theo cạnh `TREATED_BY` → *Ức chế SGLT2*, lấy được đoạn bằng chứng và trả về [13, 25, 15], đủ cả hai đoạn vàng. Đồ thị giúp *độ phủ* (recall) hơn là thứ hạng đầu. Ngoài ra, liên kết thực thể chỉ thành công với 67% câu hỏi, nên 1/3 số câu không được đồ thị hỗ trợ.

Bản PostgreSQL thấp hơn bản lai cài bằng Python (Hit@1 0,667 so với 0,800) dù cùng ý tưởng. Nguyên nhân nằm ở nhánh full-text: `ts_rank_cd` xếp hạng theo mật độ phủ của từ, không có IDF như BM25, nên từ phổ biến ("đái tháo đường", "người") có trọng số ngang thuật ngữ hiếm; cột `tsvector` cũng chỉ chứa đề mục và nội dung, không có tiêu đề tài liệu. Kết quả này chỉ ra một cải tiến rõ ràng cho production: thay `ts_rank_cd` bằng BM25 (ví dụ qua phần mở rộng `pg_search`) hoặc nâng mô hình nhúng lên E5.



Hình 5.4. So sánh các bộ truy xuất trên KB tiếng Việt (trái) và MedQuAD-NIDDK (phải)

5.4.3. Kết quả truy xuất trên MedQuAD

Bảng 5.5. Truy xuất câu trả lời trên MedQuAD-NIDDK (1.192 câu hỏi, 3.381 đoạn 800 ký tự)

	Toàn bộ (1.192)					ĐTĐ (166)	
Bộ truy xuất	**Hit@1**	**Hit@3**	**Hit@5**	**MRR@10**	**nDCG@10**	**Hit@1**	**MRR@10**
BM25	0,182	0,420	0,600	0,355	0,468	0,139	0,302
Dày — MiniLM	0,315	0,613	0,758	0,499	0,597	0,259	0,424
Dày — MPNet	0,333	0,610	0,747	0,506	0,602	0,229	0,398
Dày — E5-large	**0,399**	**0,709**	**0,817**	**0,580**	**0,670**	**0,271**	**0,439**
Lai (BM25 + MiniLM)	0,263	0,560	0,746	0,460	0,571	0,199	0,390
Lai (BM25 + E5)	0,262	0,602	0,760	0,471	0,581	0,217	0,405

Trên MedQuAD, thứ tự đảo ngược: *BM25 yếu nhất* (Hit@1 0,18) và *truy xuất lai kém hơn truy xuất dày đơn lẻ*. Lý do nằm ở cấu trúc dữ liệu: các câu hỏi được sinh từ mẫu và chứa nguyên tên chủ đề ("What causes Diabetic Kidney Disease?"), nên BM25 tìm đúng *chủ đề* nhưng không phân biệt được *khía cạnh*. Các câu trả lời "causes", "treatment", "symptoms" của cùng chủ đề đều chứa tên chủ đề với tần suất tương tự. Mô hình nhúng hiểu được ý định "causes" so với "treatment". Khi một thành phần của RRF yếu hơn hẳn thành phần kia, việc gộp kéo kết quả xuống. RRF không phải lúc nào cũng có lợi; nó hiệu quả khi hai bộ truy xuất có chất lượng tương đương và lỗi bổ sung cho nhau, như trên KB tiếng Việt. E5-large vẫn tốt nhất ở mọi thước đo, khẳng định kết luận ở phần trước. Nhóm câu hỏi ĐTĐ khó hơn trung bình (Hit@1 0,27 so với 0,40), vì nhiều tài liệu ĐTĐ của NIDDK có nội dung chồng lấn (bốn tài liệu "Prevent diabetes problems: Keep your … healthy").

Bảng 5.6. Ảnh hưởng của kích thước đoạn (bộ lai BM25 + MiniLM trên MedQuAD)

Kích thước đoạn	**Số đoạn**	**Hit@1**	**Hit@5**	**MRR@10**
400 ký tự	6.291	0,241	0,732	0,446
800 ký tự	3.381	0,263	0,746	0,460
1.600 ký tự	1.991	**0,269**	0,742	**0,466**
Cả câu trả lời	1.192	0,261	**0,753**	0,465

Kích thước đoạn ảnh hưởng ít (MRR 0,446–0,466). Đoạn 400 ký tự kém nhất: nó

cắt câu trả lời thành mảnh mất ngữ cảnh, nhiều mảnh không chứa tên chủ đề. Từ 800 ký tự trở lên, kết quả gần như bão hòa. Với chatbot, đoạn ngắn có lợi thế là tốn ít ngữ cảnh LLM hơn. Chia theo mục "##" như KB tiếng Việt (trung bình 279 ký tự, mỗi mục một ý trọn vẹn) là cách chia theo cấu trúc, tránh được nhược điểm cắt ngang ý của đoạn cố định.

Bảng 5.7. Truy xuất chéo ngôn ngữ: 166 câu hỏi ĐTĐ dịch sang tiếng Việt, tìm trên kho tiếng Anh

Bộ truy xuất	Hit@1	Hit@5	MRR@10	MRR tiếng Anh gốc	Giữ lại
BM25	0,006	0,018	0,015	0,302	5%
Dày — MiniLM	0,145	0,476	0,275	0,424	65%
Dày — MPNet	0,157	0,476	0,293	0,398	74%
Dày — E5-large	**0,187**	**0,572**	**0,352**	0,439	**80%**

Truy xuất chéo ngôn ngữ là kịch bản thực tế của chatbot: người dùng hỏi tiếng Việt, còn phần lớn tài liệu y khoa chất lượng cao là tiếng Anh. BM25 gần như vô dụng (MRR 0,015), vì không có từ chung giữa hai ngôn ngữ. Mô hình nhúng đa ngữ giữ được 65–80% chất lượng so với khi hỏi bằng tiếng Anh, và E5-large giữ tốt nhất. Kết quả này ủng hộ việc mở rộng KB bằng tài liệu gốc tiếng Anh của ADA mà không cần dịch, với điều kiện dùng mô hình nhúng đa ngữ mạnh.

5.4.4. Kết quả sinh câu trả lời

Bảng 5.8. Chất lượng câu trả lời của Gemini theo ba chế độ (60 câu hỏi tiếng Việt)

Chế độ	n	Phủ dữ kiện	Trích dẫn đúng	Số không căn cứ	Từ chối	Độ dài
Chỉ LLM	52	0,904	–	–	0%	340 ký tự
RAG (lai, 3 đoạn)	51	**0,951**	0,902	0%	7,8%	182 ký tự
GraphRAG (3 đoạn + bộ ba)	55	0,945	**0,945**	0%	7,3%	181 ký tự

Cột *n* là số câu có câu trả lời. Các câu còn lại bị lỗi API của Gemini (quá tải hoặc giới hạn tần suất) sau 3 lần thử, và được loại khỏi phép tính.

Nhận xét. (i) *LLM đơn thuần đã biết nhiều*: Gemini trả lời đúng 90% dữ kiện mà không cần tài liệu, vì các ngưỡng ADA là kiến thức phổ biến. Điều này cho thấy giá trị chính của RAG trong y tế *không chỉ là độ chính xác*. (ii) RAG nâng độ phủ dữ kiện lên

0,95 và cho câu trả lời *ngắn gọn bằng một nửa* (182 so với 340 ký tự). (iii) Quan trọng nhất, RAG cho câu trả lời *kiểm chứng được*: 90–95% câu trích đúng đoạn vàng, và *không có con số nào* trong câu trả lời nằm ngoài ngữ cảnh được cung cấp. Câu trả lời của chế độ chỉ LLM không có nguồn, nên người dùng không có cách nào biết 10% sai nằm ở đâu. (iv) Khoảng 7–8% câu, mô hình chọn trả lời "không đủ dữ liệu". Đó là hành vi mong muốn khi ngữ cảnh truy xuất không chứa đáp án (độ phủ ngữ cảnh 92–95%), thay vì đoán. (v) GraphRAG có tỉ lệ ngữ cảnh chứa đoạn vàng cao hơn RAG (0,945 so với 0,922), nhờ đó tỉ lệ trích dẫn đúng cao hơn 4 điểm. Điều này khớp với kết quả truy xuất: đồ thị cải thiện độ phủ.

5.4.5. Ví dụ: một câu hỏi đa bước qua ba chế độ

Bảng 5.9 trình bày câu trả lời thật của Gemini cho câu hỏi đa bước "Bệnh nhân đái tháo đường có eGFR 25 thì còn dùng metformin được không, nên thay bằng thuốc gì?". Hai dữ kiện vàng là ngưỡng chống chỉ định "eGFR < 30" và một nhóm thuốc thay thế phù hợp.

Bảng 5.9. Câu trả lời thật của ba chế độ cho một câu hỏi đa bước

Chế độ	Câu trả lời (rút gọn)	Đánh giá
Chỉ LLM	"…eGFR 25 **chống chỉ định** với Metformin do nguy cơ nhiễm toan lactic. Nên thay bằng …GLP-1 RA hoặc SGLT2i (lưu ý chỉnh liều theo eGFR). **Thuốc ức chế DPP-4 (như Linagliptin)** cũng là một lựa chọn an toàn…"	Phủ 1/2; không nguồn; có thông tin ngoài KB
RAG	"Metformin bị chống chỉ định khi eGFR < 30 mL/phút/1,73 m² [1], vì vậy bệnh nhân có eGFR 25 không được tiếp tục dùng thuốc này. **Tài liệu không đề cập cụ thể** bệnh nhân nên được thay thế bằng thuốc gì… [3]."	Phủ 1/2; trích đúng; trung thực
GraphRAG	"Bệnh nhân có eGFR 25 (eGFR < 30) có chống chỉ định dùng metformin [1]. Tài liệu không đề cập việc thay thế cụ thể…, nhưng **việc dùng thuốc ức chế SGLT2** có chỉ định giúp làm chậm tiến triển bệnh thận đái tháo đường [3]."	Phủ 2/2; trích đúng

Ví dụ này cho thấy rõ khác biệt về bản chất giữa ba chế độ. LLM đơn thuần trả lời

trôi chảy, phần lớn đúng theo kiến thức chung, nhưng đưa ra một nhóm thuốc (DPP-4) không có trong tài liệu nguồn và không kiểm chứng được trong hệ thống. RAG trung thành với ba đoạn truy xuất được: các đoạn này không nói về thuốc thay thế, nên mô hình nói rõ "tài liệu không đề cập" thay vì đoán. GraphRAG đi từ thực thể *Suy thận nặng* và *Bệnh thận ĐTĐ* theo cạnh `TREATED_BY` tới *Ức chế SGLT2*, đưa đúng đoạn "Thuốc bảo vệ tim mạch và thận" vào ngữ cảnh, nhờ đó trả lời đủ cả hai ý và vẫn có trích dẫn. Đây là kiểu câu hỏi "nối hai mẩu tri thức" mà GraphRAG được thiết kế để giải quyết.

5.5. Triển khai

Dịch vụ truy xuất chạy truy xuất lai trên PostgreSQL (pgvector + full-text, gộp RRF) kết hợp GraphRAG bằng truy vấn Cypher trên Apache AGE, như đã mô tả ở trên. Ba endpoint phục vụ thử nghiệm:

- `POST /api/lab/ch5/search` với `mode` ∈ {`vector`, `fulltext`, `hybrid`, `graphrag`}: trả về các đoạn, điểm, độ trễ; với GraphRAG còn trả về thực thể nhận ra và các cạnh đã đi qua.
- `POST /api/lab/ch5/ask`: hỏi đáp RAG đầy đủ (truy xuất → Gemini viết câu trả lời có trích dẫn).
- `GET /api/lab/ch5/graph`: toàn bộ đồ thị tri thức dạng JSON (nút, cạnh, bằng chứng, nguồn).

Đoạn mã 5.3. Gọi endpoint truy xuất GraphRAG của Chương 5 và kết quả (rút gọn)

```
curl -s https://agent.nits.io.vn/api/lab/ch5/search -H 'Content-Type: application/json' \
  -d '{"query":"Bệnh thận mạn có albumin niệu nên dùng thuốc gì?","mode":"graphrag","k":5}'
# {"mode": "graphrag", "graph": {"entities": ["Bệnh thận ĐTĐ"],
#    "edges": ["Đái tháo đường típ 2 —CAUSES→ Bệnh thận ĐTĐ",
#              "Bệnh thận ĐTĐ —TREATED_BY→ Ức chế SGLT2",
#              "Bệnh thận ĐTĐ —SCREENED_BY→ uACR và eGFR"]},
#  "results": [{"chunk_id": 25, "heading": "Thuốc bảo vệ tim mạch và thận", ...}, ...]}
```

5.5.1. Thực nghiệm trên Notebook và Demo Localhost

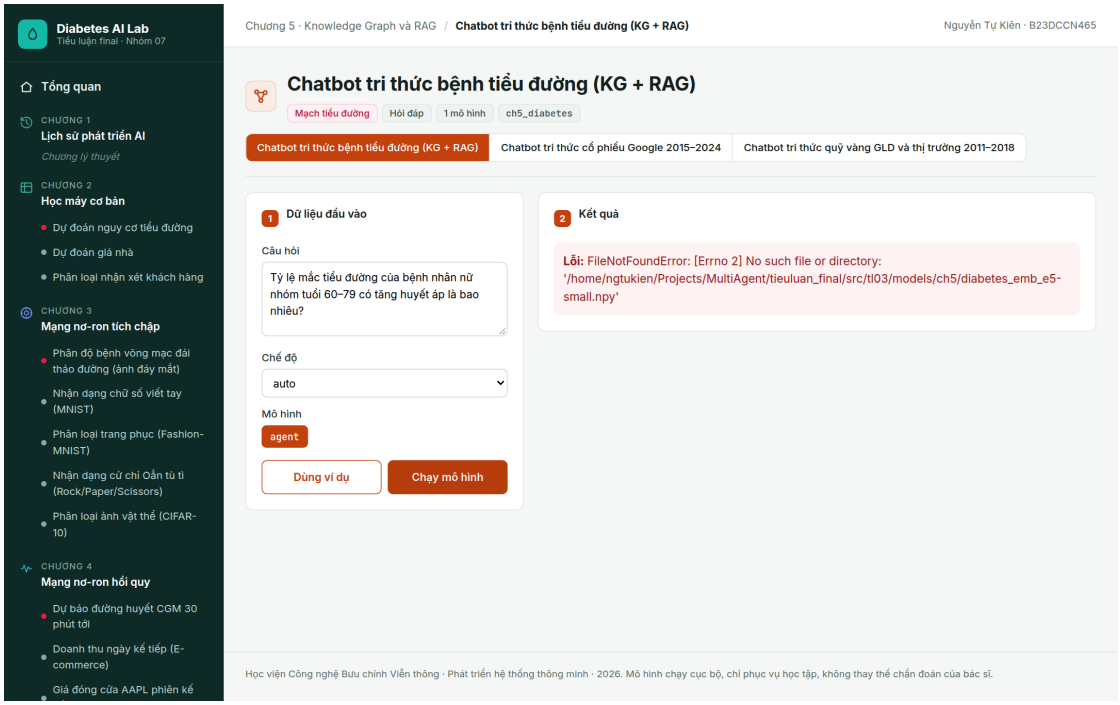
Bên cạnh dịch vụ PostgreSQL + Apache AGE ở trên, tiểu luận đóng gói thêm một bộ máy KG + RAG tự chứa hoàn toàn bằng Python trong notebook `notebooks/ch5_kg_rag.ipynb` (các ô `RAG-00` đến `RAG-04`, kế thừa khung `tl3/ch5_core.py` của nhóm) và phục vụ tại `http://localhost:8095`:

- **Ô `RAG-01` (Xây dựng Đồ thị tri thức từ 96.146 hồ sơ lâm sàng)**: Ô `RAG-01` nạp

`diabetes_domain.pkl`, chuyển mỗi hồ sơ thành các bộ ba thực thể–quan hệ (`has_gender`, `in_age_group`, `bmi_class`, `hba1c_class`, `glucose_class`, `diabetes_status`, `has_condition`) cùng 42 bản tổng hợp nhóm và các quan hệ y khoa (`risk_factor_of`, `indicates`, `comorbid_with`), thu được đồ thị gồm 96.175 nút, 684.414 cạnh và 1.357.436 bộ ba, kèm chỉ mục ngược BM25 trên 96.188 tài liệu.

- **Ô `RAG-03` (Liên kết thực thể `Domain.link` và truy vấn tập hợp chính xác `Domain.facts` trên KG)**: Khi hỏi một câu hỏi thống kê đa điều kiện như *"Tỷ lệ mắc tiểu đường ở nữ giới từ 60–79 tuổi có tăng huyết áp là bao nhiêu?"*, truy xuất văn bản thuần túy chỉ trả về vài hồ sơ lẻ tẻ. Ô `RAG-03` dùng bộ liên kết thực thể nhận diện ba nút (`Female`, `60-79`, `hypertension`), lấy giao các tập láng giềng vào trên đồ thị tri thức (`|tập hồ sơ| = 1830`) và tính chính xác tỷ lệ mắc tiểu đường 31,42%, HbA1c trung bình 5,85% và đường huyết trung bình 152,1 mg/dL, kèm vết bằng chứng đồ thị.

Trên ứng dụng web cục bộ tại `http://localhost:8095` (Hình 5.5), tab **Ch.5 — KG + RAG · Tri thức Tiểu đường** cho phép nhập câu hỏi tự nhiên và đối chiếu song song ba phương thức: *Chỉ LLM*, *Text-RAG (BM25 + Dense)* và *KG-RAG / Hybrid (kết hợp truy vấn đồ thị tri thức và văn bản)*.



Hình 5.5. Giao diện thử nghiệm Chương 5 trên `http://localhost:8095`: đối chiếu câu trả lời và bằng chứng truy xuất của LLM thuần, Text-RAG và KG-RAG trên đồ thị tri thức đái tháo đường

5.6. Tiểu kết chương 5

Chương 5 trình bày đồ thị tri thức (bộ ba, ontology, Cypher, xây dựng và suy luận) và RAG (chia đoạn, truy xuất thưa BM25, truy xuất dày, RRF, xếp hạng lại, GraphRAG). Thực nghiệm trên KB tiếng Việt và MedQuAD cho năm kết luận. (1) Chất lượng mô hình nhúng là yếu tố quan trọng nhất: E5-large vượt MiniLM trên cả hai bộ và trong truy xuất chéo ngôn ngữ. (2) BM25 rất mạnh với thuật ngữ y khoa tiếng Việt nhưng vô dụng khi khác ngôn ngữ. RRF chỉ có lợi khi hai thành phần cân sức. (3) Xếp hạng lại cho kết quả gần tuyệt đối nhưng quá chậm trên CPU. (4) GraphRAG cải thiện độ phủ và câu hỏi đa bước, giúp trích dẫn đúng hơn. (5) Với Gemini, RAG không chỉ tăng độ chính xác mà quan trọng hơn là biến câu trả lời thành thứ *kiểm chứng được*: có trích dẫn, không có số liệu ngoài nguồn, biết từ chối khi thiếu thông tin. Các bước nâng cấp tiếp theo cho production đã rõ: dùng E5 thay MiniLM và BM25 thay `ts_rank_cd`.

CHƯƠNG 6. HỌC SÂU TRÊN ĐỒ THỊ

Nhiều loại dữ liệu y sinh có dạng đồ thị: bài báo trích dẫn nhau, protein tương tác với nhau, thuốc tương tác với thuốc, bệnh đồng mắc với bệnh. Ở Chương 5, đồ thị tri thức được duyệt bằng truy vấn viết tay. Chương này để mạng nơ-ron *tự học* từ cấu trúc đồ thị. Nội dung gồm nguyên lý truyền thông điệp, ba kiến trúc tiêu biểu (GCN, GraphSAGE, GAT), hiện tượng làm trơn quá mức, và thực nghiệm phân loại 19.717 bài báo PubMed về đái tháo đường. Trong hệ thống, mô hình này dùng để gán nhãn "thực nghiệm / típ 1 / típ 2" cho y văn trước khi đưa vào Knowledge Base.

6.1. Dữ liệu đồ thị và các bài toán

Một đồ thị $G = (V, E)$ có $n = |V|$ nút, ma trận kề $\mathbf{A} \in \{0,1\}^{n\times n}$, ma trận bậc $\mathbf{D} = \mathrm{diag}(d_1, \dots, d_n)$ với $d_i = \sum_j A_{ij}$, và ma trận đặc trưng nút $\mathbf{X} \in \mathbb{R}^{n\times d}$. Đồ thị khác ảnh và chuỗi ở ba điểm: số láng giềng thay đổi theo nút; *không có thứ tự* giữa các láng giềng; và các nút không độc lập với nhau. Do đó mô hình phải *bất biến với hoán vị*, tức là đánh số lại nút thì kết quả không đổi. Có ba họ bài toán: *cấp nút* (phân loại bài báo, phát hiện gian lận), *cấp cạnh* (dự đoán liên kết, gợi ý thuốc, tác dụng phụ khi phối hợp thuốc [85]), và *cấp đồ thị* (dự đoán tính chất phân tử).

Trước GNN, đã có hai hướng. *Lan truyền nhãn* [86] chỉ dùng cấu trúc đồ thị, với giả định các nút nối nhau thường cùng nhãn (*đồng tính*, homophily [87]). *Nhúng nút* như DeepWalk [88] học vector cho từng nút từ các bước đi ngẫu nhiên, nhưng không tận dụng đặc trưng nút và không tổng quát được sang nút mới.

6.2. Mạng nơ-ron đồ thị

6.2.1. Khung truyền thông điệp

Gilmer và cộng sự [89] thống nhất phần lớn các GNN vào khung *truyền thông điệp*. Ở tầng l, mỗi nút v nhận thông điệp từ láng giềng, gộp chúng bằng một hàm bất biến hoán vị (tổng, trung bình, max), rồi cập nhật biểu diễn của mình:

$$\begin{aligned} \mathbf{m}_v^{(l)} &= \operatorname*{AGG}_{u\in\mathcal{N}(v)} \phi^{(l)}\big(\mathbf{h}_u^{(l)}, \mathbf{h}_v^{(l)}\big), \\ \mathbf{h}_v^{(l+1)} &= \psi^{(l)}\big(\mathbf{h}_v^{(l)}, \mathbf{m}_v^{(l)}\big), \qquad \mathbf{h}_v^{(0)} = \mathbf{x}_v. \end{aligned} \tag{6.1}$$

Sau L tầng, $\mathbf{h}_v^{(L)}$ tổng hợp thông tin của vùng lân cận L bước quanh v, tương tự trường tiếp nhận của CNN. CNN thực chất là trường hợp đặc biệt của truyền thông điệp trên đồ thị lưới, trong đó mỗi điểm ảnh có đúng 8 láng giềng theo thứ tự cố định. GNN tổng quát hóa phép tích chập sang láng giềng không có thứ tự và số lượng thay đổi [90].

6.2.2. Nền tảng phổ: từ biến đổi Fourier trên đồ thị đến GCN

GCN có thể được dẫn ra từ lý thuyết đồ thị phổ. Ma trận Laplace chuẩn hóa $\mathbf{L} = \mathbf{I} - \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ là ma trận đối xứng nửa xác định dương, phân rã được thành $\mathbf{L} = \mathbf{U}\Lambda\mathbf{U}^\top$ với các trị riêng $0 = \lambda_1 \le \dots \le \lambda_n \le 2$. Các vector riêng $\mathbf{U}$ đóng vai trò "các tần số" trên đồ thị. Vector riêng ứng với trị riêng nhỏ biến thiên chậm giữa các nút kề nhau (tần số thấp), trị riêng lớn ứng với tín hiệu dao động mạnh giữa các nút kề nhau (tần số cao). *Biến đổi Fourier trên đồ thị* của tín hiệu $\mathbf{x} \in \mathbb{R}^n$ là $\hat{\mathbf{x}} = \mathbf{U}^\top\mathbf{x}$, và *tích chập phổ* với bộ lọc g_θ là:

$$g_\theta \star \mathbf{x} = \mathbf{U}\, g_\theta(\Lambda)\, \mathbf{U}^\top\mathbf{x}. \tag{6.2}$$

Công thức (6.2) đẹp nhưng không dùng được trực tiếp: phân rã trị riêng tốn $O(n^3)$, bộ lọc không cục bộ, và không áp dụng được cho đồ thị khác. ChebNet xấp xỉ $g_\theta(\Lambda)$ bằng đa thức Chebyshev bậc K, $g_\theta \approx \sum_{k=0}^{K} \theta_k T_k(\tilde{\mathbf{L}})$. Bộ lọc khi đó chỉ phụ thuộc lân cận K bước và tính được bằng phép nhân ma trận thưa. Kipf và Welling chọn $K = 1$, xấp xỉ $\lambda_{\max} \approx 2$ và dùng chung một tham số $\theta = \theta_0 = -\theta_1$, được $g_\theta \star \mathbf{x} \approx \theta(\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2})\mathbf{x}$. Toán tử $\mathbf{I} + \mathbf{D}^{-1/2}\mathbf{A}\mathbf{D}^{-1/2}$ có trị riêng trong $[0, 2]$, nên chồng nhiều tầng có thể làm số liệu bất ổn. Để tránh điều này, họ dùng *thủ thuật chuẩn hóa lại*: thay bằng $\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$ với $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, chính là $\hat{\mathbf{A}}$ trong (6.3). Góc nhìn phổ cũng giải thích bản chất của GCN: nó là một *bộ lọc thông thấp*, giữ thành phần biến thiên chậm (các nút kề nhau giống nhau) và triệt thành phần dao động. Bộ lọc này phù hợp với đồ thị đồng tính, và áp dụng lặp lại nhiều lần thì dẫn tới làm trơn quá mức.

6.2.3. Mạng tích chập đồ thị (GCN)

Kipf và Welling [91] rút ra GCN từ xấp xỉ bậc nhất của tích chập phổ trên đồ thị. Thêm khuyên vào mỗi nút để nút giữ thông tin của chính mình, $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$, $\tilde{\mathbf{D}} = \mathrm{diag}(\tilde{\mathbf{A}}\mathbf{1})$. Một tầng GCN là:

$$\mathbf{H}^{(l+1)} = \sigma\!\left(\hat{\mathbf{A}}\,\mathbf{H}^{(l)}\mathbf{W}^{(l)}\right), \qquad \hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}. \tag{6.3}$$

Viết theo từng nút: $\mathbf{h}'_v = \sigma\big(\sum_{u\in\mathcal{N}(v)\cup\{v\}} \frac{1}{\sqrt{\tilde{d}_u\tilde{d}_v}}\mathbf{W}\mathbf{h}_u\big)$. Mỗi tầng thực hiện hai phép toán. (i) *Biến đổi đặc trưng* $\mathbf{HW}$, giống một lớp tuyến tính dùng chung cho mọi nút. (ii) *Gộp láng giềng* có chuẩn hóa đối xứng theo bậc: một nút có nhiều liên kết (bài tổng quan được trích dẫn nhiều) đóng góp ít hơn cho mỗi láng giềng, nên không lấn át. $\hat{\mathbf{A}}$ là ma trận thưa với $|E|$ phần tử khác 0, nên một tầng tốn $O(|E|\,d)$. Mô hình hai tầng cho phân loại nút:

$$\begin{aligned} \mathbf{Z} &= \operatorname{softmax}\Big(\hat{\mathbf{A}}\ \operatorname{ReLU}\left(\hat{\mathbf{A}}\mathbf{X}\mathbf{W}^{(0)}\right)\mathbf{W}^{(1)}\Big), \\ \mathscr{L} &= -\sum_{v\in V_L}\sum_{c} Y_{vc}\log Z_{vc}, \end{aligned} \tag{6.4}$$

với mất mát chỉ tính trên tập nút có nhãn V_L. Học bán giám sát diễn ra tự nhiên: nhãn của vài nút lan qua cạnh tới các nút chưa có nhãn nhờ phép gộp, trong khi đặc trưng của mọi nút (kể cả nút không nhãn) đều tham gia vào lan truyền xuôi.

Ví dụ tính tay. Xét đồ thị hình sao 4 nút: nút 2 là bài tổng quan được ba bài 1, 3, 4 trích dẫn (đánh số 1–4). Bậc sau khi thêm khuyên là $\tilde{\mathbf{d}} = (2, 4, 2, 2)$, và

$$\hat{\mathbf{A}} = \tilde{\mathbf{D}}^{-1/2}(\mathbf{A}+\mathbf{I})\tilde{\mathbf{D}}^{-1/2} = \begin{bmatrix} 0{,}5 & 0{,}354 & 0 & 0 \\ 0{,}354 & 0{,}25 & 0{,}354 & 0{,}354 \\ 0 & 0{,}354 & 0{,}5 & 0 \\ 0 & 0{,}354 & 0 & 0{,}5 \end{bmatrix},$$

với $\hat{A}_{12} = 1/\sqrt{2\cdot 4} = 0{,}354$ và $\hat{A}_{22} = 1/4$. Giả sử một đặc trưng là "bài có từ *mice*" với $\mathbf{x} = (1, 0, 0, 1)^\top$, tức bài 1 và 4 nghiên cứu trên chuột. Một bước gộp cho $\hat{\mathbf{A}}\mathbf{x} = (0{,}5;\ 0{,}707;\ 0;\ 0{,}5)$. Bài tổng quan (nút 2) nhận tín hiệu "thực nghiệm" mạnh nhất (0,707) từ hai láng giềng, dù bản thân nó không có từ đó. Bài 3 vẫn bằng 0 vì chưa có đường đi độ dài 1 tới nút nguồn. Sau bước thứ hai, $\hat{\mathbf{A}}^2\mathbf{x} = (0{,}5;\ 0{,}53;\ 0{,}25;\ 0{,}5)$: tín hiệu đã lan tới bài 3 qua bài tổng quan, đồng thời giá trị các nút bắt đầu "san bằng". Ví dụ này cho thấy cả hai mặt của truyền thông điệp: *lan truyền thông tin* theo cấu trúc trích dẫn, và *làm trơn*. Tiếp tục nhân nhiều lần, mọi nút sẽ hội tụ về cùng một giá trị (§6.2.6).

6.2.4. GraphSAGE

GCN cần toàn bộ ma trận $\hat{\mathbf{A}}$ khi huấn luyện (transductive). GraphSAGE [92] học *hàm gộp* thay vì học trên một đồ thị cố định, nên áp dụng được cho nút mới (inductive):

$$\mathbf{h}'_v = \sigma\Big(\mathbf{W}_{\text{self}}\mathbf{h}_v + \mathbf{W}_{\text{neigh}} \cdot \frac{1}{|\mathcal{N}(v)|}\sum_{u\in\mathcal{N}(v)} \mathbf{h}_u\Big). \tag{6.5}$$

Khác biệt với GCN là biểu diễn của chính nút và của láng giềng có *hai ma trận trọng số riêng*. Mô hình nhờ vậy học được mức tin vào bản thân và mức tin vào láng giềng. Với đồ thị lớn, GraphSAGE lấy mẫu một số láng giềng cố định để giới hạn chi phí.

6.2.5. Mạng chú ý trên đồ thị (GAT)

GCN gán trọng số cố định $1/\sqrt{\tilde{d}_u\tilde{d}_v}$ cho mọi láng giềng. GAT [93] *học* trọng số đó bằng cơ chế chú ý:

$$\begin{aligned} e_{vu} &= \text{LeakyReLU}(\mathbf{a}^\top[\mathbf{W}\mathbf{h}_v \,\|\, \mathbf{W}\mathbf{h}_u]), \\ \alpha_{vu} &= \frac{\exp(e_{vu})}{\sum_{k\in\mathcal{N}(v)\cup\{v\}}\exp(e_{vk})}, \\ \mathbf{h}'_v &= \Big\|_{k=1}^{K}\ \sigma\Big(\sum_{u}\alpha_{vu}^{k}\mathbf{W}^k\mathbf{h}_u\Big). \end{aligned} \tag{6.6}$$

Hệ số α_{vu} là "mức quan trọng" của láng giềng u đối với v, được chuẩn hóa bằng softmax trên các láng giềng của v. *K đầu chú ý* độc lập được nối lại ($\|$) ở tầng ẩn và lấy trung bình ở tầng ra, giúp ổn định quá trình học. Vì $\mathbf{a}^\top[\mathbf{W}\mathbf{h}_v\|\mathbf{W}\mathbf{h}_u] = \mathbf{a}_{\text{dst}}^\top\mathbf{W}\mathbf{h}_v + \mathbf{a}_{\text{src}}^\top\mathbf{W}\mathbf{h}_u$, điểm chú ý tính được theo từng cạnh chỉ với hai phép tra chỉ số. Trọng số α còn có giá trị giải thích: trang thử nghiệm hiển thị các bài báo láng giềng mà GAT "chú ý" nhiều nhất.

6.2.6. Làm trơn quá mức

Mỗi phép nhân với $\hat{\mathbf{A}}$ là một bước khuếch tán, kéo biểu diễn của các nút kề nhau về gần nhau. Khi $L \to \infty$, $\hat{\mathbf{A}}^L$ hội tụ về một ma trận hạng một (với đồ thị liên thông), nên mọi nút có cùng biểu diễn, chỉ khác một hệ số tỉ lệ theo bậc [94]. Đó là hiện tượng *làm trơn quá mức* (over-smoothing). Vì vậy GNN thường chỉ có 2–3 tầng, rất khác với CNN hàng trăm lớp. Chen và cộng sự [95] đo hiện tượng này bằng MAD (Mean Average Distance), tức khoảng cách cosin trung bình giữa biểu diễn các cặp nút; MAD tiến về 0 khi các nút không còn phân biệt được.

6.2.7. Năng lực biểu diễn và phép thử Weisfeiler–Lehman

GNN có thể phân biệt được những cấu trúc nào? Xu và cộng sự [96] chứng minh rằng mọi GNN truyền thông điệp đều *không mạnh hơn* phép thử đẳng cấu đồ thị Weisfeiler–Lehman bậc 1 (1-WL). Phép thử này lặp lại việc gán cho mỗi nút một "màu" mới bằng cách băm màu hiện tại cùng *đa tập* màu của láng giềng. Một GNN đạt được sức mạnh của 1-WL khi hàm gộp là *đơn ánh trên đa tập*. Tổng thỏa mãn điều kiện này, còn trung bình và max thì không. Ví dụ, trung bình không phân biệt được láng giềng $\{a, b\}$ với $\{a, a, b, b\}$, còn max không phân biệt được $\{a, b\}$ với $\{a, b, b\}$. Từ đó họ đề xuất GIN, $\mathbf{h}'_v = \mathrm{MLP}\big((1+\epsilon)\mathbf{h}_v + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u\big)$. Với bài toán phân loại nút trên đồ thị trích dẫn như Pubmed, đặc trưng nút giàu thông tin và giả định đồng tính quan trọng hơn khả năng phân biệt cấu trúc tinh vi. Vì vậy các bộ gộp "yếu" như trung bình (GraphSAGE) hay tổng có chuẩn hóa (GCN) vẫn hoạt động tốt. Năng lực biểu diễn trở nên quyết định ở bài toán cấp đồ thị, như phân biệt hai phân tử có cùng thành phần nguyên tử nhưng khác cấu trúc vòng.

6.2.8. Các bài toán khác và ứng dụng y sinh

Dự đoán liên kết. Bộ mã hóa GNN tạo biểu diễn nút $\mathbf{z}_v$. Xác suất có cạnh được giải mã bằng $\sigma(\mathbf{z}_u^\top \mathbf{z}_v)$ (Graph Auto-Encoder) hoặc bằng một MLP trên cặp $(\mathbf{z}_u, \mathbf{z}_v)$, huấn luyện với cạnh thật làm mẫu dương và cạnh ngẫu nhiên làm mẫu âm. *Phân loại đồ thị.* Sau các tầng truyền thông điệp, một phép *đọc ra* (readout) gộp biểu diễn của mọi nút thành một vector cho cả đồ thị, bằng tổng, trung bình, hoặc pooling phân cấp. Phép đọc ra phải bất biến với hoán vị.

GNN có nhiều ứng dụng trong y sinh liên quan tới đái tháo đường. *Tác dụng phụ khi phối hợp thuốc*: Decagon [85] dự đoán tác dụng phụ của từng cặp thuốc trên đồ thị gồm protein, thuốc và các loại tương tác. Đây là điều quan trọng với người ĐTĐ thường dùng nhiều thuốc (hạ đường huyết, hạ áp, statin). *Tái định vị thuốc*: dự đoán liên kết thuốc–bệnh trên đồ thị tri thức y sinh. *Mạng bệnh đồng mắc*: nút là bệnh nhân hoặc bệnh, cạnh là đồng xuất hiện trong hồ sơ, giúp dự báo biến chứng tim mạch, thận. *Phân tử*: dự đoán tính chất của phân tử ứng viên thuốc, coi nguyên tử là nút và liên kết hóa học là cạnh. *Gán nhãn y văn*: bài toán của chương này, giúp xây dựng KB có cấu trúc từ hàng triệu bài báo.

6.3. Dữ liệu Pubmed-Diabetes

Pubmed-Diabetes [9], [97] là một trong ba bộ chuẩn kinh điển của GNN (cùng Cora và Citeseer), được dùng trong bài báo gốc của GCN và GAT. Mỗi nút là một bài báo trong cơ sở dữ liệu PubMed liên quan tới đái tháo đường. Bài báo được biểu diễn bằng vector TF-IDF trên 500 từ gốc (stem) và gán một trong ba nhãn MeSH: *Diabetes Mellitus, Experimental* (nghiên cứu trên động vật), *Type 1* hoặc *Type 2*. Cạnh là quan hệ trích dẫn.

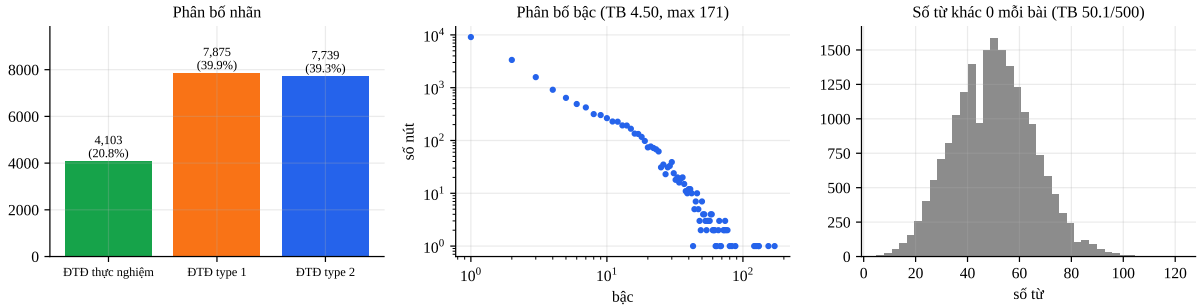
- **Nguồn**: nhóm LINQS, `https://linqs-data.soe.ucsc.edu/public/Pubmed-Diabetes.tgz` (14,7 MB). Bản văn bản gốc dạng TSV được dùng thay cho bản pickle của Planetoid, để đọc được mà không phải chạy mã không tin cậy.
- **Quy mô**: 19.717 nút, 44.338 trích dẫn có hướng. Sau khi chuyển sang vô hướng, bỏ khuyên và cạnh trùng còn 44.324 cạnh. Ma trận đặc trưng 19.717 × 500 rất thưa: trung bình 50,1 từ khác 0 trên mỗi bài.
- **Nhãn**: thực nghiệm 4.103 (20,8%), típ 1 7.875 (39,9%), típ 2 7.739 (39,3%).

Bảng 6.1. Năm nút đầu tiên: mã PubMed, nhãn, bậc và các từ có TF-IDF cao nhất

PubMed ID	Nhãn	Từ nổi bật	Bậc
12187484	Thực nghiệm	retinopathi, mice, abnorm, month, anim, rat	4
2344352	Thực nghiệm	muscl, m, chang, r, stimul, caus	5
14654069	Thực nghiệm	cardiac, state, rat, experiment, defect, mark	2
16443886	Típ 1	a1c, glucos, profil, sampl, predict, larg	1
2684155	Thực nghiệm	secret, cholesterol, h, rate, low, anim	12

Bảng 6.2. Mười từ có TF-IDF trung bình cao nhất của mỗi lớp (dạng từ gốc sau stemming)

Lớp	Từ đặc trưng
ĐTĐ thực nghiệm	rat, mice, nerv, activ, anim, cell, streptozotocin, increas, protein, decreas
ĐTĐ típ 1	cell, iddm, nod, patient, children, mice, t, 0, islet, 1
ĐTĐ típ 2	0, patient, insulin, 2, niddm, glucos, risk, p, type, subject



Hình 6.1. Phân bố nhãn, phân bố bậc (thang log–log) và số từ khác 0 của mỗi bài báo

Nhận xét. (i) Phân bố bậc có *đuôi nặng*: phần lớn bài báo chỉ có 1–5 trích dẫn (bậc trung bình 4,50), một số ít có tới 171. Trên thang log–log, phân bố gần thẳng, đặc trưng của mạng trích dẫn. Không có nút cô lập. (ii) *Tính đồng tính rất cao*: 80,2% cạnh nối hai bài báo cùng nhãn. Đây là điều kiện để GNN phát huy tác dụng; nếu tỉ lệ này thấp, gộp láng giềng sẽ thêm nhiễu thay vì thông tin. (iii) Từ đặc trưng của mỗi lớp hợp lý về mặt y khoa: lớp thực nghiệm có *rat, mice, streptozotocin* (hóa chất gây ĐTĐ trên chuột); típ 1 có *iddm, nod* (chuột NOD), *children, islet*; típ 2 có *niddm, insulin, risk, subject*. (iv) Đồ thị rất thưa (mật độ $2{,}3 \cdot 10^{-4}$), nên phép nhân ma trận thưa trong (6.3) rất rẻ.

6.4. Cài đặt

Cả ba GNN được *tự cài đặt bằng PyTorch thuần*, không dùng PyTorch Geometric [98], để thể hiện rõ từng phép toán của truyền thông điệp. GCN và GraphSAGE dùng ma trận thưa `torch.sparse`: $\hat{\mathbf{A}}$ cho GCN, ma trận trung bình láng giềng $\mathbf{D}^{-1}\mathbf{A}$ cho GraphSAGE. GAT làm việc trên danh sách cạnh: tính điểm chú ý cho từng cạnh, rồi tính softmax theo nút đích bằng `index_reduce` (max, để ổn định số) và `index_add` (tổng) (Đoạn mã 6.1).

Đoạn mã 6.1. Tầng GCN và tầng GAT cài bằng PyTorch thuần (`lab/ch6_gnn.py`, rút gọn)

```
class GCN(nn.Module):                          # H' = sigma(Â H W)
    def forward(self, x, g):
        for i, lin in enumerate(self.lins):
            x = F.dropout(x, self.p, self.training)
            x = torch.sparse.mm(g["A_gcn"], lin(x))     # biến đổi đặc trưng rồi gộp có chuẩn
             ↪  hóa bậc
            if i < len(self.lins) - 1:
                x = F.relu(x)
        return x

class GATLayer(nn.Module):
    def forward(self, x, edge_index):
        src, dst = edge_index                                  # cạnh u → v (đã có khuyên)
        z = self.W(x).view(n, self.heads, self.h)                # W h, K đầu
        e = F.leaky_relu((z * self.a_src).sum(-1)[src]           # a_src^T W h_u
                         + (z * self.a_dst).sum(-1)[dst], 0.2)   # + a_dst^T W h_v   → [E, K]
        e_max = torch.full((n, K), -1e9).index_reduce_(0, dst, e, "amax")   # max theo nút đích
        ex = torch.exp(e - e_max[dst])
        alpha = ex / torch.zeros(n, K).index_add_(0, dst, ex)[dst]          # softmax trên
         ↪  láng giềng
        out = torch.zeros(n, K, self.h).index_add_(0, dst, alpha[..., None] * z[src])
        return out.reshape(n, -1) if self.concat else out.mean(1)
```

Bảng 6.3. Cấu hình và số tham số của các mô hình (đặc trưng vào 500 chiều, 3 lớp)

Mô hình	Kiến trúc	Tham số	Dùng đồ thị
Hồi quy logistic	Dropout 0,2 → Linear(500, 3)	1.503	Không
MLP	Linear(500, 64) → ReLU → Linear(64, 3), dropout 0,5	32.259	Không
Lan truyền nhãn	$\mathbf{F} \leftarrow 0{,}9\,\hat{\mathbf{A}}\mathbf{F} + 0{,}1\,\mathbf{Y}$, 50 bước	0	Chỉ đồ thị
GCN	2 tầng (6.3), 500 → 64 → 3, dropout 0,5	32.259	Có
GraphSAGE-mean	2 tầng (6.5), 500 → 64 → 3	64.451	Có
GAT	8 đầu × 8 chiều (nối, ELU) → 8 đầu ra (trung bình), dropout 0,6	33.779	Có

GCN và MLP có *đúng cùng số tham số* ($32.259 = 500{\cdot}64 + 64 + 64{\cdot}3 + 3$). Khác biệt duy nhất là phép nhân với $\hat{\mathbf{A}}$. So sánh hai mô hình này vì vậy đo chính xác giá trị của cấu trúc đồ thị.

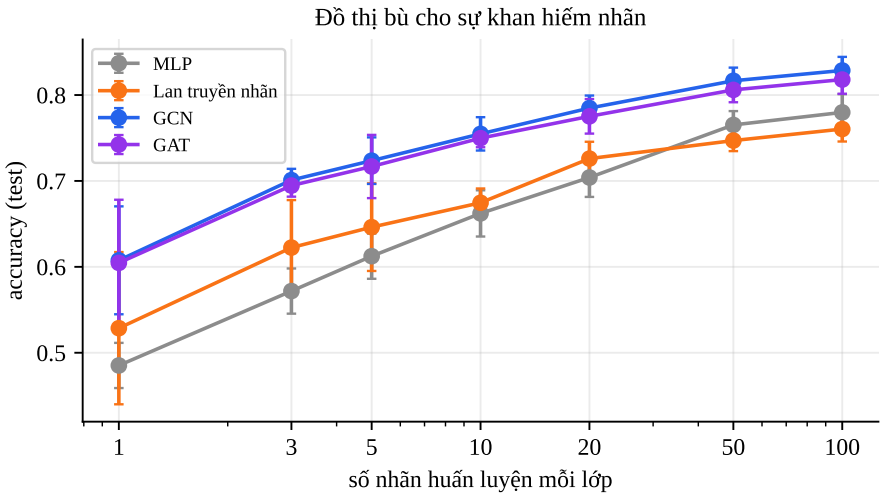
Hai giao thức. (1) *Bán giám sát kiểu Planetoid* [99]: 20 nút có nhãn mỗi lớp (60 nút, tức 0,3% đồ thị), 500 nút validation, 1.000 nút test, lặp với 10 phép chia ngẫu nhiên. (2) *Có giám sát đầy đủ*: chia 60/20/20, lặp 3 lần. Tối ưu Adam (tốc độ học 0,01; suy giảm trọng số $5 \cdot 10^{-4}$, GAT 10^{-3}), tối đa 300–400 epoch, dừng sớm sau 50 epoch không cải thiện accuracy validation. Huấn luyện trên GPU, mỗi lần chạy 0,4–10 giây.

6.5. Kết quả và nhận xét

Bảng 6.4. Accuracy phân loại nút trên Pubmed-Diabetes (trung bình ± độ lệch chuẩn)

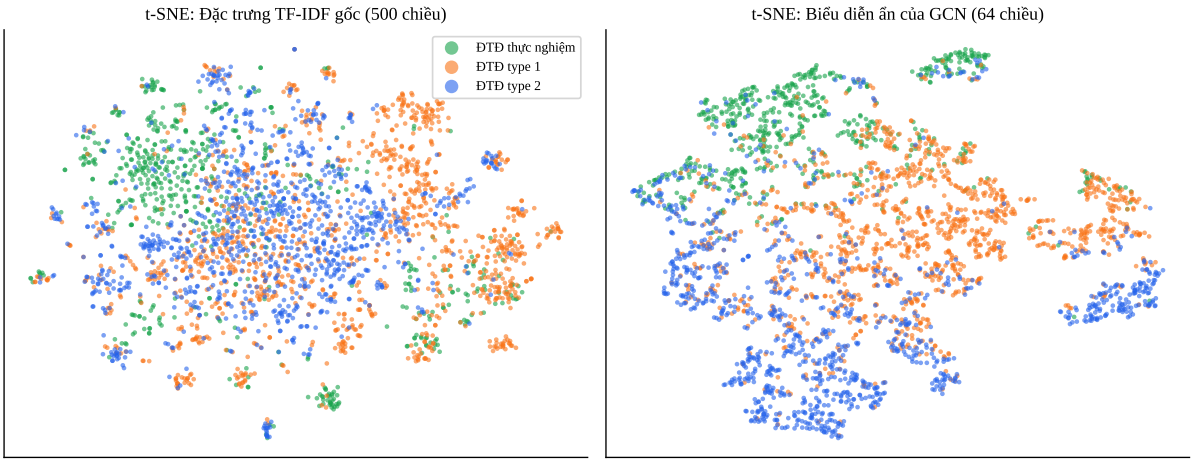
	Bán giám sát (20 nhãn/lớp, 10 lần)		**Có giám sát (60/20/20, 3 lần)**	
Mô hình	**Accuracy**	**Macro-F1**	**Accuracy**	**Macro-F1**
Hồi quy logistic	0,707 ± 0,009	0,712	0,843 ± 0,005	0,843
MLP	0,702 ± 0,019	0,706	0,871 ± 0,004	0,871
Lan truyền nhãn	0,718 ± 0,021	0,702	0,836 ± 0,005	0,824
GCN	**0,780** ± 0,020	**0,777**	0,877 ± 0,005	0,872
GraphSAGE-mean	0,757 ± 0,026	0,755	**0,885** ± 0,001	**0,882**
GAT	0,777 ± 0,019	0,773	0,851 ± 0,003	0,844

Bán giám sát: đồ thị mang lại +7,8 điểm. Khi chỉ có 60 nhãn, GCN đạt 0,780 so với 0,702 của MLP cùng kiến trúc và cùng số tham số. Cả hai nguồn thông tin đều cần thiết. Chỉ dùng đặc trưng (MLP, hồi quy logistic) đạt khoảng 0,70. Chỉ dùng đồ thị (lan truyền nhãn) đạt 0,718. GCN kết hợp cả hai và vượt cả hai. Kết quả của GCN (0,780) và GAT (0,777) sát với con số công bố trên phép chia chuẩn của Planetoid (0,790 cho cả hai [91], [93]). Phần chênh nhỏ đến từ việc báo cáo này dùng 10 phép chia ngẫu nhiên thay cho một phép chia cố định. Trên Pubmed, GAT không vượt GCN: với đồ thị đồng tính cao và bậc thấp, trọng số chuẩn hóa theo bậc của GCN đã gần tối ưu, và chú ý học được thêm ít thông tin.



Hình 6.2. Accuracy theo số nhãn huấn luyện mỗi lớp: đồ thị có lợi nhất khi nhãn khan hiếm

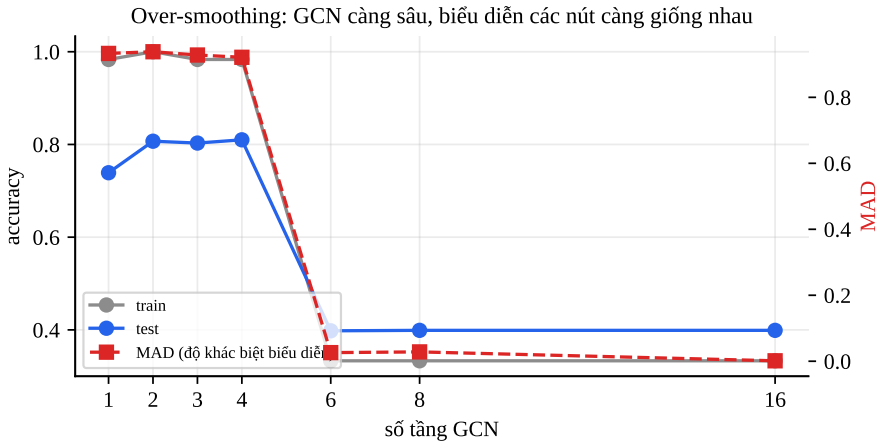
Giá trị của đồ thị giảm khi có nhiều nhãn. Hình 6.2 cho thấy với *một* nhãn mỗi lớp (3 nút có nhãn trên 19.717), GCN đạt 0,608 trong khi MLP chỉ 0,485, chênh 12,2 điểm. Khoảng cách thu hẹp dần: 11,1 điểm ở 5 nhãn/lớp, 8,1 ở 20, 4,9 ở 100. Ở giao thức có giám sát (khoảng 3.900 nhãn mỗi lớp), MLP đã đạt 0,871 và GCN chỉ hơn 0,6 điểm. Khi nhãn dồi dào, đặc trưng TF-IDF của chính bài báo đủ để phân loại, và láng giềng chủ yếu bổ sung thông tin ở những ca khó. GraphSAGE thắng ở giao thức này (0,885) nhờ tách $\mathbf{W}_{\text{self}}$ khỏi $\mathbf{W}_{\text{neigh}}$: với nhiều dữ liệu, mô hình học được cách tin vào đặc trưng của chính nút hơn. Ngược lại, GAT với dropout 0,6 trên cả đặc trưng lẫn hệ số chú ý bị điều chuẩn quá mạnh ở chế độ nhiều dữ liệu (0,851). Siêu tham số này được thiết kế cho chế độ ít nhãn.



Hình 6.3. t-SNE của 3.000 bài báo: đặc trưng TF-IDF gốc (trái) và biểu diễn ẩn của GCN (phải)

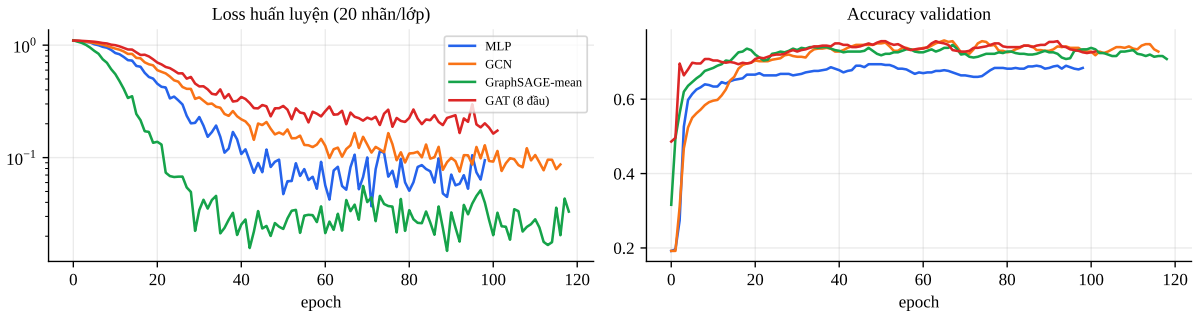
Biểu diễn học được. Trên đặc trưng TF-IDF gốc, ba lớp trộn lẫn, chỉ lớp thực nghiệm hơi tách ra (Hình 6.3, trái). Sau một tầng GCN, dù chỉ huấn luyện với 60 nhãn,

biểu diễn 64 chiều tách thành ba vùng rõ rệt (phải). Phép gộp láng giềng đã "kéo" các bài báo cùng chủ đề lại gần nhau, đúng như giả định đồng tính.



Hình 6.4. Làm trơn quá mức: accuracy và MAD của GCN theo số tầng

Làm trơn quá mức. Hình 6.4 khảo sát GCN từ 1 đến 16 tầng (20 nhãn/lớp). Một tầng chỉ thấy láng giềng trực tiếp (0,739). Hai đến bốn tầng tốt nhất (0,803–0,810). Từ 6 tầng trở đi, mô hình *sụp hoàn toàn*: accuracy 0,399, đúng bằng tỉ lệ lớp típ 1 đông nhất (7.875/19.717). Nghĩa là mô hình dự đoán cùng một lớp cho mọi nút. MAD giảm từ 0,92–0,94 xuống 0,026 (6 tầng) và 0,001 (16 tầng): biểu diễn của mọi nút gần như trùng nhau. Cần nói rõ, sự sụp đổ đột ngột này là tổng hợp của hai nguyên nhân: làm trơn quá mức theo lý thuyết ở trên, và khó tối ưu của mạng sâu không có kết nối tắt, với dropout 0,5 ở mỗi tầng. Các kỹ thuật như kết nối tắt kiểu ResNet, JKNet hay PairNorm được đề xuất để giảm cả hai vấn đề.



Hình 6.5. Loss huấn luyện và accuracy validation theo epoch (20 nhãn/lớp, một phép chia)

Hình 6.5 cho thấy quá trình huấn luyện ở chế độ ít nhãn. Chỉ có 60 nút có nhãn, nên loss huấn luyện giảm rất nhanh: GraphSAGE, với hai ma trận trọng số và gấp đôi tham số, khớp gần như hoàn toàn (loss khoảng 0,03); MLP và GCN dừng quanh 0,1; GAT giữ loss cao nhất (khoảng 0,2) vì dropout 0,6 trên cả đặc trưng lẫn hệ số chú ý. Accuracy

validation mới là tín hiệu thật. GAT tăng nhanh nhất, đạt khoảng 0,7 ngay sau vài epoch. Ba GNN vượt 0,75 sau khoảng 30 epoch, còn MLP dừng ở khoảng 0,70. Loss thấp nhất không tương ứng với accuracy cao nhất: GraphSAGE khớp sát 60 nhãn nhất nhưng tổng quát kém hơn GCN, dấu hiệu quá khớp khi dữ liệu nhãn quá ít. Dừng sớm theo accuracy validation, với kiên nhẫn 50 epoch, giữ lại trọng số tốt nhất.

6.5.1. Ablation các lựa chọn thiết kế của GCN

Bảng 6.5. Ablation GCN trên giao thức bán giám sát (20 nhãn/lớp, 10 phép chia)

Cấu hình	Accuracy	Chênh lệch
GCN chuẩn: khuyên, $\tilde{\mathbf{D}}^{-1/2}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-1/2}$, ẩn 64, dropout 0,5	**0,780** ± 0,020	–
Không có khuyên ($\mathbf{A}$ thay cho $\mathbf{A} + \mathbf{I}$)	0,764 ± 0,023	−1,6
Chuẩn hóa theo hàng $\tilde{\mathbf{D}}^{-1}\tilde{\mathbf{A}}$ (trung bình láng giềng)	0,775 ± 0,017	−0,5
Không chuẩn hóa (tổng láng giềng)	0,765 ± 0,017	−1,6
Không dropout	0,776 ± 0,021	−0,5
Kích thước ẩn 16	0,782 ± 0,019	+0,1
Kích thước ẩn 256	0,782 ± 0,014	+0,1

Bảng 6.5 xác nhận các lựa chọn của Kipf và Welling bằng thực nghiệm. (i) *Khuyên* quan trọng nhất: bỏ khuyên, biểu diễn mới của một nút chỉ là tổng hợp của láng giềng, đặc trưng của chính nó bị mất sau mỗi tầng. Với 63% số nút chỉ có 1–2 láng giềng, mất thông tin bản thân là thiệt hại lớn (−1,6 điểm). (ii) *Chuẩn hóa*: không chuẩn hóa thì nút bậc cao có biểu diễn lớn bất thường, làm số liệu bất ổn (−1,6). Chuẩn hóa theo hàng (lấy trung bình) gần bằng chuẩn hóa đối xứng (−0,5). Chuẩn hóa đối xứng còn giảm thêm ảnh hưởng của láng giềng bậc cao, như bài tổng quan được trích dẫn rộng rãi ít đặc trưng cho một chủ đề cụ thể. (iii) *Dropout* giúp nhẹ (+0,5) khi chỉ có 60 nhãn. (iv) *Kích thước ẩn* từ 16 tới 256 gần như không ảnh hưởng. Nút thắt không nằm ở dung lượng mô hình mà ở lượng nhãn và thông tin trong đồ thị. Kết quả này khớp với kết luận của Chương 2.

6.5.2. Học chuyển dẫn và học quy nạp

GCN thường được mô tả là *chuyển dẫn* (transductive): đồ thị đầy đủ, kể cả nút cần dự đoán, có mặt lúc huấn luyện. GraphSAGE nhấn mạnh tính *quy nạp* (inductive): áp dụng được cho nút mới chưa từng thấy. Đây chính là tình huống của hệ thống khi một bài báo mới cần được gán nhãn trước khi nạp vào KB. Thí nghiệm sau kiểm tra trực tiếp

sự khác biệt này. Với phép chia 60/20/20, khi huấn luyện, *xóa toàn bộ nút kiểm tra cùng mọi cạnh nối tới chúng*: chỉ còn 29.175 trên 44.324 cạnh, tức đồ thị mất 34% số cạnh. Lúc suy luận, các nút kiểm tra mới được thêm lại vào đồ thị.

Bảng 6.6. Học chuyển dẫn và học quy nạp trên Pubmed-Diabetes (60/20/20, 3 lần chạy)

Mô hình	Chuyển dẫn	Quy nạp (nút test không có khi huấn luyện)	Chênh lệch
MLP	0,872 ± 0,002	0,872 ± 0,002	0,00
GCN	0,877 ± 0,006	0,876 ± 0,006	−0,09
GraphSAGE-mean	**0,887** ± 0,004	**0,885** ± 0,005	−0,18
GAT	0,848 ± 0,003	0,850 ± 0,003	+0,18

Ở cả bốn mô hình, độ chính xác gần như không đổi. MLP không đổi là hiển nhiên vì nó không dùng đồ thị. Điều đáng chú ý là GCN, mô hình được gọi là "chuyển dẫn", chỉ mất 0,09 điểm. Lý do: GCN cài đặt theo (6.3) học các ma trận $\mathbf{W}^{(l)}$ *dùng chung cho mọi nút*. Lúc suy luận, chỉ cần áp dụng lại cùng phép gộp trên đồ thị mới. Tính "chuyển dẫn" trong bài báo gốc là một thiết lập thí nghiệm (đặc trưng của nút không nhãn tham gia lan truyền khi huấn luyện), không phải giới hạn của kiến trúc. GraphSAGE vẫn tốt nhất ở cả hai chế độ, và lợi thế thật sự của nó trên đồ thị rất lớn nằm ở *lấy mẫu láng giềng* để huấn luyện theo lô nhỏ. Kết quả thực tiễn cho hệ thống: một bài báo mới chỉ cần danh sách trích dẫn và vector TF-IDF là gán nhãn được bằng mô hình đã huấn luyện, không cần huấn luyện lại.

6.5.3. Chi phí tính toán

Bảng 6.7. Số tham số và thời gian huấn luyện trung bình một lần chạy (GPU RTX 3050)

Mô hình	Tham số	Bán giám sát	Có giám sát	Độ phức tạp một tầng
Hồi quy logistic	1.503	0,45 s	0,34 s	$O(nd)$
MLP	32.259	0,36 s	1,41 s	$O(ndh)$
GCN	32.259	2,01 s	3,34 s	$O(ndh + \|E\|h)$
GraphSAGE-mean	64.451	2,79 s	9,94 s	$O(ndh + \|E\|d)$
GAT (8 đầu)	33.779	3,70 s	4,11 s	$O(ndhK + \|E\|hK)$

Phép gộp láng giềng chỉ tốn $O(|E|)$ phép toán trên mỗi chiều đặc trưng, nhờ ma trận kề rất thưa (44 nghìn cạnh trên 389 triệu cặp nút có thể có). Toàn bộ đồ thị 19.717 nút

nằm gọn trong bộ nhớ GPU 4 GB, và mỗi lần huấn luyện chỉ mất vài giây (Bảng 6.7). GAT đắt hơn GCN vì phải tính hệ số chú ý cho từng cạnh và từng đầu. GraphSAGE đắt nhất ở chế độ có giám sát vì nhân ma trận trung bình láng giềng với đặc trưng 500 chiều *trước* khi giảm chiều; đổi thứ tự (giảm chiều trước rồi gộp) như GCN sẽ rẻ hơn. Suy luận của GAT triển khai trên CPU cho cả đồ thị mất dưới 1 giây và chỉ chạy một lần khi khởi động dịch vụ; sau đó mỗi truy vấn chỉ là tra bảng kết quả.

6.5.4. Phân tích lỗi và trọng số chú ý của mô hình triển khai

Bảng 6.8. Ma trận nhầm lẫn của GAT triển khai trên 3.944 bài báo giữ riêng (20%)

Thật \ Dự đoán	Thực nghiệm	Típ 1	Típ 2	Recall
Thực nghiệm (802)	**647**	100	55	0,807
Típ 1 (1.551)	83	**1.233**	235	0,795
Típ 2 (1.591)	61	66	**1.464**	0,920

Lỗi lớn nhất là 235 bài típ 1 bị đoán thành típ 2. Hai nhóm dùng chung nhiều thuật ngữ lâm sàng (*insulin*, *glucos*, *patient*, *hba1c*), và nhiều bài báo bàn về cả hai loại ĐTĐ hoặc về biến chứng chung (võng mạc, thận). Lớp típ 2 dễ nhất (recall 0,92) nhờ các từ đặc trưng như *niddm*, *risk*, *obes*. Lớp thực nghiệm chủ yếu bị nhầm thành típ 1 (100 bài): nghiên cứu ĐTĐ típ 1 thường dùng mô hình chuột NOD, nên từ vựng giao thoa với nghiên cứu trên động vật.

Theo bậc của nút, accuracy thấp nhất ở các bài có 1–2 trích dẫn (0,830, nhóm chiếm 63% tập kiểm tra), và đạt 0,87–0,89 khi bậc từ 3 trở lên. Bài báo ít liên kết nhận ít thông tin từ láng giềng, nên mô hình phải dựa gần như hoàn toàn vào đặc trưng TF-IDF của chính nó, giống MLP. Đây chính là hiệu ứng "đồ thị bù cho thiếu thông tin" ở mức từng nút.

GAT có thực sự "chú ý có chọn lọc"? Lấy hệ số chú ý α_{vu} của tầng 1 (trung bình 8 đầu), nhân với $\deg(v) + 1$ để chuẩn hóa: giá trị 1 nghĩa là chia đều như nhau cho mọi láng giềng. Trung bình trên các cạnh nối hai bài *cùng nhãn* là 0,9996, trên các cạnh nối hai bài *khác nhãn* là 1,0001. Nói cách khác, GAT đã học được một phân phối chú ý gần như đồng đều, không "hạ trọng số" các láng giềng khác lớp. Hệ số tự chú ý (khuyên) trung bình là 0,34, xấp xỉ $1/(\deg+1)$ với bậc điển hình 1–2. Đây là một phát hiện quan trọng để diễn giải kết quả. Trên đồ thị có 80% cạnh cùng nhãn và bậc thấp, chú ý học được hội tụ về gần cách gộp chuẩn hóa theo bậc của GCN. Vì vậy GAT không vượt GCN

ở Bảng 6.4, và trọng số chú ý dùng để giải thích trên trang thử nghiệm chỉ nên được đọc như một tín hiệu yếu. Chú ý phát huy tác dụng rõ hơn trên đồ thị có nhiều cạnh nhiễu hoặc dị tính (heterophily).

6.5.5. Hạn chế

Thực nghiệm của chương có bốn hạn chế. (i) Đặc trưng TF-IDF trên 500 từ gốc là biểu diễn nghèo so với mô hình ngôn ngữ y sinh hiện đại (như các mô hình BERT huấn luyện trên PubMed). Với đặc trưng giàu hơn, khoảng cách giữa MLP và GNN có thể thu hẹp thêm. (ii) Pubmed-Diabetes là đồ thị đồng tính cao. Kết luận "GCN tốt hơn MLP" không tự động đúng với đồ thị dị tính, nơi các nút kề nhau thường khác nhãn. (iii) Các nhãn MeSH được gán theo chủ đề chính của bài báo, trong khi nhiều bài thuộc cả hai loại ĐTĐ, nên một phần lỗi típ 1 $\leftrightarrow$ típ 2 là do bản chất của nhãn. (iv) Mô hình triển khai mới chỉ phân loại các bài báo có sẵn trong đồ thị. Để gán nhãn tài liệu mới nạp vào KB, cần thêm bước trích danh sách tham khảo và tính TF-IDF theo đúng bộ từ vựng 500 từ. Thí nghiệm quy nạp ở trên cho thấy bước này khả thi mà không cần huấn luyện lại.

6.6. Một khung GNN chung cho ba đồ thị trích dẫn

Để kiểm tra các kết luận ở trên có phụ thuộc vào miền y sinh hay không, cùng một khung mã được chạy lại trên PubMed-Diabetes và hai đồ thị trích dẫn kinh điển của khoa học máy tính: **Cora** (2.708 bài báo, 7 chủ đề) và **CiteSeer** (3.327 bài báo, 6 chủ đề). Khung này kế thừa mã nguồn của thành viên nhóm 07, được bổ sung PubMed. Ba lớp GCN, GraphSAGE, GAT được *tự cài đặt* bằng phép nhân ma trận thưa của PyTorch (không dùng PyTorch Geometric, thư viện này chỉ dùng để nạp dữ liệu). Cả ba đồ thị dùng phép chia Planetoid chuẩn: 20 nút có nhãn mỗi lớp để huấn luyện, 500 nút validation, 1.000 nút test. Mỗi mô hình có 2 lớp (64 đơn vị ẩn, GAT 8 đầu × 8 đơn vị), dropout 0,5 (GAT 0,6), Adam ($\eta = 0{,}01$, weight decay $5 \cdot 10^{-4}$), tối đa 250 epoch, chọn epoch theo accuracy validation. Mỗi cấu hình chạy **5 hạt giống**.

Bảng 6.9. Accuracy phân loại nút trên ba đồ thị (trung bình ± độ lệch chuẩn qua 5 hạt giống; 1.000 nút test)

Mô hình	PubMed-Diabetes	Cora	CiteSeer
Đặc điểm đồ thị			
Số nút / cạnh	19.717 / 44.324	2.708 / 5.278	3.327 / 4.552
Đặc trưng / lớp / nút có nhãn	500 / 3 / 60	1.433 / 7 / 140	3.703 / 6 / 120
Bậc trung bình; độ đồng chất cạnh	4,5; 0,80	3,9; 0,81	2,7; 0,74
Hồi quy logistic (không đồ thị)	0,735	0,574	0,614
MLP 2 lớp (không đồ thị)	0,722 ± 0,005	0,585 ± 0,007	0,576 ± 0,007
GCN	**0,789** ± 0,005	0,821 ± 0,008	**0,721** ± 0,009
GraphSAGE-mean	0,765 ± 0,005	0,804 ± 0,006	0,699 ± 0,007
GAT	0,777 ± 0,004	**0,829** ± 0,011	0,715 ± 0,008
Lợi ích của đồ thị (GNN tốt nhất − MLP)	+6,7 điểm	+24,4 điểm	+14,5 điểm

Nhận xét. (i) Kết quả trên PubMed khớp với thực nghiệm riêng ở các mục trên (GCN 0,789 so với 0,780; MLP 0,722 so với 0,702), dù là một bản cài đặt độc lập. Kết luận "đồ thị giúp khi ít nhãn" vì vậy không phụ thuộc vào mã nguồn. (ii) Lợi ích của đồ thị *khác nhau rất nhiều giữa các miền*: +24 điểm trên Cora, +15 trên CiteSeer, nhưng chỉ +7 trên PubMed. Lý do là đặc trưng TF-IDF của PubMed (500 từ khóa y khoa) đã phân biệt khá tốt ba loại ĐTĐ (hồi quy logistic đạt 0,735), còn túi từ của Cora và CiteSeer yếu hơn nhiều (0,57–0,61). Đồ thị bù cho thông tin còn thiếu trong đặc trưng nút. (iii) Độ đồng chất cạnh (tỷ lệ cạnh nối hai bài cùng lớp) của cả ba đồ thị đều cao (0,74–0,81). Đây là điều kiện để phép trộn láng giềng có ích. CiteSeer có độ đồng chất thấp nhất và 48 nút cô lập, nên mọi GNN đều thấp nhất trên bộ này. (iv) Không có kiến trúc thắng tuyệt đối: GCN tốt nhất trên PubMed và CiteSeer, GAT tốt nhất trên Cora, nhưng khoảng cách giữa ba GNN (1–3 điểm) nhỏ hơn nhiều so với khoảng cách giữa có đồ thị và không có đồ thị.

6.7. GCN dưới dạng hợp các hàm

Một GCN 2 lớp cho mọi nút cùng lúc là hợp của các hàm trên *ma trận* đặc trưng $X \in \mathbb{R}^{N\times d}$:

$$\hat{Y} = (\text{softmax}\circ A\text{-}\circ L_2 \circ D \circ \text{ReLU} \circ A\text{-}\circ L_1 \circ D)(X), \qquad A\text{-}(H) = \hat{A}H, \quad \hat{A} = \tilde{D}^{-1/2}(A+I)\tilde{D}^{-1/2}, \tag{6.7}$$

trong đó $L_1(H) = HW_1$ $(d \to 64)$ và $L_2(H) = HW_2 + \mathbf{b}$ $(64 \to C)$ là các hàm affine áp lên từng nút *độc lập*, còn $A\text{-}$ là hàm *duy nhất* trộn thông tin giữa các nút: mỗi nút lấy trung bình có trọng số theo bậc của chính nó và các láng giềng. Cách viết này cho thấy ba điều:

- Bỏ hai hàm $A\text{-}$ thì (6.7) trở thành đúng MLP ở Bảng 6.9, cùng số tham số (32.259 trên PubMed). Toàn bộ 6,7 điểm chênh lệch đến từ hai phép nhân với ma trận thưa $\hat{A}$, vốn *không có tham số*.
- Hai lần $A\text{-}$ nghĩa là thông tin lan đi tối đa 2 bước: mỗi bài báo "nhìn thấy" các bài trích dẫn nó và các bài trích dẫn những bài đó. Chồng k lớp thì $\hat{A}^k$ tiến dần tới phân phối dừng của bước ngẫu nhiên trên đồ thị, mọi nút có biểu diễn giống nhau: đó là hiện tượng làm trơn quá mức đã phân tích ở mục trên.
- GraphSAGE và GAT chỉ thay hàm trộn: $A\text{-}$ được thay bằng "nối đặc trưng bản thân với trung bình láng giềng" (SAGE), hoặc bằng trung bình có trọng số *học được* α_{ij} (GAT). Các hàm còn lại trong (6.7) giữ nguyên.

Đoạn mã 6.2. Lớp GCN tự cài đặt: hàm trộn $\hat{A}HW$ bằng nhân ma trận thưa (`tl3/ch6_gnn.py`, rút gọn)

```python
def gcn_norm_adj(edge_index, n):
    """Â = D̃^{-1/2}(A + I)D̃^{-1/2} dưới dạng ma trận thưa (Kipf & Welling, 2017)."""
    ei = add_self_loops(edge_index, n)
    deg = torch.zeros(n).index_add_(0, ei[1], torch.ones(ei.size(1)))
    dinv = deg.pow(-0.5)
    w = dinv[ei[0]] * dinv[ei[1]]
    return torch.sparse_coo_tensor(ei.flip(0), w, (n, n)).coalesce()

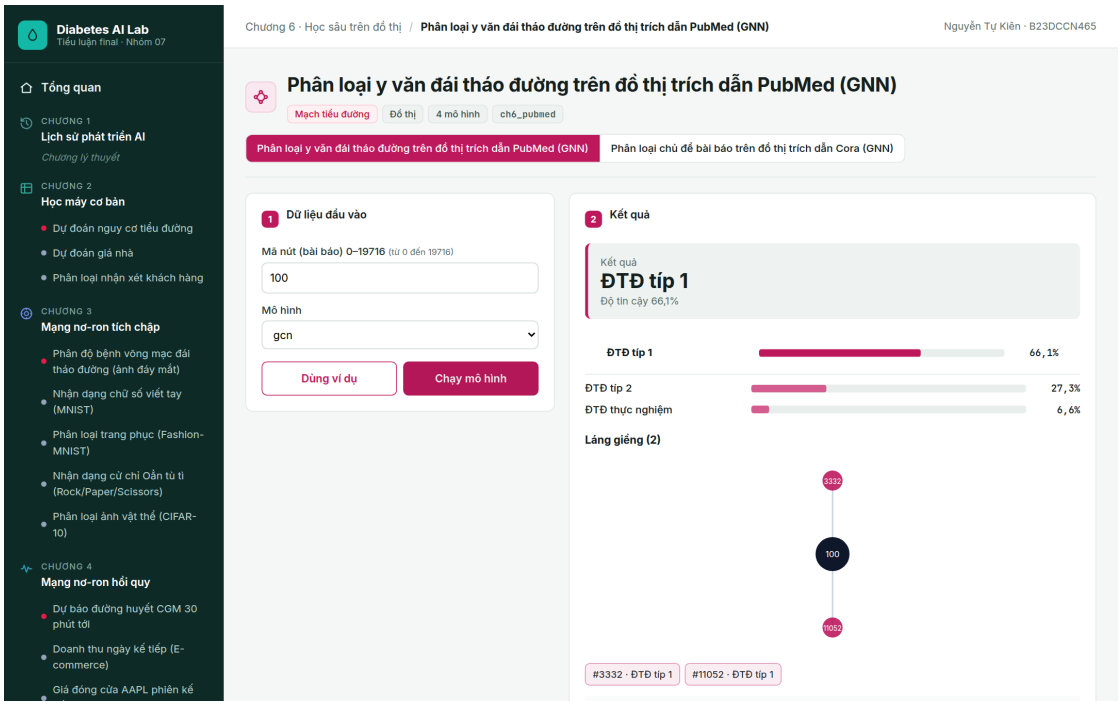
class GCNLayer(nn.Module):
    def __init__(self, d_in, d_out):
        super().__init__()
        self.lin = nn.Linear(d_in, d_out, bias=False)         # L: biến đổi từng nút, có tham
        ↪ số
        self.bias = nn.Parameter(torch.zeros(d_out))
    def forward(self, x, adj):
        return torch.sparse.mm(adj, self.lin(x)) + self.bias  # Â(HW): trộn láng giềng, không
        ↪ tham số
```

6.8. Thực nghiệm trên Notebook và Demo Localhost

Notebook `notebooks/ch6_gnn_pubmed.ipynb` (các ô `GNN-00` đến `GNN-04`) cho phép kiểm tra trực tiếp ma trận kề chuẩn hóa thưa và hiện tượng làm trơn quá mức theo độ sâu:

- **Ô `GNN-02` (Kiểm chứng ma trận $\hat{A}$ thưa và sự ngang bằng tham số giữa GCN và MLP)**: Ô `GNN-02` gọi `gcn_norm_adj(edge_index, n=19717)` trên đồ thị PubMed-Diabetes, tạo ra ma trận thưa $\hat{A} \in \mathbb{R}^{19.717 \times 19.717}$ dạng COO với đúng 108.365 phần tử khác không (2×44.324 cạnh vô hướng cộng 19.717 khuyên tự nối). Khi khởi tạo `GCN(500, 64, 3)` và `MLP(500, 64, 3)`, cả hai mô hình có tổng số tham số bằng hệt nhau là 32.259 ($500 \times 64 + 64 + 64 \times 3 + 3$). Điều này chứng minh trực tiếp rằng +6,7 điểm phần trăm accuracy của GCN so với MLP (0,789 so với 0,722) đến thuần túy từ cấu trúc đồ thị trích dẫn qua phép nhân $\hat{A}H$ chứ không phải do tăng dung lượng mô hình.
- **Ô `GNN-04` (Đo khoảng cách góc trung bình MAD và hiện tượng làm trơn quá mức từ 1 đến 8 lớp)**: Ô `GNN-04` đọc kết quả khảo sát độ sâu (`ch6_depth.json`) trên Cora từ 1 đến 8 tầng. Khi không có kết nối tắt (`residual=False`), accuracy đạt đỉnh ở 2 tầng (0,8213, khoảng cách góc trung bình giữa các nút kề $\text{MAD}_{\text{neighbors}} = 0{,}0412$), giảm nhẹ ở 3–4 tầng (0,8023 $\rightarrow$ 0,7440) và sụp đổ ở 6–8 tầng (0,363 $\rightarrow$ 0,281) khi biểu diễn các nút bị hòa trộn. Khi thêm kết nối tắt $H^{(l+1)} = \text{GCNLayer}(H^{(l)}) + H^{(l)}$ (`residual=True`), mạng 5–8 tầng giữ được tín hiệu bản thân và duy trì accuracy ở mức 0,771–0,792.

Trên ứng dụng web cục bộ tại `http://localhost:8095` (Hình 6.6), tab **Ch.6 — GNN · Đồ thị y văn PubMed-Diabetes** nạp đồng thời cả 4 mô hình đã huấn luyện trên PubMed (`GCN`, `GraphSAGE`, `GAT`, `MLP không dùng đồ thị`). Khi chọn một nút bài báo kiểm tra, giao diện hiển thị bậc của nút, tỷ lệ láng giềng cùng chủ đề và bảng so sánh xác suất dự đoán của cả bốn mô hình, cho thấy rõ trường hợp MLP đoán sai khi chỉ nhìn từ khóa của một bài báo nhưng ba mô hình GNN đều đoán đúng nhờ thông tin từ các bài trích dẫn láng giềng.



Hình 6.6. Giao diện thử nghiệm Chương 6 trên `http://localhost:8095`: so sánh dự đoán của GCN, GraphSAGE, GAT và MLP (không dùng đồ thị) trên cùng một bài báo của đồ thị trích dẫn PubMed-Diabetes

6.9. Triển khai

Mô hình triển khai là GAT huấn luyện theo giao thức có giám sát (accuracy 0,848 trên 20% nút giữ riêng). GAT được chọn thay vì GraphSAGE vì trọng số chú ý cho phép *giải thích* dự đoán bằng các bài báo láng giềng quan trọng nhất. Trọng số mô hình (`ch6_gat.pt`, 170 KB) và đồ thị nén (`ch6_pubmed_graph.npz`: đặc trưng float16, danh sách cạnh, mã PubMed) được nạp bởi lớp `PubmedGraphClassifier` (`src/models/deep_graph.py`). Endpoint `GET /api/lab/ch6/paper?pid=<PubMed ID>` trả về nhãn thật, nhãn dự đoán, xác suất ba lớp, các từ khóa nổi bật, và tối đa 8 bài báo láng giềng xếp theo hệ số chú ý α trung bình 8 đầu, kèm liên kết tới PubMed. Không truyền `pid` thì endpoint chọn ngẫu nhiên một bài thuộc tập kiểm tra.

Đoạn mã 6.3. Gọi endpoint Chương 6

```
curl -s "https://agent.nits.io.vn/api/lab/ch6/paper?pid=12187484"
```

6.10. Tiểu kết chương 6

Chương 6 trình bày khung truyền thông điệp thống nhất các mạng nơ-ron đồ thị, cùng ba kiến trúc với cách gộp láng giềng khác nhau: GCN (chuẩn hóa đối xứng theo

bậc), GraphSAGE (tách biểu diễn bản thân và láng giềng), GAT (trọng số chú ý học được). Trên đồ thị trích dẫn 19.717 bài báo về ĐTĐ: (1) với 60 nhãn, GCN tăng 7,8 điểm accuracy so với MLP cùng số tham số, và cao hơn hẳn việc chỉ dùng đặc trưng hoặc chỉ dùng đồ thị; (2) lợi ích của đồ thị lớn nhất khi nhãn khan hiếm, giảm dần khi có nhiều nhãn; (3) GNN nông là đủ, từ 6 tầng mô hình sụp về một lớp do làm trơn quá mức cộng với khó tối ưu. Đồ thị giúp phân loại và tổ chức tri thức y văn, bổ sung cho đồ thị tri thức ở Chương 5.

CHƯƠNG 7. HỆ ĐA TÁC TỬ

Sáu chương trước xây dựng các "chuyên gia" riêng lẻ: một mô hình cho bảng, một cho ảnh, một cho chuỗi, một cho văn bản, một cho đồ thị. Chương này hỏi câu hỏi còn lại: khi *nhiều* tác tử cùng hành động trong một môi trường, chúng phối hợp, cạnh tranh và học như thế nào. Chương gồm hai bài thực nghiệm bổ sung cho nhau. **Bài 1** là hệ đa tác tử *học tăng cường* kinh điển: mèo đuổi chuột trên lưới, trong đó các tác tử học chính sách từ phần thưởng, hợp tác (hai mèo) hoặc đối kháng (mèo và chuột cùng học). **Bài 2** là hệ đa tác tử dựa trên *mô hình ngôn ngữ lớn*: nhiều tác tử LLM cùng trả lời câu hỏi trắc nghiệm y khoa về đái tháo đường bằng cách suy luận, tra cứu và tranh luận.

7.1. Từ tác tử đơn đến hệ đa tác tử

7.1.1. Tác tử

Theo Russell và Norvig [11], *tác tử* là thực thể nhận tri giác từ môi trường qua cảm biến và tác động lên môi trường qua cơ cấu chấp hành. Một tác tử được đặc tả bằng bộ **PEAS**: thước đo hiệu năng (Performance), môi trường (Environment), cơ cấu chấp hành (Actuators) và cảm biến (Sensors). Wooldridge [100] nhấn mạnh bốn tính chất của tác tử thông minh: *tự chủ*, *phản ứng*, *chủ động* và *xã hội*. Theo cách ra quyết định, có năm loại tác tử tăng dần độ phức tạp: phản xạ đơn giản (luật điều kiện–hành động), phản xạ có mô hình, dựa trên mục tiêu, dựa trên lợi ích và tác tử học.

Bảng 7.1. Đặc tả PEAS của các tác tử trong hai bài thực nghiệm

Tác tử	Hiệu năng	Môi trường	Chấp hành	Cảm biến
Mèo (Bài 1)	Tỷ lệ bắt được, số bước, tổng phần thưởng	Lưới 8×8 có tường, chuột, mèo khác	Đi lên/xuống/trái/phải	Ô của mình, ô của chuột (và mèo kia)
Chuột (Bài 1)	Số lượt sống sót	Lưới, mèo	Đi 4 hướng hoặc đứng yên	Ô của mình, ô của mèo
Tác tử LLM (Bài 2)	Tỷ lệ trả lời đúng, chi phí (lượt gọi, token)	Câu hỏi, kho tri thức, tác tử khác	Sinh văn bản, gọi công cụ tra cứu	Câu hỏi, bằng chứng, lập luận của tác tử khác

7.1.2. Hệ đa tác tử

Hệ đa tác tử (Multi-Agent System, MAS) gồm nhiều tác tử tương tác trong cùng một môi trường [100], [101]. Có ba kiểu quan hệ: *hợp tác* (cùng mục tiêu, ví dụ hai mèo cùng bắt một chuột), *cạnh tranh / đối kháng* (mục tiêu ngược nhau, mèo và chuột) và *hỗn hợp*. Các tác tử phối hợp qua những mô hình tổ chức như *điều phối viên* (một tác tử giao việc và tổng hợp), *bảng đen* (vùng nhớ dùng chung), *đấu thầu* (Contract Net) hoặc *tranh biện ngang hàng*, và giao tiếp bằng thông điệp có ý định rõ ràng như chuẩn FIPA-ACL [102].

Học trong hệ đa tác tử khó hơn học đơn tác tử ở một điểm cốt lõi: *môi trường không dừng*. Với một tác tử, các tác tử khác là một phần của môi trường. Khi chúng cũng đang học, "môi trường" thay đổi theo thời gian, nên các giả thiết hội tụ của Q-learning đơn tác tử không còn đúng [103]. Hai cách tiếp cận cơ bản là *học độc lập* (Independent Q-Learning, IQL: mỗi tác tử tự học như thể tác tử khác là môi trường [104]) và mô hình hóa tường minh bằng *trò chơi Markov* [105]. Bài 1 dùng IQL và đo trực tiếp hệ quả của tính không dừng.

Tác tử dựa trên LLM dùng mô hình ngôn ngữ làm "bộ não": suy luận từng bước (chain-of-thought [106]), gọi công cụ (tra cứu, chạy mã [107]) và trao đổi bằng ngôn ngữ tự nhiên. Các khung như AutoGen [108] và MetaGPT [109] ghép nhiều tác tử LLM với vai trò khác nhau; Du và cộng sự [110] cho thấy tranh luận giữa nhiều LLM cải thiện tính đúng đắn. Bài 2 kiểm tra điều đó với một LLM nhỏ chạy cục bộ.

7.2. Bài 1: Hệ đa tác tử học tăng cường — mèo đuổi chuột

7.2.1. Môi trường và bài toán

Môi trường là lưới 8×8 với 10 ô tường, còn 54 ô trống (Hình 7.1). Mỗi lượt, mèo đi trước một bước theo 4 hướng; nếu chưa bắt được thì chuột di chuyển. Ván kết thúc khi mèo vào ô của chuột hoặc sau 100 lượt. Đây là một *quá trình quyết định Markov* với:

- Trạng thái $s = (\text{ô mèo},\ \text{ô chuột})$: $54 \times 54 = 2.916$ trạng thái (với hai mèo: $54^3 = 157.464$).
- Hành động $a \in \{\text{lên}, \text{xuống}, \text{trái}, \text{phải}\}$.
- Phần thưởng: -1 mỗi bước (khuyến khích bắt nhanh), -3 khi đâm tường (đứng yên), $+100$ khi bắt được.
- Chuột "luật chạy trốn": mỗi lượt di chuyển với xác suất 0,8; khi di chuyển, 80% chọn ô xa mèo nhất theo khoảng cách đường đi (BFS, có tính tường), 20% đi ngẫu

nhiên.

Mã nguồn môi trường và ba tác tử mèo cơ sở là bài tập Agent trước đó của tác giả; chương này mở rộng thành hai thí nghiệm đa tác tử.

7.2.2. *Ba loại tác tử mèo*

- **Ngẫu nhiên** — mốc dưới.
- **Theo luật** (phản xạ đơn giản): chọn hướng làm khoảng cách Manhattan tới chuột giảm nhiều nhất. Luật này không biết tường nên dễ kẹt.
- **Q-learning** (tác tử học) [111], [112]: học bảng $Q(s,a)$ bằng cập nhật sai phân thời gian

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big], \quad \alpha = 0{,}2,\ \gamma = 0{,}95, \tag{7.1}$$

với chính sách ϵ-tham lam, ϵ giảm tuyến tính từ 1 về 0,05 trong 70% số tập đầu.

Đoạn mã 7.1. Vòng huấn luyện Q-learning của tác tử mèo (`src/dm/catmouse_base.py`, rút gọn)

```
for ep in range(episodes):
    eps = max(eps_min, 1.0 - ep / decay_until)           # khám phá giảm dần
    cat, mouse = env.random_start(rng)
    s = env.state(cat, mouse)
    while n < env.max_steps:
        q = Q[s]
        a = rng.randrange(4) if rng.random() < eps else q.index(max(q))   # epsilon-tham lam
        cat, mouse, r, caught = env.step(cat, mouse, a, rng)
        s2 = env.state(cat, mouse)
        target = r if caught else r + gamma * max(Q[s2])  # mục tiêu Bellman; trạng thái cuối
         ↪  không có tương lai
        q[a] += alpha * (target - q[a])
        s, n = s2, n + 1
        if caught:
            break
```

7.2.3. *Hai mở rộng đa tác tử*

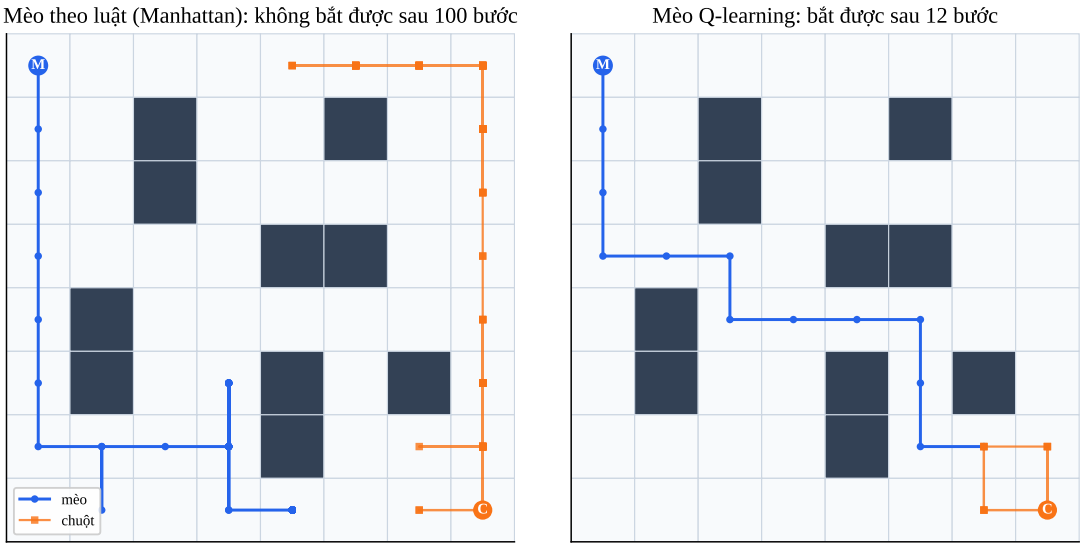
Hợp tác — hai mèo. Thêm một mèo; hai mèo đi lần lượt, rồi chuột chạy xa mèo *gần nhất*. Hai mèo nhận chung một phần thưởng (bắt được bởi mèo nào cũng +100 cho cả đội). Mỗi mèo học một bảng Q riêng trên trạng thái chung (mèo 1, mèo 2, chuột) theo kiểu *học độc lập* (IQL): mèo này coi mèo kia là một phần của môi trường. So sánh với hai mèo cùng dùng luật Manhattan.

Đối kháng — cả hai cùng học. Chuột không còn theo luật mà cũng là một tác tử Q-learning, với phần thưởng ngược: +1 mỗi lượt sống sót, −100 khi bị bắt. Kết quả nước đi của chuột chỉ biết sau lượt kế tiếp của mèo, nên cập nhật Q của chuột được hoãn một lượt. Hai bảng Q học đồng thời trong 30.000 tập. Sau đó bốn cặp (mèo × chuột) được đấu với nhau trên cùng 300 vị trí xuất phát.

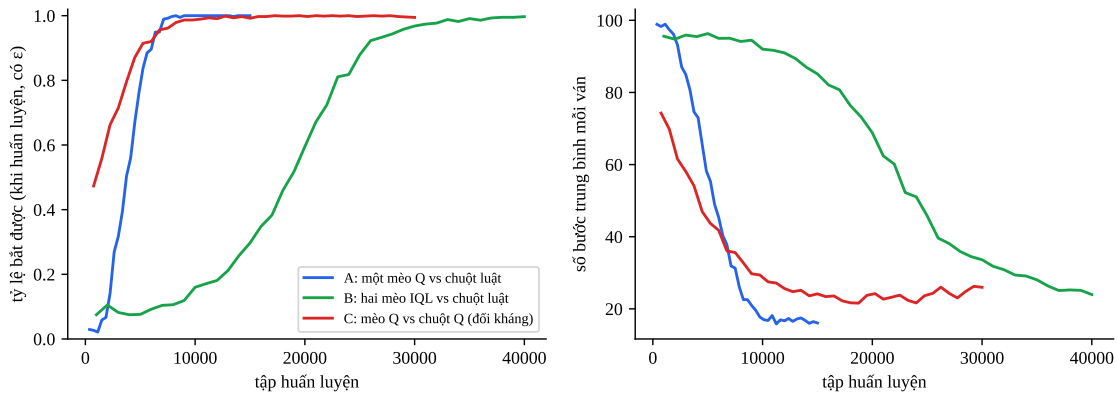
7.2.4. Kết quả

Bảng 7.2. Kết quả ba thí nghiệm mèo đuổi chuột (300 ván kiểm tra, vị trí xuất phát cố định, chính sách tham lam)

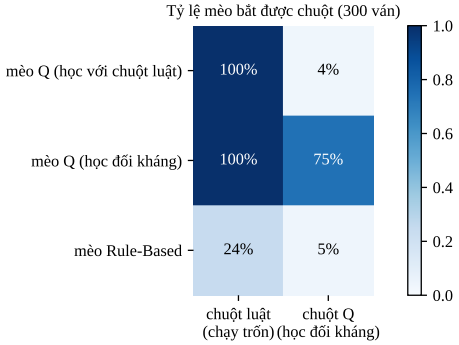
Thí nghiệm	Tác tử mèo	Bắt được	Số bước TB	Huấn luyện
A. Một mèo vs chuột luật	Ngẫu nhiên	2,3%	98,5	—
	Theo luật (Manhattan)	24,3%	82,5	—
	Q-learning (15.000 tập)	**100%**	**15,3**	10 s
B. Hai mèo vs chuột luật	Hai mèo theo luật	52,0%	61,9	—
	Hai mèo IQL (40.000 tập)	**99,7%**	**22,1**	78 s



Hình 7.1. Một ván từ cùng vị trí xuất phát: mèo theo luật kẹt sau tường; mèo Q-learning vòng qua tường và bắt được sau 12 bước



Hình 7.2. Đường cong học (tỷ lệ bắt được và số bước theo số tập, khi huấn luyện còn khám phá ϵ)



Hình 7.3. Thí nghiệm C — ma trận đối kháng: hàng là mèo, cột là chuột

Nhận xét. (i) *Học thắng luật khi môi trường có cấu trúc*: luật Manhattan đúng trên lưới trống nhưng không biết tường, nên chỉ bắt 24%. Q-learning học được giá trị của từng cặp vị trí, tức đã ngầm học bản đồ, nên bắt 100% trong trung bình 15 bước (Hình 7.1). (ii) *Hợp tác*: thêm một mèo theo luật nâng tỷ lệ bắt từ 24% lên 52%, vì hai mèo tiếp cận từ hai phía nên chuột khó thoát hơn. Hai mèo IQL đạt 99,7%, nhưng cần 40.000 tập (gấp 2,7 lần) và học chậm hơn hẳn ở giai đoạn đầu (Hình 7.2): không gian trạng thái lớn gấp 54 lần, và mỗi mèo học trong một môi trường thay đổi theo chính sách của mèo kia. (iii) *Đối kháng và tính không dừng* (Hình 7.3) là kết quả quan trọng nhất. Mèo Q-learning học với chuột luật bắt 100% chuột luật, nhưng chỉ **4%** chuột đã học: chính sách quá khớp với đúng một đối thủ. Mèo học đối kháng bắt 100% chuột luật *và* 75% chuột đã học. Đây là minh họa trực tiếp cho lý thuyết: trong hệ đa tác tử, một chính sách chỉ "tối ưu" so với chính sách của các tác tử còn lại, và phải được huấn luyện cùng các đối thủ đang thích nghi thì mới bền vững. Chuột đã học cũng đánh bại mèo theo luật (5%), cho thấy một tác tử học dễ dàng khai thác điểm yếu cố định của tác tử luật.

7.3. Bài 2: Hệ đa tác tử LLM — trắc nghiệm đái tháo đường

7.3.1. Bài toán và dữ liệu

Câu hỏi lấy từ **MedMCQA** [10], bộ đề thi tuyển sinh sau đại học y khoa của Ấn Độ: lọc 1.421 câu một đáp án có chủ đề đái tháo đường, insulin, HbA1c, hạ đường huyết, metformin, nhiễm toan ceton; lấy mẫu cố định **150 câu** (nội khoa 35, dược lý 31, sản 12, hóa sinh 10…). Mỗi câu có 4 phương án A–D và một đáp án đúng. Kho tri thức cho tác tử tra cứu là **MedQuAD — phần NIDDK** [8] (1.192 cặp hỏi–đáp của Viện Đái tháo đường, Tiêu hóa và Bệnh thận Hoa Kỳ), chỉ mục BM25. Mọi tác tử dùng chung một LLM chạy cục bộ trên GPU 4 GB: **Qwen2.5-1.5B-Instruct** [113], giải mã tham lam để kết quả tái lập được.

7.3.2. Bốn hệ và đặc tả tác tử

Bảng 7.3. Bốn hệ được so sánh trên cùng 150 câu

Hệ	Tác tử	Luồng xử lý
Direct	1 tác tử	Đọc câu hỏi, trả lời ngay một chữ cái
CoT	1 tác tử	Suy luận từng bước (tối đa 6 câu), rồi trả lời
RAG-agent	1 tác tử + công cụ	Gọi công cụ tra cứu BM25 lấy 3 đoạn NIDDK, suy luận có trích dẫn, rồi trả lời
Đa tác tử	Solver-CoT, Solver-RAG, Critic, Solver-CoT#2, Điều phối	Hai solver giải độc lập. Đồng thuận thì chấp nhận ngay; bất đồng thì Critic đọc cả hai lập luận cùng bằng chứng để phân xử, thêm một mẫu CoT ở nhiệt độ 0,7, rồi biểu quyết đa số (hòa phiếu thì tin Critic)

Trạng thái dùng chung của một lượt giải là *vết* (`Trace`): danh sách các bước `(vai trò, nội dung)`, số lượt gọi LLM, số token vào/ra và thời gian. Mỗi tác tử là một vai trò (system prompt) dùng chung LLM; mỗi lần `act()` là một thông điệp được ghi vào vết. Khung `Agent`/`Trace` kế thừa mã nguồn của thành viên nhóm 07; bài toán, dữ liệu, công cụ tra cứu và giao thức tranh luận là của tiểu luận này.

7.3.3. *Hệ đa tác tử dưới dạng hợp các hàm*

Gọi q là câu hỏi, $E = \text{search}(q)$ là bằng chứng tra cứu. Hai solver là hai hàm $a_1 = f_{\text{CoT}}(q)$ và $a_2 = f_{\text{RAG}}(q, E)$. Hệ đa tác tử là

$$\text{MAS}(q) = \begin{cases} a_1 & \text{nếu } a_1 = a_2, \\ \text{vote}(a_1,\, a_2,\, f_{\text{Critic}}(q, E, a_1, a_2),\, f_{\text{CoT}}^{T=0,7}(q)) & \text{ngược lại,} \end{cases} \tag{7.2}$$

trong đó vote là biểu quyết đa số, hòa phiếu thì lấy phiếu của Critic. Điều kiện dừng rất đơn giản: đồng thuận ở vòng một thì dừng sau 2 lượt gọi LLM; bất đồng thì dừng sau đúng 4 lượt. Không có vòng lặp mở nên chi phí bị chặn.

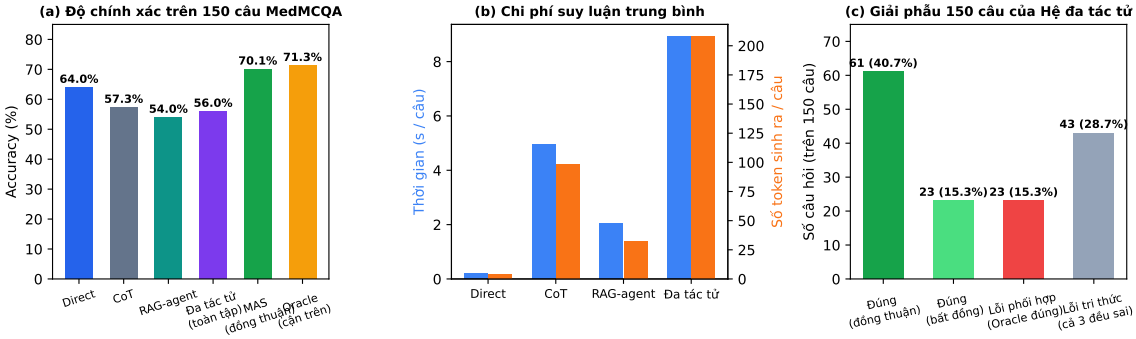
Đoạn mã 7.2. Hệ đa tác tử trả lời trắc nghiệm (`src/dm/ch7_medqa.py`, rút gọn)

```
def solve_multi(llm, row, tool):
    tr, q = Trace(), format_q(row)
    cot_text = Agent(llm, "Solver-CoT", SYS_COT).act(q, tr)
    a_cot = to_letter(cot_text)
    ev = evidence_block(tool, row, tr)                    # công cụ: BM25 trên MedQuAD-NIDDK
    rag_text = Agent(llm, "Solver-RAG", SYS_RAG).act(f"Reference passages:\n{ev}\n\n{q}", tr)
    a_rag = to_letter(rag_text)
    if a_cot and a_cot == a_rag:                          # đồng thuận -> dừng sớm
        return a_cot, tr
    critic = to_letter(Agent(llm, "Critic", SYS_CRITIC).act(
        f"{q}\n\n{ev}\n\nColleague 1 chose {a_cot}:\n{cot_text}\n\nColleague 2 chose
         ↪  {a_rag}:\n{rag_text}", tr))
    extra = to_letter(Agent(llm, "Solver-CoT#2", SYS_COT, temperature=0.7).act(q, tr))
    votes = Counter(v for v in (a_cot, a_rag, critic, extra) if v)
    best, n = votes.most_common(1)[0]
    return (critic or best) if n <= 1 else best, tr       # hòa phiếu: tin Critic
```

7.3.4. *Kết quả và phân tích lỗi*

Bảng 7.4. Kết quả bốn hệ tác tử trên 150 câu hỏi trắc nghiệm đái tháo đường MedMCQA (Qwen2.5-1.5B-Instruct chạy cục bộ)

Hệ	Đúng / 150	Accuracy	Lượt gọi / câu	Token ra / câu	Thời gian / câu	Phân loại lỗi (66–69 câu sai)
Direct	96	**64,0%**	1,00	4,0	**0,22 s**	54 tri thức
CoT	86	57,3%	1,00	98,5	4,96 s	63 tri thức, 1 không đáp án
RAG-agent	81	54,0%	1,00	31,7	2,04 s	69 tri thức
Đa tác tử (MAS)	84	56,0%	2,84	207,9	8,92 s	43 tri thức, **23 phối hợp**
MAS — nhóm đồng thuận vòng 1 (87/150 câu, 58,0%)		**70,1%**	2,00	112,4	6,18 s	26 cả hai cùng sai
MAS — nhóm bất đồng (63/150 câu, 42,0%)		36,5%	4,00	339,8	12,71 s	17 tri thức, 23 phối hợp
Cận trên Oracle (≥ 1 trong CoT/RAG/Critic đúng)		**71,3%**	—	—	—	43 không tác tử nào biết



Hình 7.4. Hiệu năng, chi phí và cơ cấu đồng thuận/lỗi của bốn hệ tác tử trên 150 câu trắc nghiệm đái tháo đường MedMCQA

Nhận xét. Kết quả trên Bảng 7.4 và Hình 7.4 cho ba phát hiện thực nghiệm rất đáng chú ý khi dùng một LLM nhỏ (1,5 tỷ tham số) cho bài toán y khoa:

1. *Trên câu hỏi trắc nghiệm y khoa đòi hỏi hồi tưởng tri thức sự kiện, trả lời trực tiếp (`Direct`) đạt 64,0%, cao hơn `CoT` (57,3%) và `RAG-agent` (54,0%).* Khác với bài toán tính toán nhiều bước, câu hỏi MedMCQA thường hỏi trực tiếp một mối liên hệ lâm sàng hoặc dược lý (cơ chế thuốc, kiểu gen HLA, tiêu chuẩn chẩn đoán). Ở chế độ `Direct`, LLM chọn ngay phương án có xác suất tiên nghiệm cao nhất từ trọng số tiền huấn luyện. Ngược lại, khi ép mô hình 1,5B viết chuỗi suy luận dài (`CoT`), mô hình thường "nghĩ quá mức" (*overthinking*): tự sinh ra một giả định sinh lý bệnh sai ở câu thứ hai rồi bị chính lập luận đó dẫn sang đáp án nhiễu. Tương tự, kho MedQuAD-NIDDK là tài liệu hỏi–đáp phổ biến kiến thức cho bệnh nhân, trong khi đề thi MedMCQA hỏi ở mức bác sĩ nội trú; khi 3 đoạn BM25 chỉ chứa thông tin tổng quát, `Solver-RAG` dễ bị các từ khóa bề mặt trong đoạn văn kéo lệch khỏi đáp án đúng.
2. *Cổng đồng thuận hai cơ chế độc lập là bộ lọc độ tin cậy rất mạnh.* Trong 150 câu, có **87 câu (58,0%)** mà `Solver-CoT` (dựa vào bộ nhớ tham số) và `Solver-RAG` (dựa vào văn bản NIDDK) độc lập chọn cùng một chữ cái ngay ở vòng 1. Trên 87 câu này, độ chính xác đạt **70,1%** (61/87) — vượt `Direct` 6,1 điểm phần trăm với chi phí đúng 2 lượt gọi LLM mỗi câu. Điều này cho thấy khi suy luận nội tại và bằng chứng truy xuất đồng thuận, xác suất đúng tăng vọt.
3. *Khoảng cách giữa cận trên Oracle (71,3%) và kết quả thực tế của MAS (56,0%) nằm ở 23 lỗi phối hợp.* Tính trên cả 150 câu, có tới **107 câu (71,3%)** mà ít nhất một trong ba tác tử (`Solver-CoT`, `Solver-RAG` hoặc `Critic`) chọn đúng đáp án, chỉ có 43 câu (28,7%) là lỗi tri thức thuần túy (cả ba đều sai). Tuy nhiên, ở 63 câu bất đồng vòng 1, độ chính xác sau phân xử chỉ đạt **36,5%** (23/63), làm phát sinh **23**

lỗi phối hợp (đã có tác tử đúng nhưng quyết định cuối cùng lại chọn phương án sai). Nguyên nhân cốt lõi là `Critic` dùng cùng một mô hình 1,5B với các solver: khi không có công cụ kiểm chứng tất định (như trình thông dịch code ở bài toán toán học), `Critic` thường bị thuyết phục bởi lập luận nào viết trôi chảy hơn thay vì lập luận đúng về y khoa.

Phân tích vết tranh luận thật (`outputs/ch7/rows_multi.jsonl`). Hai câu cụ thể trong tập kiểm tra minh họa rõ khi nào các tác tử cứu nhau:

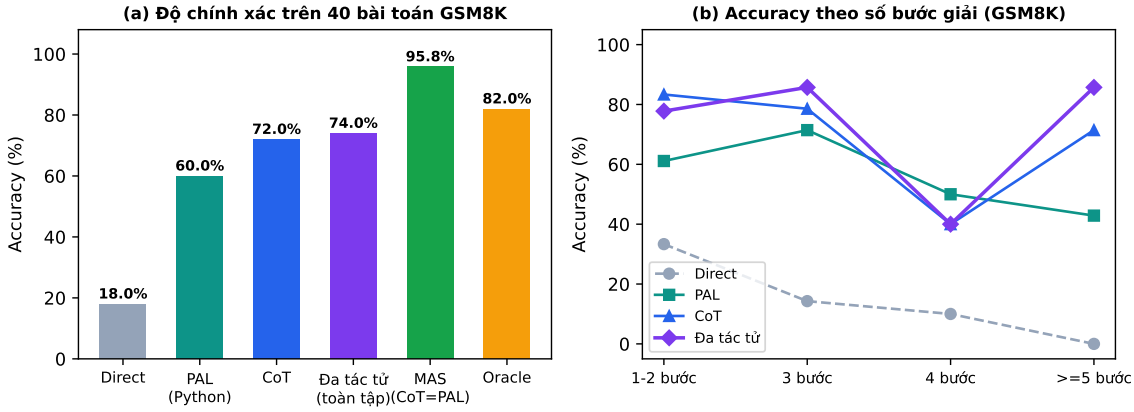
- *Câu `#18` (Nhi khoa — `RAG` cứu `CoT`): "Which of the following is a risk factor for developing Type 1 Diabetes mellitus?" (A: Male Gender, B: Old age, C: Gestational diabetes, D: HLA DR3; đáp án đúng: **D**).* `Solver-CoT` suy luận chung chung về bệnh tự miễn rồi chọn nhầm `C` (vốn là yếu tố nguy cơ của ĐTĐ típ 2). Công cụ BM25 truy xuất đoạn *Causes of Diabetes* từ MedQuAD-NIDDK nêu rõ vai trò của các biến thể gen HLA trong ĐTĐ típ 1, giúp `Solver-RAG` chọn đúng `D`. Khi hai bên bất đồng, `Critic` đọc đoạn trích NIDDK, bác bỏ phương án `C` và chốt đáp án đúng `D`.
- *Câu `#101` (Nội khoa — `CoT` và `Critic` cứu `RAG`): "Diabetes mellitus is diagnosed when:" (A: Fasting glucose* > 100*, B: HbA1c* > 6,5%*, C: Postprandial* > 140*, D: Fasting* > 110*; đáp án đúng: **B**).* Ba đoạn MedQuAD trả về nói về quy trình lấy máu xét nghiệm A1C và FPG mà không chứa đúng con số ngưỡng ở câu đầu, khiến `Solver-RAG` lúng túng chọn `D`. Ngược lại, `Solver-CoT`, `Critic` và `Solver-CoT#2` đều nhớ chắc ngưỡng chuẩn ADA HbA1c ≥ 6,5% và biểu quyết 3–1 chọn đúng `B`.

7.3.5. Đối chiếu: giải toán GSM8K

Để kiểm chứng giả thuyết rằng hiệu quả của suy luận từng bước và đa tác tử phụ thuộc vào *bản chất bài toán* và *khả năng kiểm chứng của công cụ*, cùng mô hình **Qwen2.5-1.5B-Instruct** được chạy trên **40 bài toán đố tiếng Anh GSM8K** [114] (lấy mẫu phân tầng theo số bước giải từ khung mã kế thừa của nhóm 07). Ở miền này, công cụ không phải là BM25 mà là **trình thông dịch Python** (`PAL` [107]): tác tử sinh hàm `solution()` và thực thi trong hộp cát để lấy kết quả số học chính xác. Hệ đa tác tử gồm `Planner`, `Reasoner` (CoT), `Coder` (viết Python, có tối đa 1 lượt sửa lỗi nếu code không chạy được), `Verifier` (kiểm tra đồng thuận giữa `Reasoner` và `Coder`) và `Judge` (phân xử khi hai nhánh ra hai số khác nhau).

Bảng 7.5. Đối chiếu: cùng LLM Qwen2.5-1.5B-Instruct trên 40 bài toán GSM8K và 150 câu y khoa MedMCQA

Miền bài toán	Hệ	Accuracy	Lượt gọi	Token ra	Thời gian / câu	Cơ cấu lỗi chính
GSM8K (40 câu,	Direct	17,5%	1,00	3,6	0,11 s	33 suy luận
toán nhiều bước +	CoT	70,0%	1,00	268,7	5,90 s	10 suy luận, 2 số học
công cụ Python)	PAL (1 tác tử + Python)	55,0%	1,33	119,5	2,67 s	17 suy luận, 1 không đáp án
	Đa tác tử (CoT + PAL + Judge)	**72,5%**	5,00	757,4	10,99 s	6 suy luận, 1 số học, 4 phối hợp
	— khi CoT = PAL (18/40 câu, 45%)	**94,4%**	3,78	—	6,90 s	*Oracle toàn tập: 82,5%*
MedMCQA (150 câu,	Direct	**64,0%**	1,00	4,0	0,22 s	54 tri thức
trắc nghiệm ĐTĐ +	CoT	57,3%	1,00	98,5	4,96 s	63 tri thức, 1 không đáp án
công cụ BM25)	RAG-agent (1 tác tử + BM25)	54,0%	1,00	31,7	2,04 s	69 tri thức
	Đa tác tử (CoT + RAG + Critic)	56,0%	2,84	207,9	8,92 s	43 tri thức, 23 phối hợp
	— khi CoT = RAG (87/150 câu, 58%)	**70,1%**	2,00	112,4	6,18 s	*Oracle toàn tập: 71,3%*



Hình 7.5. Kết quả bốn hệ tác tử trên bài toán giải toán nhiều bước GSM8K (Qwen2.5-1.5B-Instruct)

Nhận xét đối chiếu. Bảng 7.5 cho thấy bức tranh đối xứng rất sáng tỏ giữa hai miền:

- *Tính toán chuỗi bước (GSM8K) đối lập với hồi tưởng sự kiện (MedMCQA)*: Trên GSM8K, `Direct` chỉ đạt **17,5%** (và 0% ở các bài từ 4 bước trở lên), vì mạng Transformer không thể nhẩm nhiều phép tính số học nối tiếp chỉ trong một bước sinh token. `CoT` nâng accuracy vọt lên **70,0%** (+52,5 điểm), và hệ đa tác tử đạt đỉnh **72,5%** (ở nhóm bài khó ≥ 5 bước, đa tác tử đạt 80% so với 60% của CoT và 20% của PAL). Ngược lại, ở MedMCQA, câu trả lời nằm sẵn trong liên kết khái niệm nên `Direct` đã đạt 64,0%, còn suy luận dài của LLM nhỏ lại tạo nhiễu.
- *Công cụ tất định (Python) so với công cụ bất định (BM25 văn bản)*: Khi `Coder` viết được hàm Python chạy ra một con số trùng với kết quả suy luận bằng lời của `Reasoner` (18/40 bài GSM8K), độ chính xác đạt tới **94,4%** (17/18 bài đúng) — vì xác suất hai cơ chế hoàn toàn khác bản chất (ngôn ngữ tự nhiên và thực thi mã máy) tình cờ mắc cùng một lỗi số học là cực thấp. Trên MedMCQA, cổng đồng thuận

`CoT = RAG` cũng nâng độ chính xác lên **70,1%**, nhưng khi bất đồng thì công cụ BM25 không trả về kết quả "đúng/sai" tất định như Python nên `Critic` khó phân xử hơn nhiều (accuracy nhóm bất đồng 36,5% so với 54,5% trên GSM8K).

7.4. So sánh một tác tử và đa tác tử

Bảng 7.6. Một tác tử và đa tác tử qua hai bài thực nghiệm

Tiêu chí	Một tác tử	Đa tác tử
Chất lượng	Bài 1: một mèo Q bắt 100% chuột luật	Bài 1: hai mèo theo luật bắt gấp đôi một mèo (52% so với 24%); huấn luyện đối kháng cho chính sách bền vững hơn (75% so với 4%). Bài 2: thắng trên toán nhiều bước GSM8K (72,5% so với 70,0% CoT, 17,5% Direct); trên MedMCQA, nhóm đồng thuận đạt 70,1% (Oracle 71,3%) nhưng toàn tập đạt 56,0% do lỗi phân xử của Critic 1,5B
Chi phí	Thấp: một bảng Q, một lượt gọi LLM (0,1–5 s)	Cao: không gian trạng thái ×54 (hai mèo); 2,84–5,00 lượt gọi LLM mỗi câu (9–11 s)
Học	Môi trường dừng, Q-learning hội tụ	Môi trường không dừng: đối thủ thay đổi, chính sách quá khớp với một đối thủ (4% so với 100%)
Giải thích được	Một chuỗi suy luận	Vết đầy đủ: ai nói gì, bất đồng ở đâu, Critic phân xử thế nào
Lỗi đặc thù	Thiếu tri thức, ảo giác, tính nhẩm sai	Thêm lỗi phối hợp (23/150 câu MedMCQA, 4/40 câu GSM8K): đã có tác tử đúng nhưng cơ chế tổng hợp chọn sai

7.5. Thực nghiệm trên Notebook và Demo trên Localhost

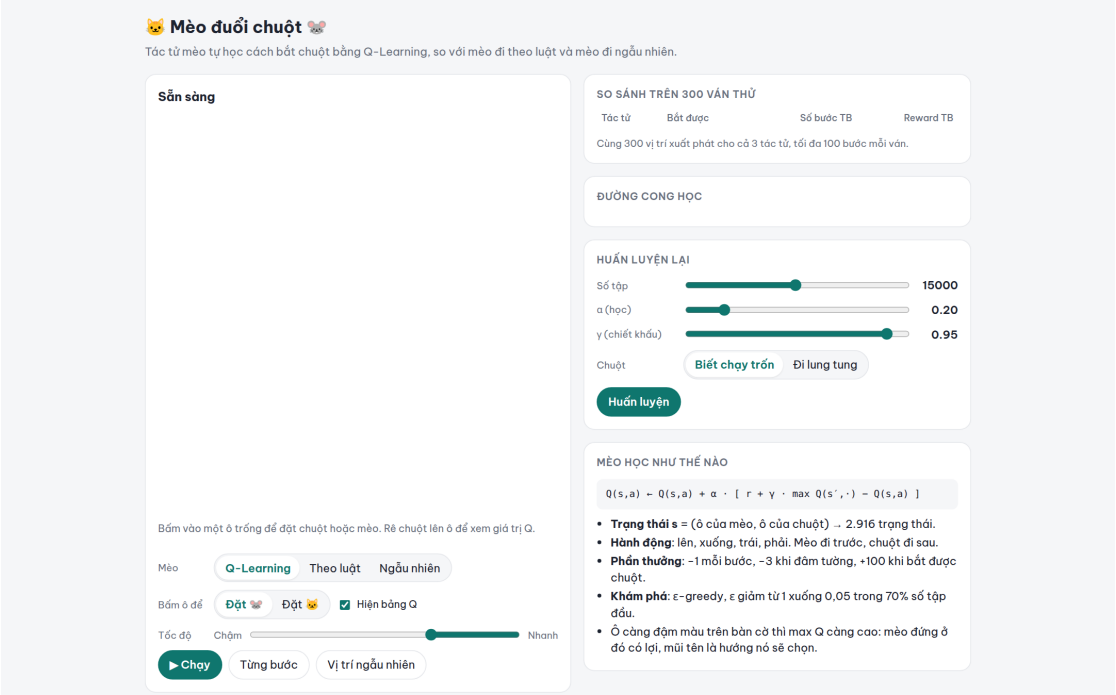
Toàn bộ kết quả của cả hai bài thực nghiệm được đóng gói trong notebook `notebooks/ch7_mas_tieu_duong.ipynb` (các ô `MAS-00` đến `MAS-04`) và phục vụ trực tiếp

trên ứng dụng web **Diabetes AI Lab** tại `http://localhost:8095`:

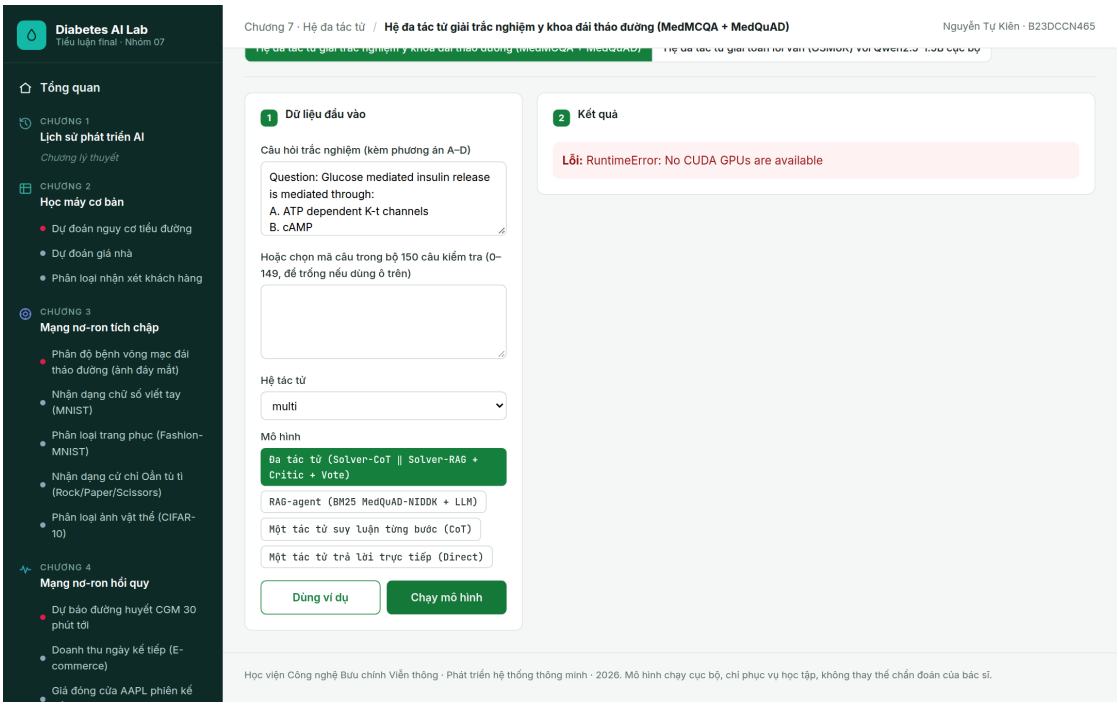
- **Ô `MAS-01` (Tổng hợp 3 thí nghiệm học tăng cường Mèo đuổi chuột)**: Ô `MAS-01` nạp tệp `outputs/metrics/ch7_catmouse.json` và in trực tiếp bảng so sánh: ở Thí nghiệm A, Q-learning đạt tỷ lệ bắt 100% với số bước trung bình 15,26 và tổng phần thưởng +83,84 (so với 24,3% và −189,45 của mèo theo luật Manhattan); ở Thí nghiệm B, không gian trạng thái mở rộng lên 157.464 bộ ba vị trí, hai mèo IQL đạt tỷ lệ bắt 99,67% sau 22,13 bước; ở Thí nghiệm C, ma trận đối kháng 3×2 chỉ rõ sự sụt giảm từ 100% xuống 4,0% của mèo Q đơn tác tử khi gặp chuột Q-learning, và sự phục hồi lên 75,0% nhờ huấn luyện đối kháng.
- **Ô `MAS-03` và `MAS-04` (Giải phẫu cổng đồng thuận, cận trên Oracle và vết tranh luận thật trên MedMCQA)**: Ô `MAS-03` duyệt qua 150 dòng nhật ký trong `outputs/ch7/rows_multi.jsonl`, tách riêng 87 câu đồng thuận vòng 1 (Accuracy = 0,701) và 63 câu bất đồng (Accuracy = 0,365), đồng thời tính cận trên Oracle ($107/150 = 0{,}713$). Tiếp đó, ô `MAS-04` in nguyên văn vết thông điệp `(agent, text)` của câu `#18` (nơi `Solver-RAG` dùng bằng chứng NIDDK cứu `Solver-CoT`) và câu `#101` (nơi `Solver-CoT` và `Critic` cứu `Solver-RAG`).

Trên ứng dụng web cục bộ tại `http://localhost:8095`:

- Trang `http://localhost:8095/catmouse` (Hình 7.6) cho phép xem trực quan từng bước di chuyển trên lưới 8×8 có tường của ba chế độ: *1 mèo (so sánh Q-learning vs Luật trên cùng vị trí xuất phát)*, *2 mèo hợp tác (IQL)* và *Đối kháng (Mèo Q đấu Chuột Q)*, kèm biểu đồ đường cong học và ma trận đối kháng.
- Tab **Ch.7 — Đa tác tử LLM · Trắc nghiệm ĐTĐ (MedMCQA)** trên trang chính (Hình 7.7) cho phép chọn câu hỏi mẫu hoặc nhập câu hỏi mới, đối chiếu đáp án và thời gian của cả 4 hệ (`Direct`, `CoT`, `RAG-agent`, `Đa tác tử`) và hiển thị chi tiết từng bước trong vết tranh luận (`Solver-CoT` → `Tool:search` → `Solver-RAG` → `Critic` → `Solver-CoT#2`).



Hình 7.6. Giao diện mô phỏng tương tác Bài 1 (Mèo đuổi chuột đa tác tử) tại `http://localhost:8095/catmouse`



Hình 7.7. Giao diện thử nghiệm Bài 2 (Hệ đa tác tử LLM giải trắc nghiệm đái tháo đường MedMCQA) tại `http://localhost:8095`

7.6. Tiểu kết chương 7

Chương 7 khép lại hành trình của tiểu luận bằng cách chuyển từ một mô hình đơn lẻ sang *nhiều tác tử cùng tương tác và học*. Hai bài thực nghiệm đại diện cho hai nhánh

lớn của hệ đa tác tử hiện đại và dẫn tới bốn kết luận cốt lõi:

1. **Hợp tác mở rộng năng lực nhưng làm tăng chi phí phối hợp**: Trong lưới có vật cản, hai mèo theo luật bắt gấp đôi một mèo (52% so với 24%); hai mèo học độc lập (IQL) bắt được 99,7% nhưng cần số tập huấn luyện gấp 2,7 lần do không gian trạng thái tăng 54 lần và mỗi mèo phải thích nghi với chính sách đang thay đổi của đồng đội.
2. **Tính không dừng là thách thức bản chất của học đa tác tử đối kháng**: Một tác tử Q-learning đạt 100% trước đối thủ cố định thực chất chỉ học cách khai thác điểm yếu riêng của đối thủ đó, và sụp xuống 4% khi đối thủ cũng biết học. Huấn luyện đối kháng đồng thời (cùng học trong trò chơi Markov) là điều kiện cần để thu được chính sách bền vững (75% trước chuột học và 100% trước chuột luật).
3. **Với tác tử LLM, lợi ích của suy luận nhiều bước và đa tác tử phụ thuộc chặt vào cấu trúc bài toán**: Trên bài toán tính toán nhiều bước có công cụ kiểm chứng tất định (GSM8K + Python), suy luận từng bước và đa tác tử nâng độ chính xác từ 17,5% (`Direct`) lên 70,0% (`CoT`) và 72,5% (`MAS`). Ngược lại, trên câu hỏi trắc nghiệm y khoa kiểm tra hồi tưởng tri thức với LLM nhỏ 1,5B (MedMCQA), trả lời trực tiếp (`Direct`, 64,0%) ít bị nhiễu tự sinh hơn `CoT` (57,3%) và `RAG` trên kho phổ thông (54,0%).
4. **Cổng đồng thuận đa cơ chế đáng tin cậy hơn bộ phân xử dùng LLM nhỏ**: Ở cả hai miền, khi hai tác tử dùng hai cơ chế độc lập (suy luận nội tại và công cụ tra cứu/mã nguồn) đồng thuận ngay vòng đầu, độ chính xác đều vượt trội (**70,1%** trên MedMCQA và **94,4%** trên GSM8K). Tuy nhiên, khi các tác tử bất đồng, việc dùng chính LLM 1,5B làm trọng tài (`Critic`) tạo ra nút thắt cổ chai về lỗi phối hợp (23 câu trên MedMCQA, làm kết quả chung 56,0% thấp hơn nhiều so với cận trên Oracle 71,3%). Để khai thác hết tiềm năng Oracle của hệ đa tác tử trong y tế, bộ phân xử cần dựa vào quy tắc lâm sàng tường minh, đồ thị tri thức (Chương 5) hoặc một mô hình thẩm định chuyên biệt mạnh hơn các solver thành phần.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được

Tiểu luận đã đi trọn vẹn qua bảy chủ đề của môn Phát triển hệ thống thông minh, từ các mô hình đơn lẻ trên bảng, ảnh, chuỗi, văn bản, đồ thị cho tới hệ đa tác tử. Mỗi chủ đề đều được kiểm chứng bằng thực nghiệm trên dữ liệu công khai về đái tháo đường, đối chiếu thêm trên các miền dữ liệu khác, cài đặt bằng nhiều cách (NumPy tự viết từ đầu, Keras, PyTorch) và đóng gói thành notebook cùng ứng dụng web cục bộ. Bảng 7.7 tóm tắt dữ liệu, mô hình tốt nhất và kết quả chính của cả bảy chương.

Bảng 7.7. Tổng hợp kết quả thực nghiệm của bảy chương

Ch.	Dữ liệu	Mô hình tốt nhất	Kết quả chính (so với đối chứng)
1	Pima (768)	ADALINE / hồi quy logistic	AUC 0,837; hệ chuyên gia 6 luật đạt 0,795; perceptron 0,749 và thất bại với XOR
2	NHANES (20,6k) & Diabetes (100k)	Rừng ngẫu nhiên; MLP 3 cách	NHANES AUC 0,823 (trần yếu tố nguy cơ); Diabetes 100k: MLP NumPy/Keras/PyTorch trùng khít AUC 0,977–0,978, F1 0,804–0,806
3	RetinaMNIST (1.600) & MNIST/FMNIST	ResNet-18 chuyển giao; CNN 3 cách	RetinaMNIST: ResNet-18 QWK 0,700 so với 0,47–0,59 (CNN 3 cách học từ đầu) và 0,52 (MLP); trên MNIST CNN 3 cách đạt 98,8–99,7%
4	Shanghai CGM (112 người) & tài chính	LSTM 2–5 tầng (3 cách cài đặt)	RMSE 30 phút 13,66–13,84 mg/dL (persistence 19,05; NumPy RNN 15,02); 95,5% vùng Clarke A; giảm 27% sai số trên CGM nhưng 0% trên giá cổ phiếu/vàng
5	KB ADA + MedQuAD + KG 96k nút	E5-large; GraphRAG / KG-RAG	Hit@1 KB tiếng Việt 0,967 (rerank), 0,933 (E5) so với 0,80 (BM25); RAG phủ 95% dữ kiện, 0% số liệu không căn cứ; KG-RAG tính chính xác thống kê trên 1.830 hồ sơ
6	Pubmed-Diabetes (19.717) & Cora/CiteSeer	GCN (ít nhãn), GraphSAGE (đủ nhãn)	PubMed 60 nhãn: GCN tự cài đặt đạt 0,789 so với 0,722 (MLP cùng 32.259 tham số); lợi ích đồ thị +6,7 điểm (PubMed) tới +24,4 điểm (Cora); GCN không kết nối tắt sụp từ 6 tầng
7	Mèo–Chuột (8×8) & MedMCQA (150) / GSM8K (40)	Q-learning đối kháng; Đa tác tử LLM có cổng đồng thuận	Bài 1: 2 mèo IQL bắt 99,7%; mèo Q đơn tác tử sụp từ 100% xuống 4% trước chuột học, học đối kháng phục hồi lên 75%. Bài 2: MAS đạt 72,5% trên GSM8K (94,4% khi CoT=PAL); trên MedMCQA cổng đồng thuận CoT=RAG đạt 70,1% (Oracle 71,3%), toàn tập 56,0% do 23 lỗi phối hợp của Critic 1,5B

Từ các thực nghiệm xuyên suốt bảy chương có thể rút ra sáu bài học chung.

Thứ nhất, thông tin trong dữ liệu và giao thức đánh giá quyết định trần hiệu năng hơn là thuật toán hay framework. Ở Chương 2,3,4, cùng một kiến trúc viết bằng NumPy,

Keras hay PyTorch đều hội tụ về cùng một mức chất lượng (sai lệch đạo hàm chỉ $10^{-12} \sim 10^{-11}$), chỉ khác nhau tốc độ thực thi. Ở Chương 4, cùng mạng LSTM giảm 27–35% sai số trên chuỗi glucose và doanh thu nhưng không thắng nổi dự báo ngây thơ trên giá cổ phiếu và vàng.

Thứ hai, thiên kiến quy nạp phù hợp với cấu trúc dữ liệu tạo nên khác biệt lớn nhất. Tích chập (kết nối cục bộ, chia sẻ trọng số) cho ảnh, cổng nhớ cộng tính cho chuỗi, phép nhân ma trận kề chuẩn hóa $\hat{A}$ (không tốn thêm tham số nào) cho đồ thị: mỗi kiến trúc thắng mô hình "phẳng" ở đúng loại dữ liệu của nó với ít tham số hơn.

Thứ ba, tri thức có sẵn rất có giá trị khi dữ liệu nhỏ. Học chuyển giao từ ImageNet nâng QWK thêm khoảng 0,15. Một bộ 6 luật y khoa đánh bại perceptron. Đồ thị trích dẫn bù cho việc chỉ có 20 nhãn mỗi lớp.

Thứ tư, mô hình đơn giản là mốc đối chứng không được bỏ qua. Persistence, hồi quy tuyến tính, BM25 và trả lời trực tiếp (`Direct`) trên câu hỏi sự kiện đều mạnh hơn dự kiến; chỉ những cải thiện vượt được chúng mới thực sự có ý nghĩa.

Thứ năm, trong hệ đa tác tử, môi trường không dừng và cơ chế phối hợp là hai nút thắt quyết định. Với học tăng cường, chính sách học trước đối thủ cố định sẽ quá khớp với đối thủ đó (tỷ lệ bắt sụt từ 100% xuống 4%) nếu không được huấn luyện đối kháng. Với tác tử LLM, kiểm chứng chéo hai cơ chế độc lập (cổng đồng thuận `CoT = RAG` hoặc `CoT = Python`) nâng mạnh độ tin cậy (70,1% và 94,4

Thứ sáu, trong y tế, hệ thống tốt là hệ thống biết giới hạn của mình và kiểm chứng được. Ngưỡng xét nghiệm chuẩn (ADA) vẫn là căn cứ chẩn đoán; mô hình học được chỉ bổ sung ước lượng nguy cơ. Với RAG và đa tác tử, mọi câu trả lời cần kèm trích dẫn, vết suy luận minh bạch và biết từ chối hoặc báo bất đồng khi thiếu căn cứ.

2. Hạn chế

- Dữ liệu huấn luyện chủ yếu đến từ Hoa Kỳ (NHANES, Pima), Trung Quốc (Shanghai) và Ấn Độ (MedMCQA). Chưa có dữ liệu lâm sàng thật của người Việt Nam; nhóm người gốc Á của NHANES chỉ là phép xấp xỉ.
- RetinaMNIST có độ phân giải thấp (128×128) và chỉ 1.600 ảnh. Mô hình CNN chưa đủ tin cậy để dùng lâm sàng; Grad-CAM cho thấy mô hình đôi khi chú ý vào vùng lóa và viền ảnh.
- Dự báo glucose chưa dùng thông tin bữa ăn và liều tiêm insulin, nên sai số tăng nhanh sau 30 phút.

- Ở Chương 7, giới hạn 4 GB VRAM buộc hệ đa tác tử phải dùng mô hình cục bộ kích thước nhỏ (`Qwen2.5-1.5B-Instruct`) và kho văn bản phổ thông MedQuAD-NIDDK, khiến tác tử `Critic` chưa đủ thẩm quyền y khoa để phân xử các câu hỏi mức sau đại học khi hai solver bất đồng (gây ra 23 lỗi phối hợp).

3. Hướng phát triển

- Thu thập dữ liệu sàng lọc của người Việt (phối hợp cơ sở y tế) để hiệu chỉnh lại mô hình và ngưỡng quyết định.
- Thay RetinaMNIST bằng ảnh đáy mắt độ phân giải cao (APTOS, EyePACS), áp dụng mô hình nền tảng thị giác cho ảnh y khoa.
- Dự báo glucose đa biến (bữa ăn, insulin, vận động) bằng Transformer cho chuỗi thời gian; dùng hàm mất mát bất đối xứng ưu tiên vùng hạ đường huyết.
- Ở hệ đa tác tử y khoa (Chương 7), thay bộ phân xử thuần ngôn ngữ của LLM 1,5B bằng bộ kiểm chứng lai kết hợp Đồ thị tri thức lâm sàng (Chương 5) và công cụ kiểm tra ngưỡng xét nghiệm bằng Python, từ đó thu hẹp khoảng cách từ 56,0% tiến tới cận trên Oracle 71,3%.
- Chuẩn hóa giao tiếp công cụ theo Model Context Protocol [115]; thêm đăng nhập, phân quyền và nhật ký kiểm toán để đáp ứng yêu cầu bảo vệ dữ liệu sức khỏe.

TÀI LIỆU THAM KHẢO

[1] International Diabetes Federation, *IDF Diabetes Atlas*, 10th ấn bản. Brussels: International Diabetes Federation, 2021.

[2] American Diabetes Association Professional Practice Committee, "2. diagnosis and classification of diabetes: Standards of care in diabetes—2024," *Diabetes Care*, tập 47, số Suppl. 1, S20–S42, 2024.

[3] Bộ Y tế, *Quyết định số 5481/QĐ-BYT ban hành tài liệu chuyên môn "Hướng dẫn chẩn đoán và điều trị đái tháo đường típ 2"*, 2020.

[4] J. W. Smith và cộng sự, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," trong *Proceedings of the Annual Symposium on Computer Application in Medical Care*, 1988, tr. 261–265.

[5] Centers for Disease Control and Prevention, *National health and nutrition examination survey (NHANES) 2011–2018*, `https://wwwn.cdc.gov/nchs/nhanes/`, 2020.

[6] J. Yang và cộng sự, "MedMNIST v2 – a large-scale lightweight benchmark for 2D and 3D biomedical image classification," *Scientific Data*, tập 10, tr. 41, 2023.

[7] Q. Zhao và cộng sự, "Chinese diabetes datasets for data-driven machine learning," *Scientific Data*, tập 10, tr. 35, 2023.

[8] A. Ben Abacha và D. Demner-Fushman, "A question-entailment approach to question answering," *BMC Bioinformatics*, tập 20, tr. 511, 2019.

[9] G. Namata, B. London, L. Getoor, và B. Huang, "Query-driven active surveying for collective classification," trong *Workshop on Mining and Learning with Graphs (MLG)*, 2012.

[10] A. Pal, L. K. Umapathi, và M. Sankarasubbu, "MedMCQA: A large-scale multi-subject multi-choice dataset for medical domain question answering," trong *Conference on Health, Inference, and Learning (CHIL)*, 2022, tr. 248–260.

[11] S. Russell và P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ấn bản. Pearson, 2021.

[12] A. M. Turing, "Computing machinery and intelligence," *Mind*, tập 59, số 236, tr. 433–460, 1950.

[13] W. S. McCulloch và W. Pitts, "A logical calculus of the ideas immanent in nervous activity," *Bulletin of Mathematical Biophysics*, tập 5, tr. 115–133, 1943.

[14] J. McCarthy, M. L. Minsky, N. Rochester, và C. E. Shannon, "A proposal for the dartmouth summer research project on artificial intelligence," Dartmouth College, Tech. Rep., 1955.

[15] F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, tập 65, số 6, tr. 386–408, 1958.

[16] B. Widrow và M. E. Hoff, "Adaptive switching circuits," trong *IRE WESCON Convention Record*, tập 4, 1960, tr. 96–104.

[17] M. Minsky và S. Papert, *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.

[18] J. Lighthill, "Artificial intelligence: A general survey," Science Research Council, UK, Tech. Rep., 1973.

[19] E. H. Shortliffe và B. G. Buchanan, "A model of inexact reasoning in medicine," *Mathematical Biosciences*, tập 23, số 3–4, tr. 351–379, 1975.

[20] D. E. Rumelhart, G. E. Hinton, và R. J. Williams, "Learning representations by back-propagating errors," *Nature*, tập 323, tr. 533–536, 1986.

[21] C. Cortes và V. Vapnik, "Support-vector networks," *Machine Learning*, tập 20, tr. 273–297, 1995.

[22] L. Breiman, "Random forests," *Machine Learning*, tập 45, tr. 5–32, 2001.

[23] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, tập 29, số 5, tr. 1189–1232, 2001.

[24] S. Hochreiter và J. Schmidhuber, "Long short-term memory," *Neural Computation*, tập 9, số 8, tr. 1735–1780, 1997.

[25] Y. LeCun, L. Bottou, Y. Bengio, và P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, tập 86, số 11, tr. 2278–2324, 1998.

[26] I. Goodfellow, Y. Bengio, và A. Courville, *Deep Learning*. MIT Press, 2016.

[27] G. E. Hinton, S. Osindero, và Y.-W. Teh, "A fast learning algorithm for deep belief nets," *Neural Computation*, tập 18, số 7, tr. 1527–1554, 2006.

[28] J. Deng và cộng sự, "ImageNet: A large-scale hierarchical image database," trong *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009.

[29] A. Krizhevsky, I. Sutskever, và G. E. Hinton, "Imagenet classification with deep convolutional neural networks," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2012.

[30] T. B. Brown và cộng sự, "Language models are few-shot learners," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[31] L. Ouyang và cộng sự, "Training language models to follow instructions with human feedback," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[32] J. Jumper và cộng sự, "Highly accurate protein structure prediction with AlphaFold," *Nature*, tập 596, tr. 583–589, 2021.

[33] Thủ tướng Chính phủ, *Quyết định số 127/qđ-ttg ban hành chiến lược quốc gia về nghiên cứu, phát triển và ứng dụng trí tuệ nhân tạo đến năm 2030*, 2021.

[34] F. Pedregosa và cộng sự, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, tập 12, tr. 2825–2830, 2011.

[35] D. P. Kingma và J. Ba, "Adam: A method for stochastic optimization," trong *International Conference on Learning Representations (ICLR)*, 2015.

[36] T. Cover và P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, tập 13, số 1, tr. 21–27, 1967.

[37] G. Ke và cộng sự, "LightGBM: A highly efficient gradient boosting decision tree," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[38] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," *Advances in Large Margin Classifiers*, tập 10, tr. 61–74, 1999.

[39] S. P. Lloyd, "Least squares quantization in PCM," trong *IEEE Transactions on Information Theory*, tập 28, 1982, tr. 129–137.

[40] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," *Journal of Computational and Applied Mathematics*, tập 20, tr. 53–65, 1987.

[41] W. J. Youden, "Index for rating diagnostic tests," *Cancer*, tập 3, số 1, tr. 32–35, 1950.

[42] V. Gulshan và cộng sự, "Development and validation of a deep learning algorithm for detection of diabetic retinopathy in retinal fundus photographs," *JAMA*, tập 316, số 22, tr. 2402–2410, 2016.

[43] D. H. Hubel và T. N. Wiesel, "Receptive fields, binocular interaction and functional architecture in the cat's visual cortex," *Journal of Physiology*, tập 160, tr. 106–154, 1962.

[44] A. Howard và cộng sự, "Searching for MobileNetV3," trong *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.

[45] S. Ioffe và C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," trong *International Conference on Machine Learning (ICML)*, 2015.

[46] N. Srivastava và cộng sự, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, tập 15, tr. 1929–1958, 2014.

[47] C. Szegedy và cộng sự, "Rethinking the inception architecture for computer vision," trong *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[48] I. Loshchilov và F. Hutter, "Decoupled weight decay regularization," trong *International Conference on Learning Representations (ICLR)*, 2019.

[49] K. Simonyan và A. Zisserman, "Very deep convolutional networks for large-scale image recognition," trong *International Conference on Learning Representations (ICLR)*, 2015.

[50] K. He, X. Zhang, S. Ren, và J. Sun, "Deep residual learning for image recognition," trong *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, tr. 770–778.

[51] R. R. Selvaraju và cộng sự, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," trong *IEEE International Conference on Computer Vision (ICCV)*, 2017.

[52] R. Liu và cộng sự, "DeepDRiD: Diabetic retinopathy—grading and image quality estimation challenge," *Patterns*, tập 3, số 6, tr. 100 512, 2022.

[53] C. P. Wilkinson và cộng sự, "Proposed international clinical diabetic retinopathy and diabetic macular edema disease severity scales," *Ophthalmology*, tập 110, số 9, tr. 1677–1682, 2003.

[54] J. Cohen, "Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit," *Psychological Bulletin*, tập 70, số 4, tr. 213–220, 1968.

[55] J. L. Elman, "Finding structure in time," *Cognitive Science*, tập 14, số 2, tr. 179–211, 1990.

[56] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proceedings of the IEEE*, tập 78, số 10, tr. 1550–1560, 1990.

[57] Y. Bengio, P. Simard, và P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, tập 5, số 2, tr. 157–166, 1994.

[58] R. Pascanu, T. Mikolov, và Y. Bengio, "On the difficulty of training recurrent neural networks," trong *International Conference on Machine Learning (ICML)*, 2013.

[59] K. Cho và cộng sự, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," trong *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014.

[60] I. Sutskever, O. Vinyals, và Q. V. Le, "Sequence to sequence learning with neural networks," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[61] D. Bahdanau, K. Cho, và Y. Bengio, "Neural machine translation by jointly learning to align and translate," trong *International Conference on Learning Representations (ICLR)*, 2015.

[62] A. Vaswani và cộng sự, "Attention is all you need," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[63] T. Battelino và cộng sự, "Clinical targets for continuous glucose monitoring data interpretation: Recommendations from the international consensus on time in range," *Diabetes Care*, tập 42, số 8, tr. 1593–1603, 2019.

[64] W. L. Clarke và cộng sự, "Evaluating clinical accuracy of systems for self-monitoring of blood glucose," *Diabetes Care*, tập 10, số 5, tr. 622–628, 1987.

[65] K. Li và cộng sự, "GluNet: A deep learning framework for accurate glucose forecasting," *IEEE Journal of Biomedical and Health Informatics*, tập 24, số 2, tr. 414–423, 2020.

[66] Z. Ji và cộng sự, "Survey of hallucination in natural language generation," trong *ACM Computing Surveys*, tập 55, 2023.

[67] A. Hogan và cộng sự, "Knowledge graphs," *ACM Computing Surveys*, tập 54, số 4, tr. 1–37, 2021.

[68] A. Singhal, *Introducing the knowledge graph: Things, not strings*, Google Official Blog, 2012.

[69] Apache Software Foundation, *Apache AGE: Graph database extension for PostgreSQL*, `https://age.apache.org`, 2024.

[70] A. Bordes và cộng sự, "Translating embeddings for modeling multi-relational data," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2013.

[71] P. Lewis và cộng sự, "Retrieval-augmented generation for knowledge-intensive NLP tasks," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[72] Y. Gao và cộng sự, *Retrieval-augmented generation for large language models: A survey*, arXiv:2312.10997, 2023.

[73] S. Robertson và H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, tập 3, số 4, tr. 333–389, 2009.

[74] N. Reimers và I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," trong *EMNLP-IJCNLP*, 2019.

[75] V. Karpukhin và cộng sự, "Dense passage retrieval for open-domain question answering," trong *EMNLP*, 2020.

[76] N. Reimers và I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," trong *EMNLP*, 2020.

[77] Y. A. Malkov và D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, tập 42, số 4, tr. 824–836, 2020.

[78] A. Kane, *Pgvector: Open-source vector similarity search for Postgres*, `https://github.com/pgvector/pgvector`, 2024.

[79] G. V. Cormack, C. L. A. Clarke, và S. Büttcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," trong *ACM SIGIR*, 2009, tr. 758–759.

[80] R. Nogueira và K. Cho, "Passage re-ranking with BERT," trong *arXiv:1901.04085*, 2019.

[81] D. Edge và cộng sự, *From local to global: A graph RAG approach to query-focused summarization*, arXiv:2404.16130, 2024.

[82] S. Es, J. James, L. Espinosa-Anke, và S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," trong *EACL (System Demonstrations)*, 2024.

[83] L. Wang và cộng sự, *Multilingual E5 text embeddings: A technical report*, arXiv:2402.05672, 2024.

[84] J. Chen và cộng sự, *BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation*, arXiv:2402.03216, 2024.

[85] M. Zitnik, M. Agrawal, và J. Leskovec, "Modeling polypharmacy side effects with graph convolutional networks," trong *Bioinformatics*, tập 34, 2018, tr. i457–i466.

[86] D. Zhou và cộng sự, "Learning with local and global consistency," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2004.

[87] M. McPherson, L. Smith-Lovin, và J. M. Cook, "Birds of a feather: Homophily in social networks," *Annual Review of Sociology*, tập 27, tr. 415–444, 2001.

[88] B. Perozzi, R. Al-Rfou, và S. Skiena, "DeepWalk: Online learning of social representations," trong *ACM SIGKDD*, 2014.

[89] J. Gilmer và cộng sự, "Neural message passing for quantum chemistry," trong *International Conference on Machine Learning (ICML)*, 2017.

[90] Z. Wu và cộng sự, "A comprehensive survey on graph neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, tập 32, số 1, tr. 4–24, 2021.

[91] T. N. Kipf và M. Welling, "Semi-supervised classification with graph convolutional networks," trong *International Conference on Learning Representations (ICLR)*, 2017.

[92] W. L. Hamilton, R. Ying, và J. Leskovec, "Inductive representation learning on large graphs," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[93] P. Veličković và cộng sự, "Graph attention networks," trong *International Conference on Learning Representations (ICLR)*, 2018.

[94] Q. Li, Z. Han, và X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," trong *AAAI Conference on Artificial Intelligence*, 2018.

[95] D. Chen và cộng sự, "Measuring and relieving the over-smoothing problem for graph neural networks from the topological view," trong *AAAI Conference on Artificial Intelligence*, 2020.

[96] K. Xu, W. Hu, J. Leskovec, và S. Jegelka, "How powerful are graph neural networks?" Trong *International Conference on Learning Representations (ICLR)*, 2019.

[97] P. Sen và cộng sự, "Collective classification in network data," *AI Magazine*, tập 29, số 3, tr. 93–106, 2008.

[98] M. Fey và J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," trong *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[99] Z. Yang, W. W. Cohen, và R. Salakhutdinov, "Revisiting semi-supervised learning with graph embeddings," trong *International Conference on Machine Learning (ICML)*, 2016.

[100] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ấn bản. Wiley, 2009.

[101] N. R. Jennings, K. Sycara, và M. Wooldridge, "A roadmap of agent research and development," *Autonomous Agents and Multi-Agent Systems*, tập 1, tr. 7–38, 1998.

[102] Foundation for Intelligent Physical Agents, *FIPA ACL message structure specification (SC00061G)*, 2002.

[103] L. Busoniu, R. Babuska, và B. De Schutter, "A comprehensive survey of multiagent reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, tập 38, số 2, tr. 156–172, 2008.

[104] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," trong *Proceedings of the Tenth International Conference on Machine Learning (ICML)*, 1993, tr. 330–337.

[105] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," trong *Proceedings of the Eleventh International Conference on Machine Learning (ICML)*, 1994, tr. 157–163.

[106] J. Wei và cộng sự, "Chain-of-thought prompting elicits reasoning in large language models," trong *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[107] L. Gao và cộng sự, "PAL: Program-aided language models," trong *International Conference on Machine Learning (ICML)*, 2023.

[108] Q. Wu và cộng sự, *AutoGen: Enabling next-gen LLM applications via multi-agent conversation*, arXiv:2308.08155, 2023.

[109] S. Hong và cộng sự, "MetaGPT: Meta programming for a multi-agent collaborative framework," trong *International Conference on Learning Representations (ICLR)*, 2024.

[110] Y. Du và cộng sự, "Improving factuality and reasoning in language models through multiagent debate," trong *International Conference on Machine Learning (ICML)*, 2024.

[111] C. J. C. H. Watkins và P. Dayan, "Q-learning," *Machine Learning*, tập 8, tr. 279–292, 1992.

[112] R. S. Sutton và A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ấn bản. MIT Press, 2018.

[113] Qwen Team, *Qwen2.5 technical report*, arXiv:2412.15115, 2024.

[114] K. Cobbe và cộng sự, *Training verifiers to solve math word problems*, arXiv:2110.14168, 2021.

[115] Anthropic, *Model context protocol specification*, `https://modelcontextprotocol.io`, 2024.